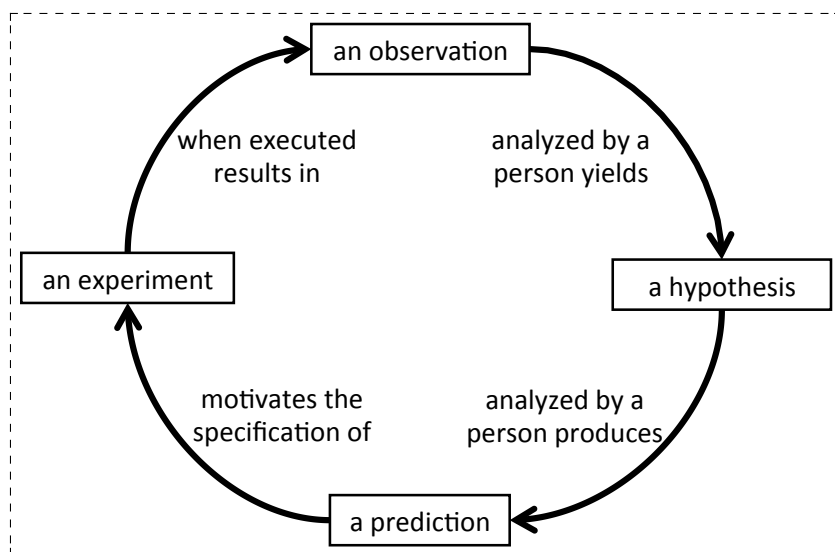


Category Theory for Scientists

David I. Spivak

April 1, 2018



How can mathematics make this diagram meaningful?

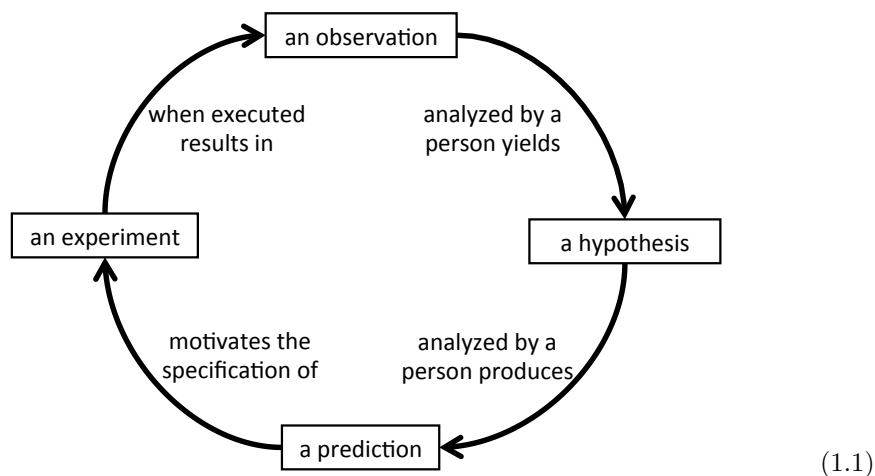
Contents

1	Introduction	5
1.1	A brief history of category theory	7
1.2	Intention of this book	8
1.3	What is requested from the student	10
1.4	Category theory references	10
1.5	Acknowledgments	10
2	The category of sets	13
2.1	Sets and functions	13
2.2	Commutative diagrams	19
2.3	Ologs	20
2.4	Products and coproducts	30
2.5	Finite limits in Set	38
2.6	Finite colimits in Set	47
2.7	Other notions in Set	52
3	Categories and functors, without admitting it	63
3.1	Monoids	63
3.2	Groups	75
3.3	Graphs	78
3.4	Orders	84
3.5	Databases: schemas and instances	91
4	Basic category theory	101
4.1	Categories and Functors	101
4.2	More basic examples from mathematics	114
4.3	Natural transformations	127
4.4	Categories and schemas are equivalent, Cat $\simeq$ Sch	146
4.5	Limits and colimits	150
4.6	Other notions in Cat	171
5	Categories at work	181
5.1	Adjunctions	181
5.2	Categories of functors	193
5.3	Monads	203
5.4	Operads	212

Chapter 1

Introduction

The title page of this book contains a graphic that we reproduce here.



It is intended to evoke thoughts of the scientific method.

A hypothesis analyzed by a person produces a prediction, which motivates the specification of an experiment, which when executed results in an observation, which analyzed by a person yields a hypothesis.

This sounds valid, and a good graphic can be exceptionally useful for leading a reader through the story that the author wishes to tell.

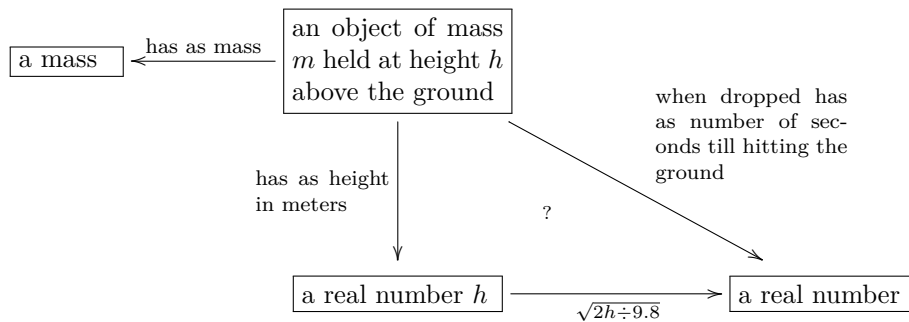
Interestingly, a graphic has the power to evoke feelings of understanding, without really meaning much. The same is true for text: it is possible to use a language such as English to express ideas that are never made rigorous or clear. When someone says “I believe in free will,” what does she believe in? We may all have some concept of what she’s saying—something we can conceptually work with and discuss or argue about. But to what extent are we all discussing the same thing, the thing she intended to convey?

Science is about agreement. When we supply a convincing argument, the result of this convincing is agreement. When in an experiment the observation matches the hypothesis—success!—that is agreement. When my methods make sense to you, that is agreement. When practice does not agree with theory, that is disagreement. Agreement is the good stuff in science; it’s the high fives.

But it is easy to think we're in agreement, when really we're not. Modeling our thoughts on heuristics and pictures may be convenient for quick travel down the road, but we're liable to miss our turnoff at the first mile. The danger is in mistaking our convenient conceptualizations for what's actually there. It is imperative that we have the ability at any time to ground out in reality. What does that mean?

Data. Hard evidence. The physical world. It is here that science touches down and heuristics evaporate. So let's look again at the diagram on the cover. It is intended to evoke an idea of how science is performed. Is there hard evidence and data to back this theory up? Can we set up an experiment to find out whether science is actually performed according to such a protocol? To do so we have to shake off the stupor evoked by the diagram and ask the question: "what does this diagram intend to communicate?"

In this course I will use a mathematical tool called *ologs*, or ontology logs, to give some structure to the kinds of ideas that are often communicated in pictures like the one on the cover. Each olog inherently offers a framework in which to record data about the subject. More precisely it encompasses a *database schema*, which means a system of interconnected tables that are initially empty but into which data can be entered. For example consider the olog below



This olog represents a framework in which to record data about objects held above the ground, their mass, their height, and a comparison (the ?-mark in the middle) between the number of seconds till they hit the ground and a certain real-valued function of their height. We will discuss ologs in detail throughout this course.

The picture in (1.1) looks like an olog, but it does not conform to the rules that we lay out for ologs in Section 2.3. In an olog, every arrow is intended to represent a mathematical function. It is difficult to imagine a function that takes in predictions and outputs experiments, but such a function is necessary in order for (1.1) to make sense. To produce an experiment design from a prediction probably requires an expert, and even then the expert may be motivated to specify a different experiment on Tuesday than he is on Monday. But perhaps our criticism has led to a way forward: if we say that every arrow represents a function *when in the context of a specific expert who is actually doing the science at a specific time*, then figure 1.1 begins to make sense. In fact, we will return to the figure in Section 5.3 (specifically Example 5.3.3.9), where background methodological context is discussed in earnest.

This course is an attempt to extol the virtues of a new branch of mathematics, called *category theory*, which was invented for powerful communication of ideas between different fields and subfields within mathematics. By powerful communication of ideas I actually mean something precise. Different branches of mathematics can be formalized into categories. These categories can then be connected together by functors. And the

sense in which these functors provide powerful communication of ideas is that facts and theorems proven in one category can be transferred through a connecting functor to yield proofs of an analogous theorem in another category. A functor is like a conductor of mathematical truth.

I believe that the language and toolset of category theory can be useful throughout science. We build scientific understanding by developing models, and category theory is the study of basic conceptual building blocks and how they cleanly fit together to make such models. Certain structures and conceptual frameworks show up again and again in our understanding of reality. No one would dispute that vector spaces are ubiquitous. But so are hierarchies, symmetries, actions of agents on objects, data models, global behavior emerging as the aggregate of local behavior, self-similarity, and the effect of methodological context.

Some ideas are so common that our use of them goes virtually undetected, such as set-theoretic intersections. For example, when we speak of a material that is both lightweight and ductile, we are intersecting two sets. But what is the use of even mentioning this fact? The answer is that when we formalize our ideas, our understanding is almost always clarified. Our ability to communicate with others is enhanced, and the possibility for developing new insights expands. And if we are ever to get to the point that we can input our ideas into computers, we will need to be able to formalize these ideas first.

It is my hope that this course will offer scientists a new vocabulary in which to think and communicate, and a new pipeline to the vast array of theorems that exist and are considered immensely powerful within mathematics. These theorems have not made their way out into the world of science, but they are directly applicable there. Hierarchies are posets, symmetries are group elements, data models are categories, agent actions are monoid actions, local-global principles are sheaves, self-similarity is modeled by operads, context can be modeled by monads.

1.1 A brief history of category theory

The paradigm shift brought on by Einstein's theory of relativity brought on the realization that there is no single perspective from which to view the world. There is no background framework that we need to find; there are infinitely many different frameworks and perspectives, and the real power lies in being able to translate between them. It is in this historical context that category theory got its start.¹

Category theory was invented in the early 1940s by Samuel Eilenberg and Saunders Mac Lane. It was specifically designed to bridge what may appear to be two quite different fields: topology and algebra. Topology is the study of abstract shapes such as 7-dimensional spheres; algebra is the study of abstract equations such as $y^2z = x^3 - xz^2$. People had already created important and useful links (e.g. cohomology theory) between these fields, but Eilenberg and Mac Lane needed to precisely compare different links with one another. To do so they first needed to boil down and extract the fundamental nature of these two fields. But the result was a structure that fit not only topology and algebra, but many other mathematical disciplines as well.

At first category theory was little more than a deeply clarifying language for existing difficult mathematical ideas. However, in 1957 Alexander Grothendieck used category

¹The following history of category theory is far too brief, and it may leave out important contributions or credit some with what should more aptly be credited to others. If you happen to know of such an error, please [contact the author](#).

theory to build new mathematical machinery (new cohomology theories) that granted unprecedented insight into the behavior of algebraic equations. Since that time, categories have been built specifically to zoom in on particular features of mathematical subjects and study them with a level of acuity that is simply unavailable elsewhere.

Bill Lawvere saw category theory as a new foundation for all mathematical thought. Mathematicians had been searching for foundations in the 19th century and were reasonably satisfied with set theory as *the foundation*. But Lawvere showed that the category of sets is simply a category with certain nice properties, not necessarily the center of the mathematical universe. He explained how whole algebraic theories can be viewed as examples of a single system. He and others went on to show that logic was beautifully captured in the setting of category theory (or more specifically toposes). It is here also that Grothendieck and his school worked out major results in algebraic geometry.

In 1980 Joachim Lambek showed that the types and programs used in computer science form a specific kind of category. This provided a new semantics for talking about programs, allowing people to investigate how programs combine and compose to create other programs, without caring about the specifics of implementation. Eugenio Moggi brought the category theoretic notion of monads into computer science to encapsulate ideas that up to that point were considered outside the realm of such theory.

It is difficult to explain the clarity and beauty brought to category theory by people like Daniel Kan and André Joyal. They have each repeatedly extracted the essence of a whole mathematical subject to reveal and formalize a stunningly simple yet extremely powerful pattern of thinking, revolutionizing how mathematics is done.

All this time, however, category theory was consistently seen by much of the mathematical community as ridiculously abstract. But in the 21st century it has finally come to find healthy respect within the larger community of pure mathematics. It is the language of choice for graduate-level algebra and topology courses, and in my opinion will continue to establish itself as the basic framework in which mathematics is done.

As mentioned above category theory has branched out into certain areas of science as well. Baez and Dolan have shown its value in making sense of quantum physics, it is well established in computer science, and it has found proponents in several other fields as well. But to my mind, we are the very beginning of its venture into scientific methodology. Category theory was invented as a bridge and it will continue to serve in that role.

1.2 Intention of this book

The world of *applied mathematics* is much smaller than the world of *applicable mathematics*. This course is intended to create a bridge to the vast array of mathematical concepts that are used daily by mathematicians to describe all manner of phenomena that arise in our studies, and which I have found naturally arise in thinking about physics, computation, and neuroscience.

To the pure mathematician I'll try to prove that concepts such as categories, functors, natural transformations, limits, colimits, functor categories, sheaves, monads, and operads—concepts that are often considered too abstract for even math majors—can be communicated to scientists with no math background beyond linear algebra. If this material is as teachable as I think, it means that category theory is not esoteric but somehow well-aligned with ideas that already make sense to the scientific mind. Note, however, that this book is example-based rather than proof-based, so it may not be suitable as a reference for students of pure mathematics.

To the scientist I'll try to prove the claim that category theory includes a formal treatment of conceptual structures that the scientist sees often perhaps without realizing that there is well-oiled mathematical machinery to be employed. We will work on the structure of information; how data is made meaningful by its connections, both internal and outreaching, to other data. Note, however, that this book should most certainly not be taken as a reference on scientific disciplines. One should assume that any account of physics, materials science, chemistry, etc. has been oversimplified. The intention is to give a flavor of how category theory may help us model scientific ideas, not to explain these ideas in a serious way.

Data gathering is ubiquitous in science. Giant databases are currently being mined for unknown patterns, but in fact there are many (many) known patterns that simply have not been catalogued. Consider the well-known case of medical records. A patient's medical history is often known by various individual doctor-offices but quite inadequately shared between them. Sharing medical records often means faxing a hand-written note or a filled-in house-created form between offices.

Similarly, in science there exists substantial expertise making brilliant connections between concepts, but it is being conveyed in silos of English prose known as journals articles. Every paper has a methods section, but it is almost impossible to read a methods section and subsequently repeat the experiment—the English is inadequate to concisely convey what is being done.

The first thing to understand is that reusable methodologies can be formalized, and that doing so is inherently valuable. Consider the following analogy. Suppose you want to add up the area of a region in space (or the area under a curve). You break the region down into small squares, each of which you know has area A ; then you count the number of squares, say n , and the result is that the region has an area of about nA . If you want a more precise and accurate result you repeat the process with half-size squares. This methodology can be used for any area-finding problem (of which there are more than a first-year calculus student generally realizes) and thus it deserves to be formalized. But once we have formalized this methodology, it can be taken to its limit and out comes integration by Riemann sums.

I intend to show that category theory is incredibly efficient as a language for experimental design patterns, introducing formality while remaining flexible. It forms a rich and tightly woven conceptual fabric that allows the scientist to maneuver between different perspectives whenever the need arises. Once one builds that fabric for oneself, he or she has an ability to think about models in a way that simply would not occur without it. Moreover, putting ideas into the language of category theory forces a person to clarify their assumptions. This is highly valuable both for the researcher and for his or her audience.

What must be recognized in order to find value in this course is that conceptual chaos is a major problem. Creativity demands clarity of thinking, and to think clearly about a subject requires an organized understanding of how it fits together. Organization and clarity also lead to better communication with others. Academics often say they are paid to think and understand, but that is not true. They are paid to think, understand, and *communicate their findings*. Universal languages for science—languages such as calculus and differential equations, matrices, or simply graphs and pie-charts—grant us a cohesiveness that makes scientific research worthwhile. In this book I will attempt to show that category theory can be similarly useful in describing complex scientific understandings.

1.3 What is requested from the student

I will do my best to make clear the value of category theory in science, but I am not a scientist. To that end I am asking for your help in exploring how category theory may be useful in your specific field.

I also want you to recognize that the value of mathematics is not generally obvious at first. A good student learning a good subject with a good teacher will see something compelling almost immediately, but may not see how it will be useful in real life. This will come later. I hope you will work hard to understand even without yet knowing what its actual value in your life and research will be. Like a student of soccer is encouraged to spend hours juggling the ball when he or she could be practicing penalty shots, it is important to gain facility with the materials you will be using. Doing exercises is imperative for learning mathematics.

1.4 Category theory references

I wrote this book because the available books on category theory are almost all written for mathematicians (the rest are written for computer scientists). There is one book by Lawvere and Schanuel, called *Conceptual Mathematics* [LS], that offers category theory to a wider audience, but its style is not appropriate for this course. Still, it is very well written and clear.

The “bible” of category theory is *Categories for the working mathematician* by Mac Lane [Mac]. But as the title suggests, it was written for working mathematicians and will be quite opaque to my target audience. However, once a person has read this book, Mac Lane’s book may become a valuable reference.

Other good books include Steve Awodey’s book *Category theory* [Awo] and Barr and Wells book *Category theory for computing science*, [BW]. And online I find wikipedia and a site called *the nlab* <http://ncatlab.org/nlab/show/HomePage> to be quite useful.

This book attempts to explain category theory by example and exercise rather than by theorems and proofs. I hope this approach will be valuable to the working scientist.

1.5 Acknowledgments

I would like to express my deep appreciation for the many scientists who I have worked with over the past five years. It all started with Paea LePendou who first taught me databases when I was naively knocking on doors in the University of Oregon computer science department. This book would never have been written if Tristan Nguyen and Dave Balaban had not noticed my work and encouraged me to continue. Dave Balaban and Peter Gates have been my scientific partners since the beginning, working hard to understand what I’m offering and working just as hard to help me understand all that I’m missing. Peter Gates has deepened my understanding of data in profound ways.

I have also been tremendously lucky to know Haynes Miller, who made it possible for me set down at MIT, with the help of Clark Barwick and Jacob Lurie. I knew that MIT would be the best place in the world for me to pursue this type of research, and it has really come through. Researchers like Markus Buehler and his graduate students Tristan Giesa and Dieter Brommer have been a pleasure to work with, and the many materials science examples scattered throughout this book is a testament to how much our work together has influenced my thinking.

I'd also like to thank my collaborators and conversation partners with whom I have discussed subjects written about in this book. Other than people mentioned above, these include Steve Awodey, Allen Brown, Adam Chlipala, Carlo Curino, Dan Dugger, Henrik Forsell, David Gepner, Jason Gross, Bob Harper, Ralph Hutchison, Robert Kent, Jack Morava, Scott Morrison, David Platt, Joey Perricone, Dylan Rupel, Guarav Singh, Sam Shames, Nat Stapleton, Patrick Schultz, Ka Yu Tam, Ryan Wisnesky, Jesse Wolfson, and Elizabeth Wood.

I would like to thank Peter Kleinhenz and Peter Gates for reading this book and providing invaluable feedback. The cover image is a mild alteration of something Gates sent me, and the text is littered with examples he and Kleinhenz provided.

I would like to thank my teacher, Peter Ralston, who taught me to repeatedly question the obvious. My ability to commit to a project like this one and to see it to fruition has certainly been enhanced since studying with him.

Finally, I acknowledge my appreciation for support from the Office of Naval Research² without which this book would not have been remotely possible. I believe that their funding of basic research is an excellent way of ensuring that the US remains a global leader in the years to come.

²Grant numbers: N000140910466, N000141010841, N000141310260

Chapter 2

The category of sets

The theory of sets was invented as a foundation for all of mathematics. The notion of sets and functions serves as a basis on which to build our intuition about categories in general. In this chapter we will give examples of sets and functions and then move on to discuss commutative diagrams. At this point we can introduce ologs which will allow us to use the language of category theory to speak about real world concepts. Then we will introduce limits and colimits, and their universal properties. All of this material is basic set theory, but it can also be taken as an investigation of our first category, the *category of sets*, which we call **Set**. We will end this chapter with some other interesting constructions in **Set** that do not fit into the previous sections.

2.1 Sets and functions

2.1.1 Sets

In this course I'll assume you know what a set is. We can think of a set X as a collection of things $x \in X$, each of which is recognizable as being in X and such that for each pair of named elements $x, x' \in X$ we can tell if $x = x'$ or not.¹ The set of pendulums is the collection of things we agree to call pendulums, each of which is recognizable as being a pendulum, and for any two people pointing at pendulums we can tell if they're pointing at the same pendulum or not.

Notation 2.1.1.1. The symbol $\emptyset$ denotes the set with no elements. The symbol $\mathbb{N}$ denotes the set of natural numbers, which we can write as

$$\mathbb{N} := \{0, 1, 2, 3, 4, \dots, 877, \dots\}.$$

The symbol $\mathbb{Z}$ denotes the set of integers, which contains both the natural numbers and their negatives,

$$\mathbb{Z} := \{\dots, -551, \dots, -2, -1, 0, 1, 2, \dots\}.$$

If A and B are sets, we say that A is a *subset* of B , and write $A \subseteq B$, if every element of A is an element of B . So we have $\mathbb{N} \subseteq \mathbb{Z}$. Checking the definition, one sees that

¹Note that the symbol x' , read “x-prime”, has nothing to do with calculus or derivatives. It is simply notation that we use to name a symbol that is suggested as being somehow like x . This suggestion of kinship between x and x' is meant only as an aid for human cognition, and not as part of the mathematics.

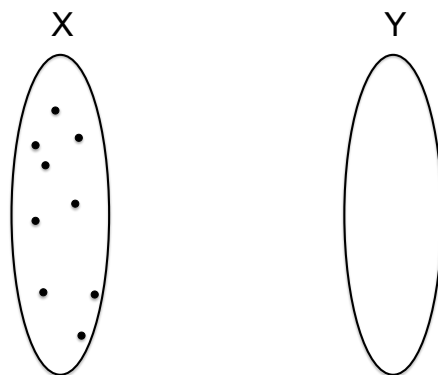


Figure 2.1: A set X with 9 elements and a set Y with no elements, $Y = \emptyset$.

for any set A , we have (perhaps uninteresting) subsets $\emptyset \subseteq A$ and $A \subseteq A$. We can use *set-builder notation* to denote subsets. For example the set of even integers can be written $\{n \in \mathbb{Z} \mid n \text{ is even}\}$. The set of integers greater than 2 can be written in many ways, such as

$$\{n \in \mathbb{Z} \mid n > 2\} \quad \text{or} \quad \{n \in \mathbb{N} \mid n > 2\} \quad \text{or} \quad \{n \in \mathbb{N} \mid n \geq 3\}.$$

The symbol $\exists$ means “there exists”. So we could write the set of even integers as

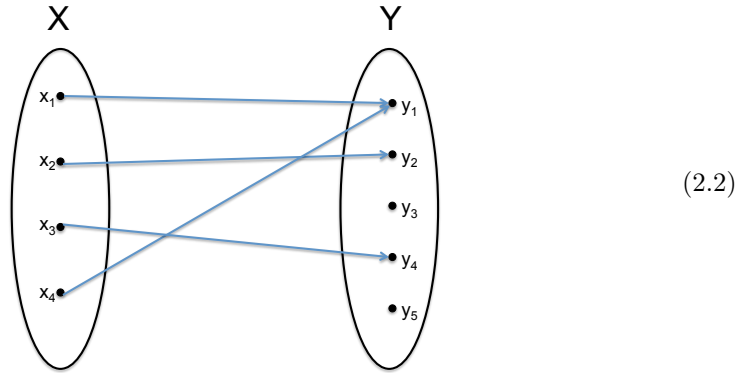
$$\{n \in \mathbb{Z} \mid n \text{ is even}\} = \{n \in \mathbb{Z} \mid \exists m \in \mathbb{Z} \text{ such that } 2m = n\}.$$

The symbol $\exists!$ means “there exists a unique”. So the statement “ $\exists! x \in \mathbb{R}$ such that $x^2 = 0$ ” means that there is one and only one number whose square is 0. Finally, the symbol $\forall$ means “for all”. So the statement “ $\forall m \in \mathbb{N} \exists n \in \mathbb{N}$ such that $m < n$ ” means that for every number there is a bigger one.

Exercise 2.1.1.2. Let $A = \{1, 2, 3\}$. What are all the subsets of A ? Hint: there are 8. $\diamond$

2.1.2 Functions

If X and Y are sets, then a *function* f from X to Y , denoted $f: X \rightarrow Y$, is a mapping that sends each element $x \in X$ to an element of Y , denoted $f(x) \in Y$. We call X the *domain* of the function f and we call Y the *codomain* of f .



Note that for every element $x \in X$, there is exactly one arrow emanating from x , but for an element $y \in Y$, there can be several arrows pointing to y , or there can be no arrows pointing to y .

Application 2.1.2.1. In studying the mechanics of materials, one wishes to know how a material responds to tension. For example a rubber band responds to tension differently than a spring does. To each material we can associate a **force-extension curve**, recording how much the force the material carries when extended to various lengths. Once we fix a methodology for performing experiments, finding a material's force-extension curve would ideally constitute a function from the set of materials to the set of curves. ²

◇◇

Exercise 2.1.2.2. Here is a simplified account of how the **brain receives light**. The eye contains about 100 million photoreceptor (PR) cells. Each connects to a retinal ganglion (RG) cell. No PR cell connects to two different RG cells, but usually many PR cells can attach to a single RG cell.

Let PR denote the set of photoreceptor cells and let RG denote the set of retinal ganglion cells.

- a.) According to the above account, does the connection pattern constitute a function $RG \rightarrow PR$, a function $PR \rightarrow RG$ or neither one?
- b.) Would you guess that the connection pattern that exists between other areas of the brain are “function-like”?

◇

Example 2.1.2.3. Suppose that X is a set and $X' \subseteq X$ is a subset. Then we can consider the function $X' \rightarrow X$ given by sending every element of X' to “itself” as an element of X . For example if $X = \{a, b, c, d, e, f\}$ and $X' = \{b, d, e\}$ then $X' \subseteq X$ and we turn that into the function $X \rightarrow X'$ given by $b \mapsto b, d \mapsto d, e \mapsto e$.

As a matter of notation, we may sometimes say something like the following: Let X be a set and let $i: X' \subseteq X$ be a subset. The good thing about this is that given a function $f: X \rightarrow Y$, we can compose it with $i: X' \rightarrow X$ and get a function $f \circ i: X' \rightarrow Y$. This is often called *the restriction of f to X'* , and we have obtained it simply by function composition.

²In reality, different samples of the same material, say samples of different sizes or at different temperatures, may have different force-extension curves. If we want to see this as a true function whose codomain is curves it should have as domain something like the set of material samples.

Given a function $f: X \rightarrow Y$, the elements of Y that have at least one arrow pointing to them are said to be *in the image* of f ; that is we have

$$\text{im}(f) := \{y \in Y \mid \exists x \in X \text{ such that } f(x) = y\}. \quad (2.3)$$

Exercise 2.1.2.4. If $f: X \rightarrow Y$ is depicted by (2.2) above, write its image, $\text{im}(f)$ as a set. $\diamond$

Given a function $f: X \rightarrow Y$ and a function $g: Y \rightarrow Z$, where the codomain of f is the same set as the domain of g (namely Y), we say that f and g are composable

$$X \xrightarrow{f} Y \xrightarrow{g} Z.$$

The *composition* of f and g is denoted by $g \circ f: X \rightarrow Z$.

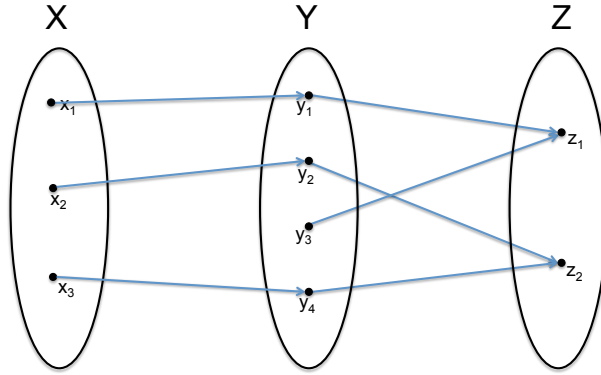


Figure 2.4: Functions $f: X \rightarrow Y$ and $g: Y \rightarrow Z$ compose to a function $g \circ f: X \rightarrow Z$; just follow the arrows.

Let X and Y be sets. We write $\text{Hom}_{\mathbf{Set}}(X, Y)$ to denote the set of functions $X \rightarrow Y$.
³ Note that two functions $f, g: X \rightarrow Y$ are equal if and only if for every element $x \in X$ we have $f(x) = g(x)$.

Exercise 2.1.2.5. Let $A = \{1, 2, 3, 4, 5\}$ and $B = \{x, y\}$.

- How many elements does $\text{Hom}_{\mathbf{Set}}(A, B)$ have?
- How many elements does $\text{Hom}_{\mathbf{Set}}(B, A)$ have?

$\diamond$

Exercise 2.1.2.6. a.) Find a set A such that for all sets X there is exactly one element in $\text{Hom}_{\mathbf{Set}}(X, A)$. Hint: draw a picture of proposed A 's and X 's.

b.) Find a set B such that for all sets X there is exactly one element in $\text{Hom}_{\mathbf{Set}}(B, X)$.

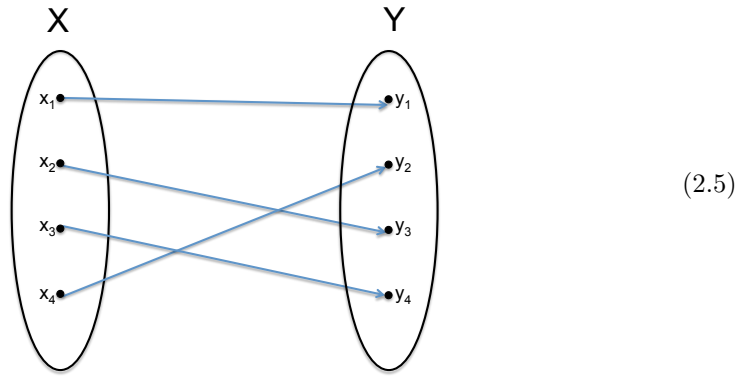
$\diamond$

For any set X , we define the *identity function on X* , denoted $\text{id}_X: X \rightarrow X$, to be the function such that for all $x \in X$ we have $\text{id}_X(x) = x$.

³This strange notation will make more sense later.

Definition 2.1.2.7. Let X and Y be sets. A function $f: X \rightarrow Y$ is called an *isomorphism*, denoted $f: X \xrightarrow{\cong} Y$, if there exists a function $g: Y \rightarrow X$ such that $g \circ f = \text{id}_X$ and $f \circ g = \text{id}_Y$. We also say that f is *invertible* and we say that g is *the inverse* of f . If there exists an isomorphism $X \xrightarrow{\cong} Y$ we say that X and Y are *isomorphic* sets and may write $X \cong Y$.

Example 2.1.2.8. If X and Y are sets and $f: X \rightarrow Y$ is an isomorphism then the analogue of Diagram 2.2 will look like a perfect matching, more often called a *one-to-one correspondence*. That means that no two arrows will hit the same element of Y , and every element of Y will be in the image. For example, the following depicts an isomorphism $X \xrightarrow{\cong} Y$.



Application 2.1.2.9. There is an isomorphism between the set Nuc_{DNA} of **nucleotides** found in DNA and the set Nuc_{RNA} of nucleotides found in RNA. Indeed both sets have four elements, so there are 24 different isomorphisms. But only one is useful. Before we say which one it is, let us say there is also an isomorphism $\text{Nuc}_{\text{DNA}} \cong \{A, C, G, T\}$ and an isomorphism $\text{Nuc}_{\text{RNA}} \cong \{A, C, G, U\}$, and we will use the letters as abbreviations for the nucleotides.

The convenient isomorphism $\text{Nuc}_{\text{DNA}} \xrightarrow{\cong} \text{Nuc}_{\text{RNA}}$ is that given by RNA transcription; it sends $A \mapsto A, C \mapsto C, G \mapsto G, T \mapsto U$. (See also Application 4.1.2.16.)

Protein production can be modeled as a function from the set of 3-nucleotide sequences to the set of eukaryotic amino acids. However, it cannot be an isomorphism because there are $4^3 = 64$ triplets of RNA nucleotides, but only 21 eukaryotic amino acids.

◇◇

Exercise 2.1.2.10. Let $n \in \mathbb{N}$ be a natural number and let X be a set with exactly n elements.

- a.) How many isomorphisms are there from X to itself?
- b.) Does your formula from part a.) hold when $n = 0$?

◇

Lemma 2.1.2.11. *The following facts hold about isomorphism.*

1. Any set A is isomorphic to itself; i.e. there exists an isomorphism $A \xrightarrow{\cong} A$.

2. For any sets A and B , if A is isomorphic to B then B is isomorphic to A .
3. For any sets A, B , and C , if A is isomorphic to B and B is isomorphic to C then A is isomorphic to C .

Proof. 1. The identity function $\text{id}_A: A \rightarrow A$ is invertible; its inverse is id_A because $\text{id}_A \circ \text{id}_A = \text{id}_A$.

2. If $f: A \rightarrow B$ is invertible with inverse $g: B \rightarrow A$ then g is an isomorphism with inverse f .
3. If $f: A \rightarrow B$ and $f': B \rightarrow C$ are each invertible with inverses $g: B \rightarrow A$ and $g': C \rightarrow B$ then the following calculations show that $f' \circ f$ is invertible with inverse $g \circ g'$:

$$\begin{aligned} (f' \circ f) \circ (g \circ g') &= f' \circ (f \circ g) \circ g' = f' \circ \text{id}_B \circ g' = f' \circ g' = \text{id}_C \\ (g \circ g') \circ (f' \circ f) &= g \circ (g' \circ f') \circ f = g \circ \text{id}_B \circ f = g \circ f = \text{id}_A \end{aligned}$$

□

Exercise 2.1.2.12. Let A and B be the sets drawn below:



Note that the sets A and B are isomorphic. Supposing that $f: B \rightarrow \{1, 2, 3, 4, 5\}$ sends “Bob” to 1, sends ♣ to 3, and sends $r8$ to 4, is there a canonical function $A \rightarrow \{1, 2, 3, 4, 5\}$ corresponding to f ? ⁴ ◇

Exercise 2.1.2.13. Find a set A such that for any set X there is a isomorphism of sets

$$X \cong \text{Hom}_{\mathbf{Set}}(A, X).$$

Hint: draw a picture of proposed A ’s and X ’s. ◇

For any natural number $n \in \mathbb{N}$, define a set

$$\underline{n} := \{1, 2, 3, \dots, n\}. \quad (2.6)$$

So, in particular, $\underline{2} = \{1, 2\}$, $\underline{1} = \{1\}$, and $\underline{0} = \emptyset$.

Let A be any set. A function $f: \underline{n} \rightarrow A$ can be written as a sequence

$$f = (f(1), f(2), \dots, f(n)).$$

Exercise 2.1.2.14. a.) Let $A = \{a, b, c, d\}$. If $f: \underline{10} \rightarrow A$ is given by $(a, b, c, c, b, a, d, d, a, b)$, what is $f(4)$?

⁴Canonical means something like “best choice”, a choice that stands out as the only reasonable one.

b.) Let $s: \mathbb{N} \rightarrow \mathbb{N}$ be given by $s(i) = i^2$. Write s out as a sequence.

◇

Definition 2.1.2.15. [Cardinality of finite sets]

Let A be a set and $n \in \mathbb{N}$ a natural number. We say that A has *cardinality* n , denoted

$$|A| = n,$$

if there exists an isomorphism of sets $A \cong \underline{n}$. If there exists some $n \in \mathbb{N}$ such that A has cardinality n then we say that A is *finite*. Otherwise, we say that A is *infinite* and write $|A| \geq \infty$.

Exercise 2.1.2.16. a.) Let $A = \{5, 6, 7\}$. What is $|A|$?

b.) What is $|\mathbb{N}|$?

c.) What is $|\{n \in \mathbb{N} \mid n \leq 5\}|$?

◇

Lemma 2.1.2.17. *Let A and B be finite sets. If there is an isomorphism of sets $f: A \rightarrow B$ then the two sets have the same cardinality, $|A| = |B|$.*

Proof. Suppose $f: A \rightarrow B$ is an isomorphism. If there exists natural numbers $m, n \in \mathbb{N}$ and isomorphisms $a: \underline{m} \xrightarrow{\cong} A$ and $b: \underline{n} \xrightarrow{\cong} B$ then $\underline{m} \xrightarrow{a^{-1}} A \xrightarrow{f} B \xrightarrow{b} \underline{n}$ is an isomorphism. One can prove by induction that the sets $\underline{m}$ and $\underline{n}$ are isomorphic if and only if $m = n$.

□

2.2 Commutative diagrams

At this point it is difficult to precisely define diagrams or commutative diagrams in general, but we can give the heuristic idea.⁵ Consider the following picture:

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ & \searrow h & \downarrow g \\ & & C \end{array} \quad (2.7)$$

We say this is a *diagram of sets* if each of A, B, C is a set and each of f, g, h is a function. We say this diagram *commutes* if $g \circ f = h$. In this case we refer to it as a commutative triangle of sets.

Application 2.2.1.1. **The central dogma of molecular biology** is that “DNA makes RNA makes protein”. That is, there is a function from DNA triplets to RNA triplets and a function from RNA triplets to amino acids. But sometimes we just want to discuss the translation from DNA to amino acids, and this is the composite of the other two. The commutative diagram is a picture of this fact.

◇◇

⁵We will define commutative diagrams precisely in Section 4.5.2.

Consider the following picture:

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ h \downarrow & & \downarrow g \\ C & \xrightarrow{i} & D \end{array}$$

We say this is a *diagram of sets* if each of A, B, C, D is a set and each of f, g, h, i is a function. We say this diagram *commutes* if $g \circ f = i \circ h$. In this case we refer to it as a commutative square of sets.

Application 2.2.1.2. Given a physical system S , there may be two mathematical approaches $f: S \rightarrow A$ and $g: S \rightarrow B$ that can be applied to it. Either of those results in a prediction of the same sort, $f': A \rightarrow P$ and $g': B \rightarrow P$. For example, in **mechanics** we can use either Lagrangian approach or the Hamiltonian approach to predict future states. To say that the diagram

$$\begin{array}{ccc} S & \longrightarrow & A \\ \downarrow & & \downarrow \\ B & \longrightarrow & P \end{array}$$

commutes would say that these approaches give the same result.

◇◇

And so on. Note that diagram (2.7) is considered to be the same diagram as each of the following:

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ h \downarrow & \searrow g & \\ C & & \end{array}$$

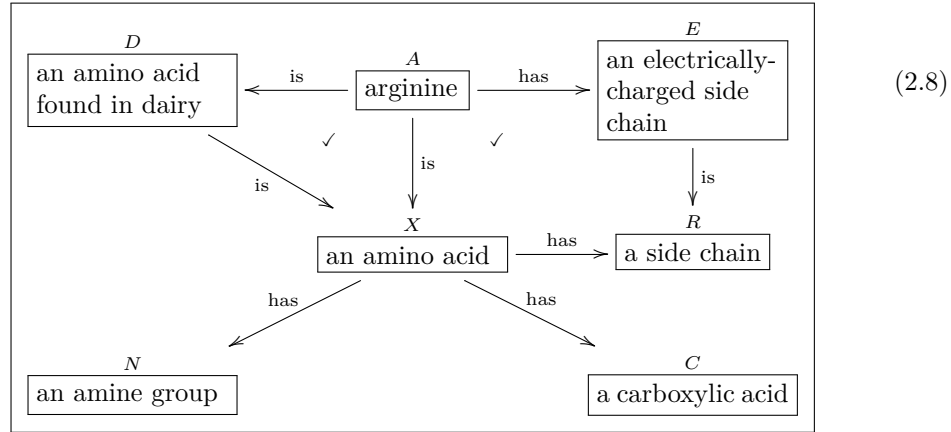
$$\begin{array}{ccccc} A & \xrightarrow{f} & B & \xrightarrow{g} & C \\ & \searrow h & & \nearrow & \end{array}$$

$$\begin{array}{ccc} & B & \\ & \searrow g & \\ f \uparrow & & C \\ A & \nearrow h & \end{array}$$

2.3 Ologs

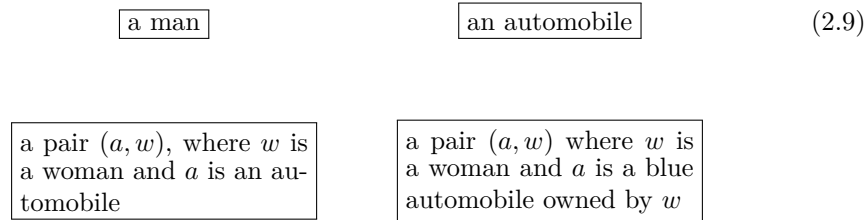
In this course we will ground the mathematical ideas in applications whenever possible. To that end we introduce ologs, which will serve as a bridge between mathematics and various conceptual landscapes. The following material is taken from [SK], an introduction

to ologs.



2.3.1 Types

A type is an abstract concept, a distinction the author has made. We represent each type as a box containing a *singular indefinite noun phrase*. Each of the following four boxes is a type:

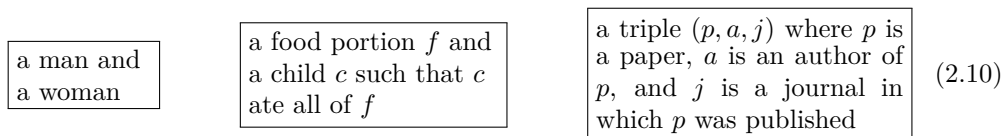


Each of the four boxes in (2.9) represents a type of thing, a whole class of things, and the label on that box is what one should call *each example* of that class. Thus 'a man' does not represent a single man, but the set of men, each example of which is called "a man". Similarly, the bottom right box represents an abstract type of thing, which probably has more than a million examples, but the label on the box indicates the common name for each such example.

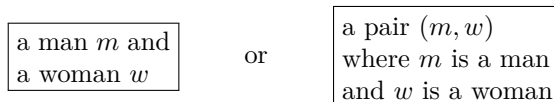
Typographical problems emerge when writing a text-box in a line of text, e.g. the text-box a man seems out of place here, and the more in-line text-boxes there are, the worse it gets. To remedy this, I will denote types which occur in a line of text with corner-symbols; e.g. I will write 'a man' instead of a man.

2.3.1.1 Types with compound structures

Many types have compound structures; i.e. they are composed of smaller units. Examples include



It is good practice to declare the variables in a “compound type”, as I did in the last two cases of (2.10). In other words, it is preferable to replace the first box above with something like



so that the variables (m, w) are clear.

Rules of good practice 2.3.1.2. A type is presented as a text box. The text in that box should

- (i) begin with the word “a” or “an”;
- (ii) refer to a distinction made and recognizable by the olog’s author;
- (iii) refer to a distinction for which instances can be documented;
- (iv) declare all variables in a compound structure.

The first, second, and third rules ensure that the class of things represented by each box appears to the author as a well-defined set. The fourth rule encourages good “readability” of arrows, as will be discussed next in Section 2.3.2.

I will not always follow the rules of good practice throughout this document. I think of these rules being followed “in the background” but that I have “nicknamed” various boxes. So ‘Steve’ may stand as a nickname for ‘a thing classified as Steve’ and ‘arginine’ as a nickname for ‘a molecule of arginine’. However, when pressed, one should always be able to rename each type according to the rules of good practice.

2.3.2 Aspects

An aspect of a thing x is a way of viewing it, a particular way in which x can be regarded or measured. For example, a woman can be regarded as a person; hence “being a person” is an aspect of a woman. A molecule has a molecular mass (say in daltons), so “having a molecular mass” is an aspect of a molecule. In other words, by *aspect* we simply mean a function. The domain A of the function $f: A \rightarrow B$ is the thing we are measuring, and the codomain is the set of possible “answers” or results of the measurement.

$$\boxed{\text{a woman}} \xrightarrow{\text{is}} \boxed{\text{a person}} \quad (2.11)$$

$$\boxed{\text{a molecule}} \xrightarrow{\text{has as molecular mass (Da)}} \boxed{\text{a positive real number}} \quad (2.12)$$

So for the arrow in (2.11), the domain is the set of women (a set with perhaps 3 billion elements); the codomain is the set of persons (a set with perhaps 6 billion elements). We can imagine drawing an arrow from each dot in the “woman” set to a unique dot in the “person” set, just as in (2.2). No woman points to two different people, nor to zero people — each woman is exactly one person — so the rules for a function are satisfied. Let us now concentrate briefly on the arrow in (2.12). The domain is the set of molecules, the codomain is the set $\mathbb{R}_{>0}$ of positive real numbers. We can imagine drawing an arrow from each dot in the “molecule” set to a single dot in the “positive real number” set. No molecule points to two different masses, nor can a molecule have no mass: each molecule has exactly one mass. Note however that two different molecules can point to the same mass.

2.3.2.1 Invalid aspects

I tried above to clarify what it is that makes an aspect “valid”, namely that it must be a “functional relationship.” In this subsection I will show two arrows which on their face may appear to be aspects, but which on closer inspection are not functional (and hence are not valid as aspects).

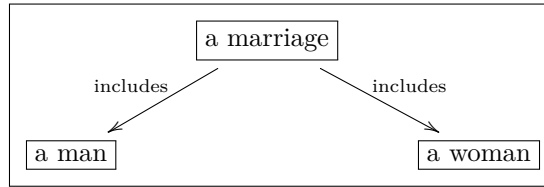
Consider the following two arrows:

$$\boxed{\text{a person}} \xrightarrow{\text{has}} \boxed{\text{a child}} \quad (2.13^*)$$

$$\boxed{\text{a mechanical pencil}} \xrightarrow{\text{uses}} \boxed{\text{a piece of lead}} \quad (2.14^*)$$

A person may have no children or may have more than one child, so the first arrow is invalid: it is not a function. Similarly, if we drew an arrow from each mechanical pencil to each piece of lead it uses, it would not be a function.

Warning 2.3.2.2. The author of an olog has a world-view, some fragment of which is captured in the olog. When person A examines the olog of person B, person A may or may not “agree with it.” For example, person B may have the following olog



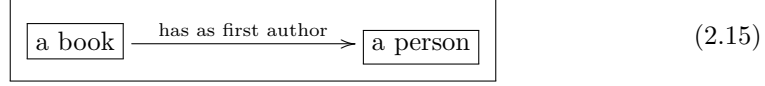
which associates to each marriage a man and a woman. Person A may take the position that some marriages involve two men or two women, and thus see B’s olog as “wrong.” Such disputes are not “problems” with either A’s olog or B’s olog, they are discrepancies between world-views. Hence, throughout this paper, a reader R may see a displayed olog and notice a discrepancy between R’s world-view and my own, but R should not worry that this is a problem. This is not to say that ologs need not follow rules, but instead that the rules are enforced to ensure that an olog is structurally sound, rather than that it “correctly reflects reality,” whatever that may mean.

Consider the aspect ‘an object’ $\xrightarrow{\text{has}}$ ‘a weight’. At some point in history, this would have been considered a valid function. Now we know that the same object would have a different weight on the moon than it has on earth. Thus as world-views change, we often need to add more information to our olog. Even the validity of ‘an object on earth’ $\xrightarrow{\text{has}}$ ‘a weight’ is questionable. However to build a model we need to choose a level of granularity and try to stay within it, or the whole model evaporates into the nothingness of truth!

Remark 2.3.2.3. In keeping with Warning 2.3.2.2, the arrows (2.13*) and (2.14*) may not be wrong but simply reflect that the author has a strange world-view or a strange vocabulary. Maybe the author believes that every mechanical pencil uses exactly one piece of lead. If this is so, then ‘a mechanical pencil’ $\xrightarrow{\text{uses}}$ ‘a piece of lead’ is indeed a valid aspect! Similarly, suppose the author meant to say that each person *was once* a child, or that a person has an inner child. Since every person has one and only one inner child (according to the author), the map ‘a person’ $\xrightarrow{\text{has as inner child}}$ ‘a child’ is a valid aspect. We cannot fault the olog if the author has a view, but note that we have changed the name of the label to make his or her intention more explicit.

2.3.2.4 Reading aspects and paths as English phrases

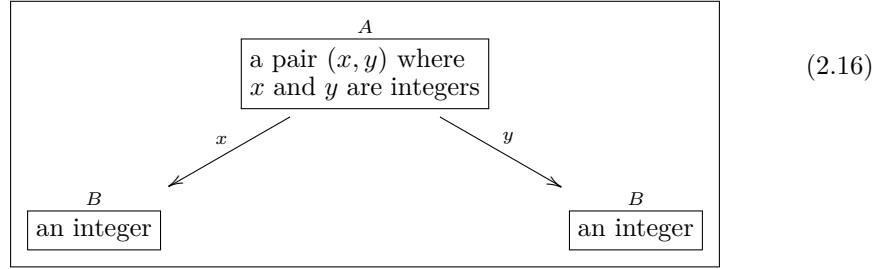
Each arrow (aspect) $X \xrightarrow{f} Y$ can be read by first reading the label on its source box (domain of definition) X , then the label on the arrow f , and finally the label on its target box (set of values) Y . For example, the arrow



is read “a book has as first author a person”.

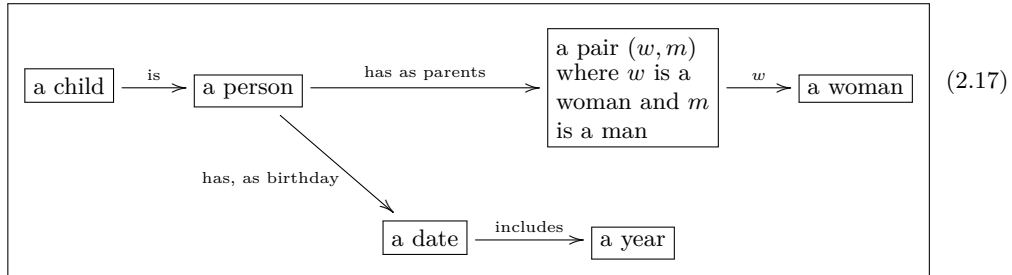
Remark 2.3.2.5. Note that the map in (2.15) is a valid aspect, but that a similarly benign-looking map ‘a book’ $\xrightarrow{\text{has as author}}$ ‘a person’ would not be valid, because it is not functional. The authors of an olog must be vigilant about this type of mistake because it is easy to miss and it can corrupt the olog.

Sometimes the label on an arrow can be shortened or dropped altogether if it is obvious from context. We will discuss this more in Section 2.3.3 but here is a common example from the way I write ologs.



Neither arrow is readable by the protocol given above (e.g. “a pair (x, y) where x and y are integers x an integer” is not an English sentence), and yet it is obvious what each map means. For example, given $(8, 11)$ in A , arrow x would yield 8 and arrow y would yield 11. The label x can be thought of as a nickname for the full name “yields, via the value of x ,” and similarly for y . I do not generally use the full name for fear that the olog would become cluttered with text.

One can also read paths through an olog by inserting the word “which” after each intermediate box.⁶ For example the following olog has two paths of length 3 (counting arrows in a chain):



The top path is read “a child is a person, who has as parents a pair (w, m) where w is a woman and m is a man, which yields, via the value of w , a woman.” The reader should

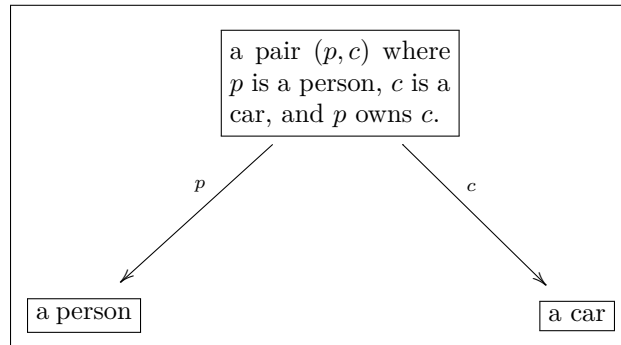
⁶If the intended elements of an intermediate box are humans, it is polite to use “who” rather than “which”, and other such conventions may be upheld if one so desires.

read and understand the content of the bottom path, which associates to every child a year.

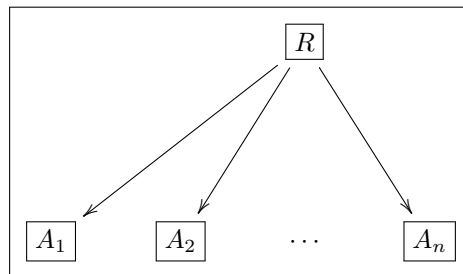
2.3.2.6 Converting non-functional relationships to aspects

There are many relationships that are not functional, and these cannot be considered aspects. Often the word “has” indicates a relationship — sometimes it is functional as in ‘a person’ $\xrightarrow{\text{has}}$ ‘a stomach’, and sometimes it is not, as in ‘a father’ $\xrightarrow{\text{has}}$ ‘a child’. Obviously, a father may have more than one child. This one is easily fixed by realizing that the arrow should go the other way: there is a function ‘a child’ $\xrightarrow{\text{has}}$ ‘a father’.

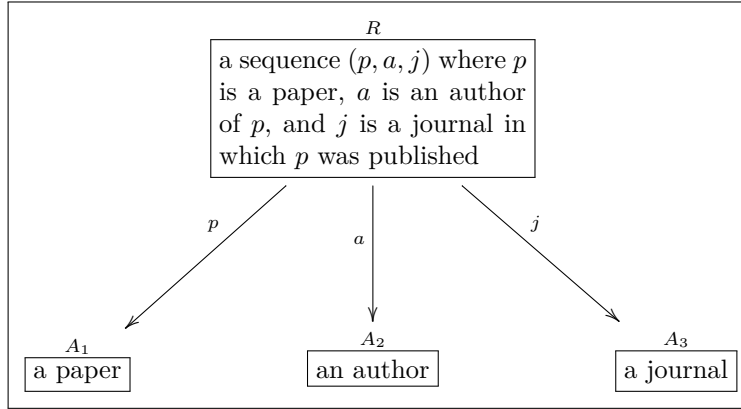
What about ‘a person’ $\xrightarrow{\text{owns}}$ ‘a car’. Again, a person may own no cars or more than one car, but this time a car can be owned by more than one person too. A quick fix would be to replace it by ‘a person’ $\xrightarrow{\text{owns}}$ ‘a set of cars’. This is ok, but the relationship between ‘a car’ and ‘a set of cars’ then becomes an issue to deal with later. There is another way to indicate such “non-functional” relationships. In this case it would look like this:



This setup will ensure that everything is properly organized. In general, relationships can involve more than two types, and the general situation looks like this



For example,



Exercise 2.3.2.7. On page 23 we indicate a so-called invalid aspect, namely

$$\boxed{\text{a person}} \xrightarrow{\text{has}} \boxed{\text{a child}} \quad (2.13^*)$$

Create a (valid) olog that captures the parent-child relationship; your olog should still have boxes 'a person' and 'a child' but may have an additional box. $\diamond$

Rules of good practice 2.3.2.8. An aspect is presented as a labeled arrow, pointing from a source box to a target box. The arrow text should

- (i) begin with a verb;
- (ii) yield an English sentence, when the source-box text followed by the arrow text followed by the target-box text is read; and
- (iii) refer to a functional relationship: each instance of the source type should give rise to a specific instance of the target type.

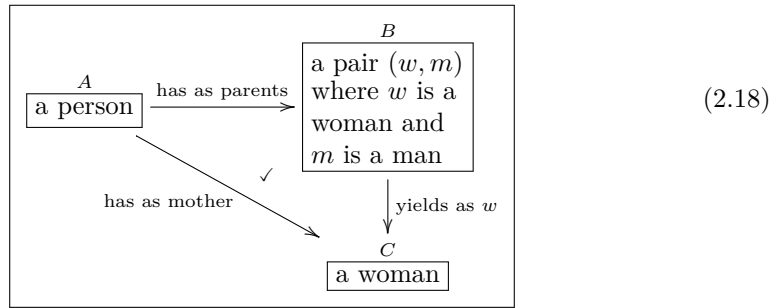
2.3.3 Facts

In this section I will discuss facts, which are simply “path equivalences” in an olog. It is the notion of path equivalences that make category theory so powerful.

A *path* in an olog is a head-to-tail sequence of arrows. That is, any path starts at some box B_0 , then follows an arrow emanating from B_0 (moving in the appropriate direction), at which point it lands at another box B_1 , then follows any arrow emanating from B_1 , etc, eventually landing at a box B_n and stopping there. The number of arrows is the *length* of the path. So a path of length 1 is just an arrow, and a path of length 0 is just a box. We call B_0 the *source* and B_n the *target* of the path.

Given an olog, the author may want to declare that two paths are equivalent. For

example consider the two paths from A to C in the olog



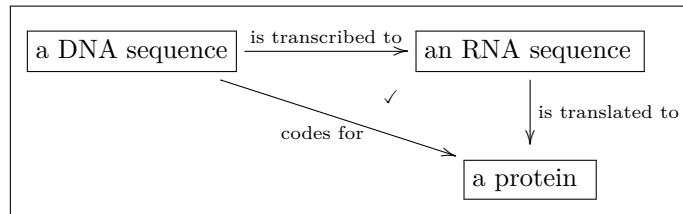
We know as English speakers that a woman parent is called a mother, so these two paths $A \rightarrow C$ should be equivalent. A more mathematical way to say this is that the triangle in Olog (2.18) *commutes*. That is, path equivalences are simply commutative diagrams as in Section 2.2. In the example above we concisely say “a woman parent is equivalent to a mother.” We declare this by defining the diagonal map in (2.18) to be *the composition* of the horizontal map and the vertical map.

I generally prefer to indicate a commutative diagram by drawing a check-mark, $\checkmark$, in the region bounded by the two paths, as in Olog (2.18). Sometimes, however, one cannot do this unambiguously on the 2-dimensional page. In such a case I will indicate the commutative diagrams (fact) by writing an equation. For example to say that the diagram

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ h \downarrow & & \downarrow g \\ C & \xrightarrow{i} & D \end{array}$$

commutes, we could either draw a checkmark inside the square or write the equation $A f g \simeq A h i$ above it. Either way, it means that “ f then g ” is equivalent to “ h then i ”.

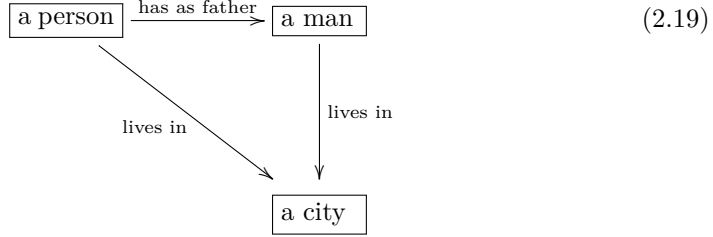
Here is another, more scientific example:



Note how this diagram gives us the established terminology for the various ways in which DNA, RNA, and protein are related in this context.

Exercise 2.3.3.1. Create an olog for human nuclear biological families that includes the concept of person, man, woman, parent, father, mother, and child. Make sure to label all the arrows, and make sure each arrow indicates a valid aspect in the sense of Section 2.3.2.1. Indicate with check-marks ($\checkmark$) the diagrams that are intended to commute. If the 2-dimensionality of the page prevents a check-mark from being unambiguous, indicate the intended commutativity with an equation. $\diamond$

Example 2.3.3.2 (Non-commuting diagram). In my conception of the world, the following diagram does not commute:



The non-commutativity of Diagram (2.19) does not imply that, in my conception, no person lives in the same city as his or her father. Rather it implies that, in my conception, it is not the case that *every* person lives in the same city as his or her father.

Exercise 2.3.3.3. Create an olog about a scientific subject, preferably one you think about often. The olog should have at least five boxes, five arrows, and one commutative diagram. $\diamond$

2.3.3.4 A formula for writing facts as English

Every fact consists of two paths, say P and Q , that are to be declared equivalent. The paths P and Q will necessarily have the same source, say s , and target, say t , but their lengths may be different, say m and n respectively. ⁷ We draw these paths as

$$\begin{array}{lcl}
 P : & a_0=s \xrightarrow{f_1} \bullet \xrightarrow{f_2} a_2 \xrightarrow{f_3} \dots \xrightarrow{f_{m-1}} a_{m-1} \xrightarrow{f_m} a_m=t & (2.20) \\
 Q : & b_0=s \xrightarrow{g_1} \bullet \xrightarrow{g_2} b_2 \xrightarrow{g_3} \dots \xrightarrow{g_{n-1}} b_{n-1} \xrightarrow{g_n} b_n=t &
 \end{array}$$

Every part ℓ of an olog (i.e. every box and every arrow) has an associated English phrase, which we write as “ ℓ ”. Using a dummy variable x we can convert a fact into English too. The following general formula is a bit difficult to understand, see Example 2.3.3.5, but here goes. The fact $P \simeq Q$ from (2.20) can be Englishified as follows:

Given x , “ s ”, consider the following. We know that x is “ s ”, (2.21)

which “ f_1 ” “ a_1 ”, which “ f_2 ” “ a_2 ”, which $\dots$ “ f_{m-1} ” “ a_{m-1} ”, which “ f_m ” “ t ”
that we’ll call $P(x)$.

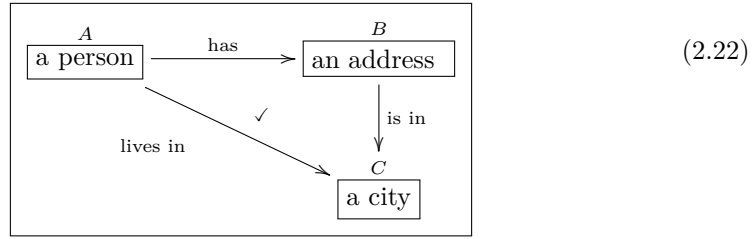
We also know that x is “ s ”,

which “ g_1 ” “ b_1 ”, which “ g_2 ” “ b_2 ”, which $\dots$ “ g_{n-1} ” “ b_{n-1} ”, which “ g_n ” “ t ”
that we’ll call $Q(x)$.

Fact: whenever x is “ s ”, we will have $P(x) = Q(x)$.

⁷If the source equals the target, $s = t$, then it is possible to have $m = 0$ or $n = 0$, and the ideas below still make sense.

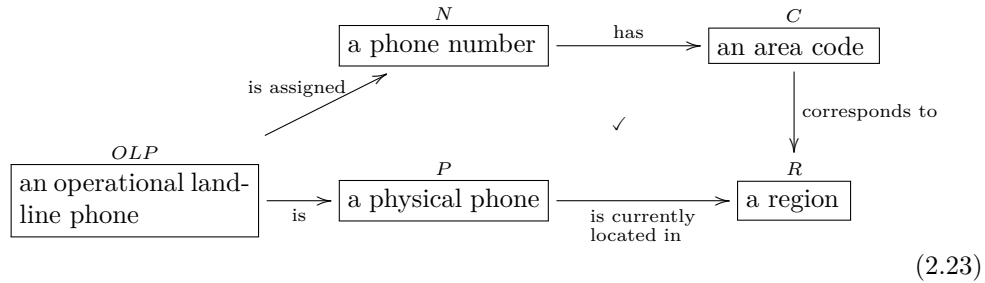
Example 2.3.3.5. Consider the olog



To put the fact that Diagram 2.22 commutes into English, we first Englishify the two paths: F = “a person has an address which is in a city” and G = “a person lives in a city”. The source of both is s = “a person” and the target of both is t = “a city”. write:

Given x , a person, consider the following. We know that x is a person,
 which has an address, which is in a city
 that we'll call $P(x)$.
 We also know that x is a person,
 which lives in a city
 that we'll call $Q(x)$.
 Fact: whenever x is a person, we will have $P(x) = Q(x)$.

Exercise 2.3.3.6. This olog was taken from [Sp1].



It says that a landline phone is physically located in the region that its phone number is assigned. Translate this fact into English using the formula from 2.21. $\diamond$

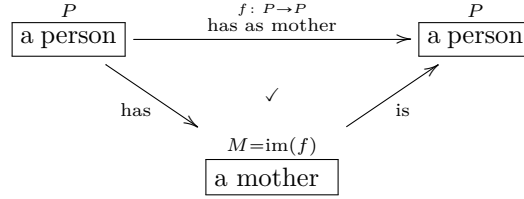
Exercise 2.3.3.7. In the above olog (2.23), suppose that the box ‘an operational landline phone’ is replaced with the box ‘an operational mobile phone’. Would the diagram still commute? $\diamond$

2.3.3.8 Images

In this section we discuss a specific kind of fact, generated by any aspect. Recall that every function has an image, meaning the subset of elements in the codomain that are “hit” by the function. For example the function $f(x) = 2 * x: \mathbb{Z} \rightarrow \mathbb{Z}$ has as image the set of all even numbers.

Similarly the set of mothers arises as is the image of the “has as mother” function,

as shown below



Exercise 2.3.3.9. For each of the following types, write down a function for which it is the image, or say “not clearly an image type”

- a.) ‘a book’
- b.) ‘a material that has been fabricated by a process of type T ’
- c.) ‘a bicycle owner’
- d.) ‘a child’
- e.) ‘a used book’
- f.) ‘an inhabited residence’

◇

2.4 Products and coproducts

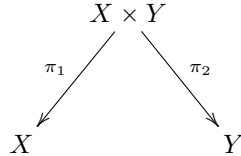
In this section we introduce two concepts that are likely to be familiar, although perhaps not by their category-theoretic names, product and coproduct. Each is an example of a large class of ideas that exist far beyond the realm of sets.

2.4.1 Products

Definition 2.4.1.1. Let X and Y be sets. The *product of X and Y* , denoted $X \times Y$, is defined as the set of ordered pairs (x, y) where $x \in X$ and $y \in Y$. Symbolically,

$$X \times Y = \{(x, y) \mid x \in X, y \in Y\}.$$

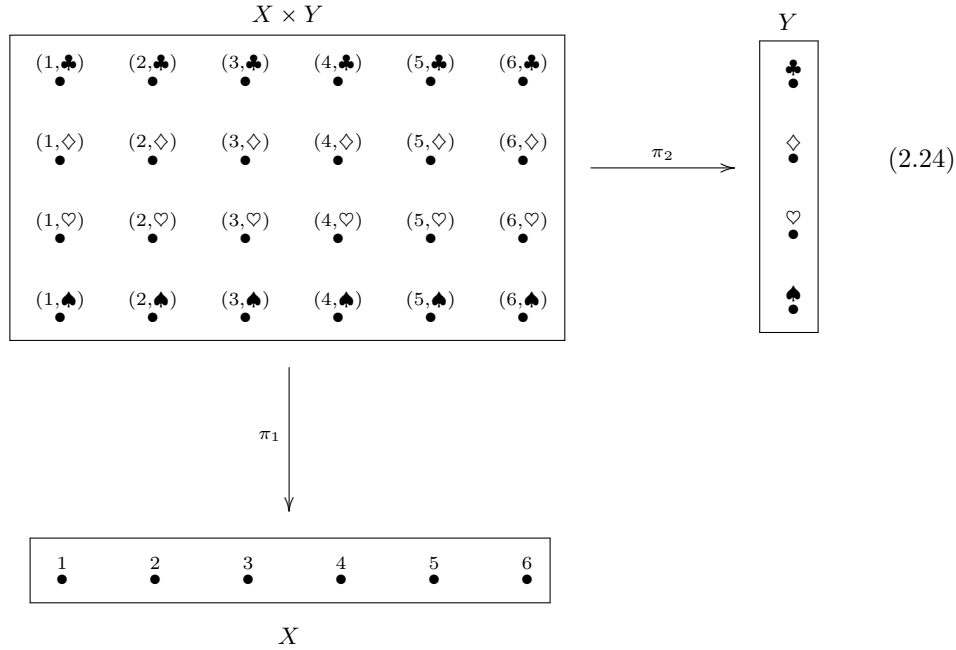
There are two natural *projection functions* $\pi_1: X \times Y \rightarrow X$ and $\pi_2: X \times Y \rightarrow Y$.



Example 2.4.1.2. [Grid of dots]

Let $X = \{1, 2, 3, 4, 5, 6\}$ and $Y = \{\clubsuit, \diamondsuit, \heartsuit, \spadesuit\}$. Then we can draw $X \times Y$ as a 6-by-4

grid of dots, and the projections as projections



Application 2.4.1.3. A traditional (Mendelian) way to predict the genotype of offspring based on the genotype of its parents is by the use of **Punnett squares**. If F is the set of possible genotypes for the female parent and M is the set of possible genotypes of the male parent, then $F \times M$ is drawn as a square, called a Punnett square, in which every combination is drawn. $\diamond\diamond$

Exercise 2.4.1.4. How many elements does the set $\{a, b, c, d\} \times \{1, 2, 3\}$ have? $\diamond$

Application 2.4.1.5. Suppose we are conducting experiments about the mechanical properties of materials, as in Application 2.1.2.1. For each material sample we will produce multiple data points in the set 'extension' $\times$ 'force' $\cong \mathbb{R} \times \mathbb{R}$. $\diamond\diamond$

Remark 2.4.1.6. It is possible to take the product of more than two sets as well. For example, if A, B , and C are sets then $A \times B \times C$ is the set of triples,

$$A \times B \times C := \{(a, b, c) \mid a \in A, b \in B, c \in C\}.$$

This kind of generality is useful in understanding multiple dimensions, e.g. what physicists mean by 10-dimensional space. It comes under the heading of *limits*, which we will see in Section 4.5.3.

Example 2.4.1.7. Let $\mathbb{R}$ be the set of real numbers. By $\mathbb{R}^2$ we mean $\mathbb{R} \times \mathbb{R}$ (though see Exercise 2.7.2.6). Similarly, for any $n \in \mathbb{N}$, we define $\mathbb{R}^n$ to be the product of n copies of $\mathbb{R}$.

According to [Pen], Aristotle seems to have conceived of space as something like $S := \mathbb{R}^3$ and of time as something like $T := \mathbb{R}$. Spacetime, had he conceived of it, would probably have been $S \times T \cong \mathbb{R}^4$. He of course did not have access to this kind of abstraction, which was probably due to Descartes.

Exercise 2.4.1.8. Let $\mathbb{Z}$ denote the set of integers, and let $+: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ denote the addition function and $\cdot: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ denote the multiplication function. Which of the following diagrams commute?

a.)

$$\begin{array}{ccc}
 \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z} & \xrightarrow{(a,b,c) \mapsto (a \cdot b, a \cdot c)} & \mathbb{Z} \times \mathbb{Z} \\
 (a,b,c) \mapsto (a+b,c) \downarrow & & \downarrow (x,y) \mapsto x+y \\
 \mathbb{Z} \times \mathbb{Z} & \xrightarrow{(x,y) \mapsto xy} & \mathbb{Z}
 \end{array}$$

b.)

$$\begin{array}{ccc}
 \mathbb{Z} & \xrightarrow{x \mapsto (x,0)} & \mathbb{Z} \times \mathbb{Z} \\
 \searrow \text{id}_{\mathbb{Z}} & & \downarrow (a,b) \mapsto a \cdot b \\
 & & \mathbb{Z}
 \end{array}$$

c.)

$$\begin{array}{ccc}
 \mathbb{Z} & \xrightarrow{x \mapsto (x,1)} & \mathbb{Z} \times \mathbb{Z} \\
 \searrow \text{id}_{\mathbb{Z}} & & \downarrow (a,b) \mapsto a \cdot b \\
 & & \mathbb{Z}
 \end{array}$$

◇

2.4.1.9 Universal property for products

Lemma 2.4.1.10 (Universal property for product). *Let X and Y be sets. For any set A and functions $f: A \rightarrow X$ and $g: A \rightarrow Y$, there exists a unique function $A \rightarrow X \times Y$ such that the following diagram commutes*⁸

$$\begin{array}{ccccc}
 & & X \times Y & & \\
 & \swarrow \pi_1 & \uparrow & \searrow \pi_2 & \\
 X & & \text{---} & & Y \\
 & \swarrow \forall f & \downarrow \exists! & \searrow \forall g & \\
 & & A & &
 \end{array} \tag{2.25}$$

We might write the unique function as

$$f \times g: A \rightarrow X \times Y.$$

Proof. Suppose given f, g as above. To provide a function $\ell: A \rightarrow X \times Y$ is equivalent to providing an element $\ell(a) \in X \times Y$ for each $a \in A$. We need such a function for which

⁸The symbol $\forall$ is read “for all”; the symbol $\exists$ is read “there exists”, and the symbol $\exists!$ is read “there exists a unique”. So this diagram is intended to express the idea that for any functions $f: A \rightarrow X$ and $g: A \rightarrow Y$, there exists a unique function $A \rightarrow X \times Y$ for which the two triangles commute.

$\pi_1 \circ \ell = f$ and $\pi_2 \circ \ell = g$. An element of $X \times Y$ is an ordered pair (x, y) , and we can use $\ell(a) = (x, y)$ if and only if $x = \pi_1(x, y) = f(a)$ and $y = \pi_2(x, y) = g(a)$. So it is necessary and sufficient to define

$$(f \times g)(a) = (f(a), g(a))$$

for all $a \in A$.

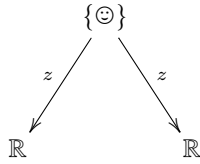
□

Example 2.4.1.11. [Grid of dots, continued]

We need to see the universal property of products as completely intuitive. Recall that if X and Y are sets, say of cardinalities $|X| = m$ and $|Y| = n$ respectively, then $X \times Y$ is an $m \times n$ grid of dots, and it comes with two canonical projections $X \xleftarrow{\pi_1} X \times Y \xrightarrow{\pi_2} Y$. These allow us to extract from every grid element $z \in X \times Y$ its column $\pi_1(z) \in X$ and its row $\pi_2(z) \in Y$.

Suppose that each person in a classroom picks an element of X and an element of Y . Thus we have functions $f: C \rightarrow X$ and $g: C \rightarrow Y$. But isn't picking a column and a row the same thing as picking an element in the grid? The two functions f and g induce a unique function $C \rightarrow X \times Y$. And how does this function $C \rightarrow X \times Y$ compare with the original functions f and g ? The commutative diagram (2.25) sums up the obvious connection.

Example 2.4.1.12. Let $\mathbb{R}$ be the set of real numbers. The origin in $\mathbb{R}$ is an element of $\mathbb{R}$. As you showed in Exercise 2.1.2.13, we can view this (or any) element of $\mathbb{R}$ as a function $z: \{\odot\} \rightarrow \mathbb{R}$, where $\{\odot\}$ is any set with one element. Our function z “picks out the origin”. Thus we can draw functions



The universal property for products guarantees a function $\{\odot\} \rightarrow \mathbb{R} \times \mathbb{R}$, which will be the origin in $\mathbb{R}^2$.

Remark 2.4.1.13. Given sets X, Y , and A , and functions $f: A \rightarrow X$ and $g: A \rightarrow Y$, there is a unique function $A \rightarrow X \times Y$ that commutes with f and g . We call it *the induced function* $A \rightarrow X \times Y$, meaning the one that arises in light of f and g .

Exercise 2.4.1.14. For every set A there is some nice relationship between the following three sets:

$$\text{Hom}_{\text{Set}}(A, X), \quad \text{Hom}_{\text{Set}}(A, Y), \quad \text{and} \quad \text{Hom}_{\text{Set}}(A, X \times Y).$$

What is it?

Hint: Do not be alarmed: this problem is a bit “recursive” in that you’ll use products in your formula. ◇

Exercise 2.4.1.15. a.) Let X and Y be sets. Construct the “swap map” $s: X \times Y \rightarrow Y \times X$ using only the universal property for products. If $\pi_1: X \times Y \rightarrow X$ and $\pi_2: X \times Y \rightarrow Y$ are the projection functions, write s in terms of the symbols $\pi_1, \pi_2, \times$, and $\circ$.

b.) Can you prove that s is a isomorphism using only the universal property for product?

◇

Exercise 2.4.1.16. Suppose given sets A, A', B, B' and functions $f: A \rightarrow A'$ and $g: B \rightarrow B'$. Use the universal property of products to construct a function $s: A \times B \rightarrow A' \times B'$. If $\pi_1: A \times B \rightarrow A$ and $\pi_2: A \times B \rightarrow B$ are the projection functions, write s in terms of the symbols f, g, π_1, π_2 , and $\times$. ◇

2.4.1.17 Ologging products

Given two objects c, d in an olog, there is a canonical label “ $c \times d$ ” for their product $c \times d$, written in terms of the labels “ c ” and “ d ”. Namely,

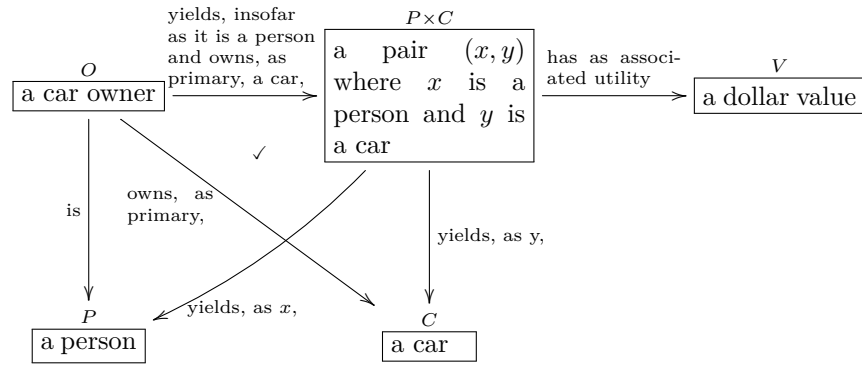
$$“c \times d” := \text{a pair } (x, y) \text{ where } x \text{ is “} c \text{” and } y \text{ is “} d \text{”}.$$

The projections $c \leftarrow c \times d \rightarrow d$ can be labeled “yields, as x ,” and “yields, as y ,” respectively.

Suppose that e is another object and $p: e \rightarrow c$ and $q: e \rightarrow d$ are two arrows. By the universal property of products (Lemma 2.4.1.10), p and q induce a unique arrow $e \rightarrow c \times d$ making the evident diagrams commute. This arrow can be labeled

yields, insofar as it “ p ” “ c ” and “ q ” “ d ”,

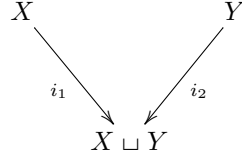
Example 2.4.1.18. Every car owner owns at least one car, but there is no obvious function ‘a car owner’ $\rightarrow$ ‘a car’ because he or she may own more than one. One good choice would be the car that the person drives most often, which we’ll call his or her primary car. Also, given a person and a car, an economist could ask how much utility the person would get out of the car. From all this we can put together the following olog involving products:



2.4.2 Coproducts

Definition 2.4.2.1. Let X and Y be sets. The *coproduct of X and Y* , denoted $X \sqcup Y$, is defined as the “disjoint union” of X and Y , i.e. the set for which an element is either an element of X or an element of Y . If something is an element of both X and Y then we include both copies, and distinguish between them, in $X \sqcup Y$. See Example 2.4.2.2

There are two natural inclusion functions $i_1: X \rightarrow X \sqcup Y$ and $i_2: Y \rightarrow X \sqcup Y$.



Example 2.4.2.2. The coproduct of $X := \{a, b, c, d\}$ and $Y := \{1, 2, 3\}$ is

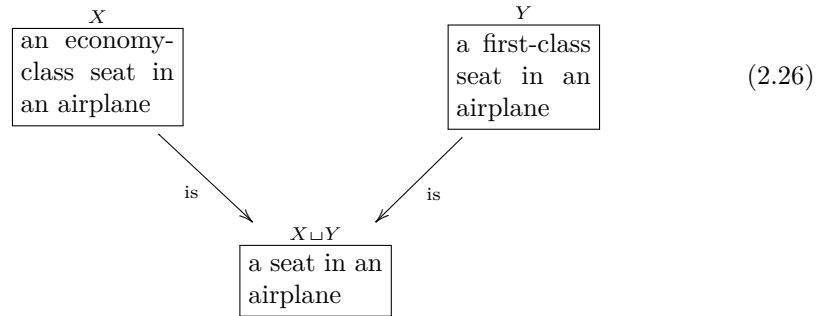
$$X \sqcup Y \cong \{a, b, c, d, 1, 2, 3\}.$$

The coproduct of X and itself is

$$X \sqcup X \cong \{i_1a, i_1b, i_1c, i_1d, i_2a, i_2b, i_2c, i_2d\}$$

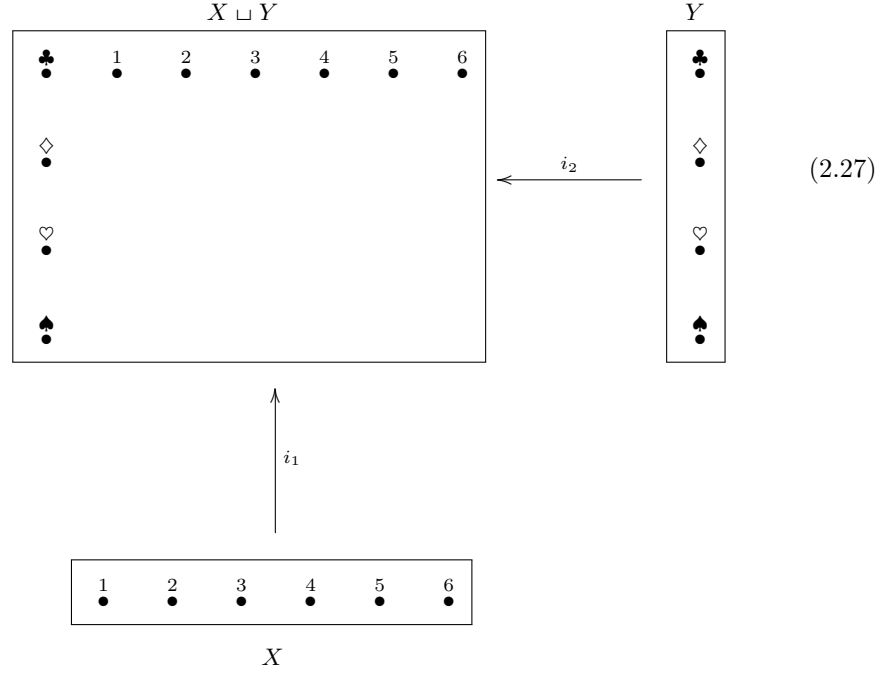
The names of the elements in $X \sqcup Y$ are not so important. What's important are the inclusion maps i_1, i_2 , which ensure that we know where each element of $X \sqcup Y$ came from.

Example 2.4.2.3. [Airplane seats]



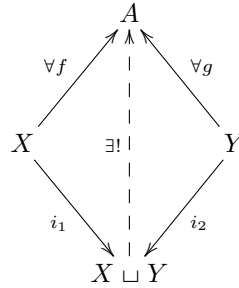
Exercise 2.4.2.4. Would you say that 'a phone' is the coproduct of 'a cellphone' and 'a landline phone'? $\diamond$

Example 2.4.2.5 (Disjoint union of dots).



2.4.2.6 Universal property for coproducts

Lemma 2.4.2.7 (Universal property for coproduct). *Let X and Y be sets. For any set A and functions $f: X \rightarrow A$ and $g: Y \rightarrow A$, there exists a unique function $X \sqcup Y \rightarrow A$ such that the following diagram commutes*



We might write the unique function as

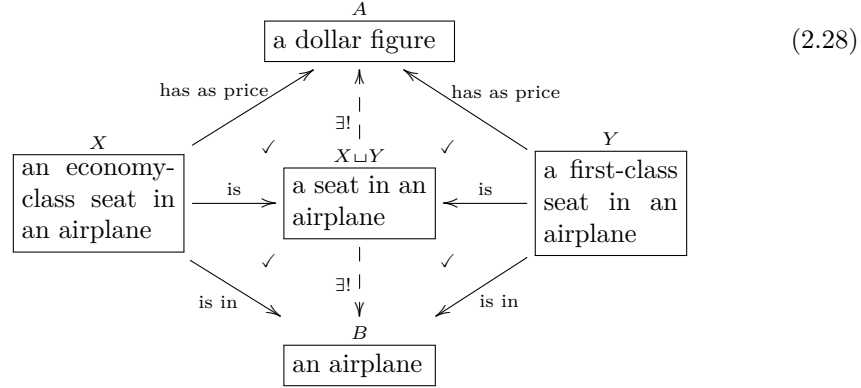
$$f \sqcup g: X \sqcup Y \rightarrow A.$$

Proof. Suppose given f, g as above. To provide a function $\ell: X \sqcup Y \rightarrow A$ is equivalent to providing an element $f(m) \in A$ for each $m \in X \sqcup Y$. We need such a function such that $\ell \circ i_1 = f$ and $\ell \circ i_2 = g$. But each element $m \in X \sqcup Y$ is either of the form i_1x or i_2y , and cannot be of both forms. So we assign

$$(f \sqcup g)(m) = \begin{cases} f(x) & \text{if } m = i_1x, \\ g(y) & \text{if } m = i_2y. \end{cases}$$

This assignment is necessary and sufficient to make all relevant diagrams commute. $\square$

Example 2.4.2.8 (Airplane seats, continued). The universal property of coproducts says the following. Any time we have a function $X \rightarrow A$ and a function $Y \rightarrow A$, we get a unique function $X \sqcup Y \rightarrow A$. For example, every economy class seat in an airplane and every first class seat in an airplane is actually *in a particular airplane*. Every economy class seat has a price, as does every first class seat.



The universal property of coproducts formalizes the following intuitively obvious fact:

If we know how economy class seats are priced and we know how first class seats are priced, and if we know that every seat is either economy class or first class, then we automatically know how all seats are priced.

To say it another way (and using the other induced map):

If we keep track of which airplane every economy class seat is in and we keep track of which airplane every first class seat is in, and if we know that every seat is either economy class or first class, then we require no additional tracking for any airplane seat whatsoever.

Application 2.4.2.9 (Piecewise defined curves). In science, curves are often defined or considered piecewise. For example in testing the mechanical properties of a material, we might be interested in various regions of **deformation**, such as elastic, plastic, or post-fracture. These are three intervals on which the material displays different kinds of properties.

For real numbers $a < b \in \mathbb{R}$, let $[a, b] := \{x \in \mathbb{R} \mid a \leq x \leq b\}$ denote the closed interval. Given a function $[a, b] \rightarrow \mathbb{R}$ and a function $[c, d] \rightarrow \mathbb{R}$, the universal property of coproducts implies that they extend uniquely to a function $[a, b] \sqcup [c, d] \rightarrow \mathbb{R}$, which will appear as a piecewise defined curve.

Often we are given a curve on $[a, b]$ and another on $[b, c]$, where the two curves agree at the point b . This situation is described by pushouts, which are mild generalizations of coproducts; see Section 2.6.2.

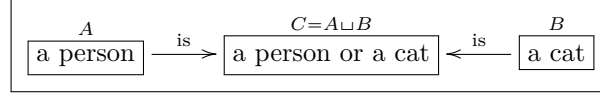
$\diamond\diamond$

Exercise 2.4.2.10. Write the universal property for coproduct in terms of a relationship between the following three sets:

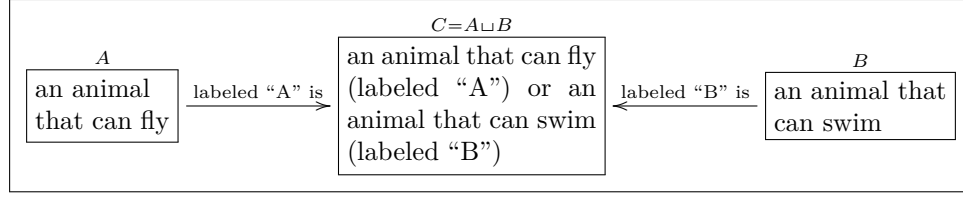
$$\text{Hom}_{\text{Set}}(X, A), \quad \text{Hom}_{\text{Set}}(Y, A), \quad \text{and} \quad \text{Hom}_{\text{Set}}(X \sqcup Y, A).$$

◇

Example 2.4.2.11. In the following olog the types A and B are disjoint, so the coproduct $C = A \sqcup B$ is just the union.



Example 2.4.2.12. In the following olog, A and B are not disjoint, so care must be taken to differentiate common elements.



Since ducks can both swim and fly, each duck is found twice in C , once labeled as a flyer and once labeled as a swimmer. The types A and B are kept disjoint in C , which justifies the name “disjoint union.”

Exercise 2.4.2.13. Understand Example 2.4.2.12 and see if a similar idea would make sense for particles and waves. Make an olog, and choose your wording in accordance with Rules 2.3.1.2. How do photons, which exhibit properties of both waves and particles, fit into the coproduct in your olog?

◇

Exercise 2.4.2.14. Following the section above, “Ologging products” page 34, come up with a naming system for coproducts, the inclusions, and the universal maps. Try it out by making an olog (involving coproducts) discussing the idea that both a .wav file and a .mp3 file can be played on a modern computer. Be careful that your arrows are valid in the sense of Section 2.3.2.1.

◇

2.5 Finite limits in Set

In this section we discuss what are called *limits* of variously-shaped diagrams of sets. We will make all this much more precise when we discuss limits in arbitrary categories in Section 4.5.3.

2.5.1 Pullbacks

Definition 2.5.1.1 (Pullback). Suppose given the diagram of sets and functions below.

$$\begin{array}{ccc} & Y & \\ & \downarrow g & \\ X & \xrightarrow{f} & Z \end{array} \quad (2.29)$$

Its *fiber product* is the set

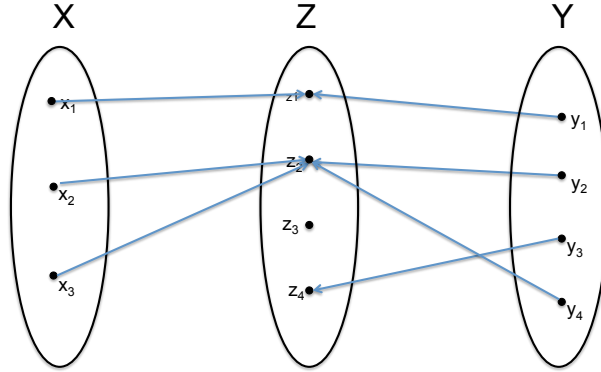
$$X \times_Z Y := \{(x, w, y) \mid f(x) = w = g(y)\}.$$

There are obvious projections $\pi_1: X \times_Z Y \rightarrow X$ and $\pi_2: X \times_Z Y \rightarrow Y$ (e.g. $\pi_2(x, w, y) = y$). Note that if $W = X \times_Z Y$ then the diagram

$$\begin{array}{ccc} W & \xrightarrow{\pi_2} & Y \\ \pi_1 \downarrow & \lrcorner & \downarrow g \\ X & \xrightarrow{f} & Z \end{array} \quad (2.30)$$

commutes. Given the setup of Diagram 2.29 we define the *pullback of X and Y over Z* to be any set W for which we have an isomorphism $W \xrightarrow{\cong} X \times_Z Y$. The corner symbol $\lrcorner$ in Diagram 2.30 indicates that W is the pullback.

Exercise 2.5.1.2. Let X, Y, Z be as drawn and $f: X \rightarrow Z$ and $g: Y \rightarrow Z$ the indicated functions.



What is the pullback of the diagram $X \xrightarrow{f} Z \xleftarrow{g} Y$? ◇

Exercise 2.5.1.3. a.) Draw a set X with five elements and a set Y with three elements. Color each element of X and each element of Y either red, blue, or yellow,⁹ and do so in a “random-looking” way. Considering your coloring of X as a function $X \rightarrow C$, where $C = \{\text{red, blue, yellow}\}$, and similarly obtaining a function $Y \rightarrow C$, draw the fiber product $X \times_C Y$. Make sure it is colored appropriately.

b.) The universal property for products guarantees a function $X \times_C Y \rightarrow X \times Y$, which I can tell you will be an injection. This means that the drawing you made of the fiber product can be imbedded into the 5×3 grid; please draw the grid and indicate this subset. ◇

Remark 2.5.1.4. Some may prefer to denote this fiber product by $f \times_Z g$ rather than $X \times_Z Y$. The former is mathematically better notation, but human-readability is often enhanced by the latter, which is also more common in the literature. We use whichever is more convenient.

Exercise 2.5.1.5. a.) Suppose that $Y = \emptyset$; what can you say about $X \times_Z Y$?

b.) Suppose now that Y is any set but that Z has exactly one element; what can you say about $X \times_Z Y$? ◇

⁹You can use shadings rather than coloring, if coloring would be annoying.

Exercise 2.5.1.6. Let $S = \mathbb{R}^3$, $T = \mathbb{R}$, and think of them as (Aristotilian) space and time, with the origin in $S \times T$ given by the center of mass of MIT at the time of its founding. Let $Y = S \times T$ and let $g_1: Y \rightarrow S$ be one projection and $g_2: Y \rightarrow T$ the other projection. Let $X = \{\odot\}$ be a set with one element and let $f_1: X \rightarrow S$ and $f_2: X \rightarrow T$ be given by the origin in both cases.

a.) What are the fiber products W_1 and W_2 :

$$\begin{array}{ccc} W_1 & \longrightarrow & Y \\ \downarrow j & & \downarrow g_1 \\ X & \xrightarrow{f_1} & S \end{array} \qquad \begin{array}{ccc} W_2 & \longrightarrow & Y \\ \downarrow j & & \downarrow g_2 \\ X & \xrightarrow{f_2} & T \end{array}$$

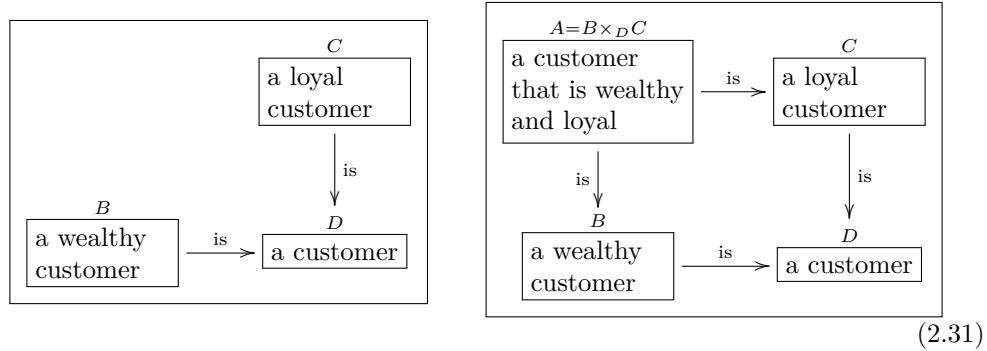
b.) Interpret these sets in terms of the center of mass of MIT at the time of its founding.

◇

2.5.1.7 Using pullbacks to define new ideas from old

In this section we will see that the fiber product of a diagram can serve to define a new concept. For example, in (2.33) we define what it means for a cellphone to have a bad battery, in terms of the length of time for which it remains charged. By being explicit, we reduce the chance of misunderstandings between different groups of people. This can be useful in situations like audits and those in which one is trying to reuse or understand data gathered by others.

Example 2.5.1.8. Consider the following two ologs. The one on the right is the pullback of the one on the left.

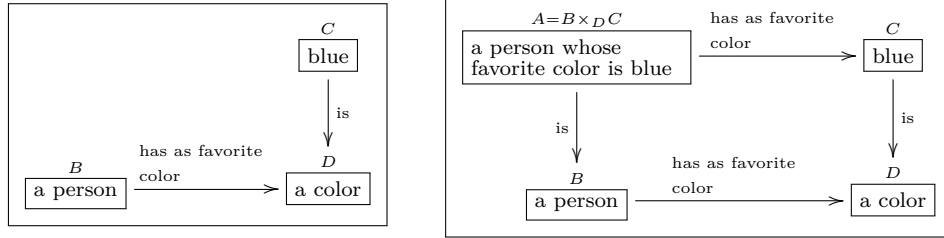


Check from Definition 2.5.1.1 that the label, “a customer that is wealthy and loyal”, is fair and straightforward as a label for the fiber product $A = B \times_D C$, given the labels on B, C , and D .

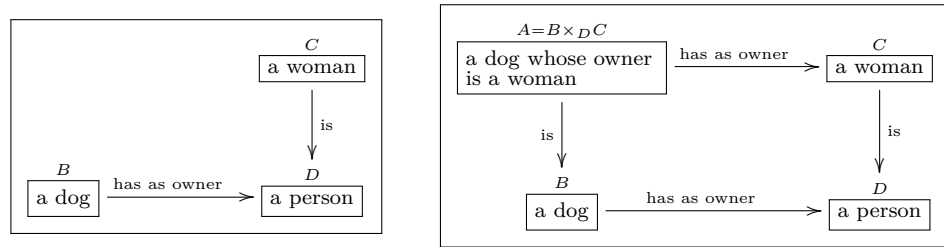
Remark 2.5.1.9. Note that in Diagram (2.31) the top-left box could have been (non-canonically named) ‘a good customer’. If it was taken to be the fiber product, then the author would be effectively *defining* a good customer to be one that is wealthy and loyal.

Exercise 2.5.1.10. For each of the following, an author has proposed that the diagram on the right is a pullback. Do you think their labels are appropriate or misleading; that is, is the label on the upper-left box reasonable given the rest of the olog, or is it suspect in some way?

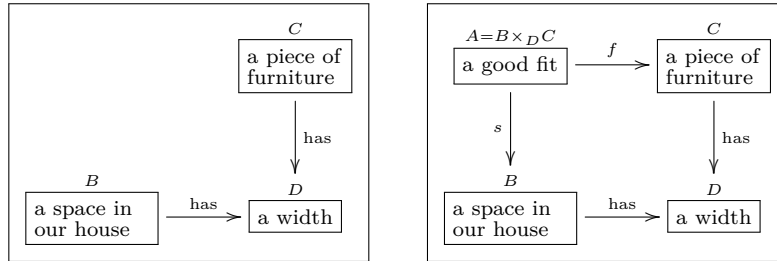
a.)



b.)



c.)



◇

Exercise 2.5.1.11. a.) Consider your olog from Exercise 2.3.3.1. Are any of the commutative squares there actually pullback squares?

b.) Now use ologs with products and pullbacks to define what a brother is and what a sister is (again in a human biological nuclear family), in terms of types such as 'an offspring of mating pair (a, b) ', 'a person', 'a male person', 'a female person', and so on.

◇

Definition 2.5.1.12 (Preimage). Let $f: X \rightarrow Y$ be a function and $y \in Y$ an element. The *preimage of y under f* , denoted $f^{-1}(y)$, is the subset $f^{-1}(y) := \{x \in X \mid f(x) = y\}$.

Exercise 2.5.1.13. Let $f: X \rightarrow Y$ be a function and $y \in Y$ an element. Draw a pullback diagram in which the fiber product is isomorphic to the preimage $f^{-1}(y)$.

◇

Lemma 2.5.1.14 (Universal property for pullback). *Suppose given the diagram of sets and functions as below.*

$$\begin{array}{ccc} & Y & \\ & \downarrow u & \\ X & \xrightarrow{t} & Z \end{array}$$

For any set A and commutative solid arrow diagram as below (i.e. functions $f: A \rightarrow X$ and $g: A \rightarrow Y$ such that $t \circ f = u \circ g$),

$$\begin{array}{ccccc} & & X \times_Z Y & & \\ & \swarrow \pi_1 & \uparrow \exists! & \searrow \pi_2 & \\ & A & & & \\ & \swarrow \forall f & & \searrow \forall g & \\ X & & & & Y \\ & \searrow t & & \swarrow u & \\ & & Z & & \end{array} \quad (2.32)$$

there exists a unique arrow $f \times_Z g: A \rightarrow X \times_Z Y$ making everything commute,

$$f = \pi_1 \circ (f \times_Z g) \quad \text{and} \quad g = \pi_2 \circ (f \times_Z g).$$

Exercise 2.5.1.15. Create an olog whose underlying shape is a commutative square. Now add the fiber product so that the shape is the same as that of Diagram (2.32). Assign English labels to the projections π_1, π_2 and to the dotted map $A \xrightarrow{f \times_Z g} X \times_Z Y$, such that these labels are as canonical as possible. $\diamond$

2.5.1.16 Pasting diagrams for pullback

Consider the diagram drawn below, which includes a left-hand square, a right-hand square, and a big rectangle.

$$\begin{array}{ccccc} A' & \xrightarrow{f'} & B' & \xrightarrow{g'} & C' \\ \downarrow i & \lrcorner & \downarrow j & \lrcorner & \downarrow k \\ A & \xrightarrow{f} & B & \xrightarrow{g} & C \end{array}$$

The right-hand square has a corner symbol indicating that $B' \cong B \times_C C'$ is a pullback. But the corner symbol on the left is ambiguous; it might be indicating that the left-hand square is a pullback, or it might be indicating that the big rectangle is a pullback. It turns out that if $B' \cong B \times_C C'$ then it is not ambiguous because the left-hand square is a pullback if and only if the big rectangle is.

Proposition 2.5.1.17. *Consider the diagram drawn below*

$$\begin{array}{ccccc}
 & & B' & \xrightarrow{g'} & C' \\
 & & \downarrow j & \lrcorner & \downarrow k \\
 A & \xrightarrow{f} & B & \xrightarrow{g} & C
 \end{array}$$

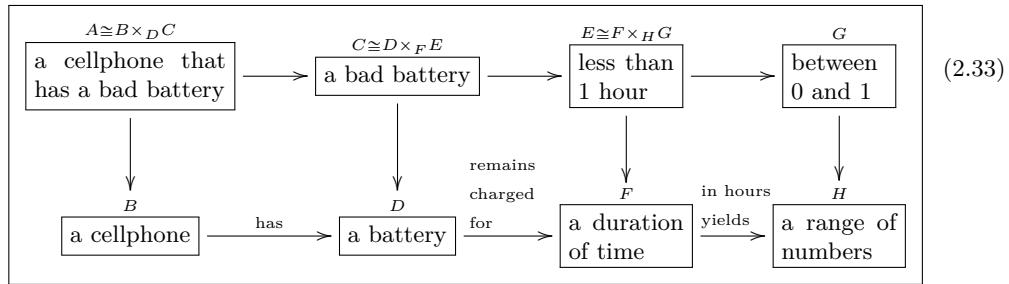
where $B' \cong B \times_C C'$ is a pullback. Then there is an isomorphism $A \times_B B' \cong A \times_C C'$. Said another way,

$$A \times_B (B \times_C C') \cong A \times_C C'.$$

Proof. We first provide a map $\phi: A \times_B (B \times_C C') \rightarrow A \times_C C'$. An element of $A \times_B (B \times_C C')$ is of the form $(a, b, (b, c, c'))$ such that $f(a) = b, g(b) = c$ and $k(c') = c$. But this implies that $g \circ f(a) = c = k(c')$ so we put $\phi(a, b, (b, c, c')) := (a, c, c') \in A \times_C C'$. Now we provide a proposed inverse, $\psi: A \times_C C' \rightarrow A \times_B (B \times_C C')$. Given (a, c, c') with $g \circ f(a) = c = k(c')$, let $b = f(a)$ and note that (b, c, c') is an element of $B \times_C C'$. So we can define $\psi(a, c, c') = (a, b, (b, c, c'))$. It is easy to see that ϕ and ψ are inverse.

□

Proposition 2.5.1.17 can be useful in authoring ologs. For example, the type ‘a cellphone that has a bad battery’ is vague, but we can lay out precisely what it means using pullbacks:

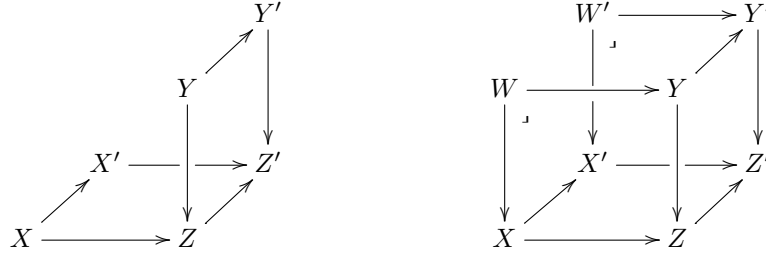


The category-theoretic fact described above says that since $A \cong B \times_D C$ and $C \cong D \times_F E$, it follows that $A \cong B \times_F E$. That is, we can deduce the definition “a cellphone that has a bad battery is defined as a cellphone that has a battery which remains charged for less than one hour.”

- Exercise 2.5.1.18.* a.) Create an olog that defines two people to be “of approximately the same height” if and only if their height difference is less than half an inch, using a pullback. Your olog can include the box ‘a real number x such that $-.5 < x < .5$ ’.
- b.) In the same olog, make a box for those people whose height is approximately the same as a person named “The Virgin Mary”. You may need to use images, as in Section 2.3.3.8.

◇

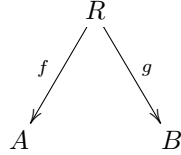
Exercise 2.5.1.19. Consider the diagram on the left below, where both squares commute.



Let $W = X \times_Z Y$ and $W' = X' \times_{Z'} Y'$, and form the diagram to the right. Use the universal property of fiber products to construct a map $W \rightarrow W'$ such that all squares commute. $\diamond$

2.5.2 Spans, experiments, and matrices

Definition 2.5.2.1. Given sets A and B , a *span on A and B* is a set R together with functions $f: R \rightarrow A$ and $g: R \rightarrow B$.



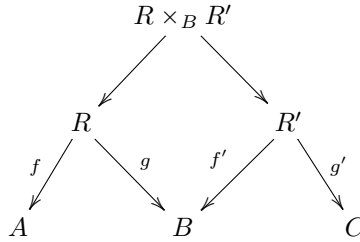
Application 2.5.2.2. Think of A and B as observables and R as a set of experiments performed on these two variables. For example, let's say T is the set of possible temperatures of a **gas** in a fixed container and let's say P is the set of possible pressures of the gas. We perform 1000 experiments in which we change and record the temperature and we simultaneously also record the pressure; this is a span $T \xleftarrow{f} E \xrightarrow{g} P$. The results might look like this:

Experiment		
ID	Temperature	Pressure
1	100	72
2	100	73
3	100	72
4	200	140
5	200	138
6	200	141
$\vdots$	$\vdots$	$\vdots$

$\diamond\diamond$

Definition 2.5.2.3. Let A, B , and C be sets, and let $A \xleftarrow{f} R \xrightarrow{g} B$ and $B \xleftarrow{f'} R' \xrightarrow{g'} C$ be spans. Their *composite span* is given by the fiber product $R \times_B R'$ as in the diagram

below:



Application 2.5.2.4. Let's look back at our lab's experiment from Application 2.5.2.2, which resulted in a span $T \xleftarrow{f} E \xrightarrow{g} P$. Suppose we notice that something looks a little wrong. The pressure should be linear in the temperature but it doesn't appear to be. We hypothesize that the volume of the container is increasing under pressure. We look up this container online and see that experiments have been done to measure the volume as the interior pressure changes. The data has generously been made available online, which gives us a span $P \xleftarrow{f'} E' \xrightarrow{g'} V$.

The composite of our lab's span with the online data span yields a span $T \leftarrow E'' \rightarrow V$, where $E'' := E \times_P E'$. What information does this span give us? In explaining it, one might say “whenever an experiment in our lab yielded the same pressure as one they recorded, let's call that a data point. Every data point has an associated temperature (from our lab) and an associated volume (from their experiment). This is the best we can do.”

The information we get this way might be seen by some as unscientific, but it certainly is the kind of information people use in business and in every day life calculation—we get our data from multiple sources and put it together. Moreover, it is scientific in the sense that it is reproducible. The way we obtained our T - V data is completely transparent.

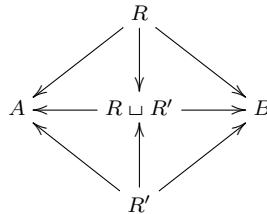
◇◇

We can relate spans to matrices of natural numbers, and see a natural “categorification” of matrix addition and matrix multiplication. If our spans come from experiments as in Applications 2.5.2.2 and 2.5.2.4 the matrices involved will look like huge but sparse matrices. Let's go through that.

Let A and B be sets and let $A \leftarrow R \rightarrow B$ be a span. By the universal property of products, we have a unique map $R \xrightarrow{p} A \times B$.

We make a matrix of natural numbers out of this data as follows. The set of rows is A , the set of columns is B . For elements $a \in A$ and $b \in B$, the (a, b) -entry is the cardinality of its preimage, $|p^{-1}(a, b)|$, i.e. the number of elements in R that are sent by p to (a, b) .

Given two spans of the same dimensions, i.e. $A \leftarrow R \rightarrow B$ and $A \leftarrow R' \rightarrow B$ we can take the disjoint union $R \sqcup R'$ and by the universal property of coproducts we have a unique span $A \leftarrow R \sqcup R' \rightarrow B$ making the requisite diagram commute.¹⁰ The matrix



¹⁰

corresponding to this new span will be the sum of the matrices corresponding to the two previous spans out of which it was made.

Given a span $A \leftarrow R \rightarrow B$ and a span $B \leftarrow S \rightarrow C$, the composite span can be formed as in Definition 2.5.2.3. It will correspond to the usual multiplication of matrices.

Construction 2.5.2.5. Given a span $A \xleftarrow{f} R \xrightarrow{g} B$, one can draw a *bipartite graph* with each element of A drawn as a dot on the left, each element of B drawn as a dot on the right, and each element $r \in R$ drawn as an arrow connecting vertex $f(r)$ on the left to vertex $g(r)$ on the right.

Exercise 2.5.2.6. a.) Draw the bipartite graph (as in Construction 2.5.2.5) corresponding to the span $T \xleftarrow{f} E \xrightarrow{g} P$ in Application 2.5.2.2.

b.) Now make up your own span $P \xleftarrow{f'} E' \xrightarrow{g'} V$ and draw it. Finally, draw the composite span below.

c.) Can you say how the composite span graph relates to the graphs of its factors?

◇

2.5.3 Equalizers and terminal objects

Definition 2.5.3.1. Suppose given two parallel arrows

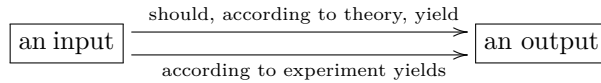
$$X \begin{array}{c} \xrightarrow{f} \\ \xrightarrow{g} \end{array} Y. \quad \quad \quad Eq(f, g) \xrightarrow{p} X \begin{array}{c} \xrightarrow{f} \\ \xrightarrow{g} \end{array} Y \quad (2.34)$$

The *equalizer of f and g* is the commutative diagram as to the right in (2.34), where we define

$$Eq(f, g) := \{x \in X \mid f(x) = g(x)\}$$

and where p is the canonical inclusion.

Example 2.5.3.2. Suppose one has designed an experiment to test a theoretical prediction. The question becomes, “when does the theory match the experiment?” The answer is given by the equalizer of the following diagram:



The equalizer is the set of all inputs for which the theory and the experiment yield the same output.

Exercise 2.5.3.3. Come up with an olog that uses equalizers in a reasonably interesting way. Alternatively, use an equalizer to specify those published authors who have published exactly one paper. Hint: find a function from authors to papers; then find another. ◇

Exercise 2.5.3.4. Find a universal property enjoyed by the equalizer of two arrows, and present it in the style of Lemmas 2.4.1.10, 2.4.2.7, and 2.5.1.14. ◇

Exercise 2.5.3.5. a.) A terminal set is a set S such that for every set X , there exists a unique function $X \rightarrow S$. Find a terminal set.

b.) Do you think that the notion *terminal set* belongs in this section (Section 2.5)? How so? If products, pullbacks, and equalizers are all limits, what do limits have in common?

◇

2.6 Finite colimits in Set

This section will parallel Section 2.5—I will introduce several types of finite colimits and hope that this gives the reader some intuition about them, without formally defining them yet. Before doing so, I must define equivalence relations and quotients.

2.6.1 Background: equivalence relations

Definition 2.6.1.1 (Equivalence relations and equivalence classes). Let X be a set. An *equivalence relation on X* is a subset $R \subseteq X \times X$ satisfying the following properties for all $x, y, z \in X$:

Reflexivity: $(x, x) \in R$;

Symmetry: $(x, y) \in R$ if and only if $(y, x) \in R$; and

Transitivity: if $(x, y) \in R$ and $(y, z) \in R$ then $(x, z) \in R$.

If R is an equivalence relation, we often write $x \sim_R y$, or simply $x \sim y$, to mean $(x, y) \in R$. For convenience we may refer to the equivalence relation by the symbol $\sim$, saying that $\sim$ is an equivalence relation on X .

An *equivalence class of $\sim$* is a subset $A \subseteq X$ such that

- A is nonempty, $A \neq \emptyset$;
- if $x \in A$ and $x' \in A$, then $x \sim x'$; and
- if $x \in A$ and $x \sim y$, then $y \in A$.

Suppose that $\sim$ is an equivalence relation on X . The *quotient of X by $\sim$* , denoted $X/\sim$ is the set of equivalence classes of $\sim$.

Example 2.6.1.2. Let $\mathbb{Z}$ denote the set of integers. Define a relation $R \subseteq \mathbb{Z} \times \mathbb{Z}$ by

$$R = \{(x, y) \mid \exists n \in \mathbb{Z} \text{ such that } x + 7n = y\}.$$

Then R is an equivalence relation because $x + 7 \cdot 0 = x$ (reflexivity); $x + 7 \cdot n = y$ if and only if $y + 7 \cdot (-n) = x$ (symmetry); and $x + 7n = y$ and $y + 7m = z$ together imply that $x + 7(m + n) = z$ (transitivity).

Exercise 2.6.1.3. Let X be the set of people on earth; define a binary relation $R \subseteq X \times X$ on X as follows. For a pair (x, y) of people, say $(x, y) \in R$ if x spends a lot of time thinking about y .

- a.) Is this relation reflexive?
- b.) Is it symmetric?
- c.) Is it transitive?

◇

Example 2.6.1.4. [Partitions]

An equivalence relation on a set X can be thought of as a way of partitioning X . A *partition of X* consists of a set I , called *the set of partitions*, and for every element $i \in I$ a subset $X_i \subseteq X$ such that two properties hold:

- every element $x \in X$ is in some subset (i.e. for all $x \in X$ there exists $i \in I$ such that $x \in X_i$); and
- no element can be found in two different subsets (i.e. if $x \in X_i$ and $x \in X_j$ then $i = j$).

Given a partition of X , we define an equivalence relation $\sim$ on X by saying $x \sim x'$ if x and x' are in the same partition (i.e. if there exists $i \in I$ such that $x, x' \in X_i$). The partitions become the equivalence classes of this relation. Conversely, given an equivalence relation, one makes a partition on X by taking I to be the set of equivalence classes and for each $i \in I$ letting X_i be the elements in that equivalence class.

Exercise 2.6.1.5. Let X and B be sets and let $f: X \rightarrow B$ be a function. Define a subset $R \subseteq X \times X$ by

$$R = \{(x, y) \mid f(x) = f(y)\}.$$

Is R an equivalence relation?

- Are all equivalence relations on X obtainable in this way (as the fibers of some function having domain X)?
- Does this viewpoint on equivalence classes relate to that of Example 2.6.1.4?

◇

Exercise 2.6.1.6. Take a set I of sets; i.e. suppose that for each element $i \in I$ you are given a set X_i . For every two elements $i, j \in I$ say that $i \sim j$ if X_i and X_j are isomorphic. Is this relation an equivalence relation on I ?

◇

Definition 2.6.1.7 (Generating equivalence relations). Let X be a set and $R \subseteq X \times X$ a subset. Define S to be the smallest subset of $X \times X$ such that S is an equivalence relation and $R \subseteq S$. We say that S is *the equivalence relation generated by R* .

Exercise 2.6.1.8. Consider the set $\mathbb{R}$ of real numbers. Draw the coordinate plane $\mathbb{R} \times \mathbb{R}$, give it coordinates x and y . A binary relation on $\mathbb{R}$ is a subset $S \subseteq \mathbb{R} \times \mathbb{R}$, which can be drawn as a set of points in the plane.

- Draw the relation $\{(x, y) \mid y = x^2\}$.
- Draw the relation $\{(x, y) \mid y \geq x^2\}$.
- Let S_0 be the equivalence relation on $\mathbb{R}$ generated (in the sense of Definition 2.6.1.7) by the empty set. Draw S as a subset of the plane.
- Consider the equivalence relation S_1 generated by $\{(1, 2), (1, 3)\}$. Draw S_1 in the plane. Highlight the equivalence class containing $(1, 2)$.
- The reflexivity property and the symmetry property have pleasing visualizations in $\mathbb{R} \times \mathbb{R}$; what are they?
- Is there a nice heuristic for visualizing the transitivity property?

◇

Exercise 2.6.1.9. Consider the binary relation $R = \{(n, n+1) \mid n \in \mathbb{Z}\} \subseteq \mathbb{Z} \times \mathbb{Z}$.

- What is the equivalence relation generated by R ?
- How many equivalence classes are there?

◇

Exercise 2.6.1.10. Suppose N is a network (or graph). Let X be the nodes of the network, and let $R \subseteq X \times X$ denote the relation such that $(x, y) \in R$ iff there exists an edge connecting x and y .

- a.) What is the equivalence relation $\sim$ generated by R ?
- b.) What is the quotient $X/\sim$?

◇

2.6.2 Pushouts

Definition 2.6.2.1 (Pushout). Suppose given the diagram of sets and functions below:

$$\begin{array}{ccc} W & \xrightarrow{f} & X \\ g \downarrow & & \\ Y & & \end{array} \quad (2.35)$$

Its *fiber sum*, denoted $X \sqcup_W Y$, is defined as the quotient of $X \sqcup W \sqcup Y$ by the equivalence relation $\sim$ generated by $w \sim f(w)$ and $w \sim g(w)$ for all $w \in W$.

$$X \sqcup_W Y := (X \sqcup W \sqcup Y) / \sim \quad \text{where } \forall w \in W, \quad w \sim f(w) \quad \text{and} \quad w \sim g(w).$$

There are obvious inclusions $i_1: X \rightarrow X \sqcup_W Y$ and $i_2: Y \rightarrow X \sqcup_W Y$.¹¹ Note that if $Z = X \sqcup_W Y$ then the diagram

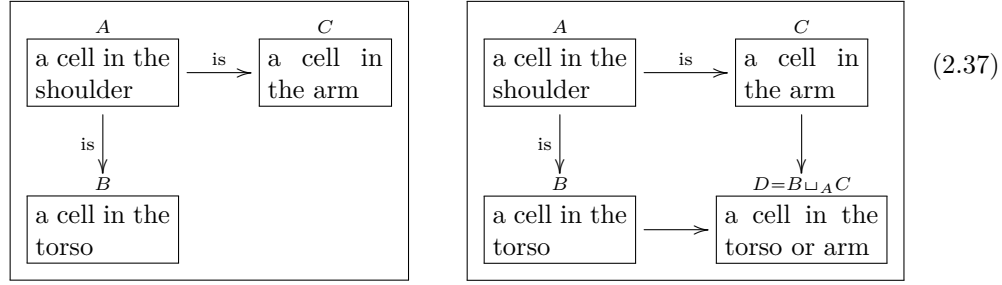
$$\begin{array}{ccc} W & \xrightarrow{g} & Y \\ f \downarrow & \lrcorner & \downarrow i_2 \\ X & \xrightarrow{i_1} & Z \end{array} \quad (2.36)$$

commutes. Given the setup of Diagram 2.35 we define the *pushout of X and Y over W* to be any set Z for which we have an isomorphism $Z \xrightarrow{\cong} X \sqcup_W Y$. The corner symbol $\lrcorner$ in Diagram 2.36 indicates that Z is the pushout.

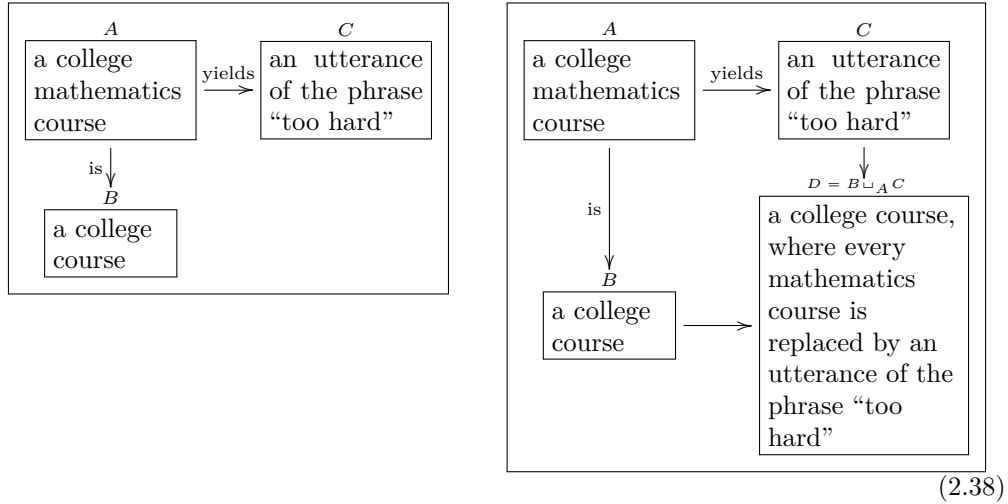
Example 2.6.2.2. Let $X = \{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$ be the set of numbers between 0 and 1, inclusive, let $Y = \{y \in \mathbb{R} \mid 1 \leq y \leq 2\}$ be the set of numbers between 1 and 2, inclusive, and let $W = \{1\}$. Then the pushout $X \xleftarrow{f} W \xrightarrow{g} Y$, where f and g are the “obvious” functions ($1 \mapsto 1$) is $X \sqcup_W Y \cong \{z \in \mathbb{R} \mid 0 \leq z \leq 2\}$, as expected. When we eventually get to general colimits, one can check that the whole real line can be made by patching together intervals in this way.

Example 2.6.2.3. In each example below, the diagram to the right is intended to be a pushout of the diagram to the left. The new object, D , is the union of B and C , but instances of A are equated to their B and C aspects. This will be discussed after the two diagrams.

¹¹Note that our term inclusions is not too good, because it seems to suggest that i_1 and i_2 are injective (see Definition 2.7.5.1) and this is not always the case.



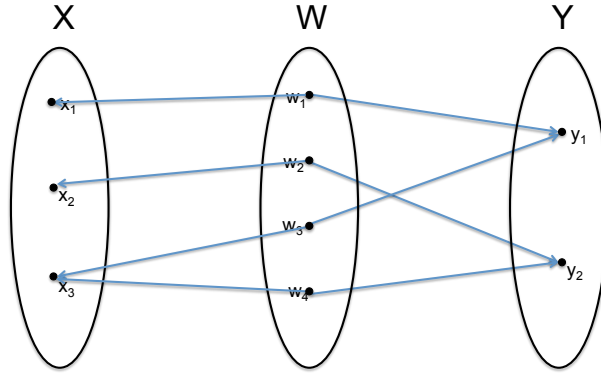
In the left-hand olog (2.37), the two arrows are inclusions: the author considers every cell in the shoulder to be both in the arm and in the torso. The pushout is then just the union, where cells in the shoulder are not double-counted.



In Olog (2.37), the shoulder is seen as part of the arm and part of the torso. When taking the union of these two parts, we do not want to “double-count” the shoulder (as would be done in the coproduct $B \sqcup C$, see Example 2.4.2.12). Thus we create a new type A for cells in the shoulder, which are considered the same whether viewed as cells in the arm or cells in the torso. In general, if one wishes to take two things and glue them together, with A as the glue and with B and C as the two things to be glued, the union is the pushout $B \sqcup_A C$. (A nice image of this can be seen in the setting of topological spaces, see Example 4.5.3.29.)

In Olog (2.38), if every mathematics course is simply “too hard,” then when reading off a list of courses, each math course will not be read aloud but simply read as “too hard.” To form D we begin by taking the union of B and C , and then we consider everything in A to be the same whether one looks at it as a course or as the phrase “too hard.” The math courses are all blurred together as one thing. Thus we see that the power to equate different things can be exercised with pushouts.

Exercise 2.6.2.4. Let W, X, Y be as drawn and $f: W \rightarrow X$ and $g: W \rightarrow Y$ the indicated functions.



The pushout of the diagram $X \xleftarrow{f} W \xrightarrow{g} Y$ is a set P . Write down the cardinality of $P \cong \underline{n}$ as a natural number $n \in \mathbb{N}$. $\diamond$

Exercise 2.6.2.5. Suppose that $W = \emptyset$; what can you say about $X \sqcup_W Z$? $\diamond$

Exercise 2.6.2.6. Let $W := \mathbb{N} = \{0, 1, 2, \dots\}$ denote the set of natural numbers, let $X = \mathbb{Z}$ denote the set of integers, and let $Y = \{\odot\}$ denote a one-element set. Define $f: W \rightarrow X$ by $f(w) = -(w + 1)$, and define $g: W \rightarrow Y$ to be the unique map. Describe the set $X \sqcup_W Y$. $\diamond$

Exercise 2.6.2.7. Let $i: R \subseteq X \times X$ be an equivalence relation (see Example 2.1.2.3 for notation). Composing with the projections $\pi_1, \pi_2: X \times X \rightarrow X$, we have two maps $\pi_1 \circ i, \pi_2 \circ i: R \rightarrow X$.

a.) What is the pushout

$$X \xleftarrow{\pi_1 \circ i} R \xrightarrow{\pi_2 \circ i} X?$$

b.) If $i: R \subseteq X \times X$ is not assumed to be an equivalence relation, we can still define the pushout above. Is there a relationship between the pushout $X \xleftarrow{\pi_1 \circ i} R \xrightarrow{\pi_2 \circ i} X$ and the equivalence relation generated by $R \subseteq X \times X$? $\diamond$

Lemma 2.6.2.8 (Universal property for pushout). *Suppose given the diagram of sets and functions as below.*

$$\begin{array}{ccc} W & \xrightarrow{u} & Y \\ \downarrow t & & \\ X & & \end{array}$$

For any set A and commutative solid arrow diagram as below (i.e. functions $f: X \rightarrow A$

and $g: Y \rightarrow A$ such that $f \circ t = g \circ u$,

$$\begin{array}{ccccc}
 & & W & & \\
 & \swarrow t & & \searrow u & \\
 X & & & & Y \\
 & \searrow f & & \swarrow g & \\
 & & A & & \\
 & \swarrow i_1 & \uparrow \exists! & \searrow i_2 & \\
 & & X \sqcup_W Y & &
 \end{array} \tag{2.39}$$

there exists a unique arrow $f \sqcup_W g: X \sqcup_W Y \rightarrow A$ making everything commute,

$$f = (f \sqcup_W g) \circ i_1 \quad \text{and} \quad g = (f \sqcup_W g) \circ i_2.$$

2.6.3 Other finite colimits

Definition 2.6.3.1. [Coequalizer]

Suppose given two parallel arrows

$$X \begin{array}{c} \xrightarrow{f} \\ \xrightarrow{g} \end{array} Y. \quad X \begin{array}{c} \xrightarrow{f} \\ \xrightarrow{g} \end{array} Y \xrightarrow{q} \text{Coeq}(f, g) \tag{2.40}$$

The *coequalizer* of f and g is the commutative diagram as to the right in (2.40), where we define

$$\text{Coeq}(f, g) := Y / f(x) \sim g(x)$$

i.e. the coequalizer of f and g is the quotient of Y by the equivalence relation generated by $\{(f(x), g(x)) \mid x \in X\} \subseteq Y \times Y$

Exercise 2.6.3.2. Let $X = \mathbb{R}$ be the set of real numbers. What is the coequalizer of the two maps $X \rightarrow X$ given by $x \mapsto x$ and $x \mapsto (x + 1)$ respectively? $\diamond$

Exercise 2.6.3.3. Find a universal property enjoyed by the coequalizer of two arrows. $\diamond$

Exercise 2.6.3.4 (Initial object). An initial set is a set S such that for every set A , there exists a unique function $S \rightarrow A$.

a.) Find an initial set.

b.) Do you think that the notion *initial set* belongs in this section (Section 2.6)? How so? If coproducts, pushouts, and coequalizers are all colimits, what do colimits have in common?

$\diamond$

2.7 Other notions in Set

In this section we discuss some left-over notions in the category of Sets.

2.7.1 Retractions

Definition 2.7.1.1. Suppose we have a function $f: X \rightarrow Y$ and a function $g: Y \rightarrow X$ such that $g \circ f = \text{id}_X$. In this case we call f a *retract section* and we call g a *retract projection*.

Exercise 2.7.1.2. Create an olog that includes sets X and Y , and functions $f: X \rightarrow Y$ and $g: Y \rightarrow X$ such that $g \circ f = \text{id}_X$ but such that $f \circ g \neq \text{id}_Y$; that is, such that f is a retract section but not an isomorphism. $\diamond$

2.7.2 Currying

Currying is the idea that when a function takes many inputs, we can input them one at a time or all at once. For example, consider the function that takes a material M and an extension E and returns the force transmitted through the material when it is pulled to that extension. This is a function $e: \text{'a material'} \times \text{'an extension'} \rightarrow \text{'a force'}$. This function takes two inputs at once, but it is convenient to “curry” the second input. Recall that $\text{Hom}_{\text{Set}}(\text{'an extension'}, \text{'a force'})$ is the set of theoretical force-extension curves. Currying transforms e into a function

$$e': \text{'a material'} \rightarrow \text{Hom}_{\text{Set}}(\text{'an extension'}, \text{'a force'}).$$

This is a more convenient way to package the same information.

In fact, it may be convenient to repackage this information another way. For any extension, we may want the function that takes a material and returns how much force it can transmit at that extension. This is a function

$$e'': \text{'an extension'} \rightarrow \text{Hom}_{\text{Set}}(\text{'a material'}, \text{'a force'}).$$

Notation 2.7.2.1. Let A and B be sets. We sometimes denote the set of functions from A to B by

$$B^A := \text{Hom}_{\text{Set}}(A, B). \quad (2.41)$$

Exercise 2.7.2.2. For a finite set A , let $|A| \in \mathbb{N}$ denote the cardinality of (number of elements in) A . If A and B are both finite (including the possibility that one or both are empty), is it always true that $|B^A| = |B|^{|A|}$? $\diamond$

Proposition 2.7.2.3 (Currying). *Let A denote a set. For any sets X, Y there is a bijection*

$$\phi: \text{Hom}_{\text{Set}}(X \times A, Y) \xrightarrow{\cong} \text{Hom}_{\text{Set}}(X, Y^A). \quad (2.42)$$

Proof. Suppose given $f: X \times A \rightarrow Y$. Define $\phi(f): X \rightarrow Y^A$ as follows: for any $x \in X$ let $\phi(f)(x): A \rightarrow Y$ be defined as follows: for any $a \in A$, let $\phi(f)(x)(a) := f(x, a)$.

We now construct the inverse, $\psi: \text{Hom}_{\text{Set}}(X, Y^A) \rightarrow \text{Hom}_{\text{Set}}(X \times A, Y)$. Suppose given $g: X \rightarrow Y^A$. Define $\psi(g): X \times A \rightarrow Y$ as follows: for any pair $(x, a) \in X \times A$ let $\psi(g)(x, a) := g(x)(a)$.

Then for any $f \in \text{Hom}_{\text{Set}}(X \times A, Y)$ we have $\psi \circ \phi(f)(x, a) = \phi(f)(x)(a) = f(x, a)$, and for any $g \in \text{Hom}_{\text{Set}}(X, Y^A)$ we have $\phi \circ \psi(g)(x)(a) = \psi(g)(x, a) = g(x)(a)$. Thus we see that ϕ is an isomorphism as desired. $\square$

Exercise 2.7.2.4. Let $X = \{1, 2\}$, $A = \{a, b\}$, and $Y = \{x, y\}$.

- a.) Write down three distinct elements of $L := \text{Hom}_{\mathbf{Set}}(X \times A, Y)$.
- b.) Write down all the elements of $M := \text{Hom}_{\mathbf{Set}}(A, Y)$.
- c.) For each element $\ell \in L$, write down the corresponding function $\phi(\ell): X \rightarrow M$ guaranteed by Proposition 2.7.2.3.

◇

Exercise 2.7.2.5. Let A and B be sets. We know that $\text{Hom}_{\mathbf{Set}}(A, B) = B^A$, so we have a function $\text{id}_{B^A}: \text{Hom}_{\mathbf{Set}}(A, B) \rightarrow B^A$. Look at Proposition 2.7.2.3, making the substitutions $X = \text{Hom}_{\mathbf{Set}}(A, B)$, $Y = B$, and $A = A$. Consider the function

$$\phi^{-1}: \text{Hom}_{\mathbf{Set}}(\text{Hom}_{\mathbf{Set}}(A, B), B^A) \rightarrow \text{Hom}_{\mathbf{Set}}(\text{Hom}_{\mathbf{Set}}(A, B) \times A, B)$$

obtained as the inverse of (2.42). We have a canonical element id_{B^A} in the domain of ϕ^{-1} . We can apply the function ϕ^{-1} and obtain an element $ev = \phi^{-1}(\text{id}_{B^A}) \in \text{Hom}_{\mathbf{Set}}(\text{Hom}_{\mathbf{Set}}(A, B) \times A, B)$, which is itself a function,

$$ev: \text{Hom}_{\mathbf{Set}}(A, B) \times A \rightarrow B.$$

- a.) Describe the function ev in terms of how it operates on elements in its domain.
- b.) Why might one be tempted to denote this function by ev ?

◇

If $n \in \mathbb{N}$ is a natural number, recall from (2.6) that there is a nice set $\underline{n} = \{1, 2, \dots, n\}$. If A is a set, we often make the abbreviation

$$A^n := A^{\underline{n}}. \quad (2.43)$$

Exercise 2.7.2.6. In Example 2.4.1.7 we said that $\mathbb{R}^2$ is an abbreviation for $\mathbb{R} \times \mathbb{R}$, but in (2.43) we say that $\mathbb{R}^2$ is an abbreviation for $\mathbb{R}^{\underline{2}}$. Use Exercise 2.1.2.13, Proposition 2.7.2.3, Exercise 2.4.2.10, and the fact that $1+1=2$, to prove that these are isomorphic, $\mathbb{R}^2 \cong (\mathbb{R} \times \mathbb{R})$.

(The answer to Exercise 2.1.2.13 was $A = \{\odot\}$: i.e. $\text{Hom}_{\mathbf{Set}}(\{\odot\}, X) \cong X$ for all X .)

◇

2.7.3 Arithmetic of sets

Proposition 2.7.3.1 summarizes the properties of products, coproducts, and exponentials, and shows them all in a familiar light, namely that of arithmetic. In fact, one can think of the natural numbers as literally being the isomorphism classes of finite sets—that’s what they are used for in counting. Consider the standard procedure for counting the elements of a set S , say cows in a field: one points to an element in S and simultaneously says “1”, points to another element in S and simultaneously says “2”, and so on until finished. This procedure amounts to nothing more than creating an isomorphism (one-to-one mapping) between S and some set $\underline{n}$.

Again, the natural numbers are the isomorphism classes of finite sets. Their behavior, i.e. the arithmetic of natural numbers, reflects the behavior of sets. For example the fact that multiplication distributes over addition is a fact about grids of dots as in Example 2.4.1.2. The following proposition lays out such arithmetic properties of sets.

In this proposition, we denote the coproduct of two sets A and B by the notation $A + B$ rather than $A \sqcup B$. It is a reasonable notation in general, and one that is often used.

Proposition 2.7.3.1. *The following isomorphisms exist for any sets A, B , and C (except for one caveat, see Exercise 2.7.3.2).*

- $A + \underline{0} \cong A$
- $A + B \cong B + A$
- $(A + B) + C \cong A + (B + C)$
- $A \times \underline{0} \cong \underline{0}$
- $A \times \underline{1} \cong A$
- $A \times B \cong B \times A$
- $(A \times B) \times C \cong A \times (B \times C)$
- $A \times (B + C) \cong (A \times B) + (A \times C)$
- $A^{\underline{0}} \cong \underline{1}$
- $A^{\underline{1}} \cong A$
- $\underline{0}^A \cong \underline{0}$
- $\underline{1}^A \cong \underline{1}$
- $A^{B+C} \cong A^B \times A^C$
- $(A^B)^C \cong A^{B \times C}$

Exercise 2.7.3.2. Everything in Proposition 2.7.3.1 is true except in one case, namely that of

$$\underline{0}^{\underline{0}}.$$

In this case, we get conflicting answers, because for any set A , including $A = \emptyset = \underline{0}$, we have claimed both that $A^{\underline{0}} \cong \underline{1}$ and that $\underline{0}^A \cong \underline{0}$.

What is the correct answer for $\underline{0}^{\underline{0}}$, based on the definitions of $\underline{0}$ and $\underline{1}$, given in (2.6), and of A^B , given in (2.41)? $\diamond$

Exercise 2.7.3.3. It is also true of natural numbers that if $a, b \in \mathbb{N}$ and $ab = 0$ then either $a = 0$ or $b = 0$. Is the analogous statement true of all sets? $\diamond$

2.7.4 Subobjects and characteristic functions

Definition 2.7.4.1. For any set B , define the *power set of B* , denoted $\mathbb{P}(B)$, to be the set of subsets of B .

Exercise 2.7.4.2. a.) How many elements does $\mathbb{P}(\emptyset)$ have?

b.) How many elements does $\mathbb{P}(\{\odot\})$ have?

c.) How many elements does $\mathbb{P}(\{1, 2, 3, 4, 5, 6\})$ have?

d.) Any idea why they may have named it “power set”? $\diamond$

2.7.4.3 Simplicial complexes

Definition 2.7.4.4. Let V be a set and let $\mathbb{P}(V)$ be its powerset. A subset $X \subseteq \mathbb{P}(V)$ is called *downward-closed* if, for every $u \in X$ and every $u' \subseteq u$, we have $u' \in X$. We say that X *contains all atoms* if for every $v \in V$ the singleton set $\{v\}$ is an element of X .

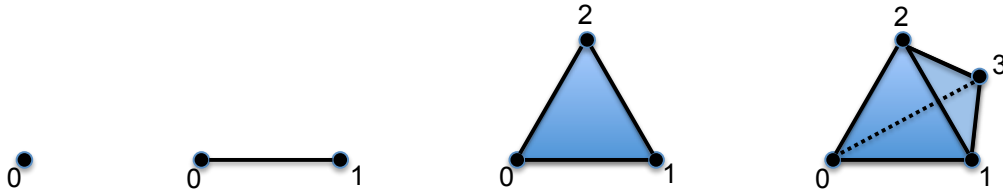
A *simplicial complex* is a pair (V, X) where V is a set and $X \subseteq \mathbb{P}(V)$ is a downward-closed subset that contains all atoms. The elements of X are called *simplices* (singular: *simplex*). Any subset $u \subseteq V$ has a cardinality $|u|$, so we have a function $X \rightarrow \mathbb{N}$ sending each simplex to its cardinality. The set of simplices with cardinality $n + 1$ is denoted X_n and each element $x \in X_n$ is called an *n -simplex*.¹² Since X contains all atoms (subsets of cardinality 1), we have $X_0 \cong V$, and we may also call the 0-simplices *vertices*. We sometimes call the 1-simplices *edges*.¹³

Since $X_0 \cong V$, we may denote a simplicial complex (V, X) simply by X .

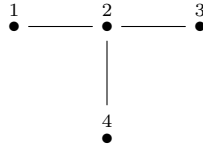
Example 2.7.4.5. Let $n \in \mathbb{N}$ be a natural number and let $V = n + 1$. Define the *n -simplex*, denoted Δ^n , to be the simplicial complex $\mathbb{P}(V) \subseteq \mathbb{P}(V)$, i.e. the whole power set, which indeed is downward-closed and contains all atoms.

We can draw a simplicial complex X by first putting all the vertices on the page as dots. Then for every $x \in X_1$, we see that $x = \{v, v'\}$ consists of 2 vertices, so we draw an edge connecting v and v' . For every $y \in X_2$ we see that $y = \{w, w', w''\}$ consists of 3 vertices, so we draw a (filled-in) triangle connecting them. All three edges will be drawn too because X is assumed to be downward closed.

Thus, the 0-simplex Δ^0 , the 1-simplex Δ^1 , the 2-simplex Δ^2 , and the 3-simplex Δ^3 are drawn here:



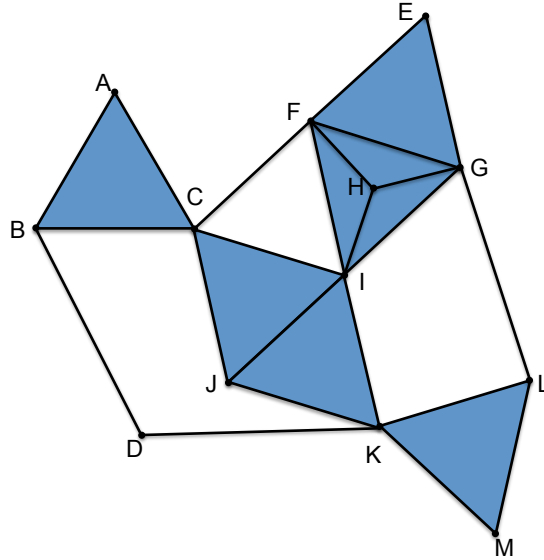
The n -simplices for various n 's are in no way all of the simplicial complexes. In general a simplicial complex is a union or “gluing together” of simplices in a prescribed manner. For example, consider the simplicial complex X with vertices $X_0 = \{1, 2, 3, 4\}$, edges $X_1 = \{\{1, 2\}, \{2, 3\}, \{2, 4\}\}$, and no higher simplices $X_2 = X_3 = \dots = \emptyset$. We might draw X as follows:



Exercise 2.7.4.6. Let X be the following simplicial complex, so that $X_0 = \{A, B, \dots, M\}$.

¹²It is annoying at first that the set of subsets with cardinality 1 is denoted X_0 , etc. But this is standard convention because as we will see, X_n will be n -dimensional.

¹³The reason we wrote $X_0 \cong V$ rather than $X_0 = V$ is that X_0 is the set of 1-element subsets of V . So if $V = \{a, b, c\}$ then $X_0 = \{\{a\}, \{b\}, \{c\}\}$. This is really just pedantry.



In this case X_1 consists of elements like $\{A, B\}$ and $\{D, K\}$ but not $\{D, J\}$.

Write out X_2 and X_3 (hint: the drawing of X indicates that X_3 should have one element. $\diamond$)

Exercise 2.7.4.7. The 2-simplex Δ^2 is drawn as a filled-in triangle with vertices $V = \{1, 2, 3\}$. There is a simplicial complex $X = \partial\Delta^2$ that would be drawn as an empty triangle with the same set of vertices. What is X ? In other words what are $X_0, X_1, X_2, X_3, \dots$? $\diamond$

2.7.4.8 Subobject classifier

Definition 2.7.4.9. Define the *subobject classifier* for **Set**, denoted Ω to be the set $\Omega := \{True, False\}$, together with the function $\{\odot\} \rightarrow \Omega$ sending the unique element to *True*.

Proposition 2.7.4.10. Let B be a set. There is an isomorphism

$$\phi: \text{Hom}_{\mathbf{Set}}(B, \Omega) \xrightarrow{\cong} \mathbb{P}(B).$$

Proof. Given a function $f: B \rightarrow \Omega$, let $\phi(f) = \{b \in B \mid f(b) = True\} \subseteq B$. We now construct a function $\psi: \mathbb{P}(B) \rightarrow \text{Hom}_{\mathbf{Set}}(B, \Omega)$ to serve as the inverse of ϕ . Given a subset $i: B' \subseteq B$, define $\psi(i): B \rightarrow \Omega$ as follows:

$$\psi(i)(b) = \begin{cases} True & \text{if } b \in B', \\ False & \text{if } b \notin B'. \end{cases}$$

One checks easily that ϕ and ψ are mutually inverse. $\square$

Definition 2.7.4.11 (Characteristic function). Given a subset $B' \subseteq B$, we call the corresponding function $B \rightarrow \Omega$ the *characteristic function of B' in B* .

Let B be any set and let $\mathbb{P}(B)$ be its power set. By Proposition 2.7.4.10 there is a bijection between $\mathbb{P}(B)$ and Ω^B . Since Ω has cardinality 2, the cardinality of $\mathbb{P}(B)$ is $2^{|B|}$, which explains the correct answer to Exercise 2.7.4.2.

Exercise 2.7.4.12. Let $f: A \rightarrow \Omega$ denote the characteristic function of some $A' \subseteq A$, and define $A'' \subseteq A$ to be its complement, $A'' := A - A'$ (i.e. $a \in A''$ if and only if $a \notin A'$).

- a.) What is the characteristic function of $A'' \subseteq A$?
- b.) Can you phrase it in terms of some function $\Omega \rightarrow \Omega$?

◇

2.7.5 Surjections, injections

The classical definition of injections and surjections involves elements, which we give now. But a more robust notion involves all maps and will be given in Proposition 2.7.5.4.

Definition 2.7.5.1. Let $f: X \rightarrow Y$ be a function. We say that f is *surjective* if, for all $y \in Y$ there exists some $x \in X$ such that $f(x) = y$. We say that f is *injective* if, for all $x \in X$ and all $x' \in X$ with $f(x) = f(x')$ we have $x = x'$.

A function that is both injective and surjective is called *bijective*.

Remark 2.7.5.2. It turns out that a function that is bijective is always an isomorphism and that all isomorphisms are bijective. We will not show that here, but it is not too hard; see for example [Big, Theorem 5.4].

Definition 2.7.5.3 (Monomorphisms, epimorphisms). Let $f: X \rightarrow Y$ be a function.

We say that f is a *monomorphism* if for all sets A and pairs of functions $h, h': A \rightarrow X$,

$$\begin{array}{ccc} A & \xrightarrow{h} & X \\ & \searrow h' & \nearrow f \\ & X & \longrightarrow Y \end{array}$$

if $f \circ h = f \circ h'$ then $h = h'$.

We say that f is an *epimorphism* if for all sets B and pairs of functions $g, g': Y \rightarrow B$,

$$\begin{array}{ccc} X & \xrightarrow{f} & Y \\ & \searrow g & \nearrow g' \\ & Y & \longrightarrow B \end{array}$$

if $g \circ f = g' \circ f$ then $g = g'$.

Proposition 2.7.5.4. Let $f: X \rightarrow Y$ be a function. Then f is injective if and only if it is a monomorphism; f is surjective if and only if it is an epimorphism.

Proof. If f is a monomorphism it is clearly injective by putting $A = \{\odot\}$. Suppose that f is injective and let $h, h': A \rightarrow X$ be functions such that $f \circ h = f \circ h'$, but suppose for contradiction that $h \neq h'$. Then there is some element $a \in A$ such that $h(a) \neq h(a') \in X$. But by injectivity $f(h(a)) \neq f(h(a'))$, contradicting $f \circ h = f \circ h'$.

Suppose that $f: X \rightarrow Y$ is an epimorphism and choose some $y_0 \in Y$ (noting that if Y is empty then the claim is vacuously true). Let $g: Y \rightarrow \Omega$ denote the characteristic

function of the subset $\{y_0\} \subseteq Y$ and let $g': Y \rightarrow \Omega$ denote the characteristic function of $Y \subseteq Y$; note that $g(y) = g'(y)$ for all $y \neq y_0$. Then since $g \neq g'$ we have $g \circ f \neq g' \circ f$, so there exists $x \in X$ with $g(f(x)) \neq g'(f(x))$, which implies that $f(x) = y_0$. This proves that f is surjective.

Finally, suppose that f is surjective, and let $g, g': Y \rightarrow B$ be functions with $g \circ f = g' \circ f$. For any $y \in Y$, there exists some $x \in X$ with $f(x) = y$, so $g(y) = g(f(x)) = g'(f(x)) = g'(y)$. This proves that f is an epimorphism. $\square$

Proposition 2.7.5.5. *Let $f: X \rightarrow Y$ be a monomorphism. Then for any function $g: A \rightarrow Y$, the top map $f': X \times_Y A \rightarrow A$ in the diagram*

$$\begin{array}{ccc} X \times_Y A & \xrightarrow{f'} & A \\ g' \downarrow & \lrcorner & \downarrow g \\ X & \xrightarrow{f} & Y \end{array}$$

is a monomorphism.

Proof. To show that f' is a monomorphism, we take an arbitrary set B and two maps $m, n: B \rightarrow X \times_Y A$ such that $f' \circ m = f' \circ n$, denote that function by $p := f' \circ m: B \rightarrow A$. Now let $q = g' \circ m$ and $r = g' \circ n$. The diagram looks like this:

$$\begin{array}{ccccc} & & p & & \\ & \curvearrowright & & \curvearrowleft & \\ B & \xrightarrow{m} & X \times_Y A & \xrightarrow{f'} & A \\ & \searrow n & \downarrow g' & \lrcorner & \downarrow g \\ & & X & \xrightarrow{f} & Y \\ & \nearrow q & & & \\ & & r & & \end{array}$$

We have that

$$f \circ q = f \circ g' \circ m = g \circ f' \circ m = g \circ f' \circ n = f \circ g' \circ n = f \circ r$$

But we assumed that f is a monomorphism so this implies that $q = r$. By the universal property of pullbacks, Lemma 2.5.1.14, we have $m = n$. $\square$

Exercise 2.7.5.6. Show, in analogy to Proposition 2.7.5.5, that pushouts preserve epimorphisms. $\diamond$

Example 2.7.5.7. Suppose an olog has a fiber product square

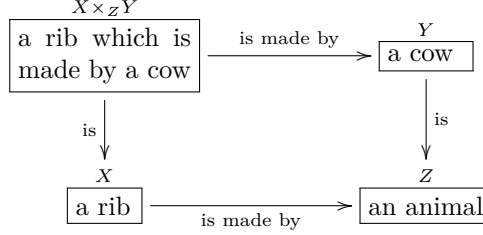
$$\begin{array}{ccc} X \times_Z Y & \xrightarrow{g'} & Y \\ f' \downarrow & & \downarrow f \\ X & \xrightarrow{g} & Z \end{array}$$

such that f is intended to be an injection and g is any map.¹⁴ In this case, there are nice labeling systems for f', g' , and $X \times_Z Y$. Namely:

¹⁴Of course, this diagram is symmetrical, so the same ideas hold if g is an injection and f is any map.

- “is” is an appropriate label for f' ,
- the label for g is an appropriate label for g' ,
- (the label for X , then “which”, then the label for g , then the label for Y) is an appropriate label for $X \times_Z Y$.

To give an explicit example,



Corollary 2.7.5.8. *Let $i: A \rightarrow X$ be a monomorphism. Then there is a fiber product square of the form*

$$\begin{array}{ccc}
 A & \xrightarrow{f'} & \{\odot\} \\
 i \downarrow & \lrcorner & \downarrow \text{True} \\
 X & \xrightarrow{f} & \Omega.
 \end{array} \tag{2.44}$$

Proof. Let $X' \subseteq X$ denote the image of i and let $f: X \rightarrow \Omega$ denote the characteristic function of $X' \subseteq X$. Then it is easy to check that Diagram 2.44 is a pullback. $\square$

Exercise 2.7.5.9. Consider the subobject classifier Ω , the singleton $\{\odot\}$ and the map $\{\odot\} \xrightarrow{\text{True}} \Omega$ from Definition 2.7.4.9. Look at diagram 2.44 and in the spirit of Exercise 2.7.5.7, come up with a label for Ω , a label for $\{\odot\}$, and a label for True . Given a label for X and a label for f , come up with a label for i and a label for f' , such that the English smoothly fits the mathematics. $\diamond$

2.7.6 Multisets, relative sets, and set-indexed sets

In this section we prepare ourselves for considering categories other than **Set**, by looking at some categories related to **Set**.

2.7.6.1 Multisets

Consider the set X of words in a given document. If $WC(X)$ is the wordcount of the document, we will not generally have $WC(X) = |X|$. The reason is that a set cannot contain the same element more than once, so words like “the” might be undercounted in $|X|$. A *multiset* is a set in which elements can be assigned a multiplicity, i.e. a number of times they are to be counted.

But if X and Y are multisets, what is the appropriate type of mapping from X to Y ? Since every set is a multiset (in which each element has multiplicity 1), let’s restrict ourselves to notions of mapping that agree with the usual one on sets. That

is, if multisets X and Y happen to be sets then our mappings $X \rightarrow Y$ should just be functions.

Exercise 2.7.6.2. a.) Come up with some notion of mapping for multisets that generalizes functions when the notion is restricted to sets. Suppose that $X = (1, 1, 2, 3)$ and $Y = (a, b, b, b)$, i.e. $X = \{1, 2, 3\}$ with 1 having multiplicity 2, and $Y = \{a, b\}$ with b having multiplicity 3.

b.) What are all the maps $X \rightarrow Y$ in your notion?

◇

In Chapter 4 we will be getting to the definition of category, and you can test whether your notion of mapping in fact defines a category. Here is my definition of mapping for multisets.

Definition 2.7.6.3. A *multiset* is a sequence $X := (E, B, \pi)$ where E and B are sets and $\pi: E \rightarrow B$ is a surjective function. We refer to E as the set of *element instances* of X , we refer to B as the set of *element names* of X , and we refer to π as the *naming function* for X . Given an element name $x \in B$, let $\pi^{-1}(x) \subseteq E$ be the preimage; the number of elements in $\pi^{-1}(x)$ is called the *multiplicity* of x .

Suppose that $X = (E, B, \pi)$ and $X' = (E', B', \pi')$ are multisets. A *mapping* from X to Y , denoted $f: X \rightarrow Y$, consists of a pair (f_1, f_0) such that $f_1: E \rightarrow E'$ and $f_0: B \rightarrow B'$ are functions and such that the following diagram commutes:

$$\begin{array}{ccc} E & \xrightarrow{f_1} & E' \\ \pi \downarrow & & \downarrow \pi' \\ B & \xrightarrow{f_0} & B' \end{array} \quad (2.45)$$

Exercise 2.7.6.4. Suppose that a pseudo-multiset is defined to be almost the same as a multiset, except that π is not required to be surjective. Write down a pseudo-multiset that is not a multi-set.

a.) Describe the difference between the two notions in terms of multiplicities.

b.) Complexity of names aside, which do you think is a more useful notion: multiset or pseudo-multisets?

◇

Exercise 2.7.6.5. Consider the multisets described in Exercise 2.7.6.2.

a.) Write each of them in the form (E, B, π) , as in Definition 2.7.6.3.

b.) In terms of the same definition, what are the mappings $X \rightarrow Y$?

c.) If we remove the restriction that diagram 2.45 must commute, how many mappings $X \rightarrow Y$ are there?

◇

2.7.6.6 Relative sets

Let's continue with our ideas from multisets, but now suppose that we have a fixed set B of names that we want to keep once and for all. Whenever someone discusses a set, each

element must have a name in B . And whenever someone discusses a mapping, it must preserve the names. For example, if B is the set of English words, then every document consists of an ordered set mapping to B (e.g. $1 \mapsto \text{Suppose}, 2 \mapsto \text{that}, 3 \mapsto \text{we}$, etc.) A mapping from document A to document B would send each word found somewhere in A to the same word found somewhere in B . This notion is defined carefully below.

Definition 2.7.6.7 (Relative set). Let B be a set. A *relative set over B* , or simply a *set over B* , is a pair (E, π) such that E is a set and $\pi: E \rightarrow B$ is a function. A *mapping of relative sets over B* , denoted $f: (E, \pi) \rightarrow (E', \pi')$, is a function $f: E \rightarrow E'$ such that the triangle below commutes, i.e. $\pi = \pi' \circ f$,

$$\begin{array}{ccc} E & \xrightarrow{f} & E' \\ & \searrow \pi & \swarrow \pi' \\ & B & \end{array}$$

Exercise 2.7.6.8. Given sets X, Y, Z and functions $f: X \rightarrow Y$ and $g: Y \rightarrow Z$, we can compose them to get a function $X \rightarrow Z$. If B is a set, if $(X, p), (Y, q)$, and (Z, r) are relative sets over B , and if $f: (X, p) \rightarrow (Y, q)$ and $g: (Y, q) \rightarrow (Z, r)$ are mappings, is there a reasonable notion of composition such that we get a mapping of relative sets $(X, p) \rightarrow (Z, r)$? Hint: draw diagrams. $\diamond$

Exercise 2.7.6.9. a.) Let $\{\odot\}$ denote a set with one element. What is the difference between sets over $\{\odot\}$ and simply sets?

b.) Describe the sets relative to $\emptyset$. How many are there?

$\diamond$

2.7.6.10 Indexed sets

Let A be a set. Suppose we want to assign to each element $a \in A$ a set S_a . This is called an A -indexed set. In category theory we are always interested in the legal mappings between two different structures of the same sort, so we need a notion of A -indexed mappings; we do the “obvious thing”.

Example 2.7.6.11. Let C be a set of classrooms. For each $c \in C$ let P_c denote the set of people in classroom c , and let S_c denote the set of seats (chairs) in classroom c . Then P and S are C -indexed sets. The appropriate kind of mapping between them respects the indexes. That is, a mapping of multi-sets $P \rightarrow S$ should, for each classroom $c \in C$, be a function $P_c \rightarrow S_c$.¹⁵

Definition 2.7.6.12. Let A be a set. An A -indexed set is a collection of sets S_a , one for each element $a \in A$; for now we denote this by $(S_a)_{a \in A}$. If $(S'_a)_{a \in A}$ is another A -indexed set, a *mapping of A -indexed sets from $(S_a)_{a \in A}$ to $(S'_a)_{a \in A}$* , denoted

$$(f_a)_{a \in A}: (S_a)_{a \in A} \rightarrow (S'_a)_{a \in A}$$

is a collection of functions $f_a: S_a \rightarrow S'_a$, one for each element $a \in A$.

Exercise 2.7.6.13. Let $\{\odot\}$ denote a one element set. What are $\{\odot\}$ -indexed sets and mappings between them? $\diamond$

Exercise 2.7.6.14. There is a strong relationship between A -indexed sets and relative sets over A . What is it? $\diamond$

¹⁵If we wanted to allow people from any classroom to choose a chair from just any classroom, category theory would tell us to reconsider P and S as sets, forgetting their indices. See Section 5.1.4.6.

Chapter 3

Categories and functors, without admitting it

In this chapter we begin to use our understanding of sets to build more interesting mathematical devices, each of which organizes our understanding of a certain kind of domain. For example, monoids organize our thoughts about agents acting on objects; groups are monoids except restricted to only allow agents to act reversibly. We will then study graphs, which are systems of nodes and arrows that can capture ideas like information flow through a network or model connections between building blocks in a material. We will discuss orders, which can be used to study taxonomies or hierarchies. Finally we take a mathematical look at databases, which actually subsume everything else in the chapter. Databases are connection patterns for structuring information.

We will see in Chapter 4 that everything we study in the present chapter is an example of a category. So is **Set**, the category of sets studied in Chapter 2. One way to think of a category is as a set of objects and a connection pattern between them; sets are objects (ovals full of dots if you wish) connected by functions. But each set is itself a category: the objects inside it are just disconnected! Just like a set has an interior view and an exterior view, so will all the categories in this chapter. Each monoid *is* a category, but there is also a category *of* monoids.

However, we will not really say the word “category” much if at all in this chapter. It seems preferable to let the ideas rise on their own accord as interesting structures in their own right before explaining that everything in site fits into a single framework. That will be the pleasant reward to come in Chapter 4.

3.1 Monoids

A common way to interpret phenomena we see around us is to say that agents are acting on objects. For example, in a computer drawing program, the user *acts on* the canvas in certain prescribed ways. Choices of actions from an available list can be performed in sequence to transform one image into another. As another example, one might investigate the notion that time *acts on* the position of hands on a clock in a prescribed way. A first rule for actions is this: the performance of a sequence of several actions is itself the performance of an action—a more complex action, but an action nonetheless.

Mathematical objects called *monoids* and *groups* are tasked with encoding the agent’s

perspective in all this, i.e. what the agent can do, and what happens when different actions are done in succession. A monoid can be construed as a set of actions, together with a formula that encodes how a sequence of actions is itself considered an action. A group is the same as a monoid, except that every action is required to be reversible. In this section we concentrate on monoids; we will get to groups in Section 3.2.

3.1.1 Definition and examples

Definition 3.1.1.1 (Monoid). A *monoid* is a sequence $(M, \text{id}, \star)$, where M is a set, $\text{id} \in M$ is an element, and $\star: M \times M \rightarrow M$ is a function, such that the following conditions hold for all $m, n, p \in M$:

- $m \star \text{id} = m$,
- $\text{id} \star m = m$, and
- $(m \star n) \star p = m \star (n \star p)$.

We refer to id as the *identity element* and to $\star$ as the *multiplication formula* for the monoid.¹ We call the first two rules *identity laws* and the third rule the *associativity law* for monoids.

Remark 3.1.1.2. To be pedantic, the conditions from Definition 3.1.1.1 should be stated

- $\star(m, \text{id}) = m$,
- $\star(\text{id}, m) = m$, and
- $\star(\star(m, n), p) = \star(m, \star(n, p))$.

The way they are written in Definition 3.1.1.1 is called *infix notation*, and we often use infix notation without mentioning it. That is, given a function $\cdot: A \times B \rightarrow C$, we may write $a \cdot b$ rather than $\cdot(a, b)$.

Example 3.1.1.3 (Additive monoid of natural numbers). Let $M = \mathbb{N}$ be the set of natural numbers. Let $\text{id} = 0$ and let $\star: M \times M \rightarrow M$ denote addition, so that $\star(4, 18) = 22$. Then the equations $m \star 0 = m$ and $0 \star m = m$ hold, and $(m \star n) \star p = m \star (n \star p)$. By assigning id and $\star$ in this way, we have “given $\mathbb{N}$ the structure of a monoid”.

Remark 3.1.1.4. Sometimes we are working with a monoid $(M, \text{id}, \star)$, and the identity id and multiplication $\star$ are somehow clear from context. In this case we might refer to the set M as though it were the whole monoid. For example, if we were discussing the monoid from Example 3.1.1.3, we might refer to it as $\mathbb{N}$. The danger comes because sets may have multiple monoid structures, as we see below in Exercise 3.1.1.5.

Exercise 3.1.1.5. Let $M = \mathbb{N}$ be the set of natural numbers. Taking $\text{id} = 1$, come up with a formula for $\star$ that gives $\mathbb{N}$ the structure of a monoid. $\diamond$

Exercise 3.1.1.6. In both Example 3.1.1.3 and Exercise 3.1.1.5, the monoids $(M, \text{id}, \star)$ satisfied an additional rule called *commutativity*, namely $m \star n = n \star m$ for every $m, n \in M$. There is a monoid $(M, \text{id}, \star)$ lurking in linear algebra textbooks that is not commutative; if you have background in linear algebra try to answer this: what M , id , and $\star$ might I be referring to? $\diamond$

¹Although the function $\star: M \times M \rightarrow M$ is called the multiplication formula, it may have nothing to do with multiplication. It is nothing more than a formula for taking two inputs and returning an output; calling it “multiplication” is suggestive of its origins, rather than prescriptive of its behavior.

Exercise 3.1.1.7. Recall the notion of commutativity for monoids from Exercise 3.1.1.6.

- a.) What is the smallest set M that you can give the structure of a non-commutative monoid?
- b.) What is the smallest set M that you can give the structure of a monoid?

◇

Example 3.1.1.8 (Trivial monoid). There is a monoid with only one element, $M = (\{\text{id}\}, \text{id}, \star)$ where $\star: \{\text{id}\} \times \{\text{id}\} \rightarrow \{\text{id}\}$ is the unique function. We call this monoid *the trivial monoid*, and sometimes denote it $\underline{1}$.

3.1.1.9 Free monoids and finitely presented monoids

Definition 3.1.1.10. Let X be a set. A *list in X* is a pair (n, f) where $n \in \mathbb{N}$ is a natural number (called the *length of the list*) and $f: \underline{n} \rightarrow X$ is a function, where $\underline{n} = \{1, 2, \dots, n\}$. We may denote such a list by

$$(n, f) = [f(1), f(2), \dots, f(n)].$$

The *empty list* is the unique list in which $n = 0$; we may denote it by $[\]$. Given an element $x \in X$ the *singleton list on x* is the list $[x]$. Given a list $L = (n, f)$ and a number $i \in \mathbb{N}$ with $i \leq n$, the *i th entry of L* is the element $f(i) \in X$.

Given two lists $L = (n, f)$ and $L' = (n', f')$, define the *concatenation of L and L'* , denoted $L ++ L'$, to be the list $(n + n', f ++ f')$, where $f ++ f': \underline{n + n'} \rightarrow X$ is given on $i \leq n + n'$ by

$$(f ++ f')(i) := \begin{cases} f(i) & \text{if } i \leq n \\ f'(i - n) & \text{if } i \geq n + 1 \end{cases}$$

Example 3.1.1.11. Let $X = \{a, b, c, \dots, z\}$. The following are elements of $\text{List}(X)$:

$$[a, b, c], [p], [p, a, a, a, p], [\], \dots$$

The concatenation of $[a, b, c]$ and $[p, a, a, a, p]$ is $[a, b, c, p, a, a, a, p]$. The concatenation of any list A with $[\]$ is just A .

Definition 3.1.1.12. Let X be a set. The *free monoid generated by X* is the sequence $M := (\text{List}(X), [\], ++)$, where $\text{List}(X)$ is the set of ordered lists of elements in X , where $[\] \in \text{List}(X)$ is the empty list, and where $++$ is the operation of list concatenation. We refer to X as the set of generators for the monoid M .

Exercise 3.1.1.13. Let $\{\odot\}$ denote a one-element set.

- a.) What is the free monoid generated by $\{\odot\}$?
- b.) What is the free monoid generated by $\emptyset$?

◇

In the definition below, we will define a monoid M by specifying some generators and some relations. Lists of generators provide us all the possible ways to write elements of M . The relations allow us to have two such ways of writing the same element. The following definition is a bit dense, so see Example 3.1.1.16 for a concrete example.

Definition 3.1.1.14 (Presented monoid). Let G be a finite set, let $n \in \mathbb{N}$ be a natural number,² and for each $1 \leq i \leq n$, let m_i and m'_i be elements of $\text{List}(G)$.³ The monoid presented by generators G and relations $\{(m_i, m'_i) \mid 1 \leq i \leq n\}$ is the monoid $(M, \text{id}, \star)$ defined as follows. Let $\sim$ denote the equivalence relation on $\text{List}(G)$ generated by $\{(xm_iy \sim xm'_iy) \mid x, y \in \text{List}(G), 1 \leq i \leq n\}$, and define $M = \text{List}(G)/\sim$. Let $\text{id} = []$ and let $a \star b$ be obtained by concatenating representing lists.

Remark 3.1.1.15. Every free monoid is a presented monoid, because we can just take the set of relations to be empty.

Example 3.1.1.16. Let $G = \{a, b, c, d\}$. Think of these as buttons that can be pressed. The free monoid $\text{List}(G)$ is the set of all ways of pressing buttons, e.g. pressing a then a then c then d corresponds to the list $[a, a, c, d]$. The idea of presented monoids is that you notice that pressing $[a, a, c]$ always gives the same result as pressing $[d, d]$. You also notice that pressing $[c, a, c, a]$ is the same thing as doing nothing.

In this case, we would have $m_1 = [a, a, c]$, $m'_1 = [d, d]$, and $m_2 = [c, a, c, a]$, $m'_2 = []$ and relations $\{(m_1, m'_1), (m_2, m'_2)\}$. Really this means that we're equating m_1 with m'_1 and m_2 with m'_2 , which for convenience we'll write out:

$$[a, a, c] = [d, d] \quad \text{and} \quad [a, c, a, c] = []$$

To see how this plays out, we give an example of a calculation in $M = \text{List}(G)/\sim$. Namely,

$$\begin{aligned} [b, c, b, d, a, c, a, a, c, d] &= [b, c, b, a, a, c, a, c, a, c, d] = [b, c, b, a, a, c, d] \\ &= [b, c, b, a, d, d, d]. \end{aligned}$$

Application 3.1.1.17 (Buffer). Let $G = \{a, b, c, \dots, z\}$. Suppose we have a buffer of 32 characters and we want to consider the set of lists of length at most 32 to be a monoid. We simply have to decide what happens when someone types a list of length more than 32.

One option is to say that the last character typed overwrites the 32nd entry,

$$[a_1, a_2, \dots, a_{31}, a_{32}, b] \sim_1 [a_1, a_2, \dots, a_{31}, b].$$

Another option is to say that any character typed after 32 entries is discarded,

$$[a_1, a_2, \dots, a_{31}, a_{32}, b] \sim_2 [a_1, a_2, \dots, a_{31}, a_{32}].$$

Both of these yield finitely presented monoids, generated by G . (In case it's useful, the number of necessary relations in both cases is 26^{33} .)

◇◇

Exercise 3.1.1.18. Let's consider the buffer concept again (see Application 3.1.1.17), but this time only having size 3 rather than size 32. Show using Definition 3.1.1.14 that with relations given by $\sim_1$ we indeed have $[a, b, c, d, e, f] = [a, b, f]$ and that with relations given by $\sim_2$ we indeed have $[a, b, c, d, e, f] = [a, b, c]$. ◇

Exercise 3.1.1.19. Let $K := \{BS, a, b, c, \dots, z\}$, a set having 27 elements. Suppose you want to think of $BS \in K$ as the “backspace key” and the elements $a, b, \dots, z \in K$ as the letter keys on a keyboard. Then the free monoid $\text{List}(K)$ is not quite appropriate as a model because we want $[a, b, d, BS] = [a, b]$.

²The number $n \in \mathbb{N}$ is going to stand for the number of relations we declare.

³Each m_i and m'_i are going to be made equal in $\mathcal{M}$.

- a.) Choose a set of relations for which the monoid presented by generators K and the chosen relations is appropriate to this application.
- b.) Under your relations, how does $[BS]$ compare with $[\]$? Is that suitable?

◇

3.1.1.20 Cyclic monoids

Definition 3.1.1.21. A monoid is called *cyclic* if it has a presentation involving only one generator.

Example 3.1.1.22. Let Q be a symbol; we look at some cyclic monoids generated by $\{Q\}$. With no relations the monoid would be the free monoid on one generator, and would have underlying set $\{[\], [Q], [Q, Q], [Q, Q, Q], \dots\}$, with identity element $[\]$ and multiplication given by concatenation (e.g. $[Q, Q, Q] ++ [Q, Q] = [Q, Q, Q, Q, Q]$). This is just $\mathbb{N}$, the additive monoid of natural numbers.

With the really strong relation $[Q] \sim [\]$ we would get the trivial monoid, a monoid having only one element (see Example 3.1.1.8).

Another possibility is given in the first part of Example 3.1.2.3, where the relation $Q^{12} \sim [\]$ is used, where Q^{12} is shorthand for $[Q, Q, Q, Q, Q, Q, Q, Q, Q, Q, Q, Q]$.

Exercise 3.1.1.23 (Classify the cyclic monoids). Classify all the cyclic monoids up to isomorphism. That is, come up with a naming system such that every cyclic monoid can be given a name in your system, such that no two non-isomorphic cyclic monoids have the same name, and such that no name exists in the system unless it refers to a cyclic monoid.

Hint: one might see a pattern in which the three monoids in Example 3.1.1.22 correspond respectively to ∞ , 1, and 12, and then think “Cyclic monoids can be classified by (i.e. systematically named by elements of) the set $\mathbb{N} \sqcup \{\infty\}$.” That idea is on the right track, but is not correct. ◇

3.1.2 Monoid actions

Definition 3.1.2.1 (Monoid action). Let $(M, \text{id}, \star)$ be a monoid and let S be a set. An *action of $(M, \text{id}, \star)$ on S* , or simply an *action of M on S* or an *M -action on S* , is a function

$$\rhd : M \times S \rightarrow S$$

such that the following conditions hold for all $m, n \in M$ and all $s \in S$:

- $\text{id} \rhd s = s$
- $m \rhd (n \rhd s) = (m \star n) \rhd s$.⁴

Remark 3.1.2.2. To be pedantic (and because it’s sometimes useful), we may rewrite $\rhd$ as $\alpha: M \times S \rightarrow S$ and restate the conditions from Definition 3.1.2.1 as

⁴ Definition 3.1.2.1 actually defines a *left action* of $(M, \text{id}, \star)$ on S . A *right action* is like a left action except the order of operations is somehow reversed. We will not really use right-actions in this text, but we briefly define it here for completeness. With notation as above, the only difference is in the second condition. We replace it by the condition that for all $m, n \in M$ and all $s \in S$ we have

$$m \rhd (n \rhd s) = (n \star m) \rhd s$$

- $\alpha(\text{id}, s) = s$, and
- $\alpha(m, \alpha(n, s)) = \alpha(m \star n, s)$.

Example 3.1.2.3. Let $S = \{0, 1, 2, \dots, 11\}$ and let $N = (\mathbb{N}, 0, +)$ be the additive monoid of natural numbers (see Example 3.1.1.3). We define a function $\curvearrowright: \mathbb{N} \times S \rightarrow S$ by taking a pair (n, s) to the remainder that appears when $n + s$ is divided by 12. For example $4 \curvearrowright 2 = 6$ and $8 \curvearrowright 9 = 5$. This function has the structure of a monoid action because the two rules from Definition 3.1.2.1 hold.

Similarly, let T denote the set of points on a circle, elements of which are denoted by a real number in the interval $[0, 12)$, i.e.

$$T = \{x \in \mathbb{R} \mid 0 \leq x < 12\}$$

and let $R = (\mathbb{R}, 0, +)$ denote the additive monoid of real numbers. Then there is an action $R \times T \rightarrow T$, similar to the one above (see Exercise 3.1.2.4).

One can think of this as an action of the monoid of time on the clock.

Exercise 3.1.2.4. a.) Realize the set $T := [0, 12) \subseteq \mathbb{R}$ as the coequalizer of a pair of arrows $\mathbb{R} \rightrightarrows \mathbb{R}$.

b.) For any $x \in \mathbb{R}$, realize the mapping $x \cdot - : T \rightarrow T$, implied by Example 3.1.2.3, using the universal property of coequalizers.

c.) Prove that it is an action.

◇

Exercise 3.1.2.5. Let B denote the set of buttons (or positions) of a video game controller (other than, say ‘start’ and ‘select’), and consider the free monoid $\text{List}(B)$ on B .

a.) What would it mean for $\text{List}(B)$ to act on the set of states of some game? Imagine a video game G' that uses the controller, but for which $\text{List}(B)$ would not be said to act on the states of G' . Now imagine a simple game G for which $\text{List}(B)$ would be said to act.

b.) Can you think of a state s of G , and two distinct elements $\ell, \ell' \in \text{List}(B)$ such that $\ell \curvearrowright s = \ell' \curvearrowright s$? In video game parlance, what would you call an element $b \in B$ such that, for every state $s \in G$, one has $b \curvearrowright s = s$?

c.) In video game parlance, what would you call a state $s \in S$ such that, for every sequence of buttons $\ell \in \text{List}(B)$, one has $\ell \curvearrowright s = s$?

◇

Application 3.1.2.6. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ be a differentiable function of which we want to find roots (points $x \in \mathbb{R}$ such that $f(x) = 0$). Let $x_0 \in \mathbb{R}$ be a starting point. For any $n \in \mathbb{N}$ we can apply **Newton’s method** to x_n to get

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

This is a monoid (namely $\mathbb{N}$, the free monoid on one generator) acting on a set (namely $\mathbb{R}$).

However, Newton’s method can get into trouble. For example at a critical point it causes division by 0, and sometimes it can oscillate or overshoot. In these cases we want to perturb a bit to the left or right. To have these actions available to us, we would add

“perturb” elements to our monoid. Now we have more available actions at any point, but the system is less deterministic.

When publishing an experimental finding, there may be some deep methodological questions that are not considered suitably important to mention. For example, one may not publish the kind solution finding method (e.g. Newton’s method or Runge-Kutta) that was used, nor the set of available actions in terms of perturbation. However, these may actually influence the reproducibility of results. By using the language of monoid actions, we can align our data model with our unspoken assumptions about how functions are analyzed.

◇◇

Remark 3.1.2.7. A monoid is useful for understanding how an agent acts on the set of states of an object, but there is only one *kind* of action. At any point, all actions are available. In reality it is often the case that contexts can change and different actions are available at different times. For example on a computer, the commands available in one application have no meaning in another. This will get us to categories in the next chapter.

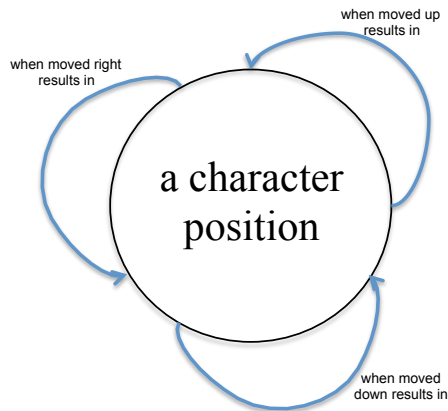
3.1.2.8 Monoids actions as ologs

If monoids are understood in terms of how they act on sets, then it is reasonable to think of them in terms of ologs. In fact, the ologs associated to monoids are precisely those ologs that have exactly one type (and possibly many arrows and commutative diagrams).

Example 3.1.2.9. In this example we show how to associate an olog to a monoid action. Consider the monoid M generated by the set $\{u, d, r\}$, standing for “up, down, right”, and subject to the relations

$$[u, d] \sim [], \quad [d, u] \sim [], \quad [u, r] = [r, u], \quad \text{and} \quad [d, r] = [r, d].$$

We might imagine that M acts on the set of positions for a character in an old video game. In that case the olog corresponding to this action should look something like the following:



Given x , a character position, consider the following. We know that x is a character position, which when moved up results in a character position, which when moved down results in a character position that we'll call $P(x)$. We also know that x is a character position that we'll call $Q(x)$. Fact: whenever x is a character position we will have $P(x)=Q(x)$. **Summary:** $[\text{up}, \text{down}] = []$

Given x , a character position, consider the following. We know that x is a character position, which when moved down results in a character position, which when moved up results in a character position that we'll call $P(x)$. We also know that x is a character position that we'll call $Q(x)$. Fact: whenever x is a character position we will have $P(x)=Q(x)$. **Summary:** $[\text{down}, \text{up}] = []$

Given x , a character position, consider the following. We know that x is a character position, which when moved up results in a character position, which when moved right results in a character position that we'll call $P(x)$. We also know that x is a character position, which when moved right results in a character position, which when moved up results in a character position that we'll call $Q(x)$. Fact: whenever x is a character position we will have $P(x)=Q(x)$. **Summary:** $[\text{up}, \text{right}] = [\text{right}, \text{up}]$

Given x , a character position, consider the following. We know that x is a character position, which when moved down results in a character position, which when moved right results in a character position that we'll call $P(x)$. We also know that x is a character position, which when moved right results in a character position, which when moved down results in a character position that we'll call $Q(x)$. Fact: whenever x is a character position we will have $P(x)=Q(x)$. **Summary:** $[\text{down}, \text{right}] = [\text{right}, \text{down}]$

3.1.2.10 Finite state machines

According to Wikipedia, a *deterministic finite state machine* is a quintuple $(\Sigma, S, s_0, \delta, F)$, where

1. Σ is a finite non-empty set of symbols, called the *input alphabet*,
2. S is a finite, non-empty set, called *the state set*,
3. $\delta: \Sigma \times S \rightarrow S$ is a function, called the *state-transition function*, and
4. $s_0 \in S$ is an element, called *the initial state*,
5. $F \subseteq S$ is a subset, called the *set of final states*.

In this book we will not worry about the initial state and the set of final states, concerning ourselves more with the interaction via δ of the alphabet Σ on the set S of states.

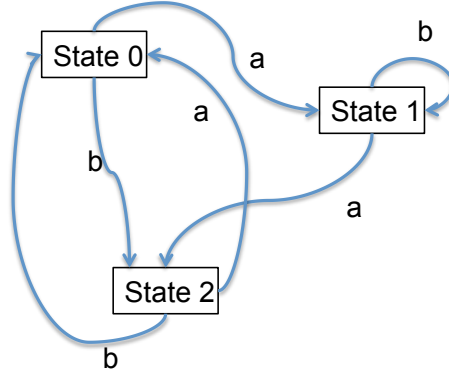


Figure 3.1: A finite state machine with alphabet $\Sigma = \{a, b\}$ and state set $S = \{\text{State 0}, \text{State 1}, \text{State 2}\}$. If pressed, we will make State 0 the initial state and $\{\text{State 2}\}$ the set of final states.

The following proposition expresses the notion of finite state automata in terms of free monoids and their actions on finite sets.

Proposition 3.1.2.11. *Let Σ, S be finite non-empty sets. Giving a function $\delta: \Sigma \times S \rightarrow S$ is equivalent to giving an action of the free monoid $\text{List}(\Sigma)$ on S .*

Proof. By Definition 3.1.2.1, we know that function $\epsilon: \text{List}(\Sigma) \times S \rightarrow S$ constitutes an action of the monoid $\text{List}(\Sigma)$ on the set S if and only if, for all $s \in S$ we have $\epsilon([], s) = s$, and for any two elements $m, m' \in \text{List}(\Sigma)$ we have $\epsilon(m, \epsilon(m', s)) = \epsilon(m \star m', s)$, where $m \star m'$ is the concatenation of lists. Let

$$A = \{\epsilon: \text{List}(\Sigma) \times S \rightarrow S \mid \epsilon \text{ constitutes an action}\}.$$

We need to prove that there is an isomorphism of sets

$$\phi: A \xrightarrow{\cong} \text{Hom}_{\text{Set}}(\Sigma \times S, S).$$

Given an element $\epsilon: \text{List}(\Sigma) \times S \rightarrow S$ in A , define $\phi(\epsilon)$ on an element $(\sigma, s) \in \Sigma \times S$ by $\phi(\epsilon)(\sigma, s) := \epsilon([\sigma], s)$, where $[\sigma]$ is the one-element list. We now define $\psi: \text{Hom}_{\mathbf{Set}}(\Sigma \times S, S) \rightarrow A$.

Given an element $f \in \text{Hom}_{\mathbf{Set}}(\Sigma \times S, S)$, define $\psi(f): \text{List}(\Sigma) \times S \rightarrow S$ on a pair $(L, s) \in \text{List}(\Sigma) \times S$, where $L = [\epsilon_1, \dots, \epsilon_n]$ as follows. By induction, if $n = 0$, put $\psi(f)(L, s) = s$; if $n \geq 1$, let $L' = [\epsilon_1, \dots, \epsilon_{n-1}]$ and put $\psi(f)(L, s) = \psi(f)(L', f(\epsilon_n, s))$. One checks easily that $\psi(f)$ satisfies the two rules above, making it an action of $\text{List}(\Sigma)$ on S .⁵ It is also easy to check that ϕ and ψ are mutually inverse, completing the proof. $\square$

We sum up the idea of this section as follows:

Slogan 3.1.2.12.

“ A finite state machine is an action of a free monoid on a finite set. ”

Exercise 3.1.2.13. Consider the functions ϕ and ψ above.

- Show that for any $f: \Sigma \times S \rightarrow S$, the map $\psi(f): \text{List}(\Sigma) \times S \rightarrow S$ constitutes an action.
- Show that ϕ and ψ are mutually inverse functions (i.e. $\phi \circ \psi = \text{id}_{\text{Hom}(\Sigma \times S, S)}$ and $\psi \circ \phi = \text{id}_A$.)

$\diamond$

3.1.3 Monoid action tables

Let M be a monoid generated by the set $G = \{g_1, \dots, g_m\}$, and with some relations, and suppose that $\alpha: M \times S \rightarrow S$ is an action of M on a set $S = \{s_1, \dots, s_n\}$. We can represent the action α using an *action table* whose columns are the elements of G and whose rows are the elements of S . In each cell (row, col) , where $\text{row} \in S$ and $\text{col} \in G$, we put the element $\alpha(\text{col}, \text{row}) \in S$.

Example 3.1.3.1. If Σ and S are the sets from Figure 3.1, the displayed action of $\text{List}(\Sigma)$ on S would be given by the action table

Action from 3.1		
ID	a	b
State 0	State 1	State 2
State 1	State 2	State 1
State 2	State 0	State 0

Example 3.1.3.2. Every monoid acts on itself by its multiplication formula, $M \times M \rightarrow M$. If G is a generating set for M , we can write the elements of G as the columns and the elements of M as rows, and call this a multiplication table. For example, let $(\mathbb{N}, 1, *)$ denote the multiplicative monoid of natural numbers. The multiplication table is as

⁵This is technically a right-action.

follows:

Multiplication of natural numbers						
$\mathbb{N}$	1	2	3	4	5	...
0	0	0	0	0	0	...
1	1	2	3	4	5	...
2	2	4	6	8	10	...
3	3	6	9	12	15	...
4	4	8	12	16	20	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
21	21	42	63	84	105	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

(3.2)

Try to understand what is meant by this: “applying column 2 and then column 2 returns the same thing as applying column 4.”

Exercise 3.1.3.3. Let $\mathbb{N}$ be the additive monoid of natural numbers, let $S = \{0, 1, 2, \dots, 11\}$, and let $\cdot : \mathbb{N} \times S \rightarrow S$ be the action given in Example 3.1.2.3. Using a nice small generating set for the monoid, write out the corresponding action table. $\diamond$

3.1.4 Monoid homomorphisms

A monoid $(M, \text{id}, \star)$ involves a set, an identity element, and a multiplication formula. For two monoids to be comparable, their sets, their identity elements, and their multiplication formulas should be appropriately comparable. For example the additive monoids $\mathbb{N}$ and $\mathbb{Z}$ should be comparable because $\mathbb{N} \subseteq \mathbb{Z}$ is a subset, the identity elements in both cases are the same $\text{id} = 0$, and the multiplication formulas are both integer addition.

Definition 3.1.4.1. Let $\mathcal{M} := (M, \text{id}, \star)$ and $\mathcal{M}' := (M', \text{id}', \star')$ be monoids. A *monoid homomorphism* f from $\mathcal{M}$ to $\mathcal{M}'$, denoted $f: \mathcal{M} \rightarrow \mathcal{M}'$, is a function $f: M \rightarrow M'$ satisfying two conditions:

- $f(\text{id}) = \text{id}'$, and
- $f(m_1 \star m_2) = f(m_1) \star' f(m_2)$, for all $m_1, m_2 \in M$.

The set of monoid homomorphisms from $\mathcal{M}$ to $\mathcal{M}'$ is denoted $\text{Hom}_{\mathbf{Mon}}(\mathcal{M}, \mathcal{M}')$.

Example 3.1.4.2 (From $\mathbb{N}$ to $\mathbb{Z}$). As stated above, the inclusion map $i: \mathbb{N} \rightarrow \mathbb{Z}$ induces a monoid homomorphism $(\mathbb{N}, 0, +) \rightarrow (\mathbb{Z}, 0, +)$ because $i(0) = 0$ and $i(n_1 + n_2) = i(n_1) + i(n_2)$.

Let $i_5: \mathbb{N} \rightarrow \mathbb{Z}$ denote the function $i_5(n) = 5 * n$, so $i_5(4) = 20$. This is also a monoid homomorphism because $i_5(0) = 5 * 0 = 0$ and $i_5(n_1 + n_2) = 5 * (n_1 + n_2) = 5 * n_1 + 5 * n_2 = i_5(n_1) + i_5(n_2)$.

Application 3.1.4.3. Let $R = \{a, c, g, u\}$ and let $T = R^3$, the set of triplets in R . Let $\mathcal{R} = \text{List}(R)$ be the free monoid on R and let $\mathcal{T} = \text{List}(T)$ denote the free monoid on T . There is a monoid homomorphism $F: \mathcal{T} \rightarrow \mathcal{R}$ given by sending $t = (r_1, r_2, r_3)$ to the list $[r_1, r_2, r_3]$.

If A be the set of amino acids and $\mathcal{A} = \text{List}(A)$ the free monoid on A , the process of **translation** gives a monoid homomorphism $G: \mathcal{T} \rightarrow \mathcal{A}$, turning a list of RNA triplets into a polypeptide. But how do we go from a list of RNA nucleotides to a polypeptide?

The answer is that there is no good way to do this mathematically. So what is going wrong?

The answer is that there should not be a monoid homomorphism $\mathcal{R} \rightarrow \mathcal{A}$ because not all sequences of nucleotides produce a polypeptide; for example if the sequence has only two elements, it does not code for a polypeptide. There are several possible remedies to this problem. One is to take the image of F , which is a submonoid $\mathcal{R}' \subseteq \mathcal{R}$. It is not hard to see that there is a monoid homomorphism $F': \mathcal{R}' \rightarrow \mathcal{T}$, and we can compose it with G to get our desired monoid homomorphism $G \circ F': \mathcal{R}' \rightarrow \mathcal{A}$.⁶

◇◇

Example 3.1.4.4. Given any monoids $\mathcal{M}$ there is a unique monoid homomorphism from $\mathcal{M}$ to the trivial monoid $\underline{1}$ (see Example 3.1.1.8). There is also a unique homomorphism $\underline{1} \rightarrow \mathcal{M}$. These facts together have an upshot: between any two monoids $\mathcal{M}$ and $\mathcal{M}'$ we can always construct a homomorphism

$$\mathcal{M} \xrightarrow{!} \underline{1} \xrightarrow{!} \mathcal{M}'$$

which we call the *trivial homomorphism* $\mathcal{M} \rightarrow \mathcal{M}'$. A morphism $\mathcal{M} \rightarrow \mathcal{M}'$ that is not trivial is called a *nontrivial homomorphism*.

Proposition 3.1.4.5. *Let $\mathcal{M} = (\mathbb{Z}, 0, +)$ and $\mathcal{M}' = (\mathbb{N}, 0, +)$. The only monoid homomorphism $f: \mathcal{M} \rightarrow \mathcal{M}'$ sends every element $m \in \mathbb{Z}$ to $0 \in \mathbb{N}$.*

Proof. Let $f: \mathcal{M} \rightarrow \mathcal{M}'$ be a monoid homomorphism, and let $n = f(1)$ and $n' = f(-1)$ in $\mathbb{N}$. Then we know that since $0 = 1 + (-1)$ in $\mathbb{Z}$ we must have $0 = f(0) = f(1 + (-1)) = f(1) + f(-1) = n + n' \in \mathbb{N}$. But if $n \geq 1$ then this is impossible, so $n = 0$. Similarly $n' = 0$. Any element $m \in \mathbb{Z}$ can be written $m = 1 + 1 + \cdots + 1$ or as $m = -1 + -1 + \cdots + -1$, and it is easy to see that $f(1) + f(1) + \cdots + f(1) = 0 = f(-1) + f(-1) + \cdots + f(-1)$. Therefore, $f(m) = 0$ for all $m \in \mathbb{Z}$.

□

Exercise 3.1.4.6. For any $m \in \mathbb{N}$ let $i_m: \mathbb{N} \rightarrow \mathbb{Z}$ be the function $i_m(n) = m * n$. All such functions are monoid homomorphisms. Do any monoid homomorphisms $(\mathbb{N}, 0, +) \rightarrow (\mathbb{Z}, 0, +)$ not come in this way? For example, what about using $n \mapsto 5 * n - 1$ or $n \mapsto n^2$, or some other function?

◇

Exercise 3.1.4.7. Let $\mathcal{M} := (\mathbb{N}, 0, +)$ be the additive monoid of natural numbers, let $\mathcal{N} := (\mathbb{R}_{\geq 0}, 0, +)$ be the additive monoid of nonnegative real numbers, and let $\mathcal{P} := (\mathbb{R}_{> 0}, 1, *)$ be the multiplicative monoid of positive real numbers. Can you think of any nontrivial monoid homomorphisms of the following sorts:

$$\mathcal{M} \rightarrow \mathcal{N}, \quad \mathcal{M} \rightarrow \mathcal{P}, \quad \mathcal{N} \rightarrow \mathcal{P}, \quad \mathcal{N} \rightarrow \mathcal{M}, \quad \mathcal{P} \rightarrow \mathcal{N}?$$

◇

3.1.4.8 Homomorphisms from free monoids

Recall that $(\mathbb{N}, 0, +)$ is the free monoid on one generator. It turns out that for any other monoid $\mathcal{M} = (M, \text{id}, \star)$, the set of monoid homomorphisms $\mathbb{N} \rightarrow \mathcal{M}$ is in bijection with the set M . This is a special case (in which G is a set with one element) of the following proposition.

⁶Adding stop-codons to the mix we can handle more of $\mathcal{R}$, e.g. sequences that don't have a multiple-of-three many nucleotides.

Proposition 3.1.4.9. *Let G be a set, let $F(G) := (\text{List}(G), [], ++)$ be the free monoid on G , and let $\mathcal{M} := (M, \text{id}, \star)$ be any monoid. There is a natural bijection*

$$\text{Hom}_{\mathbf{Mon}}(F(G), \mathcal{M}) \xrightarrow{\cong} \text{Hom}_{\mathbf{Set}}(G, M).$$

Proof. We provide a function $\phi: \text{Hom}_{\mathbf{Mon}}(F(G), \mathcal{M}) \rightarrow \text{Hom}_{\mathbf{Set}}(G, M)$ and a function $\psi: \text{Hom}_{\mathbf{Set}}(G, M) \rightarrow \text{Hom}_{\mathbf{Mon}}(F(G), \mathcal{M})$ and show that they are mutually inverse. Let us first construct ϕ . Given a monoid homomorphism $f: F(G) \rightarrow \mathcal{M}$, we need to provide $\phi(f): G \rightarrow M$. Given any $g \in G$ we define $\phi(f)(g) := f([g])$.

Now let us construct ψ . Given $p: G \rightarrow M$, we need to provide $\psi(p): \text{List}(G) \rightarrow \mathcal{M}$ such that $\psi(p)$ is a monoid homomorphism. For a list $L = [g_1, \dots, g_n] \in \text{List}(G)$, define $\psi(p)(L) := p(g_1) \star \dots \star p(g_n) \in M$. In particular, $\psi(p)([]) = \text{id}$. It is not hard to see that this is a monoid homomorphism. It is also easy to see that $\phi \circ \psi(p) = p$ for all $p \in \text{Hom}_{\mathbf{Set}}(G, M)$. We show that $\psi \circ \phi(f) = f$ for all $f \in \text{Hom}_{\mathbf{Mon}}(F(G), \mathcal{M})$. Choose $L = [g_1, \dots, g_n] \in \text{List}(G)$. Then

$$\psi(\phi(f))(L) = (\phi(f)(g_1) \star \dots \star \phi(f)(g_n)) = f[g_1] \star \dots \star f[g_n] = f([g_1, \dots, g_n]) = f(L).$$

□

Exercise 3.1.4.10. Let $G = \{a, b\}$, let $\mathcal{M} := (M, \text{id}, \star)$ be any monoid, and let $f: G \rightarrow M$ be given by $f(a) = m$ and $f(b) = n$, where $m, n \in M$. If $\psi: \text{Hom}_{\mathbf{Set}}(G, M) \rightarrow \text{Hom}_{\mathbf{Mon}}(F(G), \mathcal{M})$ is the function from the proof of Proposition 3.1.4.9 and $L = [a, a, b, a, b]$, what is $\psi(f)(L)$? ◇

3.1.4.11 Restriction of scalars

A monoid homomorphism $f: M \rightarrow M'$ (see Definition 3.1.4.1) ensures that the elements of M have a reasonable interpretation in M' ; they act the same way over in M' as they did back home in M . If we have such a homomorphism f and we have an action $\alpha: M' \times S \rightarrow S$ of M' on a set S , then we have a method for allowing M to act on S as well. Namely, we take an element of M , send it over to M' , and act on S . In terms of functions, we compose α with the function $(f, \text{id}_S): M \times S \rightarrow M' \times S$, to get a function we'll denote

$$\Delta_f(\alpha): M \times S \rightarrow S.$$

After Proposition 3.1.4.12 we will know that $\Delta_f(\alpha)$ is indeed a monoid action, and we say that it is given by *restriction of scalars along f* .

Proposition 3.1.4.12. *Let $(M, \text{id}, \star)$ and $(M', \text{id}', \star')$ be monoids, $f: M \rightarrow M'$ a monoid homomorphism, S a set, and suppose that $\alpha: M' \times S \rightarrow S$ is an action of M' on S . Then $\Delta_f(\alpha): M \times S \rightarrow S$, defined as above, is a monoid action as well.*

Proof. Refer to Remark 3.1.2.2; we assume α is a monoid action and want to show that $\Delta_f(\alpha)$ is too. We have $\Delta_f(\alpha)(\text{id}, s) = \alpha(f(\text{id}), s) = \alpha(\text{id}', s) = s$. We also have

$$\begin{aligned} \Delta_f(\alpha)(m, \Delta_f(\alpha)(n, s)) &= \alpha(f(m), \alpha(f(n), s)) = \alpha(f(m) \star' f(n), s) \\ &= \alpha(f(m \star n), s) \\ &= \Delta_f(\alpha)(m \star n, s). \end{aligned}$$

□

Example 3.1.4.13. Let $\mathbb{N}$ and $\mathbb{Z}$ denote the additive monoids of natural numbers and integers, respectively, and let $i: \mathbb{N} \rightarrow \mathbb{Z}$ be the inclusion, which we saw in Example 3.1.4.2 is a monoid homomorphism. There is an action $\alpha: \mathbb{Z} \times \mathbb{R} \rightarrow \mathbb{R}$ of the monoid $\mathbb{Z}$ on the set $\mathbb{R}$ of real numbers, given by $\alpha(n, x) = n + x$. Clearly, this action works just as well if we restrict our scalars to $\mathbb{N} \subseteq \mathbb{Z}$, allowing ourselves only to add natural numbers to reals. The action $\Delta_i \alpha: \mathbb{N} \times \mathbb{R} \rightarrow \mathbb{R}$ is given on $(n, x) \in \mathbb{N} \times \mathbb{R}$ by $\Delta_i \alpha(n, x) = \alpha(i(n), x) = \alpha(n, x) = n + x$, just as expected.

Example 3.1.4.14. Suppose that V is a complex vector space. In particular, this means that the monoid $\mathbb{C}$ of complex numbers (under multiplication) acts on the elements of V . If $i: \mathbb{R} \rightarrow \mathbb{C}$ is the inclusion of the real line inside $\mathbb{C}$, then i is a monoid homomorphism. Restriction of scalars in the above sense turns V into a real vector space, so the name “restriction of scalars” is apt.

Exercise 3.1.4.15. Let $\mathbb{N}$ be the free monoid on one generator, let $\Sigma = \{a, b\}$, and let $S = \{\text{State 0, State 1, State 2}\}$. Consider the map of monoids $f: \mathbb{N} \rightarrow \text{List}(\Sigma)$ given by sending $1 \mapsto [a, b, b]$. The monoid action $\alpha: \text{List}(\Sigma) \times S \rightarrow S$ given in Example 3.1.3.1 can be transformed by restriction of scalars along f to an action $\Delta_f(\alpha)$ of $\mathbb{N}$ on S . Write down its action table. $\diamond$

3.2 Groups

Groups are monoids in which every element has an inverse. If we think of these structures in terms of how they act on sets, the difference between groups and monoids is that the action of every group element can be undone. One way of thinking about groups is in terms of symmetries. For example, the rotations and reflections of a square form a group.

Another way to think of the difference between monoids and groups is in terms of time. Monoids are likely useful in thinking about diffusion, in which time plays a role and things cannot be undone. Groups are more likely useful in thinking about mechanics, where actions are time-reversible.

3.2.1 Definition and examples

Definition 3.2.1.1. Let $(M, \text{id}, \star)$ be a monoid. An element $m \in M$ is said to *have an inverse* if there exists an $m' \in M$ such that $mm' = \text{id}$ and $m'm = \text{id}$. A *group* is a monoid $(M, \text{id}, \star)$ in which every element $m \in M$ has an inverse.

Proposition 3.2.1.2. Suppose that $(M, \text{id}, \star)$ is a group and let $m \in M$ be an element. Then m has only one inverse.

Proof. Suppose that both m' and m'' are inverses of m ; we want to show that $m' = m''$. This follows by the associative law for monoids:

$$m' = m'(mm'') = (m'm)m'' = m''.$$

□

Example 3.2.1.3. The additive monoid $(\mathbb{N}, 0, +)$ is not a group because none of its elements are invertible, except for 0. However, the monoid of integers $(\mathbb{Z}, 0, +)$ is a group. The monoid of clock positions from Example 3.1.1.22 is also a group. For example the inverse of Q^5 is Q^7 because $Q^5 \star Q^7 = \text{id} = Q^7 \star Q^5$.

Example 3.2.1.4. Consider a square centered at the origin in $\mathbb{R}^2$. It has rotational and mirror symmetries. There are eight of these, which we denote

$$\{\text{id}, \rho, \rho^2, \rho^3, \phi, \phi\rho, \phi\rho^2, \phi\rho^3\},$$

where ρ stands for 90° counterclockwise rotation and ϕ stands for horizontal-flip (across the vertical axis). So relations include $\rho^4 = \text{id}$, $\phi^2 = \text{id}$, and $\rho^3\phi = \phi\rho$.

Example 3.2.1.5. The set of 3×3 matrices can be given the structure of a monoid, where the identity element is the 3×3 identity matrix, the multiplication is matrix multiplication. The subset of invertible matrices forms a group, called *the general linear group of dimension 3* and denoted GL_3 . Inside of GL_3 is the so-called *orthogonal group*, denoted O_3 , of matrices M such that $M^{-1} = M^\top$. These matrices correspond to symmetries of the sphere centered at the origin.

Another interesting group is the Euclidean group $E(3)$ which consists of all *isometries* of $\mathbb{R}^3$, i.e. all functions $\mathbb{R}^3 \rightarrow \mathbb{R}^3$ that preserve distances.

Application 3.2.1.6. In **crystallography** one is often concerned with the symmetries that arise in the arrangement A of atoms in a molecule. To think about symmetries in terms of groups, we first define an *atom-arrangement* to be a finite subset $i: A \subseteq \mathbb{R}^3$. A symmetry in this case is an isometry of $\mathbb{R}^3$ (see Example 3.2.1.5), say $f: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ such that there exists a dotted arrow making the diagram below commute:

$$\begin{array}{ccc} A & \xrightarrow{\quad} & A \\ i \downarrow & & \downarrow i \\ \mathbb{R}^3 & \xrightarrow{\quad f \quad} & \mathbb{R}^3 \end{array}$$

That is, it's an isometry of $\mathbb{R}^3$ such that each atom of A is sent to a position currently occupied by an atom of A . It is not hard to show that the set of such isometries forms a group, called the *space group* of the crystal.

◇◇

Exercise 3.2.1.7. Let S be a finite set. A *permutation* of S is an isomorphism $f: S \xrightarrow{\cong} S$.

- Come up with an identity, and a multiplication formula, such that the set of permutations of S forms a monoid.
- Is it a group?

◇

Exercise 3.2.1.8. In Exercise 3.1.1.23 you classified the cyclic monoids. Which of them are groups?

◇

Definition 3.2.1.9 (Group action). Let $(G, \text{id}, \star)$ be a group and S a set. An *action* of G on S is a function $\curvearrowright: G \times S \rightarrow S$ such that for all $s \in S$ and $g, g' \in G$, we have

- $\text{id} \curvearrowright s = s$ and
- $g \curvearrowright (g' \curvearrowright s) = (g \star g') \curvearrowright s$.

In other words, considering G as a monoid, it is an action in the sense of Definition 3.1.2.1.

Example 3.2.1.10. When a group acts on a set, it has the character of symmetry. For example, consider the group whose elements are angles θ . This group may be denoted $U(1)$ and is often formalized as the unit circle in $\mathbb{C}$ of complex numbers $z = a + bi$ such that $|z| = a^2 + b^2 = 1$. The set of such points is given the structure of a group $(U(1), \text{id}, \star)$ by defining the identity element to be $\text{id} := 1 + 0i$ and the group law to be complex multiplication. But for those unfamiliar with complex numbers, this is simply angle addition where we understand that $360^\circ = 0^\circ$. If $\theta_1 = 190^\circ$ and $\theta_2 = 278^\circ$, then $\theta_1 \star \theta_2 = 468 = 108$. In the language of complex numbers, $z = e^{i\theta}$.

The group $U(1)$ acts on any set that we can picture as having rotational symmetry about a fixed axis, such as the earth around the north-south axis. We will define $S = \{(x, y, z) \in \mathbb{R}^3 \mid x^2 + y^2 + z^2 = 1\}$, the unit sphere, and understand the rotational action of $U(1)$ on S .

We first show that $U(1)$ acts on $\mathbb{R}^3$ by $\theta \curvearrowright (x, y, z) = (x \cos \theta + y \sin \theta, -x \sin \theta + y \cos \theta, z)$, or with matrix notation as

$$\theta \curvearrowright (x, y, z) := \begin{pmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

Trigonometric identities ensure that this is indeed an action.

In terms of action tables, we would need infinitely many columns to express this action. Here is a sample

Action of $U(1)$ on $\mathbb{R}^3$			
$\mathbb{R}^3$	$\theta = 45^\circ$	$\theta = 90^\circ$	$\theta = 100^\circ$
$(0,0,0)$	$(0,0,0)$	$(0,0,0)$	$(0,0,0)$
$(1,0,0)$	$(.71,.71,0)$	$(0,1,0)$	$(-.17,.98,0)$
$(0,1,-4.2)$	$(-.71,.71,-4.2)$	$(-1,0,-4.2)$	$(-.98,-.17,-4.2)$
$(3,4,2)$	$(4.95,.71,2)$	$(-4,3,2)$	$(3.42,-3.65,2)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Finally, we are looking to see that the action preserves length so that if $(x, y, z) \in S$ then $\theta \curvearrowright (x, y, z) \in S$; this way we will have confirmed that $U(1)$ indeed acts on S . The calculation begins by assuming $x^2 + y^2 + z^2 = 1$ and checks

$$(x \cos \theta + y \sin \theta)^2 + (-x \sin \theta + y \cos \theta)^2 + z^2 = x^2 + y^2 + z^2 = 1.$$

Exercise 3.2.1.11. Let X be a set and consider the group of permutations of X (see Exercise 3.2.1.7), which we will denote Σ_X . Find a canonical action of Σ_X on X . $\diamond$

Definition 3.2.1.12. Let G be a group acting on a set X . For any point $x \in X$, the *orbit of x* , denoted Gx , is the set

$$Gx := \{x' \in X \mid \exists g \in G \text{ such that } gx = x'\}.$$

Application 3.2.1.13. Let S be the surface of the earth, understood as a sphere, and let $G = U(1)$ be the group of angles acting on S as in Example 3.2.1.10. The orbit of any point $p = (x, y, z) \in S$ is the set of points on the same latitude line as p .

One may also consider a small band around the earth, i.e. the set $A = \{(x, y, z) \mid 1.0 \leq x^2 + y^2 + z^2 \leq 1.05\}$. The action of $U(1) \curvearrowright S$ extends to an action $U(1) \curvearrowright A$. The

orbits are latitude-lines-at-altitude. A simplifying assumption in **climatology** may be given by assuming that $U(1)$ acts on all currents in the atmosphere in an appropriate sense. That way, instead of considering movement within the whole space A , we only allow movement that behaves the same way throughout each orbit of the group action.

◇◇

Exercise 3.2.1.14. a.) Consider the $U(1)$ action on $\mathbb{R}^3$ given in Example 3.2.1.10. Describe the set of orbits of this action.

b.) What are the orbits of the action of the permutation group $\Sigma_{\{1,2,3\}}$ on the set $\{1, 2, 3\}$? (See Exercise 3.2.1.11.)

◇

Exercise 3.2.1.15. Let G be a group and X a set on which G acts by $\curvearrowright: G \times X \rightarrow X$. Is “being in the same orbit” an equivalence relation on X ?

◇

Definition 3.2.1.16. Let G and G' be groups. A *group homomorphism* $f: G \rightarrow G'$ is defined to be a monoid homomorphism $G \rightarrow G'$, where G and G' are being regarded as monoids in accordance with Definition 3.2.1.1.

3.3 Graphs

In this course, unless otherwise specified, whenever we speak of graphs we are not talking about curves in the plane, such as parabolas, or pictures of functions generally. We are speaking of systems of vertices and arrows.

We will take our graphs to be *directed*, meaning that every arrow points *from* a vertex *to* a vertex; rather than merely connecting vertices, arrows will have direction. If a and b are vertices, there can be many arrows from a to b , or none at all. There can be arrows from a to itself. Here is the formal definition in terms of sets and functions.

3.3.1 Definition and examples

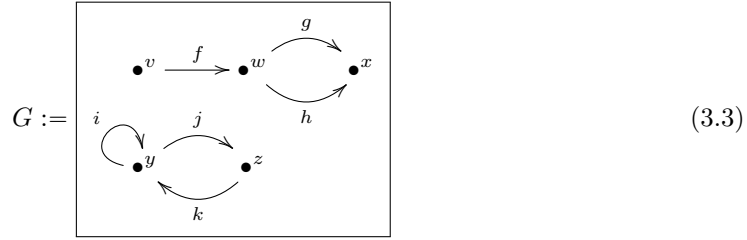
Definition 3.3.1.1. A *graph* G consists of a sequence $G := (V, A, \text{src}, \text{tgt})$ where

- V is a set, called *the set of vertices of G* (singular: *vertex*),
- A is a set, called *the set of arrows of G* ,
- $\text{src}: A \rightarrow V$ is a function, called *the source function for G* , and
- $\text{tgt}: A \rightarrow V$ is a function, called *the target function for G* .

Given an arrow $a \in A$ we refer to $\text{src}(a)$ as the *source vertex* of a and to $\text{tgt}(a)$ as the *target vertex* of a .

To draw a graph, first draw a dot for every element of V . Then for every element $a \in A$, draw an arrow connecting dot $\text{src}(a)$ to dot $\text{tgt}(a)$.

Example 3.3.1.2. Here is a picture of a graph $G = (V, A, \text{src}, \text{tgt})$:



We have $V = \{v, w, x, y, z\}$ and $A = \{f, g, h, i, j, k\}$. The source and target functions $\text{src}, \text{tgt}: A \rightarrow V$ can be captured in the table to the left below:

A	src	tgt
f	v	w
g	w	x
h	w	x
i	y	y
j	y	z
k	z	y

V
v
w
x
y
z

In fact, all of the data of the graph G is captured in the two tables above—together they tell us the sets A and V and the functions src and tgt .

Example 3.3.1.3. Every olog has an underlying graph. The additional information in an olog has to do with which pairs of paths are declared equivalent, as well as text that has certain English-readability rules.

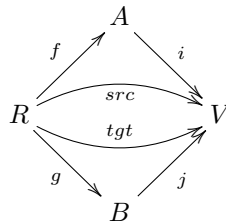
Exercise 3.3.1.4. Write down a function $f: A \rightarrow B$, where $A = \{1, 2, 3, 4, 5\}$ and $B = \{a, b, c\}$. Let $A \sqcup B$ be the coproduct of A and B (Definition 2.4.2.1) and let $A \xrightarrow{i_1} A \sqcup B \xleftarrow{i_2} B$ be the two inclusions. Consider the two functions $\text{src}, \text{tgt}: A \rightarrow A \sqcup B$, where $\text{src} = i_1$ and tgt is the composition $A \xrightarrow{f} B \xrightarrow{i_2} A \sqcup B$. Draw the associated graph $(A \sqcup B, A, \text{src}, \text{tgt})$. $\diamond$

Exercise 3.3.1.5. a.) Let V be a set. Suppose we just draw the elements of V as vertices and have no arrows between them. Is this a graph?

b.) Given V , is there any other “canonical” or somehow automatic non-random procedure for generating a graph with those vertices?

$\diamond$

Example 3.3.1.6. Recall from Construction 2.5.2.5 the notion of bipartite graph, which we defined to be a span (i.e. pair of functions, see Definition 2.5.2.1) $A \xleftarrow{f} R \xrightarrow{g} B$. Now that we have a formal definition of graph, we might hope that bipartite graphs fit in, and they do. Let $V = A \sqcup B$ and let $i: A \rightarrow V$ and $j: B \rightarrow V$ be the inclusions. Let $\text{src} = i \circ f: R \rightarrow V$ and let $\text{tgt} = j \circ g: R \rightarrow V$ be the composites.



Then $(V, R, \text{src}, \text{tgt})$ is a graph that would be drawn exactly as we specified the drawing of spans in Construction 2.5.2.5.

Example 3.3.1.7. Let $G = (V, A, \text{src}, \text{tgt})$ be a graph; we want to spread it out over discrete time, so that each arrow does not occur within a given time-slice but instead over a quantum unit of time.

Let $N = (\mathbb{N}, \mathbb{N}, n \mapsto n, n \mapsto n + 1)$ be the graph depicted

$$\bullet \xrightarrow{0} \bullet \xrightarrow{1} \bullet \xrightarrow{2} \dots$$

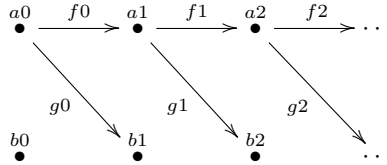
When we get to limits in a category, we will understand that products can be taken in the category of graphs (see Example 4.5.1.5, and $N \times G$ will make sense. For now, we construct it by hand.

Let $T(G) = (V \times \mathbb{N}, A \times \mathbb{N}, \text{src}', \text{cod}')$ be a new graph, where for $a \in A$ and $n \in \mathbb{N}$ we have $\text{src}'(a, n) := (\text{src}(a), n)$ and $\text{tgt}'(a, n) = (\text{tgt}(a), n + 1)$. This may be a bit much to swallow, so try to simply understand what is being done in the following example.

Let G be the graph drawn below

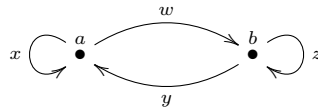


Then $T(G)$ will be the graph



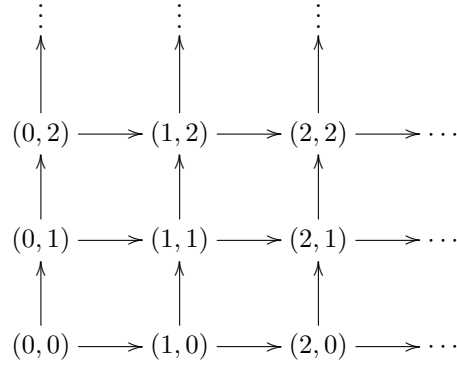
As you can see, f -arrows still take a 's to a 's and g -arrows still take a 's to b 's, but they always march forward in time.

Exercise 3.3.1.8. Let G be the graph depicted below:



Draw (using ellipses “...” if necessary) the graph $T(G)$ defined in Example 3.3.1.7. $\diamond$

Exercise 3.3.1.9. Consider the infinite graph $G = (V, A, \text{src}, \text{tgt})$ depicted below,



- a.) Write down the sets A and E .
- b.) What are the source and target function $A \rightarrow V$?

◇

Exercise 3.3.1.10. A graph is a pair of functions $E \rightrightarrows V$. This sets up the notion of equalizer and coequalizer (see Definitions 2.5.3.1 and 2.6.3.1).

- a.) What feature of a graph is captured by the equalizer of its source and target functions?
- b.) What feature of a graph is captured by the coequalizer of its source and target functions?

◇

3.3.2 Paths in a graph

We all know what a path in a graph is. The following definition makes this precise. In particular, one can have paths of any finite length $n \in \mathbb{N}$, even length 0 or 1. Also, we want to be able to talk about the source vertex and target vertex of a path, as well as composition of paths.

Definition 3.3.2.1. Let $G = (V, A, \text{src}, \text{tgt})$ be a graph. A *path of length n* in G , denoted $p \in \text{Path}_G^{(n)}$ is a head-to-tail sequence

$$p = (v_0 \xrightarrow{a_1} v_1 \xrightarrow{a_2} v_2 \xrightarrow{a_3} \dots \xrightarrow{a_n} v_n) \quad (3.4)$$

of arrows in G . In particular, $\text{Path}_G^{(1)} = A$ and $\text{Path}_G^{(0)} = V$; we refer to the path of length 0 on vertex v as the *trivial path on v* and denote it by id_v . We denote by Path_G the set of all paths on G ,

$$\text{Path}_G := \bigcup_{n \in \mathbb{N}} \text{Path}_G^{(n)}.$$

Every path $p \in \text{Path}_G$ has a source vertex and a target vertex, and we may denote these by $\overline{\text{src}}, \text{tgt}: \text{Path}_G \rightarrow V$. If p is a path with $\overline{\text{src}}(p) = v$ and $\text{tgt}(p) = w$, we may denote it by $p: v \rightarrow w$. Given two vertices $v, w \in V$, we write $\text{Path}_G(v, w)$ to denote the set of all paths $p: v \rightarrow w$.

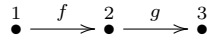
There is a concatenation operation on paths. Given a path $p: v \rightarrow w$ and $q: w \rightarrow x$, we define the concatenation, denoted $pq: v \rightarrow x$ in the obvious way. In particular, if p (resp. r) is the trivial path on vertex v (resp. vertex w) then for any path $q: v \rightarrow w$, we have $pq = q$ (resp. $qr = q$). Thus, for clarity, we may always denote a path as beginning with a trivial path on its source vertex; e.g. the path p from Diagram (3.4) may be denoted $p = \text{id}_{v_0} a_1 a_2 \cdots a_n$.

Example 3.3.2.2. In Diagram (3.3), page 79, there are no paths from v to y , one path (f) from v to w , two paths (fg and fh) from v to x , and infinitely many paths

$$\{i^{p_1}(jk)^{q_1} \cdots i^{p_n}(jk)^{q_n} \mid n, p_1, q_1, \dots, p_n, q_n \in \mathbb{N}\}$$

from y to y .

Exercise 3.3.2.3. How many paths are there in the following graph?



◇

Exercise 3.3.2.4. Let G be a graph and consider the set Path_G of paths in G . Suppose someone claimed that there is a monoid structure on the set Path_G , where the multiplication formula is given by concatenation of paths. Are they correct? Why or why not? Hint: what should be the identity element? ◇

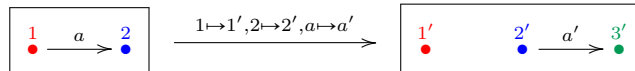
3.3.3 Graph homomorphisms

A graph $(V, A, \text{src}, \text{tgt})$ involves two sets and two functions. For two graphs to be comparable, their two sets and their two functions should be appropriately comparable.

Definition 3.3.3.1. Let $G = (V, A, \text{src}, \text{tgt})$ and $G' = (V', A', \text{src}', \text{tgt}')$ be graphs. A *graph homomorphism* f from G to G' , denoted $f: G \rightarrow G'$, consists of two functions $f_0: V \rightarrow V'$ and $f_1: A \rightarrow A'$ such that the two diagrams below commute:

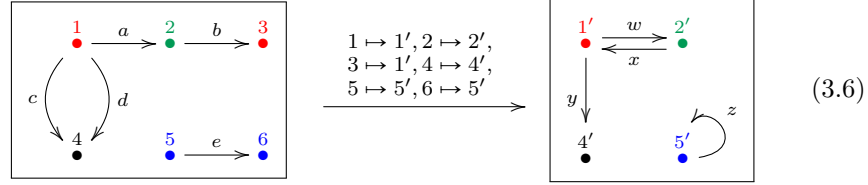
$$\begin{array}{ccc} A & \xrightarrow{f_1} & A' \\ \text{src} \downarrow & & \downarrow \text{src}' \\ V & \xrightarrow{f_0} & V' \end{array} \qquad \begin{array}{ccc} A & \xrightarrow{f_1} & A' \\ \text{tgt} \downarrow & & \downarrow \text{tgt}' \\ V & \xrightarrow{f_0} & V' \end{array} \quad (3.5)$$

Remark 3.3.3.2. The above conditions (3.5) may look abstruse at first, but they encode a very important idea, roughly stated “arrows are bound to their vertices”. Under a map of graphs $G \rightarrow G'$, one cannot flippantly send an arrow of G any old arrow of G' : it must still connect the vertices it connected before. Below is an example of a mapping that does not respect this condition: e connects 1 and 2 before, but not after:



The commutativity of the diagrams in (3.5) is exactly what is needed to ensure that arrows are handled in the expected way by a proposed graph homomorphism.

Example 3.3.3.3. Let $G = (V, A, \text{src}, \text{tgt})$ and $G' = (V', A', \text{src}', \text{tgt}')$ be the graphs drawn to the left and right (respectively) below:



The colors indicate our choice of function $f_0: V \rightarrow V'$. Given that choice, condition (3.5) imposes in this case that there is a unique choice of graph homomorphism $f: G \rightarrow G'$.

Exercise 3.3.3.4. a.) Where are a, b, c, d, e sent under $f_1: A \rightarrow A'$ in Diagram (3.6)?

b.) Choose a couple elements of A and check that they behave as specified by Diagram (3.5).

◇

Exercise 3.3.3.5. Given a morphism of graphs $f: G \rightarrow G'$, there an induced function $\text{Path}(f): \text{Path}(G) \rightarrow \text{Path}(G')$.

- Is it the case that for every $n \in \mathbb{N}$, the function $\text{Path}(f)$ carries $\text{Path}^{(n)}(G)$ to $\text{Path}^{(n)}(G')$, or can path lengths change in this process?
- Suppose that f_0 and f_1 are injective (meaning no two distinct vertices in G are sent to the same vertex (respectively for arrows) under f). Does this imply that $\text{Path}(f)$ is also injective (meaning no two distinct paths are sent to the same path under f)?
- Suppose that f_0 and f_1 are surjective (meaning every vertex in G' and every arrow in G' is in the image of f). Does this imply that $\text{Path}(f)$ is also surjective? Hint: at least one of the answers to these three questions is “no”.

◇

Exercise 3.3.3.6. Given a graph $(V, A, \text{src}, \text{tgt})$, let $i: A \rightarrow V \times V$ be function guaranteed by the universal property for products, as applied to $\text{src}, \text{tgt}: A \rightarrow V$. One might hope to summarize Condition (3.5) for graph homomorphisms by the commutativity of the single square

$$\begin{array}{ccc} A & \xrightarrow{f_1} & A' \\ i \downarrow & & \downarrow i' \\ V \times V & \xrightarrow{f_0 \times f_0} & V' \times V'. \end{array} \quad (3.7)$$

Is the commutativity of the diagram in (3.7) indeed equivalent to the commutativity of the diagrams in (3.5)?

◇

3.3.3.7 Binary relations and graphs

Definition 3.3.3.8. Let X be a set. A *binary relation* on X is a subset $R \subseteq X \times X$.

If $X = \mathbb{N}$ is the set of integers, then the usual $\leq$ defines a relation on X : given $(m, n) \in \mathbb{N} \times \mathbb{N}$, we put $(m, n) \in R$ iff $m \leq n$. As a table it might be written as to the

left

$m \leq n$		$n = 5m$		$ n - m \leq 1$		(3.8)
m	n	m	n	m	n	
0	0	0	0	0	0	
0	1	1	5	0	1	
1	1	2	10	1	0	
0	2	3	15	1	1	
1	2	4	20	1	2	
2	2	5	25	2	1	
0	3	6	30	2	2	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

The middle table is the relation $\{(m, n) \in \mathbb{N} \times \mathbb{N} \mid n = 5m\} \subseteq \mathbb{N} \times \mathbb{N}$ and the right-hand table is the relation $\{(m, n) \in \mathbb{N} \times \mathbb{N} \mid |n - m| \leq 1\} \subseteq \mathbb{N} \times \mathbb{N}$.

Exercise 3.3.3.9. A relation on $\mathbb{R}$ is a subset of $\mathbb{R} \times \mathbb{R}$, and one can indicate such a subset of the plane by shading. Choose an error bound $\epsilon > 0$ and draw the relation one might refer to as “ ϵ -approximation”. To say it another way, draw the relation “ x is within ϵ of y ”. $\diamond$

Exercise 3.3.3.10 (Binary relations to graphs). a.) If $R \subseteq S \times S$ is a binary relation, find a natural way to make a graph out of it, having vertices S .

b.) What is the set A of arrows?

c.) What are the source and target functions $src, tgt: A \rightarrow S$?

d.) Take the left-hand table in (3.8) and consider its first 7 rows (i.e. forget the $\vdots$). Draw the corresponding graph (do you see a tetrahedron?).

e.) Do the same for the right-hand table. $\diamond$

Exercise 3.3.3.11 (Graphs to binary relations). a.) If (V, A, src, tgt) is a graph, find a natural way to make a binary relation $R \subseteq V \times V$ out of it.

b.) Take the left-hand graph G from (3.6) and write out the corresponding binary relation in table form. $\diamond$

Exercise 3.3.3.12 (Going around the loops). a.) Given a binary relation $R \subseteq S \times S$, you know from Exercise 3.3.3.10 how to construct a graph out of it, and from Exercise 3.3.3.11 how to make a new binary relation out of that. How does the resulting relation compare with the original?

b.) Given a graph (V, A, src, tgt) , you know from Exercise 3.3.3.11 how to make a new binary relation out of it, and from Exercise 3.3.3.10 how to construct a new graph out of that. How does the resulting graph compare with the original? $\diamond$

3.4 Orders

People usually think of certain sets as though they just *are* ordered, e.g. that an order is ordained by God. For example the natural numbers just *are* ordered. The letters in the

alphabet just *are* ordered. But in fact we put orders on sets, and some are simply more commonly used in culture. One could order the letters in the alphabet by frequency of use and *e* would come before *a*. Given different purposes, we can put different orders on the same set. For example in Exercise 4.5.1.4 we will give a different ordering on the natural numbers that is useful in elementary number theory.

In science, we might order the set of materials in two different ways. In the first, we consider material *A* to be “before” material *B* if *A* is an ingredient or part of *B*, so water would be before concrete. But we could also order materials based on how electrically conductive they are, whereby concrete would be before water. This section is about different kinds of orders.

3.4.1 Definitions of preorder, partial order, linear order

Definition 3.4.1.1. Let S be a set and $R \subseteq S \times S$ a binary relation on S ; if $(s, s') \in R$ we will write $s \leq s'$. Then we say that R is a *preorder* if, for all $s, s', s'' \in S$ we have

Reflexivity: $s \leq s$, and

Transitivity: if $s \leq s'$ and $s' \leq s''$, then $s \leq s''$.

We say that R is a *partial order* if it is a preorder and, in addition, for all $s, s' \in S$ we have

Antisymmetry: If $s \leq s'$ and $s' \leq s$, then $s = s'$.

We say that R is a *linear order* if it is a partial order and, in addition, for all $s, s' \in S$ we have

Comparability: Either $s \leq s'$ or $s' \leq s$.

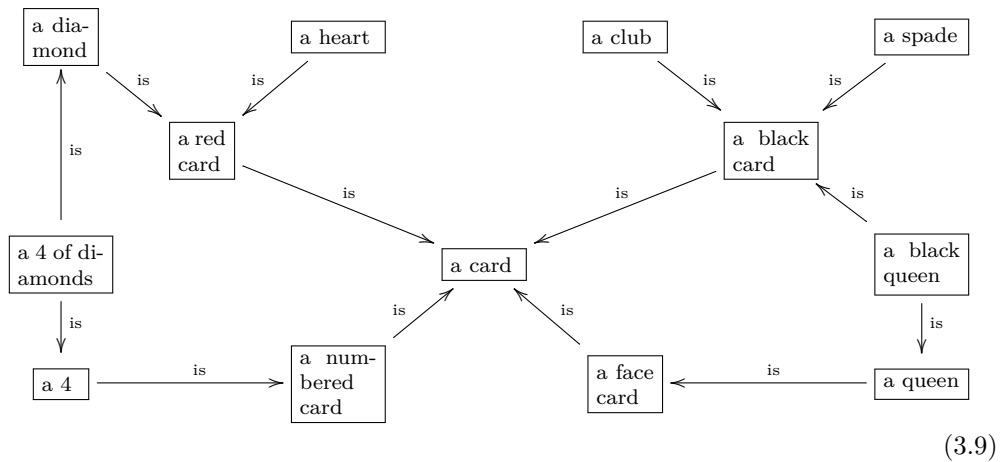
We denote such a preorder (or partial order or linear order) by $(S, \leq)$.

Exercise 3.4.1.2. a.) Decide whether the table to the left in Display (3.8) constitutes a linear order.

b.) Show that neither of the other tables are even preorders.

◇

Example 3.4.1.3 (Partial order not linear order). We will draw an olog for playing cards.



We can put a binary relation on the set of boxes here by saying $A \leq B$ if there is a path $A \rightarrow B$. One can see immediately that this is a preorder because length=0 paths give reflexivity and concatenation of paths gives transitivity. To see that it is a partial order we only note that there are no loops. But this partial order is not a linear order because there is no path (in either direction) between, e.g., ‘a 4 of diamonds’ and ‘a black queen’, so it violates the comparability condition.

Remark 3.4.1.4. Note that olog (3.9) in Example 3.4.1.3 is a good olog in the sense that given any collection of cards (e.g. choose 45 cards at random from each of 7 decks and throw them in a pile), they can be classified according to the boxes of (3.9) such that every arrow indeed constitutes a function (which happens to be injective). For example the arrow ‘a heart’ $\xrightarrow{\text{is}}$ ‘a red card’ is a function from the set of chosen hearts to the set of chosen red cards.

Example 3.4.1.5 (Preorder not partial order). Every equivalence relation is a preorder but rarely are they partial orders. For example if $S = \{1, 2\}$ and we put $R = S \times S$, then this is an equivalence relation. It is a preorder but not a partial order (because $1 \leq 2$ and $2 \leq 1$, but $1 \neq 2$, so antisymmetry fails).

Example 3.4.1.6 ([Finite linear orders]). Let $n \in \mathbb{N}$ be a natural number. Define a linear order on the set $\{0, 1, 2, \dots, n\}$ in the standard way. Pictorially,

$$[n] := \bullet^0 \longrightarrow \bullet^1 \longrightarrow \bullet^2 \longrightarrow \dots \longrightarrow \bullet^n$$

Every finite linear order, i.e. linear order on a finite set, is of the above form. That is, though the labels might change, the picture would be the same. We can make this precise when we have a notion of morphism of orders (see Definition 3.4.3.1)

Exercise 3.4.1.7. Let $S = \{1, 2, 3, 4\}$.

- Find a preorder $R \subseteq S \times S$ such that the set R is as small as possible. Is it a partial order? Is it a linear order?
- Find a preorder $R' \subseteq S \times S$ such that the set R' is as large as possible. Is it a partial order? Is it a linear order?

◇

Exercise 3.4.1.8. a.) List all the preorder relations possible on the set $\{1, 2\}$.

b.) For any $n \in \mathbb{N}$, how many linear orders exist on the set $\{1, 2, 3, \dots, n\}$.

c.) Does your formula work when $n = 0$?

◇

Remark 3.4.1.9. We can draw any preorder $(S, \leq)$ as a graph with vertices X and with an arrow $a \rightarrow b$ if $a \leq b$. These are precisely the graphs with the following two properties for any vertices $a, b \in S$:

- there is at most one arrow $a \rightarrow b$, and
- if there is a path from a to b then there is an arrow $a \rightarrow b$.

If $(S, \leq)$ is a partial order then the associated graph has an additional “no loops” property,

- if $n \in \mathbb{N}$ is an integer with $n \geq 2$ then there are no paths of length n that start at a and end at a .

If $(S, \leq)$ is a linear order then there is an additional “comparability” property,

4. for any two vertices a, b there is an arrow $a \rightarrow b$ or an arrow $b \rightarrow a$.

Given a graph G , we can create a binary relation $\leq$ on its set S of vertices as follows. Say $a \leq b$ if there is a path in G from a to b . This relation will be reflexive and transitive, so it is a preorder. If the graph satisfies Property 3 then the preorder will be a partial order, and if the graph also satisfies Property 4 then the partial order will be a linear order. Thus graphs give us a nice way to visualize orders.

Exercise 3.4.1.10. a.) Let $S = \{1, 2\}$. The subsets of S form a partial order; draw the associated graph.

b.) Repeat this for $Q = \emptyset$, $R = \{1\}$, and $T = \{1, 2, 3\}$.

c.) Do you see n -dimensional cubes?

◇

Definition 3.4.1.11. Let $(S, \leq)$ be a preorder. A *clique* is a subset $S' \subseteq S$ such that for each $a, b \in S'$ one has $a \leq b$.

Exercise 3.4.1.12. True or false: a partial order is a preorder that has no cliques. (If false, is there a “nearby” true statement?) ◇

Example 3.4.1.13. Let X be a set and $R \subseteq X \times X$ a relation. Let

$$R' := R \cup \{(x, x) \in X \times X \mid x \in X\},$$

so that R' is reflexive. Let

$$R'' := R' \cup \{(x, y) \in X \times X \mid \exists z \in X \text{ such that } (x, z) \in R', \text{ and } (z, y) \in R'\},$$

so that R'' is both transitive and reflexive, hence a preorder. We call R'' the *preorder generated by R* .

Exercise 3.4.1.14. Let X be the set of people and let $R \subseteq X \times X$ be the relation with $(x, y) \in R$ if x is the child of y . Describe the preorder generated by R . ◇

3.4.2 Meets and joins

Let X be any set. Recall from Definition 2.7.4.9 that the powerset of X , denoted $\mathbb{P}(X)$ is the set of subsets of X . There is a natural order on $\mathbb{P}(X)$ given by the subset relationship, as exemplified in Exercise 3.4.1.10. Given two elements $a, b \in \mathbb{P}(X)$ we can consider them as subsets of X and take their intersection as an element of $\mathbb{P}(X)$ which we denote $a \wedge b$. We can also consider them as subsets of X and take their union as an element of $\mathbb{P}(X)$ which we denote $a \vee b$. The intersection and union operations are generalized in the following definition.

Definition 3.4.2.1. Let $(S, \leq)$ be a preorder and let $a, b \in S$ be elements. A *meet* of a and b is an element $c \in S$ such that

- $c \leq a$ and $c \leq b$ and,
- for any $d \in S$, if $d \leq a$ and $d \leq b$ then $d \leq c$.

A *join* of a and b is an element $c \in S$ such that

- $a \leq c$ and $b \leq c$ and,
- for any $d \in S$, if $a \leq d$ and $b \leq d$ then $c \leq d$.

That is, the meet of a and b is the biggest thing smaller than both, and the join of a and b is the smallest thing bigger than both.

Exercise 3.4.2.2. Let $(S, \leq)$ be a partial order, and let $a, b \in S$ be elements. Suppose that c and c' are both meets of a and b . Then $c \leq c'$ and $c' \leq c$ so $c = c'$ by antisymmetry.

- Will the same argument style hold for joins, to show that any two joins of a and b are the same in a partial order?
- What about in a preorder—can a and b have more than one meet?

◇

Exercise 3.4.2.3. Consider the partial order from Example 3.4.1.3.

- What is the join of 'a diamond' and 'a heart'?
- What is the meet of 'a black card' and 'a queen'?
- What is the meet of 'a diamond' and 'a card'?

◇

Not every two elements in a preorder need have a meet, nor need they have a join.

Exercise 3.4.2.4. a.) Find two elements in the partial order from Example 3.4.1.3 that do not have a meet.⁷

- Find two elements that do not have a join (in that preorder).

◇

Definition 3.4.2.5. Let $\mathcal{S} := (S, \leq)$ be a preorder. The *opposite preorder*, denoted $\mathcal{S}^{\text{op}}$ is the preorder $(S, \leq^{\text{op}})$ having the same set of elements but where $s \leq^{\text{op}} s'$ iff $s' \leq s$.

Example 3.4.2.6. Recall the preorder $\mathcal{N} := (\mathbb{N}, \text{divides})$ from Exercise 4.5.1.4. Then $\mathcal{N}^{\text{op}}$ is the set of natural numbers but where $m \leq n$ iff m is a multiple of n . So $6 \leq 2$ and $6 \leq 3$.

Exercise 3.4.2.7. Suppose that $\mathcal{S} := (S, \leq)$ is an preorder.

- If $\mathcal{S}$ is a partial order, is $\mathcal{S}^{\text{op}}$ also a partial order?
- If $\mathcal{S}$ is a linear order, is $\mathcal{S}^{\text{op}}$ a linear order?

◇

3.4.3 Morphism of orders

An order $(S, \leq)$, be it a preorder, a partial order, or a linear order, involves a set and a binary relations. For two orders to be comparable, their sets and their relations should be appropriately comparable.

Definition 3.4.3.1. Let $\mathcal{S} := (S, \leq)$ and $\mathcal{S}' := (S', \leq')$ be preorders (respectively partial orders or linear orders). A *morphism of preorders* (resp. of *partial orders* or of *linear orders*) f from $\mathcal{S}$ to $\mathcal{S}'$, denoted $f: \mathcal{S} \rightarrow \mathcal{S}'$, is a function $f: S \rightarrow S'$ such that, for every pair of elements $s_1, s_2 \in S$, if $s_1 \leq s_2$ then $f(s_1) \leq' f(s_2)$.

⁷Use the displayed preorder, not any kind of “completion of what’s there”.

Example 3.4.3.2. Let X and Y be sets, let $f: X \rightarrow Y$ be a function. Then for every subset $X' \subseteq X$, its image $f(X') \subseteq Y$ is a subset (see Section 2.1.2). Thus we have a function $F: \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$, given by taking images. This is a morphism of partial orders $(\mathbb{P}(X), \subseteq) \rightarrow (\mathbb{P}(Y), \subseteq)$. Indeed, if $a \subseteq b$ in $\mathbb{P}(X)$ then $f(a) \subseteq f(b)$ in $\mathbb{P}(Y)$.

Application 3.4.3.3. It's often said that “a team is only as strong as its weakest member”. Is this true for materials? The hypothesis that a material is only as strong as its weakest constituent can be understood as follows.

Recall from the introduction to this section (see 3.4, page 84) that we can put several different orders on the set M of materials. One example there was the order given by constituency ($m \leq_C m'$ if m is an ingredient or constituent of m'). Another order is given by strength: $m \leq_S m'$ if m' is stronger than m (in some fixed setting).

Is it true that if material m is a constituent of material m' then the strength of m' is less than or equal to the strength of m ? This is the substance of our quote above. Mathematically the question would be posed, “is there a morphism of preorders $(M, \leq_C) \rightarrow (M, \leq_S^{\text{op}})$?”

◇◇

Exercise 3.4.3.4. Let X and Y be sets, let $f: X \rightarrow Y$ be a function. Then for every subset $Y' \subseteq Y$, its preimage $f^{-1}(Y') \subseteq X$ is a subset (see Definition 2.5.1.12). Thus we have a function $F: \mathbb{P}(Y) \rightarrow \mathbb{P}(X)$, given by taking preimages. Is it a morphism of partial orders?

◇

Example 3.4.3.5. Let S be a set. The smallest preorder structure that can be put on S is to say $a \leq b$ iff $a = b$. This is indeed reflexive and transitive, and it is called the *discrete preorder on S* .

The largest preorder structure that can be put on S is to say $a \leq b$ for all $a, b \in S$. This again is reflexive and transitive, and it is called the *indiscrete preorder on S* .

Exercise 3.4.3.6. Let S be a set and let $(T, \leq_T)$ be a preorder. Let $\leq_D$ be the discrete preorder on S . Given a morphism of preorders $(S, \leq_D) \rightarrow (T, \leq_T)$ we get a function $S \rightarrow T$.

- a.) Which functions $S \rightarrow T$ arise in this way?
- b.) Given a morphism of preorders $(T, \leq_T) \rightarrow (S, \leq_D)$, we get a function $T \rightarrow S$. In terms of $\leq_T$, which functions $T \rightarrow S$ arise in this way?

◇

Exercise 3.4.3.7. Let S be a set and let $(T, \leq_T)$ be a preorder. Let $\leq_I$ be the indiscrete preorder on S . Given a morphism of preorders $(S, \leq_I) \rightarrow (T, \leq_T)$ we get a function $S \rightarrow T$.

- a.) In terms of $\leq_T$, which functions $S \rightarrow T$ arise in this way?
- b.) Given a morphism of preorders $(T, \leq_T) \rightarrow (S, \leq_I)$, we get a function $T \rightarrow S$. In terms of $\leq_T$, which functions $T \rightarrow S$ arise in this way?

◇

3.4.4 Other applications

3.4.4.1 Taxonomy

Taxonomy is the science of making orders. The most commonly known such taxonomy is that of biological organisms.

Exercise 3.4.4.2. Should the taxonomy of biological organisms be a preorder, a partial order, or a linear order? Can you see any possibility of more than one decent answer? ⁸

◇

Exercise 3.4.4.3. a.) Would you guess that the taxonomic order of biological species has all meets?

b.) Would you guess it has all joins?

c.) What would a meet or a join mean?

◇

Exercise 3.4.4.4. a.) In your favorite scientific realm, are there any interesting taxonomies (or orders)?

b.) Choose one; what would meets and joins mean in that setting?

◇

3.4.4.5 Security

Security, say of sensitive information, is based on two things: a security clearance and “need to know.” The former, security clearance might have levels like “confidential”, “secret”, “top secret”. But maybe we can throw in “president” and some others too, like “plebe”.

Exercise 3.4.4.6. Does it appear that security clearance is a preorder, a partial order, or a linear order?

◇

Need-to-know is another classification of people. For each bit of information, we do not necessarily want everyone to know about it, even everyone of the specified clearance. It is only disseminated to those that need to know.

Exercise 3.4.4.7. Let P be the set of all people and let $\bar{I}$ be the set of all pieces of information known by the government. For each subset $I \subseteq \bar{I}$, let $K(I) \subseteq P$ be the set of people that need to know every piece of information in I . Let $S = \{K(I) \mid I \subseteq \bar{I}\}$ with the subset relation.

a.) Is $(P, \leq)$ a preorder? If not, find a nearby preorder.

b.) If $I_1 \subseteq I_2$ do we have $K(I_1) \subseteq K(I_2)$ or $K(I_2) \subseteq K(I_1)$ or neither?

c.) Should the preorder $(S, \leq)$ have all meets?

d.) Should it have all joins?

◇

3.4.4.8 Spaces, e.g. geography

Consider closed curves that can be drawn in the plane $\mathbb{R}^2$, e.g. circles, ellipses, and kidney-bean shaped curves. The interiors of these closed curves (not including the boundary itself) are called *basic open sets* in $\mathbb{R}^2$. The good thing about such an interior U is that any point $p \in U$ is not on the boundary, so there is a tiny little basic open set surrounding p and completely contained in U . In fact, the union of any collection of

⁸Every partial order is a preorder and every linear order is a partial order. Here we want to know whether the taxonomy is a preorder-but-not-partial-order, a partial-order-but-not-linear-order, or a linear order.

basic open sets still has this property. An *open set in $\mathbb{R}^2$* is any subset $U \subseteq \mathbb{R}^2$ that can be formed as the union of a collection of basic open sets.

Example 3.4.4.9. Let $U = \{(x, y) \in \mathbb{R}^2 \mid x > 0\}$. To see that U is open, define the following sets: for any $a, b \in \mathbb{R}$, let $S(a, b)$ be the square parallel to the axes, with side length 1, where the upper left corner is (a, b) . Let $S'(a, b)$ be the interior of $S(a, b)$. Then each $S'(a, b)$ is open, and U is the union of $S'(a, b)$ over the collection of all $a > 0$ and all b ,

$$U = \bigcup_{\substack{a, b \in \mathbb{R}, \\ a > 0}} S'(a, b).$$

The idea of open sets extends to spaces beyond $\mathbb{R}^2$. For example, on the earth one could define a basic open set to be the interior of any region one can “draw a circle around” (with a metaphorical pen), and define open sets to be unions of basic open sets.

Exercise 3.4.4.10. Let S be the set of open subsets on earth, as defined in the above paragraph.

- a.) If $\leq$ is the subset relation, is $(S, \leq)$ a preorder or a partial order?
- b.) Does it have meets, does it have joins?

◇

Exercise 3.4.4.11. Let S be the set of open subsets of earth as defined above. To each open subset of earth suppose we know the range of recorded temperature throughout s (i.e. the low and high throughout the region). Thus to each element $s \in S$ we assign an interval $T(s) := \{x \in \mathbb{R} \mid a \leq x \leq b\}$. If we order the set V of intervals of $\mathbb{R}$ by the subset relation, it gives a partial order on V .

- a.) Does our assignment $T: S \rightarrow V$ amount to a morphism of orders?
- b.) Does it preserve meets or joins? (Hint: it doesn’t preserve both.)

◇

Exercise 3.4.4.12. a.) Can you think of a space relevant to your favorite area of science for which it makes sense to assign an interval of real numbers to each open set somehow, analogously to Exercise 3.4.4.11? For example for a sample of some material under stress, perhaps the strain on each open set is somehow an interval?

- b.) Repeat the questions from Exercise 3.4.4.11.

◇

3.5 Databases: schemas and instances

The first three sections of this chapter were about classical objects from mathematics. The present section is about databases, which are classical objects from computer science. These are truly “categories and functors, without admitting it” (see Theorem 4.4.2.3).

3.5.1 What are databases?

Data, in particular the set of observations made during experiment, plays ⁹ a primary role in science of any kind. To be useful data must be organized, often in a row-and-

⁹The word data is generally considered to be the plural form of the word datum. However, individual datum elements are only useful when they are organized into structures (e.g. if one were to shuffle the

column display called a table. Columns existing in different tables can refer to the same data.

A database is a collection of tables, each table T of which consists of a set of columns and a set of rows. We roughly explain the role of tables, columns, and rows as follows. The existence of table T suggests the existence of a fixed methodology for observing objects or events of a certain type. Each column c in T prescribes a single kind or method of observation, so that the datum inhabiting any cell in column c refers to an observation of that kind. Each row r in T has a fixed sourcing event or object, which can be observed using the methods prescribed by the columns. The cell (r, c) refers to the observation of kind c made on event r . All of the rows in T should refer to uniquely identifiable objects or events of a single type, and the name of the table T should refer to that type.

Example 3.5.1.1. When graphene is strained (lengthened by a factor of $x \geq 1$), it becomes stressed (carries a force in the direction of the lengthening). The following is a made-up set of data.

Graphene sample				Supplier		
ID	Source	Stress	Strain	ID	Full name	Phone
A118-1	C Smkt	0	0	C Smkt	Carbon Supermarket	(541)781-6611
A118-2	C Smkt	0.02	20	AC	Advanced Chemical	(410) 693-0818
A118-3	C Smkt	0.05	40	C Plat	Carbon Platform	(510) 719-2857
A118-4	AC	0.04	37	McD	McDonard's Burgers	(617) 244-4400
A118-5	AC	0.1	80	APP	Acme Pen and Paper	(617) 823-5603
A118-6	C Plat	0.1	82			

(3.10)

In the first table, titled “Graphene sample”, the rows refer to graphene samples, and the table is so named. Each graphene sample can be observed according to the source supplier from which it came, the strain that it was subjected to, and the stress that it carried. These observations are the columns. In the second table, the rows refer to suppliers of various things, and the table is so named. Each supplier can be observed according to its full name and its phone number; these are the columns.

In the left-hand table it appears either that each graphene sample was used only once, or that the person recording the data did not keep track of which samples were reused. If such details become important later, the lab may want to change the layout of the first table by adding on the appropriate column. This can be accomplished using morphisms of schemas, which will be discussed in Section 4.4.1.

3.5.1.2 Primary keys, foreign keys, and data columns

There is a bit more structure in the above tables (Example 3.10) than may first meet the eye. Each table has a *primary ID column*, found on the left, as well as some *data columns* and some *foreign key columns*. The primary key column is tasked with uniquely identifying different rows. Each data column houses elementary data of a certain sort. Perhaps most interesting from a structural point of view are the foreign key columns, because they link one table to another, creating a connection pattern between tables.

cells in a spreadsheet, most would consider the data to be destroyed). It is the whole organized structure that really houses the information; the data must be in formation in order to be useful. Thus I will use the word *data* as a collective noun (akin to the word “sand”); it bridges the divide between the *individual datum elements* (akin to the grains of sand) and the *data set* (akin to a sand pile). In particular, I will often use the word *data* as a singular noun.

Each foreign key column houses data that needs to be further unpacked. It thus refers us to another *foreign* table, in particular the primary ID column of that table. In Example 3.10 the **Source** column was a foreign key to the **Supplier** table.

Here is another example, lifted from [Sp2].

Example 3.5.1.3. Consider the bookkeeping necessary to run a department store. We keep track of a set of employees and a set of departments. For each employee e , we keep track of

- E.1 the **first** name of e , which is a **FirstNameString**,
- E.2 the **last** name of e , which is a **LastNameString**,
- E.3 the **manager** of e , which is an **Employee**, and
- E.4 the department that e **works in**, which is a **Department**.

For each department d , we keep track of

- D.1 the **name** of d , which is a **DepartmentNameString**, and
- D.2 the **secretary** of d , which is an **Employee**.

Above we can suppose that E.1, E.2, and D.1 are data columns (referring to names of various sorts), and E.3, E.4, and D.2 are foreign key columns (referring to managers, secretaries, etc.).

Display (3.11) shows how such a database might look at a particular moment in time.

Employee					Department		
ID	first	last	manager	worksIn	ID	name	secretary
101	David	Hilbert	103	q10	q10	Sales	101
102	Bertrand	Russell	102	x02	x02	Production	102
103	Emmy	Noether	103	q10			

(3.11)

3.5.1.4 Business rules

Looking at the tables from Example 3.5.1.3, one may notice a few patterns. First, every employee works in the same department as his or manager. Second, every department's secretary works in that department. Perhaps the business counts on these rules for the way it structures itself. In that case the database should enforce those rules, i.e. it should check that whenever the data is updated, it conforms to the rules:

Rule 1 For every employee e , the **manager** of e **works in** the same department that e **works in**.

Rule 2 For every department d , the **secretary** of d **works in** department d .

(3.12)

3.5.1.5 Data columns as foreign keys

To make everything consistent, we could even say that data columns are specific kinds of foreign keys. That is, each data column constitutes a foreign key to some non-branching *leaf table*, which has no additional data.

Example 3.5.1.6. Consider again Example 3.5.1.3. Note that first names and last names had a particular type, which we all but ignored above. We could cease to ignore them by adding three tables, as follows.

FirstNameString	LastNameString	DepartmentNameString
ID	ID	ID
Alan	Arden	Marketing
Alice	Hilbert	Production
Bertrand	Jones	Sales
Carl	Noether	
David	Russell	
Emmy		
⋮	⋮	⋮

(3.13)

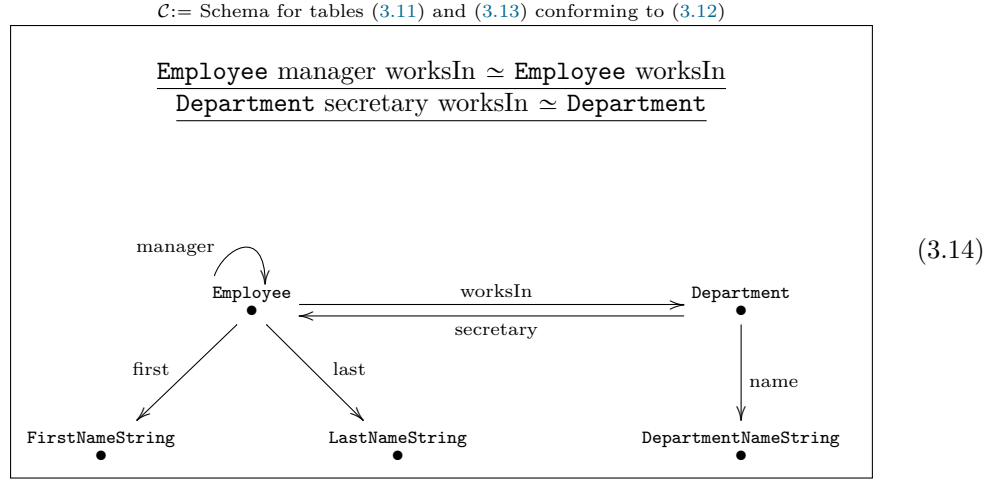
In combination, Displays (3.11) and (3.13) form a collection of tables with the property that every column is either a primary key or a foreign key. The notion of data column is now subsumed under the notion of foreign key column. Everything is either a primary key (one per table, labeled ID) or a foreign key column (everything else).

3.5.2 Schemas

The above section may all seem intuitive or reasonable in some ways, but also a bit difficult to fully grasp, perhaps. It would be nice to summarize what is happening in a picture. Such a picture, which will basically be a graph, should capture the *conceptual layout* to which the data conforms, without yet being concerned with the individual data that may populate the tables in this instant. We proceed at first by example, giving the precise definition in Definition 3.5.2.6.

Example 3.5.2.1. In Examples 3.5.1.3 and 3.5.1.6, the conceptual layout for a department store was given, and some example tables were shown. We were instructed to keep track of employees, departments, and six types of data (E.1, E.2, E.3, E.4, D.1, and D.2), and we were instructed to follow two rules (Rule 1, Rule 2). All of this is summarized in the

following picture:



The five tables from (3.11) and (3.13) are seen as five vertices; this is also the number of primary ID columns. The six foreign key columns from (3.11) and (3.13) are seen as six arrows; each points from a table to a foreign table. The two rules from (3.12) are seen as statements at the top of Display (3.14). We will explain path equivalences in Definition 3.5.2.3.

Exercise 3.5.2.2. Come up with a schema (consisting of dots and arrows) describing the conceptual layout of information presented in Example 3.5.1.1. $\diamond$

In order to define schemas, we must first define the notion of schematic equivalence relation, which is to hold on the set of paths of a graph G (see Section 3.3.2). Such an equivalence relation (in addition to being reflexive, symmetric, and transitive) has two sorts of additional properties: equivalent paths must have the same source and target, and the composition of equivalent paths with other equivalent paths must yield equivalent paths. Formally we have Definition 3.5.2.3.

Definition 3.5.2.3.

Let $G = (V, A, src, tgt)$ be a graph. A *path equivalence declaration* (or *PED*) is a phrase of the form $p \simeq q$ where $p, q \in \text{Path}_G$ have the same source and target, $src(p) = src(q)$ and $tgt(p) = tgt(q)$.

A *categorical path equivalence relation* (or *CPER*) on G is an equivalence relation $\simeq$ on Path_G that has the following properties:

1. If $p \simeq q$ then $src(p) = src(q)$.
2. If $p \simeq q$ then $tgt(p) = tgt(q)$.
3. Suppose $p, q: b \rightarrow c$ are paths, and $m: a \rightarrow b$ is an arrow. If $p \simeq q$ then $mp \simeq mq$.
4. Suppose $p, q: a \rightarrow b$ are paths, and $n: b \rightarrow c$ is an arrow. If $p \simeq q$ then $pn \simeq qn$.

Any set of categorical path equivalence declarations generates a categorical path equivalence relation (CPER). We tend to elide the difference between CPERs and sets of PEDs.

Exercise 3.5.2.4. Consider the graph shown in (3.14), and the two declarations shown at the top. They generate a CPER.

a.) Is it true that under this CPER we have

$$\text{Employee manager manager worksIn} \simeq \text{Employee worksIn?}$$

b.) What about

$$\text{Employee worksIn secretary} \simeq \text{Employee?}$$

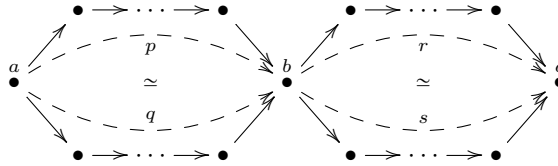
c.) What about

$$\text{Department secretary manager worksIn name} \simeq \text{Department name?}$$

◇

Lemma 3.5.2.5. *Suppose that G is a graph and $\simeq$ is a CPER on G . Suppose $p \simeq q: a \rightarrow b$ and $r \simeq s: b \rightarrow c$. Then $pr \simeq qs$.*

Proof. The picture to have in mind is this:



Applying condition (3) from Definition 3.5.2.3 to each arrow in path p , it follows by induction that $pr \simeq ps$. Applying condition (4) to each arrow in path s , it follows similarly that $ps \simeq qs$. Because $\simeq$ is an equivalence relation, it follows that $pr \simeq qs$. □

Definition 3.5.2.6. A *database schema* (or simply *schema*) $\mathcal{C}$ consists of a pair $\mathcal{C} := (G, \simeq)$ where G is a graph and $\simeq$ is a categorical path equivalence relation (CPER) on G .

Example 3.5.2.7. The picture drawn in (3.14) has the makings of a schema. Pictured is a graph with two PEDs; these generate a CPER, as discussed in Exercise 3.5.2.4.

A schema can be converted into a system of tables each with a primary key and some number of foreign keys referring to other tables, as discussed in Section 3.5.1. Definition 3.5.2.6 gives a precise conceptual understanding of what a schema is, and the following rules describe how to convert such a thing into a table layout.

Rules of good practice 3.5.2.8. Converting a schema $\mathcal{C} = (G, \simeq)$ into a table layout should be done as follows:

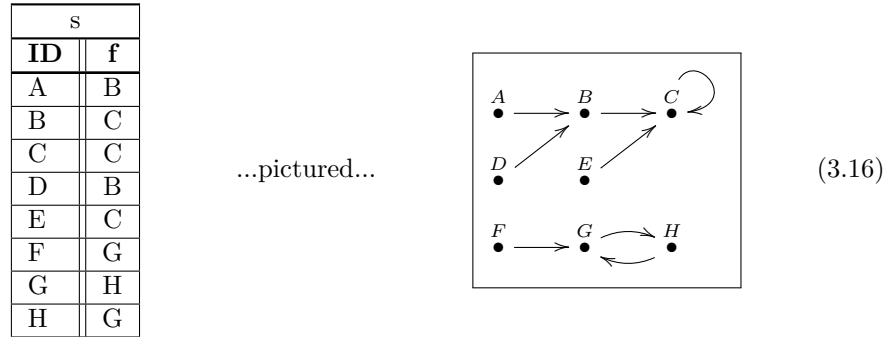
- (i) There should be a table for every vertex in G and if the vertex is named, the table should have that name;
- (ii) Each table should have a left-most column called ID, set apart from the other columns by a double vertical line; and
- (iii) To each arrow a in G having source vertex $s := \text{src}(a)$ and target vertex $t := \text{tgt}(a)$, there should be a foreign key column a in table s , referring to table t ; if the arrow a is named, column a should have that name.

Example 3.5.2.9. [Discrete dynamical system]

Consider the schema

$$\mathcal{Loop} := \boxed{\begin{array}{c} f \\ \curvearrowright \\ s \end{array}} \quad (3.15)$$

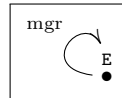
in which the CPER is trivial (i.e. generated by the empty set of PEDs.) This schema is quite interesting. It encodes a set s and a function $f: s \rightarrow s$. Such a thing is called a *discrete dynamical system*. One imagines s as the set of states and, for any state $x \in s$, a notion of “next state” $f(x) \in s$. For example



Application 3.5.2.10. Imagine a **quantum-time** universe in which there are discrete time steps. We model it as a discrete dynamical system, i.e. a table of the form (3.16). For every possible state of the universe we include a row in the table. The state in the next instant is recorded in the second column.

◇◇

Example 3.5.2.11 (Finite hierarchy). The schema $\mathcal{Loop}$ can also be used to encode hierarchies, such as the manager relation from Examples 3.5.1.3 and 3.5.2.1,



One problem with this, however, is if a schema has even one loop, then it can have infinitely many paths (corresponding, e.g. to an employee's manager's manager's manager's ... manager).

Sometimes we know that in a given company that process eventually ends, a famous example being that at Ben and Jerry's ice cream, there were only seven levels. In that case we know that an employee's 8th level manager is equal to his or her 7th level manager. This can be encoded by the PED

$$E \text{ mgr mgr mgr mgr mgr mgr mgr mgr } \simeq E \text{ mgr mgr mgr mgr mgr mgr mgr mgr}$$

or more concisely, $\text{mgr}^8 = \text{mgr}^7$.

Exercise 3.5.2.12. Is there any nontrivial PED on $\mathcal{Loop}$ that holds for the data in Example 3.5.2.9? If so, what is it and how many equivalence classes of paths in $\mathcal{Loop}$ are there after you impose that relation? ◇

Exercise 3.5.2.13. Let P be a chess-playing program. Given any position (including the choice of whose turn it is), P will make a move.

- a.) Is this an example of a discrete dynamical system?
- b.) How do the rules for ending the game in a win or draw play out in this model? (Look up online how chess games end if you don't know.)

◇

3.5.2.14 Ologging schemas

It should be clear that a database schema is nothing but an olog in disguise. The difference is basically the readability requirements for ologs. There is an important new addition in this section, namely that we can fill out an olog with data. Conversely, we have seen that databases are not any harder to understand than ologs are.

Example 3.5.2.15. Consider the olog

$$\boxed{\text{a moon}} \xrightarrow{\text{orbits}} \boxed{\text{a planet}} \quad (3.17)$$

We can document some instances of this relationship using the following tables:

orbits	
a moon	a planet
The Moon	Earth
Phobos	Mars
Deimos	Mars
Ganymede	Jupiter
Titan	Saturn

(3.18)

Clearly, this table of instances can be updated as more moons are discovered by the author (be it by telescope, conversation, or research).

Exercise 3.5.2.16. In fact, Example 3.5.2.15 did not follow Rules 3.5.2.8. Strictly following those rules, copy over the data from (3.18) into tables that are in accordance with schema (3.17). ◇

Exercise 3.5.2.17. a.) Write down a schema, in terms of the boxes 'a thing I own' and 'a place' and one additional arrow, that might help one remember where they decided to put "random" things.

b.) What is a good label for the arrow?

c.) Fill in some rows of the corresponding set of tables for your own case.

◇

Exercise 3.5.2.18. Consider the olog

$$\begin{array}{ccc} F & & C \\ \boxed{\text{a father}} & \xrightarrow{\text{has as first}} & \boxed{\text{a child}} \\ & \xleftarrow{\text{has}} & \end{array}$$

What path equivalence declarations would be appropriate for this olog? You can use $f: F \rightarrow C$ and $h: C \rightarrow F$ if you prefer. ◇

3.5.3 Instances

Given a database schema $(G, \simeq)$, an instance of it is just a bunch of tables whose data conform to the specified layout. These can be seen throughout the previous section, most explicitly in the relationship between schema (3.14) and tables (3.11) and (3.13), and between schema (3.15) and table (3.16). Below is the mathematical definition.

Definition 3.5.3.1. Let $\mathcal{C} = (G, \simeq)$ where $G = (V, A, src, tgt)$. An *instance* on $\mathcal{C}$, denoted $(PK, FK): \mathcal{C} \rightarrow \mathbf{Set}$, consists of some constituents (A. primary ID part, B. foreign key part) conforming to a law (1. preservation of CPER), as follows:

- A. a function $PK: V \rightarrow \mathbf{Set}$; i.e. to each vertex $v \in V$ we assign a set $PK(v)$;¹⁰ and
- B. for every arrow $a \in A$ with $v = src(a)$ and $w = tgt(a)$, a function $FK(a): PK(v) \rightarrow PK(w)$.¹¹

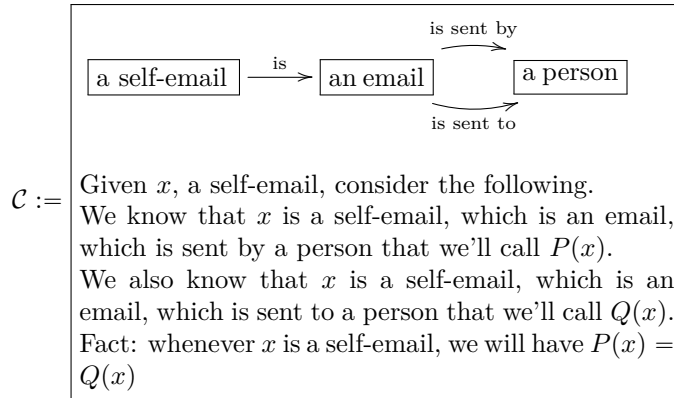
The following law holds for any vertices v, w and paths $p = [a_1, a_2, \dots, a_m]$ and $q = [a'_1, a'_2, \dots, a'_n]$ from v to w :

1. If $p \simeq q$ then for all $x \in PK(v)$, we have

$$FK(a_m) \circ \dots \circ FK(a_2) \circ FK(a_1)(x) = FK(a'_n) \circ \dots \circ FK(a'_2) \circ FK(a'_1)(x)$$

in $PK(w)$.

Exercise 3.5.3.2. Consider the olog pictured below:



a self-email	
ID	is
SEm1207	Em1207
SEm1210	Em1210
SEm1211	Em1211

an email		
ID	is sent by	is sent to
Em1206	Bob	Sue
Em1207	Carl	Carl
Em1208	Sue	Martha
Em1209	Chris	Bob
Em1210	Chris	Chris
Em1211	Julia	Julia
Em1212	Martha	Chris

a person	
ID	
Bob	
Carl	
Chris	
Julia	
Martha	
Sue	

(3.19)

¹⁰The elements of $PK(v)$ will be listed as the rows of table v , or more precisely as the leftmost cells of these rows.

¹¹The arrow a will correspond to a column, and to each row $r \in PK(v)$ the (r, a) cell will contain the datum $FK(a)(r)$.

- a.) What is the set $\text{PK}(\text{'an email'})$?
- b.) What is the set $\text{PK}(\text{'a person'})$?
- c.) What is the function $\text{FK}(\text{is sent by}): \text{PK}(\text{'an email'}) \rightarrow \text{PK}(\text{'a person'})$?
- d.) Interpret the Englishification mess at the bottom of $\mathcal{C}$ as a simple path equivalence declaration. Is it satisfied by the instance (3.19); that is, does law 1. from Definition 3.5.3.1 hold?

◇

Example 3.5.3.3. [Monoid action table]

In Example 3.1.2.9, we saw how a monoid $\mathcal{M}$ could be captured as an olog with only one object. As a database schema, this means there is only one table. Every generator of $\mathcal{M}$ would be a column of the table. The notion of database instance for such a schema is precisely the notion of action table from Section 3.1.3.

Exercise 3.5.3.4. Draw (as a graph) the schema for which Table 3.1.3.1 is an instance. ◇

Exercise 3.5.3.5. Suppose that $\mathcal{M}$ is a monoid and some instance of it is written out in table form. It's possible that $\mathcal{M}$ is a group. What evidence in an instance table for $\mathcal{M}$ might suggest that $\mathcal{M}$ is a group? ◇

3.5.3.6 Paths through a database

Let $\mathcal{C} := (G, \simeq)$ be a schema and let $(\text{PK}, \text{FK}): \mathcal{C} \rightarrow \mathbf{Set}$ be an instance on $\mathcal{C}$. Then for every arrow $a: v \rightarrow w$ in G we get a function $\text{FK}(a): \text{PK}(v) \rightarrow \text{PK}(w)$. Functions can be composed, so in fact for every path through G we get a function. Namely, if $p = [a_1, a_2, \dots, a_n]$ is a path from v_0 to v_n then the instance provides a function

$$\text{FK}(p) := \text{FK}(a_n) \circ \dots \circ \text{FK}(a_2) \circ \text{FK}(a_1): \text{PK}(v_0) \rightarrow \text{PK}(v_n),$$

which first made an appearance as part of Law 1 in Definition 3.5.3.1.

Example 3.5.3.7. Consider the department store schema from Example 3.5.2.1, and in (3.14) the path $[\text{worksIn}, \text{secretary}, \text{last}]$ which points from **Employee** to **LastNameString**. The instance will let us interpret this path as a function from the set of employees to the set of last names; this could be a useful function to have around. The instance from (3.11) would yield the following function

Employee	
ID	Secr. name
101	Hilbert
102	Russell
103	Hilbert

Exercise 3.5.3.8. Consider the path $p := [f, f]$ on the $\mathcal{Loop}$ schema from (3.15). Using the instance from (3.16), where $\text{PK}(s) = \{A, B, C, D, E, F, G, H\}$, interpret p as a function $\text{PK}(s) \rightarrow \text{PK}(s)$, and write this as a 2-column table, as above in Example 3.5.3.7. ◇

Exercise 3.5.3.9. a.) Given an instance (PK, FK) on a schema $\mathcal{C}$, and given a trivial path p (i.e. p has length 0; it starts at some vertex but doesn't go anywhere), what function does p yield?

b.) What are its domain and codomain?

◇

Chapter 4

Basic category theory

Categories, or an equivalent notion, have already been secretly introduced as ologs. One can think of a category as a graph (as in Section 3.3) in which certain paths have been declared equivalent. (Ologs demand an extra requirement that everything in sight be readable in natural language, and this cannot be part of the mathematical definition of category.) The formal definition of category is given in Definition 4.1.1.1, but it will not be obviously the same as the “graph+path equivalences” notion; the latter was given in Definition 3.5.2.6 as the definition of a *schema*. Once we talk about how different categories can be compared using functors (Definition 4.1.2.1), and how different schemas can be compared using schema mappings (Definition 4.4.1.2), we will prove that the two notions are equivalent (Theorem 4.4.2.3).

4.1 Categories and Functors

In this section we give the standard definition of categories and functors. These, together with natural transformations (Section 4.3), form the backbone of category theory. We also give some examples.

4.1.1 Categories

Definition 4.1.1.1. A *category* $\mathcal{C}$ consists of some constituents (A. objects, B. morphisms, C. identities, D. compositions) conforming to some laws (1. identity law, 2. associativity law), as follows:

- A. a collection $\text{Ob}(\mathcal{C})$, elements of which are called *objects*;
- B. for every pair $x, y \in \text{Ob}(\mathcal{C})$, a set $\text{Hom}_{\mathcal{C}}(x, y) \in \mathbf{Set}$. It is called the *hom-set from x to y* ; its elements are called *morphisms from x to y* ;
- C. for every object $x \in \text{Ob}(\mathcal{C})$, a specified morphism denoted $\text{id}_x \in \text{Hom}_{\mathcal{C}}(x, x)$ called *the identity morphism on x* ; and
- D. for every three objects $x, y, z \in \text{Ob}(\mathcal{C})$ a function

$$\circ: \text{Hom}_{\mathcal{C}}(y, z) \times \text{Hom}_{\mathcal{C}}(x, y) \rightarrow \text{Hom}_{\mathcal{C}}(x, z),$$

called *the composition formula*.

Given objects $x, y \in \text{Ob}(\mathcal{C})$, we can denote a morphism $f \in \text{Hom}_{\mathcal{C}}(x, y)$ by $f: x \rightarrow y$; we say that x is the *domain* of f and that y is the *codomain* of f . Given also $g: y \rightarrow z$, the composition formula is written using infix notation, so $g \circ f: x \rightarrow z$ means $\circ(g, f) \in \text{Hom}_{\mathcal{C}}(x, z)$.

Here are the laws that harmonize the above constituents:

1. for every $x, y \in \text{Ob}(\mathcal{C})$ and every morphism $f: x \rightarrow y$, we have

$$f \circ \text{id}_x = f \quad \text{and} \quad \text{id}_y \circ f = f;$$

and;

2. if $w, x, y, z \in \text{Ob}(\mathcal{C})$ are any objects and $f: w \rightarrow x$, $g: x \rightarrow y$, and $h: y \rightarrow z$ are any morphisms, then the two ways to compose are the same:

$$(h \circ g) \circ f = h \circ (g \circ f) \in \text{Hom}_{\mathcal{C}}(w, z).$$

Remark 4.1.1.2. There is perhaps much that is unfamiliar about Definition 4.1.1.1 but there is also one thing that is strange about it. The objects $\text{Ob}(\mathcal{C})$ of $\mathcal{C}$ are said to be a “collection” rather than a set. This is because we sometimes want to talk about the category of all sets, in which the objects are the sets, and the collection of sets is not a set. Modeling this was a sticking point in the foundations of category theory, but it was eventually fixed by Grothendieck’s notion of expanding universes. Roughly the idea is to choose some huge set κ (with certain properties making it a *universe*), to work entirely inside of it when possible, and to call anything in that world κ -small (or just *small* if κ is clear from context). When we need to look at κ itself, we choose an even bigger universe κ' and work entirely within it.

A category in which the collection $\text{Ob}(\mathcal{C})$ is a set (or in the above language, a small set) is called a *small category*. From here on out we will not take care of the difference, referring to $\text{Ob}(\mathcal{C})$ as a set. We do not think this will do any harm to scientists using category theory, at least not in the beginning phases of their learning.

Example 4.1.1.3 (The category **Set** of sets). Chapter 2 was all about the category of sets, denoted **Set**. The objects are the sets and the morphisms are the functions; we even used the current notation, referring to the set of functions $X \rightarrow Y$ as $\text{Hom}_{\mathbf{Set}}(X, Y)$. The composition formula $\circ$ is given by function composition, and for every set X , the identity function $\text{id}_X: X \rightarrow X$ serves as the identity morphism for $X \in \text{Ob}(\mathbf{Set})$. The two laws clearly hold, so **Set** is indeed a category.

Example 4.1.1.4. [The category **Mon** of monoids]

We defined monoids in Definition 3.1.1.1 and monoid homomorphisms in Definition 3.1.4.1. Every monoid $\mathcal{M} := (M, 1_M, \star_M)$ ¹ has an identity homomorphism $\text{id}_{\mathcal{M}}: \mathcal{M} \rightarrow \mathcal{M}$, given by the identity function $\text{id}_M: M \rightarrow M$. To compose two monoid homomorphisms $f: \mathcal{M} \rightarrow \mathcal{M}'$ and $g: \mathcal{M}' \rightarrow \mathcal{M}''$, we compose their underlying functions $f: M \rightarrow M'$ and $g: M' \rightarrow M''$, and check that the result $g \circ f$ is a monoid homomorphism. Indeed,

$$g \circ f(1_M) = g(1_{M'}) = 1_{M''}$$

$$g \circ f(m_1 \star_M m_2) = g(f(m_1) \star_{M'} f(m_2)) = g \circ f(m_1) \star_{M''} g \circ f(m_2).$$

It is clear that the two laws hold, so **Mon** is a category.

¹Note, we changed the notation for our monoid because the symbol id was overloaded.

Exercise 4.1.1.5 (The category **Grp** of groups). Suppose we set out to define a category **Grp**, having groups as objects and group homomorphisms as morphisms, see Definition 3.2.1.16. Show (to the level of detail of Example 4.1.1.4) that the rest of the conditions for **Grp** to be a category are satisfied. $\diamond$

Exercise 4.1.1.6 (The category **PrO** of preorders). Suppose we set out to define a category **PrO**, having preorders as objects and preorder homomorphisms as morphisms (see Definition 3.4.3.1). Show (to the level of detail of Example 4.1.1.4) that the rest of the conditions for **PrO** to be a category are satisfied. $\diamond$

Example 4.1.1.7. [The category of preorders with joins]

Suppose that we are only interested in preorders $(X, \leq)$ for which every pair of elements has a join. We saw in Exercise 3.4.2.4 that not all preorders have this property. However we can create a category $\mathcal{C}$ in which every object does have this property. To begin we put $\text{Ob}(\mathcal{C}) = \{(X, \leq) \in \text{Ob}(\mathbf{PrO}) \mid (X, \leq) \text{ has all joins}\}$. But what about morphisms?

One option would be to put in no morphisms (other than identities), and to just consider this collection of objects as having no structure other than a set.

Another option would be to put in exactly the same morphisms as in **PrO**: for any objects $a, b \in \text{Ob}(\mathcal{C})$ we consider a and b as regular old preorders, and put $\text{Hom}_{\mathcal{C}}(a, b) := \text{Hom}_{\mathbf{PrO}}(a, b)$. The resulting category of preorders with joins is called the *full subcategory of **PrO** spanned by the preorders with joins*.

A third option, and the one perhaps that would jump out to a category theorist, is to take the choice about how we define our objects as a clue to how we should define our morphisms. Namely, if we are so interested in joins, perhaps we want joins to be preserved under morphisms. That is, if $f: (X, \leq_X) \rightarrow (Y, \leq_Y)$ is a morphism of preorders then for any join $w = x \vee x'$ in X we might want to enforce that $f(w) = f(x) \vee f(x')$ in Y . Thus a third possibility for the morphisms of $\mathcal{C}$ would be

$$\text{Hom}_{\mathcal{C}}(a, b) := \{f \in \text{Hom}_{\mathbf{PrO}}(a, b) \mid f \text{ preserves joins}\}.$$

One can check easily that the identity morphisms preserve joins and that compositions of join-preserving morphisms are join-preserving, so this version of homomorphisms makes for a well-defined category.

Exercise 4.1.1.8 (Category **FLin** of finite linear orders). We have a category **PrO** of preorders, and some of its objects are finite (nonempty) linear orders. Let **FLin** be the full subcategory of **PrO** spanned by the linear orders. That is, following Definition 3.4.3.1, given linear orders X, Y , every morphism of preorders $X \rightarrow Y$ counts as a morphism in **FLin**:

$$\text{Hom}_{\mathbf{FLin}}(X, Y) = \text{Hom}_{\mathbf{PrO}}(X, Y).$$

For $n \in \mathbb{N}$, let $[n]$ be the linear order defined in Example 3.4.1.6. What are the cardinalities of the following sets:

- a.) $\text{Hom}_{\mathbf{FLin}}([0], [3]);$
- b.) $\text{Hom}_{\mathbf{FLin}}([3], [0]);$
- c.) $\text{Hom}_{\mathbf{FLin}}([2], [3]);$
- d.) $\text{Hom}_{\mathbf{FLin}}([1], [n]);$
- e.) (Challenge) $\text{Hom}_{\mathbf{FLin}}([m], [n]);$

It turns out that the category **FLin** of linear orders is sufficiently rich that much of algebraic topology (the study of arbitrary spaces, such as Mobius strips and 7-dimensional spheres) can be understood in its terms. See Example 4.6.1.6. $\diamond$

Example 4.1.1.9 (Category of graphs). We defined graphs in Definition 3.3.1.1 and graph homomorphisms in Definition 3.3.3.1. To see that these are sufficient to form a category, is routine. Since a morphism from $\mathcal{G} = (V, A, \text{src}, \text{tgt})$ to $\mathcal{G}' = (V', A', \text{src}', \text{tgt}')$ involves two functions $f_0: V \rightarrow V'$ and $f_1: A \rightarrow A'$, the identity and composition formulas will simply arise from the identity and composition formulas for sets. Associativity will follow similarly. The only thing that needs to be checked, really, is that the composition of two such things, each satisfying (3.5), will itself satisfy (3.5). Just for completeness, we check that now.

Suppose that $f = (f_0, f_1): \mathcal{G} \rightarrow \mathcal{G}'$ and $g = (g_0, g_1): \mathcal{G}' \rightarrow \mathcal{G}''$ are graph homomorphisms, where $\mathcal{G}'' = (V'', A'', \text{src}'', \text{tgt}'')$. Then in each diagram below

$$\begin{array}{ccccc} A & \xrightarrow{f_1} & A' & \xrightarrow{g_1} & A'' \\ \downarrow \text{src} & & \downarrow \text{src}' & & \downarrow \text{src}'' \\ V & \xrightarrow{f_0} & V' & \xrightarrow{g_0} & V'' \end{array} \qquad \begin{array}{ccccc} A & \xrightarrow{f_1} & A' & \xrightarrow{g_1} & A'' \\ \downarrow \text{tgt} & & \downarrow \text{tgt}' & & \downarrow \text{tgt}'' \\ V & \xrightarrow{f_0} & V' & \xrightarrow{g_0} & V'' \end{array} \quad (4.1)$$

the left-hand square commutes because f is a graph homomorphism and the right-hand square commutes because g is a graph homomorphism. Thus the whole rectangle commutes, meaning that $g \circ f$ is a graph homomorphism, as desired.

We denote the category of graphs and graph homomorphisms by **Grph**.

Exercise 4.1.1.10. Let F be a vector field on $\mathbb{R}^2$. Recall that for two points $x, x' \in \mathbb{R}^2$, any curve C with endpoints x and x' , and any parameterization $r: [a, b] \rightarrow C$, the line integral $\int_C F(r) \cdot dr$ returns a real number. It does not depend on r , except its orientation (direction). Therefore, if we think of C as having an orientation, say going from x to x' , then $\int_C F$ is a well-defined real number. If C goes from x to x' , let's suggestively write $C: x \rightarrow x'$. Define an equivalence relation $\sim$ on the set of oriented curves in $\mathbb{R}^2$ by saying $C \sim C'$ if

- C and C' start at the same point,
- C and C' end at the same point, and
- $\int_C F = \int_{C'} F$.

Suppose we try to make a category $\mathcal{C}_F$ as follows. Put $\text{Ob}(\mathcal{C}_F) = \mathbb{R}^2$, and for every pair of points $x, x' \in \mathbb{R}^2$, let $\text{Hom}_{\mathcal{C}_F}(x, x') = \{C: x \rightarrow x'\} / \sim$, where $C: x \rightarrow x'$ is an oriented curve and $\sim$ means “same line integral”, as explained above.

Is there an identity morphism and a composition formula that will make $\mathcal{C}_F$ into a category? $\diamond$

4.1.1.11 Isomorphisms

In any category we have a notion of isomorphism between objects.

Definition 4.1.1.12. Let $\mathcal{C}$ be a category and let $X, Y \in \text{Ob}(\mathcal{C})$ be objects. An *isomorphism* f from X to Y is a morphism $f: X \rightarrow Y$ in $\mathcal{C}$, such that there exists a morphism $g: Y \rightarrow X$ in $\mathcal{C}$ such that

$$g \circ f = \text{id}_X \quad \text{and} \quad f \circ g = \text{id}_Y.$$

In this case we say that the morphism f is *invertible* and that g is the *inverse* of f . We may also say that the objects X and Y are *isomorphic*.

Example 4.1.1.13. If $\mathcal{C} = \mathbf{Set}$ is the category of sets, then the above definition coincides precisely with the one given in Definition 2.1.2.7.

Exercise 4.1.1.14. Suppose that $G = (V, A, \text{src}, \text{tgt})$ and $G' = (V', A', \text{src}', \text{tgt}')$ are graphs and that $f = (f_0, f_1): G \rightarrow G'$ is a graph homomorphism (as in Definition 3.3.3.1).

- a.) If f is an isomorphism in **Grph**, does this imply that $f_0: V \rightarrow V'$ and $f_1: A \rightarrow A'$ are isomorphisms in **Set**?
- b.) If so, why; and if not, show a counterexample (where f is an isomorphism but either f_0 or f_1 is not).

◇

Exercise 4.1.1.15. Suppose that $G = (V, A, \text{src}, \text{tgt})$ and $G' = (V', A', \text{src}', \text{tgt}')$ are graphs and that $f = (f_0, f_1): G \rightarrow G'$ is a graph homomorphism (as in Definition 3.3.3.1).

- a.) If $f_0: V \rightarrow V'$ and $f_1: A \rightarrow A'$ are isomorphisms in **Set**, does this imply that f is an isomorphism in **Grph**?
- b.) If so, why; and if not, show a counterexample (where f_0 and f_1 are isomorphisms but f is not).

◇

Lemma 4.1.1.16. *Let $\mathcal{C}$ be a category and let $\sim$ be the relation on $\text{Ob}(\mathcal{C})$ given by saying $X \sim Y$ iff X and Y are isomorphic. Then $\sim$ is an equivalence relation.*

Proof. The proof of Lemma 2.1.2.11 can be mimicked in this more general setting.

□

4.1.1.17 A more formal viewpoint on categories

Here is an alternate definition of category, using the work we did in Chapter 2.

Exercise 4.1.1.18. Suppose we begin our definition of category as follows.

A *category*, $\mathcal{C}$ consists of a sequence $(\text{Ob}(\mathcal{C}), \text{Hom}_{\mathcal{C}}, \text{dom}, \text{cod}, \text{id}, \circ)$, where

1. $\text{Ob}(\mathcal{C})$ is a set,
2. $\text{Hom}_{\mathcal{C}}$ is a set, and $\text{dom}, \text{cod}: \text{Hom}_{\mathcal{C}} \rightarrow \text{Ob}(\mathcal{C})$ are functions,
3. $\text{id}: \text{Ob}(\mathcal{C}) \rightarrow \text{Hom}_{\mathcal{C}}$ is a function, and

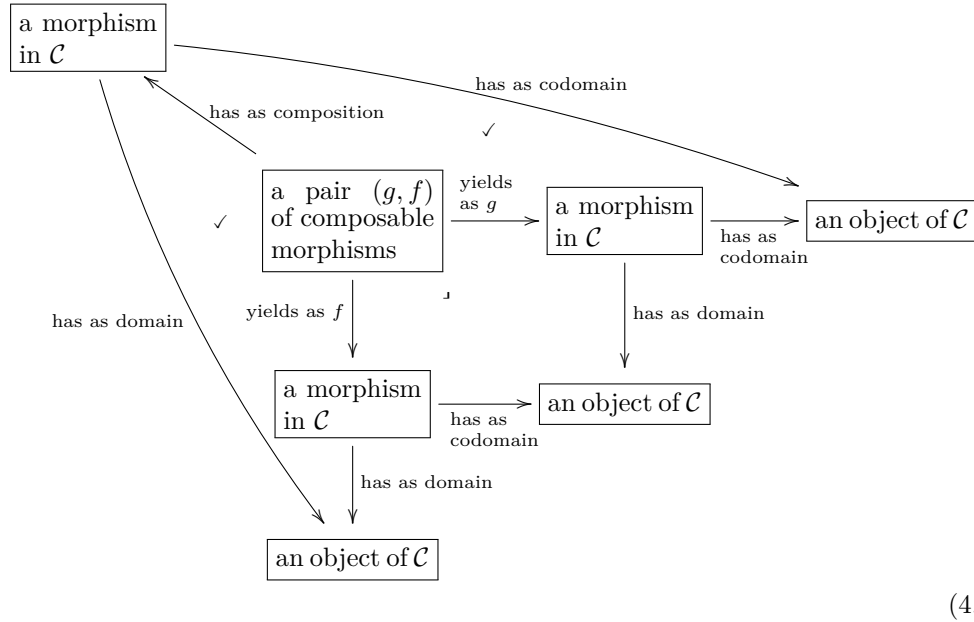
4. $\circ$ is a function as depicted in the commutative diagram below

$$\begin{array}{ccccc}
 \text{Hom}_{\mathcal{C}} & & & & \text{Ob}(\mathcal{C}) \\
 & \swarrow \circ & \searrow \checkmark & & \\
 \text{Hom}_{\mathcal{C}} \times \text{Ob}(\mathcal{C}) & \xrightarrow{\quad} & \text{Hom}_{\mathcal{C}} & \xrightarrow{\text{cod}} & \text{Ob}(\mathcal{C}) \\
 \downarrow \downarrow & \downarrow \downarrow & \downarrow \downarrow & & \\
 \text{Hom}_{\mathcal{C}} & \xrightarrow{\text{cod}} & \text{Ob}(\mathcal{C}) & & \\
 \downarrow \downarrow & & \downarrow \downarrow & & \\
 \text{Ob}(\mathcal{C}) & & \text{Ob}(\mathcal{C}) & &
 \end{array}
 \quad (4.2)$$

- Express the fact that for any $x \in \text{Ob}(\mathcal{C})$ the morphism id_x points from x to x in terms of the functions id , dom , cod .
- Express the condition that composing a morphism f with an appropriate identity morphism yields f .
- Express the associativity law in these terms (Hint: Proposition 2.5.1.17 may be useful).

◇

Example 4.1.1.19 (Partial olog for a category). Below is an olog that captures some of the essential structures of a category.



Missing from (4.3) is the notion of identity morphism (as an arrow from 'an object of $\mathcal{C}$ ' to 'a morphism in $\mathcal{C}$ ') and the associated path equivalences, as well as the identity

and associativity laws. All of these can be added to the olog, at the expense of some clutter.

Remark 4.1.1.20. Perhaps it is already clear that category theory is very wrapped up in itself. It feels like everything relates to everything, and this feeling may intensify as you go on. However, the relationships between different notions are rigorously defined, and almost everything presented here can be formalized in a proof system like Coq (the most obvious exceptions being things like the readability requirement of ologs and the modeling of scientific applications).

If you feel cognitive vertigo, look to formal definitions as the ground of your understanding.

4.1.2 Functors

A category $\mathcal{C} = (\text{Ob}(\mathcal{C}), \text{Hom}_{\mathcal{C}}, \text{dom}, \text{cod}, \text{id}, \circ)$, involves a set of objects a set of morphisms, a notion of domains and codomains, a notion of identity morphisms, and a composition formula. For two categories to be comparable, these various components should be appropriately comparable.

Definition 4.1.2.1. Let $\mathcal{C}$ and $\mathcal{C}'$ be categories. A *functor* F from $\mathcal{C}$ to $\mathcal{C}'$, denoted $F: \mathcal{C} \rightarrow \mathcal{C}'$ consists of some constituents (A. on-objects part, B. on-morphisms part) conforming to some laws (1. preservation of identities, 2. preservation of composition), as follows:

- A. a function $\text{Ob}(F): \text{Ob}(\mathcal{C}) \rightarrow \text{Ob}(\mathcal{C}')$, which we sometimes denote simply by $F: \text{Ob}(\mathcal{C}) \rightarrow \text{Ob}(\mathcal{C}')$; and
- B. for every pair of objects $c, d \in \text{Ob}(\mathcal{C})$, a function

$$\text{Hom}_F(c, d): \text{Hom}_{\mathcal{C}}(c, d) \rightarrow \text{Hom}_{\mathcal{C}'}(F(c), F(d)),$$

which we sometimes denote simply by $F: \text{Hom}_{\mathcal{C}}(c, d) \rightarrow \text{Hom}_{\mathcal{C}'}(F(c), F(d))$.

The following laws harmonize the above constituents:

- 1. Identities are preserved by F . That is, for any object $c \in \text{Ob}(\mathcal{C})$, we have $F(\text{id}_c) = \text{id}_{F(c)}$; and
- 2. Composition is preserved by F . That is, for any objects $b, c, d \in \text{Ob}(\mathcal{C})$ and morphisms $g: b \rightarrow c$ and $h: c \rightarrow d$, we have $F(h \circ g) = F(h) \circ F(g)$.

Example 4.1.2.2 (Monoids have underlying sets). Recall from Definition 3.1.1.1 that if $\mathcal{M} = (M, \text{id}, \star)$ is a monoid, then M is a set. And recall from Definition 3.1.4.1 that if $f: \mathcal{M} \rightarrow \mathcal{M}'$ is a monoid homomorphism then $f: M \rightarrow M'$ is a function. Thus we have a functor

$$U: \mathbf{Mon} \rightarrow \mathbf{Set}$$

that takes every monoid to its underlying set and every monoid homomorphism to its underlying function.

Given two monoids $\mathcal{M} = (M, \text{id}, \star)$ and $\mathcal{M}' = (M', \text{id}', \star')$, there may be many functions from M to M' that do not arise from monoid homomorphisms. It is often useful to speak of such functions. For example, one could assign to every command in one video game V a command in another video game V' , but this may not work in the “monoid way” when performing a sequence of commands. By being able to speak of M as a set, or as $\mathcal{M}$ as a monoid, and understanding the relationship U between them, we can be clear about where we stand at all times in our discussion.

Example 4.1.2.3. [Groups have underlying monoids]

Recall that a group is just a monoid $(M, \text{id}, \star)$ with the extra property that every element $m \in M$ has an inverse $m' \star m = \text{id} = m \star m'$. Thus to every group we can assign its *underlying monoid*. Similarly, a group homomorphism is just a monoid homomorphism of its underlying monoids. This means that there is a functor

$$U: \mathbf{Grp} \rightarrow \mathbf{Mon}$$

that sends every group or group homomorphism to its underlying monoid or monoid homomorphism. That identity and composition are preserved is obvious.

Application 4.1.2.4. Suppose you're a scientist working with symmetries. But then suppose that the symmetry breaks somewhere, or you add some extra observable which is not reversible under the symmetry. You want to seamlessly relax the requirement that every action be reversible without changing anything else. You want to know where you can go, or what's allowed. The answer is to simply pass from the category of groups (or group actions) to the category of monoids (or monoid actions).

◇◇

Exercise 4.1.2.5 (Forgetful functors between types of orders). A partial order is just a preorder with a special property. A linear order is just a partial order with a special property.

- a.) Is there an “obvious” functor $\mathbf{FLin} \rightarrow \mathbf{PrO}$?
- b.) Is there an “obvious” functor $\mathbf{PrO} \rightarrow \mathbf{FLin}$?

◇

Proposition 4.1.2.6. [Preorders to graphs]

Let $\mathbf{PrO}$ be the category of preorders and $\mathbf{Grph}$ be the category of graphs. There is a functor $U: \mathbf{PrO} \rightarrow \mathbf{Grph}$ such that for any preorder $\mathcal{X} = (X, \leq_X)$, the graph $U(\mathcal{X})$ has vertices X .

Proof. Given a preorder $\mathcal{X} = (X, \leq_X)$, we can make a graph $F(\mathcal{X})$ with vertices X and an arrow $x \rightarrow x'$ whenever $x \leq_X x'$, as in Remark 3.4.1.9. More precisely, the preorder $\leq_X$ is a relation, i.e. a subset $R_X \subseteq X \times X$, which we think of as a function $i: R_X \rightarrow X \times X$. Composing with projections $\pi_1, \pi_2: X \times X \rightarrow X$ gives us

$$\text{src}_X := \pi_1 \circ i: R_X \rightarrow X \quad \text{and} \quad \text{tgt}_X := \pi_2 \circ i: R_X \rightarrow X.$$

Then we put $F(\mathcal{X}) := (X, R_X, \text{src}_X, \text{tgt}_X)$. This gives us a function $F: \text{Ob}(\mathbf{PrO}) \rightarrow \text{Ob}(\mathbf{Grph})$.

Suppose now that $f: \mathcal{X} \rightarrow \mathcal{Y}$ is a preorder morphism (where $\mathcal{Y} = (Y, \leq_Y)$). This is a function $f: X \rightarrow Y$ such that for any $(x, x') \in X \times X$, if $x \leq_X x'$ then $f(x) \leq_Y f(x')$. But that's the same as saying that there exists a dotted arrow making the following diagram of sets commute

$$\begin{array}{ccc} R_X & \longrightarrow & X \times X \\ \vdots & & \downarrow f \times f \\ R_Y & \longrightarrow & Y \times Y \end{array}$$

(Note that there cannot be two different dotted arrows making that diagram commute because $R_Y \rightarrow Y \times Y$ is a monomorphism.) Our commutative square is precisely what's

needed for a graph homomorphism, as shown in Exercise 3.3.3.6. Thus, we have defined F on objects and on morphisms. It is clear that F preserves identity and composition. $\square$

Exercise 4.1.2.7. In Proposition 4.1.2.6 we gave a functor $U: \mathbf{PrO} \rightarrow \mathbf{Grph}$.

a.) Is every graph $G \in \text{Ob}(\mathbf{Grph})$ in the image of U (or more precisely, is the function

$$\text{Ob}(U): \text{Ob}(\mathbf{PrO}) \rightarrow \text{Ob}(\mathbf{Grph})$$

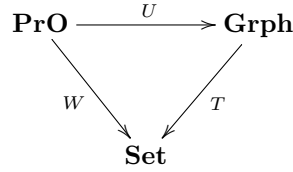
surjective)?

b.) If so, why; if not, name a graph not in the image.

c.) Suppose that $G, H \in \text{Ob}(\mathbf{Grph})$ are two graphs that are in the image of U . Is every graph homomorphism $f: G \rightarrow H$ in the image of Hom_U ? In other words, does every graph homomorphism between G and H come from a preorder homomorphism?

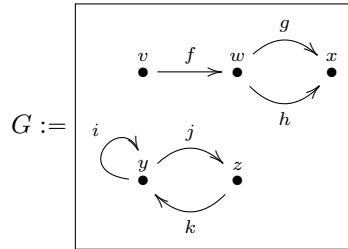
$\diamond$

Remark 4.1.2.8. There is a functor $W: \mathbf{PrO} \rightarrow \mathbf{Set}$ sending $(X, \leq)$ to X . There is a functor $T: \mathbf{Grph} \rightarrow \mathbf{Set}$ sending $(V, A, \text{src}, \text{tgt})$ to V . When we understand the category of categories (Section 4.1.2.24), it will be clear that Proposition 4.1.2.6 can be summarized as a commutative triangle in $\mathbf{Cat}$,



Exercise 4.1.2.9. [Graphs to preorders] Recall from (2.3) that every function $f: A \rightarrow B$ has an image, $\text{im}_f(A) \subseteq B$. Use this idea and Example 3.4.1.13 to construct a functor $\text{Im}: \mathbf{Grph} \rightarrow \mathbf{PrO}$ such that for any graph $G = (V, A, \text{src}, \text{tgt})$, the preorder has elements given by the vertices of G (i.e. we have $\text{Im}(G) = (V, \leq_G)$, for some ordering $\leq_G$). $\diamond$

Exercise 4.1.2.10. What is the preorder $\text{Im}(G)$ when $G \in \text{Ob}(\mathbf{Grph})$ is the following graph?



$\diamond$

Exercise 4.1.2.11. Consider the functor $\text{Im}: \mathbf{Grph} \rightarrow \mathbf{PrO}$ constructed in Exercise 4.1.2.9.

a.) Is every preorder $\mathcal{X} \in \text{Ob}(\mathbf{PrO})$ in the image of Im (or more precisely in the image of $\text{Ob}(\text{Im}): \text{Ob}(\mathbf{Grph}) \rightarrow \text{Ob}(\mathbf{PrO})$)?

- b.) If so, why; if not, name a preorder not in the image.
- c.) Suppose that $\mathcal{X}, \mathcal{Y} \in \text{Ob}(\mathbf{PrO})$ are two preorders that are in the image of Im . Is every preorder morphism $f: \mathcal{X} \rightarrow \mathcal{Y}$ in the image of Hom_{Im} ? In other words, does every preorder homomorphism between $\mathcal{X}$ and $\mathcal{Y}$ come from a graph homomorphism?

◇

Exercise 4.1.2.12. We have functors $U: \mathbf{PrO} \rightarrow \mathbf{Grph}$ and $Im: \mathbf{Grph} \rightarrow \mathbf{PrO}$.

- a.) What can you say about $Im \circ U: \mathbf{PrO} \rightarrow \mathbf{PrO}$?
- b.) What can you say about $U \circ Im: \mathbf{Grph} \rightarrow \mathbf{Grph}$?

◇

Remark 4.1.2.13. In the course announcement for 18-S996, I wrote the following:

It is often useful to focus ones study by viewing an individual thing, or a group of things, as though it exists in isolation. However, the ability to rigorously change our point of view, seeing our object of study in a different context, often yields unexpected insights. Moreover this ability to change perspective is indispensable for effectively communicating with and learning from others. It is the relationships between things, rather than the things in and by themselves, that are responsible for generating the rich variety of phenomena we observe in the physical, informational, and mathematical worlds.

This holds at many different levels. For example, one can study a group (in the sense of Definition 3.2.1.1) in isolation, trying to understand its subgroups or its automorphisms, and this is mathematically interesting. But one can also view it as a quotient of something else, or as a subgroup of something else. One can view the group as a monoid and look at monoid homomorphisms to or from it. One can look at the group in the context of symmetries by seeing how it acts on sets. These changes of viewpoint are all clearly and formally expressible within category theory. We know how the different changes of viewpoint compose and how they fit together in a larger context.

Exercise 4.1.2.14. a.) Is the above quote also true in your scientific discipline of expertise? How so?

- b.) Can you imagine a way that category theory can help catalogue the kinds of relationships or changes of viewpoint that exist in your discipline?
- c.) What kinds of structures that you use often really deserve to be better formalized? Keep this kind of question in mind for your final project.

◇

Example 4.1.2.15 (Free monoids). Let G be a set. We saw in 3.1.1.12 that $\text{List}(G)$ is a monoid, called the free monoid on G . Given a function $f: G \rightarrow G'$, there is an induced function $\text{List}(f): \text{List}(G) \rightarrow \text{List}(G')$, and this preserves the identity element $[]$ and concatenation of lists, so $\text{List}(f)$ is a monoid homomorphism. It is easy to check that $\text{List}: \mathbf{Set} \rightarrow \mathbf{Mon}$ is a functor.

Application 4.1.2.16. In Application 2.1.2.9 we discussed an isomorphism $\text{Nuc}_{\text{DNA}} \cong \text{Nuc}_{\text{RNA}}$ given by RNA transcription. Applying the functor List we get a function

$$\text{List}(\text{Nuc}_{\text{DNA}}) \xrightarrow{\cong} \text{List}(\text{Nuc}_{\text{RNA}}),$$

which will send sequences of DNA nucleotides to sequences of RNA nucleotides and vice versa. This is performed by polymerases.

◇◇

Exercise 4.1.2.17. Let $G = \{1, 2, 3, 4, 5\}$, $G' = \{a, b, c\}$, and let $f: G \rightarrow G'$ be given by the sequence (a, c, b, a, c) .² Then if $L = [1, 1, 3, 5, 4, 5, 3, 2, 4, 1]$, what is $\text{List}(f)(L)$? ◇

Exercise 4.1.2.18. We can rephrase our notion of functor in terms compatible with Exercise 4.1.1.18. We would begin by saying that a functor $F: \mathcal{C} \rightarrow \mathcal{C}'$ consists of two functions,

$$\text{Ob}(F): \text{Ob}(\mathcal{C}) \rightarrow \text{Ob}(\mathcal{C}') \quad \text{and} \quad \text{Hom}_F: \text{Hom}_{\mathcal{C}} \rightarrow \text{Hom}_{\mathcal{C}'},$$

which we call the *on-objects part* and the *on-morphisms part*, respectively. They must follow some rules, expressed by the commutativity of the following squares in **Set**:

$$\begin{array}{ccc} \text{Hom}_{\mathcal{C}} & \xrightarrow{\text{dom}} & \text{Ob}(\mathcal{C}) \\ \text{Hom}_F \downarrow & & \downarrow \text{Ob}(F) \\ \text{Hom}_{\mathcal{C}'} & \xrightarrow{\text{dom}} & \text{Ob}(\mathcal{C}') \end{array} \quad \begin{array}{ccc} \text{Hom}_{\mathcal{C}} & \xrightarrow{\text{cod}} & \text{Ob}(\mathcal{C}) \\ \text{Hom}_F \downarrow & & \downarrow \text{Ob}(F) \\ \text{Hom}_{\mathcal{C}'} & \xrightarrow{\text{cod}} & \text{Ob}(\mathcal{C}') \end{array} \quad (4.4)$$

$$\begin{array}{ccc} \text{Ob}(\mathcal{C}) & \xrightarrow{\text{id}} & \text{Hom}_{\mathcal{C}} \\ \text{Ob}(F) \downarrow & & \downarrow \text{Hom}_F \\ \text{Ob}(\mathcal{C}') & \xrightarrow{\text{id}} & \text{Hom}_{\mathcal{C}'} \end{array} \quad \begin{array}{ccc} \text{Hom}_{\mathcal{C}} \times_{\text{Ob}(\mathcal{C})} \text{Hom}_{\mathcal{C}} & \xrightarrow{\circ} & \text{Hom}_{\mathcal{C}} \\ \downarrow & & \downarrow \text{Hom}_F \\ \text{Hom}_{\mathcal{C}'} \times_{\text{Ob}(\mathcal{C}')} \text{Hom}_{\mathcal{C}'} & \xrightarrow{\circ} & \text{Hom}_{\mathcal{C}'} \end{array} \quad (4.5)$$

Where does the (unlabeled) left-hand function in the bottom right diagram come from? Hint: use Exercise 2.5.1.19.

Consider Diagram (4.2) and imagine it as though contained in a pane of glass. Then imagine a parallel pane of glass involving $\mathcal{C}'$ in place of $\mathcal{C}$ everywhere.

- Draw arrows from the $\mathcal{C}$ pane to the $\mathcal{C}'$ pane, each labeled $\text{Ob}(F)$ or Hom_F as seems appropriate.
- If F is a functor (i.e. satisfies (4.4) and (4.5), do all the squares in your drawing commute?
- Does the definition of functor involve anything not captured in this setup?

◇

Example 4.1.2.19. [paths-graph]

Let $G = (V, A, \text{src}, \text{tgt})$ be a graph. Then for any pair of vertices $v, w \in G$, there is a set $\text{Path}_G(v, w)$ of paths from v to w ; see Definition 3.3.2.1. In fact there is a set Path_G and functions $\overline{\text{src}}, \overline{\text{tgt}}: \text{Path}_G \rightarrow V$. That information is enough to define a new graph,

$$\text{Paths}(G) := (V, \text{Path}_G, \overline{\text{src}}, \overline{\text{tgt}}).$$

Moreover, given a graph homomorphism $f: G \rightarrow G'$, every path in G is sent under f to a path in G' . So $\text{Paths}: \mathbf{Grph} \rightarrow \mathbf{Grph}$ is a functor.

²See Exercise 2.1.2.14 in case there is any confusion with this.

- Exercise 4.1.2.20.* a.) Consider the graph G from Example 3.3.3.3. Draw the path-graph $\text{Paths}(G)$ for G .
- b.) Repeating the above exercise for G' from the same example would be hard, because the path graph $\text{Paths}(G')$ has infinitely many arrows. However, the graph homomorphism $f: G \rightarrow G'$ does induce a morphism of paths-graphs $\text{Paths}(f): \text{Paths}(G) \rightarrow \text{Paths}(G')$, and it is possible to say how that acts on the vertices and arrows of $\text{Paths}(G)$. Please do so.
- c.) Given a graph homomorphism $f: G \rightarrow G'$ and two paths $p: v \rightarrow w$ and $q: w \rightarrow x$ in G , is it true that $\text{Paths}(f)$ preserves the concatenation? What does that even mean?

◇

Exercise 4.1.2.21. Suppose that $\mathcal{C}$ and $\mathcal{D}$ are categories, $c, c' \in \text{Ob}(\mathcal{C})$ are objects, and $F: \mathcal{C} \rightarrow \mathcal{D}$ is a functor. Suppose that c and c' are isomorphic in $\mathcal{C}$. Show that this implies that $F(c)$ and $F(c')$ are isomorphic in $\mathcal{D}$.

◇

Example 4.1.2.22. For any graph G , we can assign its set of loops $Eq(G)$ as in Exercise 3.3.1.10. This assignment is functorial in that given a graph homomorphism $G \rightarrow G'$ there is an induced function $Eq(G) \rightarrow Eq(G')$. Similarly, we can functorially assign the set of connected components of the graph, $Coeq(G)$. In other words $Eq: \mathbf{Grph} \rightarrow \mathbf{Set}$ and $Coeq: \mathbf{Grph} \rightarrow \mathbf{Set}$ are functors. The assignment of vertex set and arrow set are two more functors $\mathbf{Grph} \rightarrow \mathbf{Set}$.

Suppose you want to decide whether two graphs G and G' are isomorphic. Supposing that the graphs have thousands of vertices and thousands of arrows, this could take a long time. However, the functors above, in combination with Exercise 4.1.2.21 give us some things to try.

The first thing to do is to count the number of loops of each, because these numbers are generally small. If the number of loops in G is different than the number of loops in G' then because functors preserve isomorphisms, G and G' cannot be isomorphic. Similarly one can count the number of connected components, again generally a small number; if the number of components in G is different than the number of components in G' then $G \not\cong G'$. Similarly, one can simply count the number of vertices or the number of arrows in G and G' . These are all isomorphism invariants.

All this is a bit like trying to decide if a number is prime by checking if it's even, if its digits add up to a multiple of 3, or it ends in a 5; these tests do not determine the answer, but they offer some level of discernment.

Remark 4.1.2.23. In the introduction I said that functors allow ideas in one domain to be rigorously imported to another. Example 4.1.2.22 is a first taste. Because functors preserve isomorphisms, we can tell graphs apart by looking at them in a simpler category, **Set**. There is relatively simple theorem in **Set** that says that for different natural numbers m, n the sets $\underline{m}$ and $\underline{n}$ are never isomorphic. This theorem is transported via our four functors to four different theorems about telling graphs apart.

4.1.2.24 The category of categories

Recall from Remark 4.1.1.2 that a small category $\mathcal{C}$ is one in which $\text{Ob}(\mathcal{C})$ is a set. We have not really been paying attention to this issue, and everything we have said so far works whether $\mathcal{C}$ is small or not. In the following definition we really ought to be a little more careful, so we are.

Proposition 4.1.2.25. *There exists a category, called the category of small categories and denoted **Cat**, in which the objects are the small categories and the morphisms are the functors,*

$$\text{Hom}_{\mathbf{Cat}}(\mathcal{C}, \mathcal{D}) = \{F: \mathcal{C} \rightarrow \mathcal{D} \mid F \text{ is a functor}\}.$$

That is, there are identity functors, functors can be composed, and the identity and associativity laws hold.

Proof. We follow Definition 4.1.1.1. We have specified $\text{Ob}(\mathbf{Cat})$ and $\text{Hom}_{\mathbf{Cat}}$ already. Given a small category $\mathcal{C}$, there is an identity functor $\text{id}_{\mathcal{C}}: \mathcal{C} \rightarrow \mathcal{C}$ that is identity on the set of objects and the set of morphisms. And given a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ and a functor $G: \mathcal{D} \rightarrow \mathcal{E}$, it is easy to check that $G \circ F: \mathcal{C} \rightarrow \mathcal{E}$, defined by composition of functions $\text{Ob}(G) \circ \text{Ob}(F): \text{Ob}(\mathcal{C}) \rightarrow \text{Ob}(\mathcal{E})$ and $\text{Hom}_G \circ \text{Hom}_F: \text{Hom}_{\mathcal{C}} \rightarrow \text{Hom}_{\mathcal{E}}$ (see Exercise 4.1.2.18), is a functor. For the same reasons, it is easy to show that functors obey the identity law and the composition formula. Therefore this specification of **Cat** satisfies the definition of being a category. $\square$

Example 4.1.2.26. [Categories have underlying graphs]

Let $\mathcal{C} = (\text{Ob}(\mathcal{C}), \text{Hom}_{\mathcal{C}}, \text{dom}, \text{cod}, \text{id}, \circ)$ be a category (see Exercise 4.1.1.18). Then $(\text{Ob}(\mathcal{C}), \text{Hom}_{\mathcal{C}}, \text{dom}, \text{cod})$ is a graph, which we will call the *graph underlying $\mathcal{C}$* and denote by $U(\mathcal{C}) \in \text{Ob}(\mathbf{Grph})$. A functor $F: \mathcal{C} \rightarrow \mathcal{D}$ induces a graph morphism $U(F): U(\mathcal{C}) \rightarrow U(\mathcal{D})$, as seen in (4.4). So we have a functor,

$$U: \mathbf{Cat} \rightarrow \mathbf{Grph}.$$

Example 4.1.2.27. [Free category on a graph]

In Example 4.1.2.19, we discussed a functor $\text{Paths}: \mathbf{Grph} \rightarrow \mathbf{Grph}$ that considered all the paths in a graph G as the arrows of a new graph $\text{Paths}(G)$. In fact, $\text{Paths}(G)$ could be construed as a category, which we will denote $F(G) \in \text{Ob}(\mathbf{Cat})$ and call *the free category generated by G* . Indeed for any vertex v the trivial path at v serves as the identity element, and paths can be concatenated to give a composition. To see that F is a functor, we need to see that a graph homomorphism $f: G \rightarrow G'$ induces a functor $F(f): F(G) \rightarrow F(G')$. But this was shown in Exercise 4.1.2.20. Thus we have a functor

$$F: \mathbf{Grph} \rightarrow \mathbf{Cat}$$

called *the free category functor*.

Exercise 4.1.2.28. Let G be the graph depicted

$$\bullet \xrightarrow{e} \bullet,$$

and let $[1] \in \text{Ob}(\mathbf{Cat})$ denote the free category on G (see Example 4.1.2.27). We call $[1]$ the *free arrow category*.

a.) What are its objects?

b.) For every pair of objects in $[1]$, write down the hom-set.

$\diamond$

Exercise 4.1.2.29. Let G be the graph whose vertices are all cities in the US and whose arrows are airplane flights connecting cities. What idea is captured by the free category on G ? $\diamond$

Exercise 4.1.2.30. Let $F: \mathbf{Grph} \rightarrow \mathbf{Cat}$ denote the free category functor from Example 4.1.2.27, and let $U: \mathbf{Cat} \rightarrow \mathbf{Grph}$ denote the underlying graph functor from Example 4.1.2.27. We have seen the composition $U \circ F: \mathbf{Grph} \rightarrow \mathbf{Grph}$ before; what was it called? $\diamond$

Exercise 4.1.2.31. Recall the graph G from Example 3.3.1.2. Let $\mathcal{C} = F(G)$ be the free category on G .

a.) What is $\text{Hom}_{\mathcal{C}}(v, x)$?

b.) What is $\text{Hom}_{\mathcal{C}}(x, v)$?

$\diamond$

Example 4.1.2.32. [Discrete graphs, discrete categories]

There is a functor $\text{Disc}: \mathbf{Set} \rightarrow \mathbf{Grph}$ that sends a set S to the graph

$$\text{Disc}(S) := (S, \emptyset, !, !),$$

where $!: \emptyset \rightarrow S$ is the unique function. We call $\text{Disc}(S)$ the *discrete graph on the set S* . It is clear that a function $S \rightarrow S'$ induces a morphism of discrete graphs. Now applying the free category functor $F: \mathbf{Grph} \rightarrow \mathbf{Cat}$, we get the so-called *discrete category on the set S* , which we also might call $\text{Disc}: \mathbf{Set} \rightarrow \mathbf{Cat}$.

Exercise 4.1.2.33. Recall the definition of the set $\underline{n}$ for any natural number $n \in \mathbb{N}$, and let $D_n := \text{Disc}(\underline{n}) \in \text{Ob}(\mathbf{Cat})$.

a.) List all the morphisms in D_4 .

b.) List all the functors $D_3 \rightarrow D_2$.

$\diamond$

Exercise 4.1.2.34 (Terminal category). Let $\mathcal{C}$ be a category. How many functors are there $\mathcal{C} \rightarrow D_1$, where $D_1 := \text{Disc}(\underline{1})$ is the discrete category on one element? $\diamond$

We sometimes refer to $\text{Disc}(\underline{1})$ as the *terminal category* (for reasons that will be made clear in Section 4.5.3), and for simplicity denote it by $\underline{1}$.

Exercise 4.1.2.35. If someone said “ Ob is a functor from $\mathbf{Cat}$ to $\mathbf{Set}$,” what might they mean? $\diamond$

4.2 More basic examples from mathematics

4.2.1 Monoids, groups, preorders, and graphs

We saw in Section 4.1.1 that there is a category $\mathbf{Mon}$ of monoids, a category $\mathbf{Grp}$ of groups, a category $\mathbf{PrO}$ of preorders, and a category $\mathbf{Grph}$ of graphs. In this section we show that each monoid $\mathcal{M}$, each group $\mathcal{G}$, and each preorder $\mathcal{P}$ can be considered as its own category. If each object in $\mathbf{Mon}$ is a category, we might hope that each morphism in $\mathbf{Mon}$ is just a functor, and this is true. The same holds for $\mathbf{Grp}$ and $\mathbf{PrO}$. We will deal with graphs in Section 4.2.1.19.

4.2.1.1 Monoids as categories

In Example 3.1.2.9 we said that to olog a monoid, we should use only one box. And again in Example 3.5.3.3 we said that a monoid action could be captured by only one table. These ideas emanated from the understanding that a monoid is perfectly modeled as a category with one object.

Each monoid as a category with one object Let $(M, \text{id}, \star)$ be a monoid. We consider it as a category $\mathcal{M}$ with one object, $\text{Ob}(\mathcal{M}) = \{\blacktriangle\}$, and

$$\text{Hom}_{\mathcal{M}}(\blacktriangle, \blacktriangle) := M.$$

The identity morphism $\text{id}_{\blacktriangle}$ serves as the monoid identity id , and the composition formula

$$\circ: \text{Hom}_{\mathcal{M}}(\blacktriangle, \blacktriangle) \times \text{Hom}_{\mathcal{M}}(\blacktriangle, \blacktriangle) \rightarrow \text{Hom}_{\mathcal{M}}(\blacktriangle, \blacktriangle)$$

is given by $\star: M \times M \rightarrow M$. The associativity and identity laws for the monoid match precisely with the associativity and identity laws for categories.

If monoids are categories with one object, is there any categorical way of phrasing the notion of monoid homomorphism? Suppose that $\mathcal{M} = (M, \text{id}_M, \star_M)$ and $\mathcal{M}' = (M', \text{id}', \star')$. We know that a monoid homomorphism is a function $f: M \rightarrow M'$ such that $f(\text{id}) = \text{id}'$ and such that for every pair $m_0, m_1 \in M$ we have $f(m_0 \star m_1) = f(m_0) \star' f(m_1)$. What is a functor $\mathcal{M} \rightarrow \mathcal{M}'$?

Each monoid homomorphism as a functor between one-object categories Say that $\text{Ob}(\mathcal{M}) = \{\blacktriangle\}$ and $\text{Ob}(\mathcal{M}') = \{\blacktriangle'\}$; and we know that $\text{Hom}_{\mathcal{M}}(\blacktriangle, \blacktriangle) = M$ and $\text{Hom}_{\mathcal{M}'}(\blacktriangle', \blacktriangle') = M'$. A functor $F: \mathcal{M} \rightarrow \mathcal{M}'$ consists first of a function $\text{Ob}(\mathcal{M}) \rightarrow \text{Ob}(\mathcal{M}')$, but these sets have only one element each, so there is nothing to say on that front. It also consists of a function $\text{Hom}_{\mathcal{M}} \rightarrow \text{Hom}_{\mathcal{M}'}$ but that is just a function $M \rightarrow M'$. The identity and composition formulas for functors match precisely with the identity and composition formula for monoid homomorphisms, as discussed above. Thus a monoid homomorphism is nothing more than a functor between one-object categories.

Slogan 4.2.1.2.

“ A monoid is a category $\mathcal{G}$ with one object. A monoid homomorphism is just a functor between one-object categories. ”

We formalize this as the following theorem.

Theorem 4.2.1.3. *There is a functor $i: \mathbf{Mon} \rightarrow \mathbf{Cat}$ with the following properties:*

- for every monoid $\mathcal{M} \in \text{Ob}(\mathbf{Mon})$, the category $i(\mathcal{M}) \in \text{Ob}(\mathbf{Cat})$ itself has exactly one object,

$$|\text{Ob}(i(\mathcal{M}))| = 1$$

- for every pair of monoids $\mathcal{M}, \mathcal{M}' \in \text{Ob}(\mathbf{Mon})$ the function

$$\text{Hom}_{\mathbf{Mon}}(\mathcal{M}, \mathcal{M}') \xrightarrow{\cong} \text{Hom}_{\mathbf{Cat}}(i(\mathcal{M}), i(\mathcal{M}')),$$

induced by the functor i , is a bijection.

Proof. This is basically the content of the preceding paragraphs. The functor i sends a monoid to the corresponding category with one object and i sends a monoid homomorphism to the corresponding functor; it is not hard to check that i preserves identities and compositions. □

Theorem 4.2.1.3 situates the theory of monoids very nicely within the world of categories. But we have other ways of thinking about monoids, namely their actions on sets. As such it would greatly strengthen the story if we could subsume monoid actions within category theory also, and we can.

Each monoid action as a set-valued functor Recall from Definition 3.1.2.1 that if $(M, \text{id}, \star)$ is a monoid, an action consists of a set S and a function $\curvearrowright: M \times S \rightarrow S$ such that $\text{id} \curvearrowright s = s$ and $m_0 \curvearrowright (m_1 \curvearrowright s) = (m_0 \star m_1) \curvearrowright s$ for all $s \in S$. How might we relate the notion of monoid actions to the notion of functors? One idea is to try asking what a functor $F: \mathcal{M} \rightarrow \mathbf{Set}$ is; this idea will work.

Since $\mathcal{M}$ has only one object, we obtain one set, $S := F(\blacktriangle) \in \text{Ob}(\mathbf{Set})$. We also obtain a function $\text{Hom}_F: \text{Hom}_{\mathcal{M}}(\blacktriangle, \blacktriangle) \rightarrow \text{Hom}_{\mathbf{Set}}(F(\blacktriangle), F(\blacktriangle))$, or more concisely, a function

$$H_F: M \rightarrow \text{Hom}_{\mathbf{Set}}(S, S).$$

By currying (see Proposition 2.7.2.3), this is the same as a function $\curvearrowright: M \times S \rightarrow S$. The rule that $\text{id} \curvearrowright s = s$ becomes the rule that functors preserve identities, $\text{Hom}_F(\text{id}_{\blacktriangle}) = \text{id}_S$. The other rule is equivalent to the composition formula for functors.

4.2.1.4 Groups as categories

A group is just a monoid $(M, \text{id}, \star)$ in which every element $m \in M$ is invertible, meaning there exists some $m' \in M$ with $m \star m' = \text{id} = m' \star m$. If a monoid is the same thing as a category $\mathcal{M}$ with one object, then a group must be a category with one object and with an additional property having to do with invertibility. The elements of M are the morphisms of the category $\mathcal{M}$, so we need a notion of invertibility for morphisms. Luckily we have such a notion already, namely isomorphism. We have the following:

Slogan 4.2.1.5.

“ A group is a category $\mathcal{G}$ with one object, such that every morphism in $\mathcal{G}$ is an isomorphism. A group homomorphism is just a functor between such categories. ”

Theorem 4.2.1.6. *There is a functor $i: \mathbf{Grp} \rightarrow \mathbf{Cat}$ with the following properties:*

- for every group $\mathcal{G} \in \text{Ob}(\mathbf{Grp})$, the category $i(\mathcal{G}) \in \text{Ob}(\mathbf{Cat})$ itself has exactly one object, and every morphism m in $i(\mathcal{G})$ is an isomorphism; and
- for every pair of groups $\mathcal{G}, \mathcal{G}' \in \text{Ob}(\mathbf{Grp})$ the function

$$\text{Hom}_{\mathbf{Grp}}(\mathcal{G}, \mathcal{G}') \xrightarrow{\cong} \text{Hom}_{\mathbf{Cat}}(i(\mathcal{G}), i(\mathcal{G}')),$$

induced by the functor i , is a bijection.

Just as with monoids, an action of some group $(G, \text{id}, \star)$ on a set $S \in \text{Ob}(\mathbf{Set})$ is the same thing as a functor $\mathcal{G} \rightarrow \mathbf{Set}$ sending the unique object of $\mathcal{G}$ to the set S .

4.2.1.7 Monoids and groups stationed at each object in a category

If a monoid is just a category with one object, we can locate monoids in any category $\mathcal{C}$ by narrowing our gaze to one object in $\mathcal{C}$. Similarly for groups.

Example 4.2.1.8 (Endomorphism monoid). Let $\mathcal{C}$ be a category and $x \in \text{Ob}(\mathcal{C})$ an object. Let $M = \text{Hom}_{\mathcal{C}}(x, x)$. Note that for any two elements $f, g \in M$ we have $f \circ g: x \rightarrow x$ in M . Let $\mathcal{M} = (M, \text{id}_x, \circ)$. It is easy to check that $\mathcal{M}$ is a monoid; it is called the *endomorphism monoid of x in $\mathcal{C}$* .

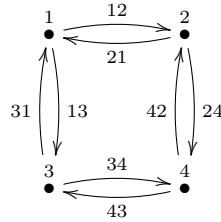
Example 4.2.1.9 (Automorphism group). Let $\mathcal{C}$ be a category and $x \in \text{Ob}(\mathcal{C})$ an object. Let $G = \{f: x \rightarrow x \mid f \text{ is an isomorphism}\}$. Let $\mathcal{G} = (G, \text{id}_x, \circ)$. It is easy to check that $\mathcal{G}$ is a group; it is called the *automorphism group of x in $\mathcal{C}$* .

Exercise 4.2.1.10. Let $S = \{1, 2, 3, 4\} \in \text{Ob}(\mathbf{Set})$.

- What is the automorphism group of S in $\mathbf{Set}$, and how many elements does this group have?
- What is the endomorphism monoid of S in $\mathbf{Set}$, and how many elements does this monoid have?
- Recall from Example 4.1.2.3 that every group has an underlying monoid $U(G)$; is the endomorphism monoid of S the underlying monoid of the automorphism group of S ?

◇

Exercise 4.2.1.11. Consider the graph G depicted below.



What is its group of automorphisms? Hint: every automorphism of G will induce an automorphism of the set $\{1, 2, 3, 4\}$; which ones will preserve the arrows? ◇

4.2.1.12 Preorders as categories

A preorder $(X, \leq)$ consists of a set X and a binary relation $\leq$ that is reflexive and transitive. We can make from $(X, \leq) \in \text{Ob}(\mathbf{PrO})$ a category $\mathcal{X} \in \text{Ob}(\mathbf{Cat})$ as follows. Define $\text{Ob}(\mathcal{X}) = X$ and for every two objects $x, y \in X$ define

$$\text{Hom}_{\mathcal{X}}(x, y) = \begin{cases} \{“x \leq y”\} & \text{if } x \leq y \\ \emptyset & \text{if } x \not\leq y \end{cases}$$

To clarify: if $x \leq y$, we assign $\text{Hom}_{\mathcal{X}}(x, y)$ to be the set containing only one element, namely the string “ $x \leq y$ ”.³ If (x, y) is not in relation $\leq$, then we assign $\text{Hom}_{\mathcal{X}}(x, y)$ to be the empty set. The composition formula

$$\circ: \text{Hom}_{\mathcal{X}}(x, y) \times \text{Hom}_{\mathcal{X}}(y, z) \rightarrow \text{Hom}_{\mathcal{X}}(x, z) \quad (4.6)$$

is completely determined because either one of two possibilities occurs. One possibility is that the left-hand side is empty (if either $x \not\leq y$ or $y \not\leq z$; in this case there is a unique function $\circ$ as in (4.6)). The other possibility is that the left-hand side is not empty in case $x \leq y$ and $y \leq z$, which implies $x \leq z$, so the right-hand side has exactly one element “ $x \leq z$ ” in which case again there is a unique function $\circ$ as in (4.6).

³The name of this morphism is completely unimportant. What matters is that $\text{Hom}_{\mathcal{X}}(x, y)$ has exactly one element iff $x \leq y$.

On the other hand, if $\mathcal{C}$ is a category having the property that for every pair of objects $x, y \in \text{Ob}(\mathcal{C})$, the set $\text{Hom}_{\mathcal{C}}(x, y)$ is either empty or has one element, then we can form a preorder out of $\mathcal{C}$. Namely, take $X = \text{Ob}(\mathcal{C})$ and say $x \leq y$ if there exists a morphism $x \rightarrow y$ in $\mathcal{C}$.

Exercise 4.2.1.13. We have seen that a preorder can be considered as a category $\mathcal{P}$. Recall from Definition 3.4.1.1 that a partial order is a preorder with an additional property. Phrase the defining property for preorders in terms of isomorphisms in the category $\mathcal{P}$. $\diamond$

Exercise 4.2.1.14. Suppose that $\mathcal{C}$ is a preorder (considered as a category). Let $x, y \in \text{Ob}(\mathcal{C})$ be objects such that $x \leq y$ and $y \leq x$. Prove that there is an isomorphism $x \rightarrow y$ in $\mathcal{C}$. $\diamond$

Example 4.2.1.15. The olog from Example 3.4.1.3 depicted a partial order, say $\mathcal{P}$. In it we have

$$\text{Hom}_{\mathcal{P}}(\text{'a diamond'}, \text{'a red card'}) = \{\text{is}\}$$

and we have

$$\text{Hom}_{\mathcal{P}}(\text{'a black queen'}, \text{'a card'}) \cong \{\text{is} \circ \text{is}\};$$

Both of these sets contain exactly one element, the name is not important. The set $\text{Hom}_{\mathcal{P}}(\text{'a 4'}, \text{'a 4 of diamonds'}) = \emptyset$.

Exercise 4.2.1.16. Every linear order is a partial order with a special property. Can you phrase this property in terms of hom-sets? $\diamond$

Proposition 4.2.1.17. *There is a functor $i: \mathbf{PrO} \rightarrow \mathbf{Cat}$ with the following properties for every preorder $(X, \leq)$:*

1. *the category $\mathcal{X} := i(X, \leq)$ has objects $\text{Ob}(\mathcal{X}) = X$; and*
2. *for each pair of elements $x, x' \in X$ the set $\text{Hom}_{\mathcal{X}}(x, x')$ has at most one element.*

Moreover, any category with property 2 is in the image of i .

Proof. We have already shown how i works on objects. To see that it is a functor we must show that for any morphism $f: (X, \leq_X) \rightarrow (Y, \leq_Y)$ of preorders, there is a functor $i(f): \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{Y} := (Y, \leq_Y)$. This follows directly from definitions. The second claim, that any category with property 2 is in the image of i follows from the discussion above. $\square$

Exercise 4.2.1.18. Recall the functor $U: \mathbf{PrO} \rightarrow \mathbf{Grph}$ from Proposition 4.1.2.6, the functor $F: \mathbf{Grph} \rightarrow \mathbf{Cat}$ from Example 4.1.2.27, and the functor $i: \mathbf{PrO} \rightarrow \mathbf{Cat}$ from Proposition 4.2.1.17.

a.) Does the following diagram of categories commute?

$$\begin{array}{ccc} \mathbf{PrO} & \xrightarrow{U} & \mathbf{Grph} \\ & \searrow i \quad \swarrow F & \\ & \mathbf{Cat} & \end{array}$$

- b.) We also had a functor $\mathbf{Grph} \rightarrow \mathbf{PrO}$. Does the following diagram of categories commute?

$$\begin{array}{ccc} \mathbf{Grph} & \xrightarrow{\quad} & \mathbf{PrO} \\ & \searrow i \quad \swarrow F & \\ & \mathbf{Cat} & \end{array} \quad ?$$

◇

4.2.1.19 Graphs as functors

Let $\mathcal{C}$ denote the category depicted below

$$\mathcal{C} := \boxed{\begin{array}{ccc} A & \xrightarrow{\text{src}} & V \\ \bullet & \xRightarrow{\quad} & \bullet \\ & \text{tgt} & \end{array}} \quad (4.7)$$

Then a functor $G: \mathcal{C} \rightarrow \mathbf{Set}$ is the same thing as two sets $G(A), G(V)$ and two functions $G(\text{src}): G(A) \rightarrow G(V)$ and $G(\text{tgt}): G(A) \rightarrow G(V)$. This is precisely what is needed for a graph; see Definition 3.3.1.1. We call $\mathcal{C}$ the *graph indexing category*.

Exercise 4.2.1.20. Consider the terminal category, $\mathbf{1}$, also known as the discrete category on one element (see Exercise 4.1.2.34). Let $\mathcal{C}$ be as in (4.7) and consider the functor $i_0: \mathbf{1} \rightarrow \mathcal{C}$ sending the object of $\mathbf{1}$ to the object $V \in \text{Ob}(\mathcal{C})$. If $G: \mathcal{C} \rightarrow \mathbf{Set}$ is a graph, what is the composite $G \circ i_0$? It consists of only one set; what set is it? For example, what set is it when G is the graph from Example 3.3.3.3. ◇

If a graph is a functor $\mathcal{C} \rightarrow \mathbf{Set}$, what is a graph homomorphism? We will see later in Example 4.3.1.12 that graph homomorphisms are homomorphisms between functors, which are called natural transformations. (Natural transformations are the highest-“level” structure that occurs in ordinary category theory.)

Example 4.2.1.21. Let $\mathcal{D}$ be the category whose underlying graph is depicted below

$$\mathcal{D} := \boxed{\begin{array}{ccc} \rho \circlearrowleft & A & \xrightarrow{\text{src}} V \\ & \bullet & \xRightarrow{\quad} \bullet \\ & & \text{tgt} \end{array}} \quad (4.8)$$

with the following composition formula:

$$\rho \circ \rho = \text{id}_A; \quad \text{src} \circ \rho = \text{tgt}; \quad \text{and} \quad \text{tgt} \circ \rho = \text{src}.$$

This category $\mathcal{D}$ is the *symmetric graph indexing category*. Just like any graph can be understood as a functor $\mathcal{C} \rightarrow \mathbf{Set}$, where $\mathcal{C}$ is the graph indexing category displayed in (4.7), any symmetric graph can be understood as a functor $\mathcal{D} \rightarrow \mathbf{Set}$, where $\mathcal{D}$ is the category drawn above.

It is customary to draw the connections in a symmetric graphs as line segments rather than arrows between vertices. However, a better heuristic is to think that these connections consists of two arrows, one pointing in each direction. More precisely, every arrow has an equal and opposite arrow.

Exercise 4.2.1.22. Which of the following graphs are symmetric:

- a.)

- b.) The graph G from (3.3)?
- c.) The graph G from Exercise 3.3.1.8?
- d.) The graph G' from (3.6)?
- e.) The graph $\mathcal{Loop}$ from (3.15), i.e. the graph having exactly one vertex and one arrow?
- f.) The graph G from Exercise 4.2.1.11?

◇

Exercise 4.2.1.23. Let $\mathcal{C}$ be the graph indexing category shown in (4.7) and let $\mathcal{D}$ be the symmetric graph indexing category displayed in (4.8).

- a.) How many functors are there of the form $\mathcal{C} \rightarrow \mathcal{D}$?
- b.) Is one more “reasonable” than the others?
- c.) Choose the one that seems most reasonable and call it $i: \mathcal{C} \rightarrow \mathcal{D}$. If a symmetric graph is a functor $S: \mathcal{D} \rightarrow \mathbf{Set}$, you can compose with i to get a functor $S \circ i: \mathcal{C} \rightarrow \mathbf{Set}$. This is a graph; what graph is it? What has changed?

◇

4.2.2 Database schemas present categories

Recall from Definition 3.5.2.6 that a database schema (or schema, for short) consists of a graph together with a certain kind of equivalence relation on its paths. In Section 4.4.1 we will define a category **Sch** that has schemas as objects and appropriately modified graph homomorphisms as morphisms. In Section 4.4.2 we prove that the category of schemas is equivalent (in the sense of Definition 4.3.4.1) to the category of categories,

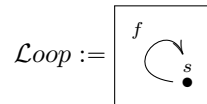
$$\mathbf{Sch} \simeq \mathbf{Cat}.$$

The difference between schemas and categories is like the difference between monoid presentations, given by generators and relations as in Definition 3.1.1.14, and the monoids themselves. The same monoid has (infinitely) many different presentations, and so it is for categories: many different schemas can *present* the same category. A schema is a compact form, and can be specified in finite space and time while generating something infinite.

Slogan 4.2.2.1.

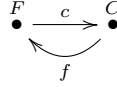
“ A database schema is a category presentation. ”

Example 4.2.2.2. The schema $\mathcal{Loop}$, depicted below, has no path equivalence declarations. As a graph it has one vertex and one arrow.



The category it generates, however, is the free monoid on one generator, $\mathbb{N}$. It has one object $\blacktriangle$ but a morphism $f^n: \blacktriangle \rightarrow \blacktriangle$ for every natural number $n \in \mathbb{N}$, thought of as “how many times to go around the loop f ”. Clearly, the schema is more compact than the infinite category it generates.

Exercise 4.2.2.3. Consider the olog from Exercise 3.5.2.18, which says that for any father x , his first child's father is x . It is redrawn below as a schema $\mathcal{S}$, and we include the desired path equivalence declaration, $F \circ c \circ f = F$,



How many morphisms are there (total) in the category generated by $\mathcal{S}$? ◇

Exercise 4.2.2.4. Suppose that G is a graph and that $\mathcal{G}$ is the schema generated by G with no PEDs. What is the relationship between the category generated by $\mathcal{G}$ and the free category $F(G) \in \text{Ob}(\mathbf{Cat})$ as defined in Example 4.1.2.27? ◇

4.2.2.5 Instances on a schema $\mathcal{C}$

If schemas are like categories, what are instances? Recall that an instance I on a schema $\mathcal{S} = (G, \simeq)$ assigns to each vertex v in G a set of rows say $I(v) \in \text{Ob}(\mathbf{Set})$. And to every arrow $a: v \rightarrow v'$ in G the instance assigns a function $I(a): I(v) \rightarrow I(v')$. The rule is that given two equivalent paths, their compositions must give the same function. Concisely, an instance is a functor $I: \mathcal{S} \rightarrow \mathbf{Set}$.

Example 4.2.2.6. We have now seen that a monoid is just a category $\mathcal{M}$ with one object and that a monoid action is a functor $\mathcal{M} \rightarrow \mathbf{Set}$. Under our understanding of database schemas as categories, $\mathcal{M}$ is a schema and so an action becomes an instance of that schema. The monoid action table from Example ex:action table was simply a manifestation of the database instance according to the Rules 3.5.2.8.

Exercise 4.2.2.7. In Section 4.2.1.19 we discuss how each graph is a functor $\mathcal{G} \rightarrow \mathbf{Set}$ for the graph indexing category depicted below:

$$\mathcal{G} := \boxed{\begin{array}{ccc} A & \xrightarrow{\text{src}} & V \\ \bullet & \xRightarrow{\quad} & \bullet \\ & \text{tgt} & \end{array}}$$

But now we know that if a graph is a set-valued functor then we can consider $\mathcal{G}$ as a database schema.

- a.) How many tables, and how many columns of each should there be (if unsure, consult Rules 3.5.2.8)?
- b.) Write out the table view of graph G from Example 3.3.3.3.

◇

4.2.3 Spaces

Category theory was invented for use in algebraic topology, and in particular to discuss natural transformations between certain functors. We will get to natural transformations more formally in Section 4.3. For now, they are ways of relating functors. In the original use, Eilenberg and Mac Lane were interested in functors that connect topological spaces (shapes like spheres, etc.) to algebraic systems (groups, etc.)

For example, there is a functor that assigns to each space X its group $\pi_1(X)$ of round-trip voyages (starting and ending at some chosen point $x \in X$), modulo some equivalence

relation. There is another functor that assigns to every space its group $H_1(X, \mathbb{Z})$ of ways to drop some (positive or negative) number of circles on X . These two functors are related, but they are not equal.

There is a relationship between the functor π_1 and the functor H_1 . For example when X is the figure-8 space (two circles joined at a point) the group $\pi_1(X)$ is much bigger than the group $H_1(X)$. Indeed $\pi_1(X)$ includes information about the order and direction of loops traveled; whereas the group $H_1(X, \mathbb{Z})$ includes only information about how many times one goes around each loop. However, there is a natural transformation of functors $\pi_1(-) \rightarrow H_1(-, \mathbb{Z})$, called the Hurewicz transformation, which “forgets” the extra information and thus yields a simplification.

Example 4.2.3.1. Given a set X , recall that $\mathbb{P}(X)$ denotes the set of subsets of X . A *topology* on X is a choice of which subsets $U \in \mathbb{P}(X)$ will be called *open sets*. The union of any number of open sets must be considered to be an open set, and the intersection of any finite number of open sets must be considered open. One could say succinctly that a topology on X is a sub-order $\text{Op}(X) \subseteq \mathbb{P}(X)$ that is closed under taking finite meets and infinite joins.

A *topological space* is a pair $(X, \text{Op}(X))$, where X is a set and $\text{Op}(X)$ is a topology on X . The elements of the set X are called *points*. A *morphism of topological spaces* (also called a *continuous map*) is a function $f: X \rightarrow Y$ such that for every $V \in \text{Op}(Y)$ the preimage $f^{-1}(V) \in \mathbb{P}(X)$ is actually in $\text{Op}(X)$. That is, such that there exists a dashed arrow making the diagram below commute:

$$\begin{array}{ccc} \text{Op}(Y) & \dashrightarrow & \text{Op}(X) \\ \downarrow & & \downarrow \\ \mathbb{P}(Y) & \xrightarrow{f^{-1}} & \mathbb{P}(X). \end{array}$$

The *category of topological spaces*, denoted **Top**, is the category having objects and morphisms as above.

Exercise 4.2.3.2. a.) Explain how “looking at points” gives a functor **Top** $\rightarrow$ **Set**.

b.) Does “looking at open sets” give a functor **Top** $\rightarrow$ **PrO**?

◇

Example 4.2.3.3. The set $\mathbb{R}$ can be given a topology in a standard way.⁴ But $(\mathbb{R}, 0, +)$ is also a monoid. Moreover, for every $x \in \mathbb{R}$ the monoid operation $+: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ is continuous.⁵ So we say that $\mathcal{R} := (\mathbb{R}, 0, +)$ is a *topological monoid*.

Recall from Section 4.2.1.1 that a monoid action is a functor $\mathcal{M} \rightarrow \mathbf{Set}$, where $\mathcal{M}$ is a monoid. Instead imagine a functor $a: \mathcal{R} \rightarrow \mathbf{Top}$? Since $\mathcal{R}$ is a category with one object, this amounts to an object $X \in \text{Ob}(\mathbf{Top})$, a space. And to every real number $t \in \mathbb{R}$ we obtain a continuous map $a(t): X \rightarrow X$. If we consider X as the set of states of some system and $\mathbb{R}$ as the time line, we have captured what is called a *continuous dynamical system*.

Example 4.2.3.4. Recall (see []) that a *real vector space* is a set X , elements of which are called *vectors*, which is closed under addition and scalar multiplication. For example $\mathbb{R}^3$

⁴The topology is given by saying that $U \subseteq \mathbb{R}$ is open iff for every $x \in U$ there exists $\epsilon > 0$ such that $\{y \in \mathbb{R} \mid |y - x| < \epsilon\} \subseteq U$. One says, “ $U \subseteq \mathbb{R}$ is open if every point in U has an epsilon-neighborhood fully contained in U ”.

⁵The topology on $\mathbb{R} \times \mathbb{R}$ is similar; a subset $U \subseteq \mathbb{R} \times \mathbb{R}$ is open if every point $x \in U$ has an epsilon-neighborhood (a disk around x of some positive radius) fully contained in U .

is a vector space. A *linear transformation from X to Y* is a function $f: X \rightarrow Y$ that appropriately preserves addition and scalar multiplication. The *category of real vector spaces*, denoted $\mathbf{Vect}_{\mathbb{R}}$, has as objects the real vector spaces and as morphisms the linear transformations.

There is a functor $\mathbf{Vect}_{\mathbb{R}} \rightarrow \mathbf{Grp}$ sending a vector space to its underlying group of vectors, where the group operation is addition of vectors and the group identity is the 0-vector.

Exercise 4.2.3.5. Every vector space has vector subspaces, ordered by inclusion (the origin is inside of any line which is inside of certain planes, etc., and all are inside of the whole space V). If you know about this topic, answer the following questions.

- a.) Does a linear transformation $V \rightarrow V'$ induce a morphism of these orders? In other words, is there a functor $\mathbf{Vect}_{\mathbb{R}} \rightarrow \mathbf{PrO}$?
- b.) Would you guess that there is a nice functor $\mathbf{Vect}_{\mathbb{R}} \rightarrow \mathbf{Top}$? By a “nice functor” I mean one that doesn’t make people roll their eyes (for example, there is a functor $\mathbf{Vect}_{\mathbb{R}} \rightarrow \mathbf{Top}$ that sends every vector space to the empty space, and that’s not really a “nice” one. If someone asked for a functor $\mathbf{Vect}_{\mathbb{R}} \rightarrow \mathbf{Top}$ for their birthday, this functor would make them sad. We’re looking for a functor $\mathbf{Vect}_{\mathbb{R}} \rightarrow \mathbf{Top}$ that would make them happy.)

◇

4.2.3.6 Groupoids

Groupoids are like groups except a groupoid can have more than one object.

Definition 4.2.3.7. A *groupoid* is a category $\mathcal{C}$ such that every morphism is an isomorphism. If $\mathcal{C}$ and $\mathcal{D}$ are groupoids, a *morphism of groupoids*, denoted $F: \mathcal{C} \rightarrow \mathcal{D}$, is simply a functor. The category of groupoids is denoted $\mathbf{Grpd}$.

Example 4.2.3.8. There is a functor $\mathbf{Grpd} \rightarrow \mathbf{Cat}$, sending a groupoid to its underlying category. There is also a functor $\mathbf{Grp} \rightarrow \mathbf{Grpd}$ sending a group to “itself as a groupoid with one object.”

Application 4.2.3.9. Let M be a material in some original state s_0 .⁶ Construct a category $\mathcal{S}_M$ whose objects are the states of M , e.g. by pulling on M in different ways, or by heating it up, etc. we obtain such states. Include a morphism from state s to state s' if there exists a physical transformation from s to s' . Physical transformations can be performed one after another, so we can compose morphisms, and perhaps we can agree this composition is associative. Note that there exists a morphism $i_s: s_0 \rightarrow s$ for any s . Note also that this category is a preorder because there either exists a physical transformation or there does not.⁷

The **elastic deformation region** of the material is the set of states s such that there exists a morphism $s \rightarrow s_0$, because any such morphism will be the inverse of $i_s: s_0 \rightarrow s$. A transformation is irreversible if there is no transformation back. If s_1 is not in the elastic deformation region, we can (inventing a term) still talk about the region that is

⁶This example may be a bit crude, in accordance with the crudeness of my understanding of materials science.

⁷Someone may choose to beef this category up to include the set of physical processes between states as the hom-set. This gives a category that is not a preorder. But there would be a functor from their category to ours.

“elastically-equivalent” to s_1 . It is all the objects in $\mathcal{S}_M$ that are isomorphic to s_1 . If we consider only elastic equivalences, we are looking at a groupoid sitting inside the larger category $\mathcal{S}_M$.

◇◇

Example 4.2.3.10. Alan Weinstein **explains** groupoids in terms of tiling patterns on a bathroom floor, see [WeA].

Example 4.2.3.11. Let $I = \{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$ denote the unit interval. It can be given a topology in a standard way, as a subset of $\mathbb{R}$ (see Example 4.2.3.3)

For any space X , a *path in X* is a continuous map $I \rightarrow X$. Two paths are called *homotopic* if one can be continuously deformed to the other, where the deformation occurs completely within X .⁸ One can prove that being homotopic is an equivalence relation on paths.

Paths in X can be composed, one after the other, and the composition is associative (up to homotopy). Moreover, for any point $x \in X$ there is a trivial path (that stays at x). Finally every path is invertible (by traversing it backwards) up to homotopy.

This all means that to any space X we can associate a groupoid, called the *fundamental groupoid of X* and denoted $\Pi_1(X) \in \text{Ob}(\mathbf{Grpd})$. The objects of $\Pi_1(X)$ are the points of X ; the morphisms in $\Pi_1(X)$ are the paths in X (up to homotopy). A continuous map $f: X \rightarrow Y$ can be composed with any path $I \rightarrow X$ to give a path $I \rightarrow Y$ and this preserves homotopy. So in fact $\Pi_1: \mathbf{Top} \rightarrow \mathbf{Grpd}$ is a functor.

Exercise 4.2.3.12. Let T denote the surface of a donut, i.e. a torus. Choose two points $p, q \in T$. Since $\Pi_1(T)$ is a groupoid, it is also a category. What would the hom-set $\text{Hom}_{\Pi_1(T)}(p, q)$ represent? ◇

Exercise 4.2.3.13. Let $U \subseteq \mathbb{R}^2$ be an open subset of the plane, and let F be any vector field on U . Following Exercise 4.1.1.10, we have a category $\mathcal{C}_F$. Take it on faith (or read **this** to see) that if two curves C, C' in U are homotopic then they have the same line integral, $\int_C F = \int_{C'} F$.

We also have a category $\Pi_1 U$, given by the fundamental groupoid, as in Example 4.2.3.11. Both categories have the same objects, $\text{Ob}(\mathcal{C}_F) = |U| = \text{Ob}(\Pi_1 U)$, the set of points in U .

- a.) Is there a functor $\mathcal{C}_F \rightarrow \Pi_1 U$ or a functor $\Pi_1 U \rightarrow \mathcal{C}_F$ that is identity on the underlying objects?
- b.) What is $\mathcal{C}_F$ if F is a conservative vector field?

◇

Exercise 4.2.3.14. Consider the set A of all (well-formed) arithmetic expressions in the symbols $\{0, \dots, 9, +, -, *, (,)\}$. For example, here are some elements of A :

$$52, \quad 52 - 7, \quad 50 + 3 * (6 - 2).$$

We can say that an equivalence between two arithmetic expressions is a justification that they give the same “final answer”, e.g. $52 + 60$ is equivalent to $10 * (5 + 6) + (2 + 0)$, which

⁸ Let $I^2 = \{(x, y) \in \mathbb{R}^2 \mid 0 \leq x \leq 1 \text{ and } 0 \leq y \leq 1\}$ denote the square. There are two inclusions $i_0, i_1: I \rightarrow I^2$ that put the interval inside the square at the left and right arrows. Two paths $f_0, f_1: I \rightarrow X$ are homotopic if there exists a continuous map $f: I \times I \rightarrow X$ such that $f_0 = f \circ i_0$ and $f_1 = f \circ i_1$,

$$I \begin{array}{c} \xrightarrow{i_0} \\ \xrightarrow{i_1} \end{array} I \times I \xrightarrow{f} X$$

is equivalent to $10 * 11 + 2$. I've basically described a groupoid. What are its objects and what are its morphisms? $\diamond$

4.2.4 Logic, set theory, and computer science

4.2.4.1 The category of propositions

Logical statements are either true or false. They are statements like

For all real numbers $x \in \mathbb{R}$ there exists a real number $y \in \mathbb{R}$ such that $y > 3x$.

Knowing that a similar statement with 3 replaced by 4 is true may or may not *imply* the above statement. There is a category **Prop** whose objects are logical statements (propositions) and whose morphisms are proofs that one statement implies another. Crudely, one might say that B *holds at least as often as* A if there is a morphism $A \rightarrow B$ (meaning whenever A holds, so does B). So the proposition “ $x \neq x$ ” holds very seldom and “ $x = x$ ” always very often.

Example 4.2.4.2. We can repeat this idea for non-mathematical statements. Take all possible statements that are verifiable by experiment as objects of a category. Given two such statements, it may be that one implies the other (e.g. “if the speed of light is fixed then there are relativistic effects”). Every statement implies itself (identity) and implication is transitive, so we have a category.

Let's consider differences in proofs to be irrelevant, so the category **Prop** becomes a preorder: either A implies B or it does not. Then it makes sense to discuss meets and joins. It turns out that meets are “and's” and joins are “or's”. That is, given propositions A, B the meet $A \wedge B$ is defined to be a proposition that holds as often as possible subject to the constraint that it implies both A and B ; the proposition “ A holds and B holds” fits the bill. Similarly, the join $A \vee B$ is given by “ A holds or B holds”.

Exercise 4.2.4.3. Consider the set of possible laws (most likely an infinite set) that can be dictated to hold throughout a jurisdiction. Consider each law as a proposition (“such and such is (dictated to be) the case”), i.e. as an object of our preorder **Prop**. Given a jurisdiction V , and a set of laws $\{\ell_1, \ell_2, \dots, \ell_n\}$ that are dictated to hold throughout V , we take their meet $L(V) := \ell_1 \wedge \ell_2 \wedge \dots \wedge \ell_n$ and consider it to be the single law of the land V . Suppose that V is a jurisdiction and $U \subseteq V$ is a sub-jurisdiction (e.g. U is a county and V is a state); then clearly any law dictated by the large jurisdiction (the state) must also hold throughout the small jurisdiction (the county).

- a.) What is the relation in **Prop** between $L(U)$ and $L(V)$?
- b.) Consider the obvious preorder J on jurisdictions, where $U \subseteq V$ means $U \leq V$. Is “the law of the land” a morphism of preorders $J \rightarrow \mathbf{Prop}$? To be a bit more high-brow, considering both J and **Prop** to be categories (by Proposition 4.2.1.17), we have a function $L: \text{Ob}(J) \rightarrow \text{Ob}(\mathbf{Prop})$; this question is asking whether L extends to a functor $J \rightarrow \mathbf{Prop}$.⁹

$\diamond$

Exercise 4.2.4.4. Take again the preorder J of jurisdictions from Exercise 4.2.4.3 and the idea that laws are propositions. But this time, let $R(V)$ be the set of all possible laws

⁹Hint: Exercises 4.2.4.3 and 4.2.4.4 will ask similar yes/no questions and at least one of these is correctly answered “no”.

(not just those dictated to hold) that are in actuality being respected, i.e. followed, by all people in V . This assigns to each jurisdiction a set.

- a.) Since preorders can be considered categories, does our “the set of respected laws” function $R: \text{Ob}(J) \rightarrow \text{Ob}(\mathbf{Set})$ extend to a functor $J \rightarrow \mathbf{Set}$?
- b.) What about if instead we take the meet of all these laws and assign to each jurisdiction the maximal law respected throughout. Does this assignment $\text{Ob}(J) \rightarrow \text{Ob}(\mathbf{Prop})$ extend to a functor $J \rightarrow \mathbf{Prop}$?⁹

◇

4.2.4.5 A categorical characterization of $\mathbf{Set}$

The category $\mathbf{Set}$ of sets is fundamental in mathematics, but instead of thinking of it as something given or somehow special, it can be shown to merely be a category with certain properties, each of which can be phrased purely categorically. This was shown by Lawvere [Law]. A very readable account is given in [Le2].

4.2.4.6 Categories in computer science

Computer science makes heavy use of trees, graphs, orders, lists, and monoids. We have seen that all of these are naturally viewed in the context of category theory, though it seems that such facts are rarely mentioned explicitly in computer science textbooks. However, categories are also used explicitly in the theory of programming languages (PL). Researchers in that field attempt to understand the connection between what programs are supposed to do (their denotation) and what they actually cause to occur (their operation). Category theory provides a useful mathematical formalism in which to study this.

The kind of category most often considered by a PL researcher is what is known as a *Cartesian closed category* or *CCC*, which means a category $\mathcal{T}$ that has products (like $A \times B$ in $\mathbf{Set}$) and exponential objects (like B^A in $\mathbf{Set}$). $\mathbf{Set}$ is an example of a CCC, but there are others that are more appropriate for actual computation. The objects in a PL person’s CCC represent the *types* of the language, types such as **integers**, **strings**, **floats**. The morphisms represent computable functions, e.g. **length: strings $\rightarrow$ integers**. The products allow one to discuss pairs (a, b) where a is of one type and b is of another type. Exponential objects allow one to consider computable functions as things that can be input to a function (e.g. given any computable function **floats $\rightarrow$ integers** one can consistently multiply its results by 2 and get a new computable function **floats $\rightarrow$ integers**). We will be getting to products in Section 4.5.1.8 and exponential objects in Section 4.3.2.

But category theory did not only offer a language for thinking about programs, it offered an unexpected tool called monads. The above CCC model for types allows researchers only to discuss functions, leading to the notion of functional programming languages; however, not all things that a computer does are functions. For example, reading input and output, changing internal state, etc. are operations that can be performed that ruin the functional-ness of programs. Monads were found in 19?? by Moggi [Mog] to provide a powerful abstraction that opens the doors to such non-functional operations without forcing the developer to leave the category-theoretic garden of eden. We will discuss monads in Section 5.3.

We have also seen in Section 4.2.2 that databases are well captured by the language of categories. We will formalize this in Section 4.4. Throughout the remainder of this book we will continue to use databases to bring clarity to concepts within standard category theory.

4.2.5 Miscellaneous

Categories are being used throughout mathematics to relate various subjects, as well as to draw out the essential structures within these subjects. For example, there is an active research for “categorifying” classical theories like that of knots, links, and braids [1].

Quantum field theory is also being categorified, with much success [2]. In this domain, one takes a category in which an object is a reasonable space, called a manifold, and a morphism is a manifold connecting two manifolds, like a cylinder connects two circles. Such connecting manifolds are called cobordisms, and as such people refer to the category as **Cob**. Topological quantum field theory is the study of functors $\mathbf{Cob} \rightarrow \mathbf{Vect}$ that assign a vector space to each manifold and a linear transformation of vector spaces to each cobordism.

Information theory¹⁰ is the study of how to ideally compress messages so that they can be sent quickly and accurately across a noisy channel.¹¹ Invented in 1948 by Claude Shannon, its main quantity of interest is the number of bits necessary to encode a piece of information. For example, the amount of information in an English sentence can be greatly reduced. The fact that **t**’s are often followed by **h**’s, or that **e**’s are much more common than **z**’s, implies that letters are not being used as efficiently as possible. The amount of bits necessary to encode a message is called its *entropy* and has been linked to the commonly used notion of the same name in physics.

In [3], Baez and Leinster show that entropy can be captured quite cleanly using category theory. They make a category **FinProb** whose objects are finite sets equipped with a probability measure, and whose morphisms are probability preserving functions. They characterize *information loss* as a way to assign numbers to such morphisms, subject to certain explicit constraints. They then show that the entropy of an object in **FinProb** is the amount of information lost under the unique map to the singleton set $\{\odot\}$. This approach explicates (by way of the explicit constraints for information loss functions) the essential idea of Shannon’s information theory, allowing it to be generalized to categories other than **FinProb**. Thus Baez and Leinster effectively *categorified* information theory.

4.3 Natural transformations

In this section we conclude our discussion of the **Big 3**, by defining natural transformations. Category theory was originally invented to discuss natural transformations. These

¹⁰To me, the subject of “information theory” is badly named. That discipline is devoted to finding ideal compression schemes for messages to be sent quickly and accurately across a noisy channel. It deliberately does not pay any attention to what the messages mean. To my mind this should be called compression theory or redundancy theory. Information is inherently meaningful—that is its purpose—any theory that is unconcerned with the meaning is not really studying information per se. The people who decide on speed limits for roads and highways may care about human health, but a study limited to deciding ideal speed limits should not be called “human health theory”.

¹¹Despite what was said above, Information theory has been extremely important in a diverse array of fields, including computer science [4], but also in neuroscience [5] and physics [6]. I’m not trying to denigrate the field; I am only frustrated with its name.

were sufficiently conceptually challenging that they required formalization and thus the invention of category theory. If we think of categories as domains (of discourse, interaction, comparability, etc.) and of functors as transformations between different domains, the natural transformations compare different transformations.

Natural transformations can seem a bit abstruse at first, but hopefully some examples and exercises will help.

4.3.1 Definition and examples

Definition 4.3.1.1. Let $\mathcal{C}$ and $\mathcal{D}$ be categories and let $F: \mathcal{C} \rightarrow \mathcal{D}$ and $G: \mathcal{C} \rightarrow \mathcal{D}$ be functors. A *natural transformation* α from F to G , denoted $\alpha: F \rightarrow G$, consists of some constituents (A. components) conforming to some laws (1. naturality squares), as follows:

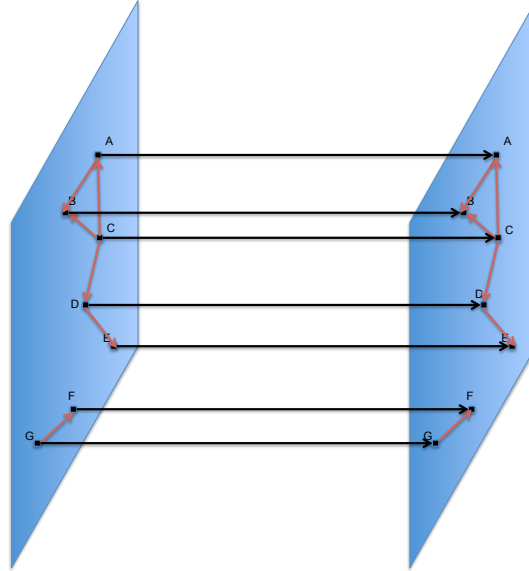
- A. For each object $c \in \text{Ob}(\mathcal{C})$ a morphism $\alpha_c: F(c) \rightarrow G(c)$ in $\mathcal{D}$, called *the c -component of α* .

The following law harmonizes the above constituents:

1. For every morphism $h: c \rightarrow c'$ in $\mathcal{C}$, the following square must commute:

$$\begin{array}{ccc} F(c) & \xrightarrow{\alpha_c} & G(c) \\ F(h) \downarrow & \checkmark & \downarrow G(h) \\ F(c') & \xrightarrow{\alpha_{c'}} & G(c') \end{array} \quad (4.9)$$

To think about a natural transformation between functors $F, G: \mathcal{C} \rightarrow \mathcal{D}$, a picture to have in mind is this:

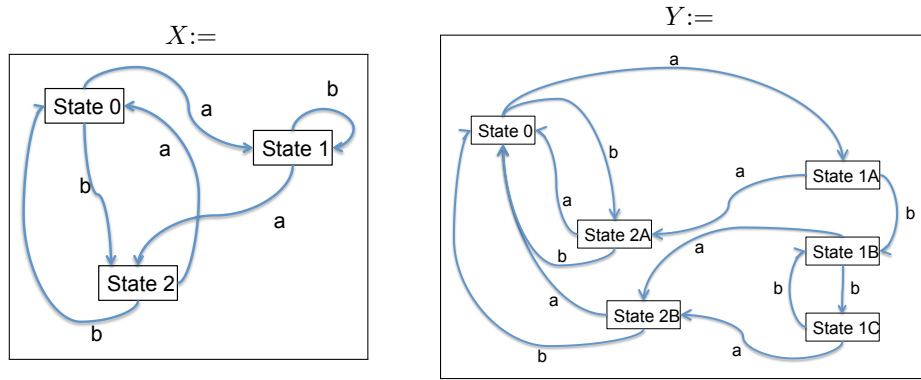


Both panes are pictures of $\mathcal{C}$, and we imagine F as a mapping from the left pane to $\mathcal{D}$ and G as a mapping from the right pane to $\mathcal{D}$. There is a horizontal arrows for

each object $c \in \text{Ob}(\mathcal{C})$ and we imagine these as the components α_c of α . To each arrow $h: c \rightarrow c'$ in $\mathcal{C}$ we see a commutative square as in (4.9): its left side is $F(h)$, its right side is $G(h)$, its top is α_c and its bottom is $\alpha_{c'}$. This picture will be made formal in Example 4.5.3.22.

Application 4.3.1.2. In Figure 3.1 we drew a **finite state machine** on alphabet $\Sigma = \{a, b\}$, and in Example 3.1.3.1 we showed the associated action table. It will be reproduced below. Imagine this was your model for understanding the behavior of some system when acted on by commands a and b . And suppose that a collaborator tells you that she has a more refined notion that fits with the same data. Her notion has 6 states rather than 3, but it's “compatible”. What might that mean?

Let's call the original state machine X and the new model Y .



The action tables for these two machines are:

Original model X		
ID	a	b
State 0	State 1	State 2
State 1	State 2	State 1
State 2	State 0	State 0

Proposed model Y		
ID	a	b
State 0	State 1A	State 2A
State 1A	State 2A	State 1B
State 1B	State 2B	State 1C
State 1C	State 2B	State 1B
State 2A	State 0	State 0
State 2B	State 0	State 0

How are these models compatible? Looking at the table for Y , if one removes the distinction between States 1A, 1B, 1C and between States 2A and 2B, then one returns with the table for X . The table for Y is more specific, but it is fully compatible with table X . The sense in which it is compatible is precisely the sense defined by their being a natural transformation.

Recall that $\mathcal{M} = (\text{List}(\Sigma), [], ++)$ is a monoid, and that a monoid is simply a category with one object, say $\text{Ob}(\mathcal{M}) = \{\blacktriangle\}$ (see Section 4.2.1). With $\Sigma = \{a, b\}$, the monoid $\mathcal{M}$ can be visualized as follows:

$$\mathcal{M} = \begin{array}{c} \text{a} \quad \text{b} \\ \text{---} \text{ } \text{---} \end{array}$$

Recall also that a state machine on $\mathcal{M}$ is simply a functor $\mathcal{M} \rightarrow \mathbf{Set}$. We thus have two such functors, X and Y . A natural transformation $\alpha: Y \rightarrow X$ would consist of a component α_m for every object $m \in \text{Ob}(\mathcal{M})$, such that certain diagrams commute. But

$\mathcal{M}$ having only one object, we need only one function $\alpha_{\blacktriangle}: Y(\blacktriangle) \rightarrow X(\blacktriangle)$, where $Y(\blacktriangle)$ is the set of (6) states of Y and $X(\blacktriangle)$ is the set of (3) states of X .

The states of Y have been named so as to make the function $\alpha_{\blacktriangle}$ particularly easy to guess.¹² We need to check that two squares commute:

$$\begin{array}{ccc}
 Y(\blacktriangle) & \xrightarrow{\alpha_{\blacktriangle}} & X(\blacktriangle) \\
 Y(a) \downarrow & & \downarrow X(a) \\
 Y(\blacktriangle) & \xrightarrow{\alpha_{\blacktriangle}} & X(\blacktriangle)
 \end{array}
 \qquad
 \begin{array}{ccc}
 Y(\blacktriangle) & \xrightarrow{\alpha_{\blacktriangle}} & X(\blacktriangle) \\
 Y(b) \downarrow & & \downarrow X(b) \\
 Y(\blacktriangle) & \xrightarrow{\alpha_{\blacktriangle}} & X(\blacktriangle)
 \end{array}
 \quad (4.10)$$

This can only be checked by going through and making sure certain things match, as specified by (4.10); we spell it out in gory detail. The columns that should match are those whose entries are written in blue.

Naturality square for $a: \blacktriangle \rightarrow \blacktriangle$				
$Y(\blacktriangle)$ [ID]	$Y(a)$	$\alpha_{\blacktriangle} \circ Y(a)$	$\alpha_{\blacktriangle}$	$X(a) \circ \alpha_{\blacktriangle}$
State 0	State 1A	State 1	State 0	State 1
State 1A	State 2A	State 2	State 1	State 2
State 1B	State 2B	State 2	State 1	State 2
State 1C	State 2B	State 2	State 1	State 2
State 2A	State 0	State 0	State 2	State 0
State 2B	State 0	State 0	State 2	State 0

(4.11)

Naturality square for $b: \blacktriangle \rightarrow \blacktriangle$				
$Y(\blacktriangle)$ [ID]	$Y(b)$	$\alpha_{\blacktriangle} \circ Y(b)$	$\alpha_{\blacktriangle}$	$X(b) \circ \alpha_{\blacktriangle}$
State 0	State 2A	State 2	State 0	State 2
State 1A	State 1B	State 1	State 1	State 1
State 1B	State 1C	State 1	State 1	State 1
State 1C	State 1B	State 1	State 1	State 1
State 2A	State 0	State 0	State 2	State 0
State 2B	State 0	State 0	State 2	State 0

(4.12)

In reality we need to check that for *every* morphism in $\mathcal{M}$, such as $[a, a, b]$, a similar diagram commutes. But this holds automatically. For example (flipping the naturality square sideways for typographical reasons)

$$\begin{array}{ccccccc}
 Y(\blacktriangle) & \xrightarrow{Y(a)} & Y(\blacktriangle) & \xrightarrow{Y(a)} & Y(\blacktriangle) & \xrightarrow{Y(b)} & Y(\blacktriangle) \\
 \downarrow \alpha_{\blacktriangle} & & \downarrow \alpha_{\blacktriangle} & & \downarrow \alpha_{\blacktriangle} & & \downarrow \alpha_{\blacktriangle} \\
 X(\blacktriangle) & \xrightarrow{X(a)} & X(\blacktriangle) & \xrightarrow{X(a)} & X(\blacktriangle) & \xrightarrow{X(b)} & X(\blacktriangle)
 \end{array}$$

Since each small square above commutes (as checked by tables 4.11 and 4.12), the big outer rectangle commutes too.

¹²The function $\alpha_{\blacktriangle}: Y(\blacktriangle) \rightarrow X(\blacktriangle)$ makes the following assignments: State 0 $\mapsto$ State 0, State 1A $\mapsto$ State 1, State 1B $\mapsto$ State 1, State 1C $\mapsto$ State 1, State 2A $\mapsto$ State 2, State 2B $\mapsto$ State 2.

To recap, the notion of compatibility between Y and X is one that can be checked and agreed upon by humans, but doing so it is left implicit, and it may be difficult to explain to an outsider what exactly was agreed to, especially in more complex situations. It is quite convenient to simply claim “there is a natural transformation from Y to X .”

◇◇

Exercise 4.3.1.3. Let $F: \mathcal{C} \rightarrow \mathcal{D}$ be a functor. Suppose someone said “the identity on F is a natural transformation from F to itself.”

a.) What might they mean?

b.) If it is somehow true, what are the components of this natural transformation?

◇

Example 4.3.1.4. Let $[1] \in \text{Ob}(\mathbf{Cat})$ be the free arrow category described in Exercise 4.1.2.28 and let $\mathcal{D}$ be any category. To specify a functor $F: [1] \rightarrow \mathcal{D}$ requires the specification of two objects, $F(v_1), F(v_2) \in \text{Ob}(\mathcal{D})$ and a morphism $F(e): F(v_1) \rightarrow F(v_2)$ in $\mathcal{D}$. The identity and composition formulas are taken care of once that much is specified. To recap, a functor $F: [1] \rightarrow \mathcal{D}$ is the same thing as a morphism in $\mathcal{D}$.

Thus, choosing two functors $F, G: [1] \rightarrow \mathcal{D}$ is precisely the same thing as choosing two morphisms in $\mathcal{D}$. Let us call them $f: a_0 \rightarrow a_1$ and $g: b_0 \rightarrow b_1$, where to be clear we have $f = F(e), a_0 = F(v_0), a_1 = F(v_1)$ and $g = G(e), b_0 = G(v_0), b_1 = G(v_1)$.

A natural transformation $\alpha: F \rightarrow G$ consists of two components, $h_0 := \alpha_{v_0}: a_0 \rightarrow b_0$ and $h_1 := \alpha_{v_1}: a_1 \rightarrow b_1$, drawn as dashed lines below:

$$\begin{array}{ccc} a_0 & \xrightarrow{h_0} & b_0 \\ f \downarrow & & \downarrow g \\ a_1 & \xrightarrow{h_1} & b_1 \end{array}$$

The condition for α to be a natural transformation is that the above square commutes.

In other words, a functor $[1] \rightarrow \mathcal{D}$ is an arrow in $\mathcal{D}$ and a natural transformation between two such functors is just a commutative square in $\mathcal{D}$.

Example 4.3.1.5. Recall that to any graph G we can associate the so-called paths-graph $\text{Paths}(G)$, as described in Example 4.1.2.19. This is a functor $\text{Paths}: \mathbf{Grph} \rightarrow \mathbf{Grph}$. There is also an identity functor $\text{id}_{\mathbf{Grph}}: \mathbf{Grph} \rightarrow \mathbf{Grph}$. A natural transformation $\eta: \text{id}_{\mathbf{Grph}} \rightarrow \text{Paths}$ would consist of a graph homomorphism $\eta_G: \text{id}_{\mathbf{Grph}}(G) \rightarrow \text{Paths}(G)$ for every graph G . But $\text{id}_{\mathbf{Grph}}(G) = G$ by definition, so we need $\eta_G: G \rightarrow \text{Paths}(G)$. Recall that $\text{Paths}(G)$ has the same vertices as G and every arrow in G counts as a path (of length 1). So there is an obvious graph homomorphism from G to $\text{Paths}(G)$. It is not hard to see that the necessary naturality squares commute.

Example 4.3.1.6. For any graph G we can associate the paths-graph $\text{Paths}(G)$, and nothing stops us from doing that twice to yield a new graph $\text{Paths}(\text{Paths}(G))$. Let's think through what a path of paths in G is. It's a head-to-tail sequence of arrows in $\text{Paths}(G)$, meaning a head-to-tail sequence of paths in G . These composable sequences of paths (or “paths of paths”) are the individual arrows in $\text{Paths}(\text{Paths}(G))$. (The vertices in $\text{Paths}(G)$ and $\text{Paths}(\text{Paths}(G))$ are the same as those in G , and all source and target functions are as expected.)

Clearly, given such a sequence of paths in G , we could compose them to one big path in G with the same endpoints. In other words, there is graph morphism $\mu_G: \text{Paths}(\text{Paths}(G)) \rightarrow$

$\text{Paths}(G)$, that one might call “concatenation”. In fact, this concatenation extends to a natural transformation

$$\mu: \text{Paths} \circ \text{Paths} \rightarrow \text{Paths}$$

between functors $\mathbf{Grph} \rightarrow \mathbf{Grph}$. In Example 4.3.1.5, we compared a graph to its paths-graph using a natural transformation $\text{id}_{\mathbf{Grph}} \rightarrow \text{Paths}$; here we are making a similar kind of comparison.

Remark 4.3.1.7. In Example 4.3.1.5 we saw that there is a natural transformation sending each graph into its paths-graph. There is a formal sense in which a category is nothing more than a kind of reverse mapping. That is, to specify a category is the same thing as to specify a graph G together with a graph homomorphism $\text{Paths}(G) \rightarrow G$. The formalities involve monads, which we will discuss in Section 5.3.

Example 4.3.1.8. For any set X , there is a set $\text{List}(X)$ consisting of all lists whose entries are elements of X . In fact $\text{List}: \mathbf{Set} \rightarrow \mathbf{Set}$ is a functor because for any function $f: X \rightarrow Y$ we can apply it entry-by-entry to a list of X ’s to get a list of Y ’s (this was worked out in Exercise 4.1.2.17). But then we can compose List with itself to get a functor $\text{List} \circ \text{List}: \mathbf{Set} \rightarrow \mathbf{Set}$. Given $X \in \text{Ob}(\mathbf{Set})$, the set $\text{ListList}(X)$ is the set of lists of lists. For example if $X = \{1, 2, 3\}$ then $\text{ListList}(X)$ contains $[[1, 2, 3, 2], [2, 2], [], [3]]$.

There is a natural transformation $B: \text{ListList} \rightarrow \text{List}$ given by taking a list of lists and flattening it to a list. For example the above element would be sent under $B_X: \text{ListList}(X) \rightarrow \text{List}(X)$ to $[1, 2, 3, 2, 2, 2, 3]$. It is not hard to see that the relevant naturality square commutes for any $f: X \rightarrow Y$.

Exercise 4.3.1.9. Let $\text{List}: \mathbf{Set} \rightarrow \mathbf{Set}$ be as above.

- a.) If someone said “singleton lists give a natural transformation from $\text{id}_{\mathbf{Set}}$ to List ”, what might they mean?
- b.) Would they be correct?

◇

Exercise 4.3.1.10. Let X and Y be sets, and let $f: X \rightarrow Y$. There is a functor $C_X: \mathbf{Grph} \rightarrow \mathbf{Set}$ that sends every graph to the set X and sends every morphism of graphs to the identity morphism $\text{id}_X: X \rightarrow X$. This functor is called *the constant functor at X* . Similarly there is a constant functor $C_Y: \mathbf{Grph} \rightarrow \mathbf{Set}$.

- a.) Use f to construct a natural transformation $C_X \rightarrow C_Y$.
- b.) What are its components?

◇

Exercise 4.3.1.11. For any graph $(V, A, \text{src}, \text{tgt})$ we can extract the set of arrows or the set of vertices. Since each morphism of graphs includes a function between their arrow sets and a function between their vertex sets, we actually have functors $Ar: \mathbf{Grph} \rightarrow \mathbf{Set}$ and $Ve: \mathbf{Grph} \rightarrow \mathbf{Set}$.

- a.) If someone said “taking source vertices gives a natural transformation from Ar to Ve ”, what natural transformation might they be referring to?
- b.) What are its components?
- c.) If a different person, say from a totally different country, were to say “taking target vertices also gives a natural transformation from Ar to Ve ,” would they also be correct?

◇

Example 4.3.1.12. [Graph homomorphisms are natural transformations]

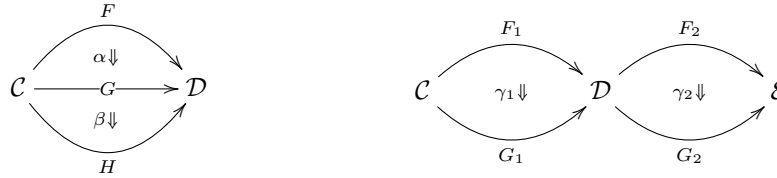
As discussed above (see Diagram 4.7), there is a category $\mathcal{C}$ for which a functor $G: \mathcal{C} \rightarrow \mathbf{Set}$ is the same thing as a graph. Namely, we have

$$\mathcal{C} := \boxed{\begin{array}{ccc} A & \xrightarrow{\text{src}} & V \\ & \text{tgt} & \bullet \end{array}}$$

A natural transformation of two such functors $\alpha: G \rightarrow G'$ involves two components, $\alpha_A: G(A) \rightarrow G'(A)$ and $\alpha_V: G(V) \rightarrow G'(V)$, and two naturality squares, one for *src* and one for *tgt*. This is precisely the same thing as a graph homomorphism, as defined in Definition 3.3.3.1.

4.3.2 Vertical and horizontal composition

In this section we discuss two types of compositions for natural transformations. The terms vertical and horizontal are used to describe them; these terms come from the following pictures:



We generally use $\circ$ to denote both kinds of composition, but if we want to be very clear we will differentiate as follows: $\beta \circ \alpha: F \rightarrow H$ for vertical composition, and $\gamma_2 \circ \gamma_1: F_2 \circ F_1 \rightarrow G_2 \circ G_1$ for horizontal composition. Of course, the actual arrangement of things on a page of text does not correlate with verticality or horizontality—these are just names. We will define them more carefully below.

4.3.2.1 Vertical composition of natural transformations

The following proposition proves that functors and natural transformations (using vertical composition) form a category.

Proposition 4.3.2.2. *Let $\mathcal{C}$ and $\mathcal{D}$ be categories. There exists a category, called the category of functors from $\mathcal{C}$ to $\mathcal{D}$ and denoted $\mathbf{Fun}(\mathcal{C}, \mathcal{D})$, whose objects are the functors $\mathcal{C} \rightarrow \mathcal{D}$ and whose morphisms are the natural transformations,*

$$\mathbf{Hom}_{\mathbf{Fun}(\mathcal{C}, \mathcal{D})}(F, G) = \{\alpha: F \rightarrow G \mid \alpha \text{ is a natural transformation}\}.$$

That is, there are identity natural transformations, natural transformations can be composed, and the identity and associativity laws hold.

Proof. We showed in Exercise 4.3.1.3 that there for any functor $F: \mathcal{C} \rightarrow \mathcal{D}$, there is an identity natural transformation $\text{id}_F: F \rightarrow F$ (its component at $c \in \text{Ob}(\mathcal{C})$ is $\text{id}_{F(c)}: F(c) \rightarrow F(c)$).

Given a natural transformation $\alpha: F \rightarrow G$ and a natural transformation $\beta: G \rightarrow H$, we propose for the composite $\beta \circ \alpha$ the transformation $\gamma: F \rightarrow H$ having components

$\beta_c \circ \alpha_c$ for every $c \in \text{Ob}(\mathcal{C})$. To see that γ is indeed a natural transformation, one simply puts together naturality squares for α and β to get naturality squares for $\beta \circ \alpha$.

The associativity and identity laws for $\text{Fun}(\mathcal{C}, \mathcal{D})$ follow from those holding for morphisms in $\mathcal{D}$. □

Notation 4.3.2.3. We sometimes denote the category $\text{Fun}(\mathcal{C}, \mathcal{D})$ by $\mathcal{D}^{\mathcal{C}}$.

Example 4.3.2.4. Recall from Exercise 4.1.2.35 that there is a functor $\text{Ob}: \mathbf{Cat} \rightarrow \mathbf{Set}$ sending a category to its set of objects. And recall from Example 4.1.2.32 that there is a functor $\text{Disc}: \mathbf{Set} \rightarrow \mathbf{Cat}$ sending a set to the discrete category with that set of objects (all morphisms in $\text{Disc}(S)$ are identity morphisms). Let $P: \mathbf{Cat} \rightarrow \mathbf{Cat}$ be the composition $P = \text{Disc} \circ \text{Ob}$. Then P takes a category and makes a new category with the same objects but no morphisms. It's like crystal meth for categories.

Let $\text{id}_{\mathbf{Cat}}: \mathbf{Cat} \rightarrow \mathbf{Cat}$ be the identity functor. There is a natural transformation $i: P \rightarrow \text{id}_{\mathbf{Cat}}$. For any category $\mathcal{C}$, the component $i_{\mathcal{C}}: P(\mathcal{C}) \rightarrow \mathcal{C}$ is pretty easily understood. It is a morphism of categories, i.e. a functor. The two categories $P(\mathcal{C})$ and $\mathcal{C}$ have the same set of objects, namely $\text{Ob}(\mathcal{C})$, so our functor is identity on objects; and $P(\mathcal{C})$ has no non-identity morphisms, so nothing else needs be specified.

Exercise 4.3.2.5. Let $\mathcal{C} = \boxed{\begin{smallmatrix} A \\ \bullet \end{smallmatrix}}$ be the category with $\text{Ob}(\mathcal{C}) = \{A\}$, and $\text{Hom}_{\mathcal{C}}(A, A) = \{\text{id}_A\}$. What is $\text{Fun}(\mathcal{C}, \mathbf{Set})$? In particular, characterize the objects and the morphisms. ◇

Exercise 4.3.2.6. Let $n \in \mathbb{N}$ and let $\underline{n}$ be the set with n elements, considered as a discrete category (i.e. we abuse notation and write $\underline{n}$ to mean what should really be called $\text{Disc}(\underline{n})$). Describe the category $\text{Fun}(\underline{3}, \underline{2})$. ◇

Exercise 4.3.2.7. Let $\underline{1}$ denote the discrete category with one object, and let $\mathcal{C}$ be any category.

a.) What are the objects of $\text{Fun}(\underline{1}, \mathcal{C})$?

b.) What are the morphisms of $\text{Fun}(\underline{1}, \mathcal{C})$? ◇

Example 4.3.2.8. Let $\underline{1}$ denote the discrete category with one object (also known as the trivial monoid). For any category $\mathcal{C}$, we investigate the category $\mathcal{D} := \text{Fun}(\mathcal{C}, \underline{1})$. Its objects are functors $\mathcal{C} \rightarrow \underline{1}$. Such a functor F assigns to each object in $\mathcal{C}$ an object in $\underline{1}$ of which there is one; so there is no choice in what F does on objects. And there is only one morphism in $\underline{1}$ so there is no choice in what F does on morphisms. The upshot is that there is only one object in $\mathcal{D}$, let's call it F , in $\mathcal{D}$, so $\mathcal{D}$ is a monoid. What are its morphisms?

A morphism $\alpha: F \rightarrow F$ in $\mathcal{D}$ is a natural transformation of functors. For every $c \in \text{Ob}(\mathcal{C})$ we need a component $\alpha_c: F(c) \rightarrow F(c)$, which is a morphism $1 \rightarrow 1$ in $\underline{1}$. But there is only one morphism in $\underline{1}$, namely id_1 , so there is no choice about what these components should be: they are all id_1 . The necessary naturality squares commute, so α is indeed a natural transformation. Thus the monoid $\mathcal{D}$ is the trivial monoid; that is, $\text{Fun}(\mathcal{C}, \underline{1}) \cong \underline{1}$ for any category $\mathcal{C}$.

Exercise 4.3.2.9. Let $\underline{0}$ represent the discrete category on 0 objects; it has no objects and no morphisms. Let $\mathcal{C}$ be any category. What is $\text{Fun}(\underline{0}, \mathcal{C})$? ◇

Exercise 4.3.2.10. Let $[1]$ denote the free arrow category as in Exercise 4.1.2.28, and let $\mathcal{C}$ be the graph indexing category from (4.7). Draw the category $\text{Fun}([1], \mathcal{C})$. Say whether any diagram you see commutes or not. $\diamond$

4.3.2.11 Natural isomorphisms

Let $\mathcal{C}$ and $\mathcal{D}$ be categories. We have defined a category $\text{Fun}(\mathcal{C}, \mathcal{D})$ whose objects are functors $\mathcal{C} \rightarrow \mathcal{D}$ and whose morphisms are natural transformations. What are the isomorphisms in this category?

Lemma 4.3.2.12. *Let $\mathcal{C}$ and $\mathcal{D}$ be categories and let $F, G: \mathcal{C} \rightarrow \mathcal{D}$ be functors. A natural transformation $\alpha: F \rightarrow G$ is an isomorphism in $\text{Fun}(\mathcal{C}, \mathcal{D})$ if and only if the component $\alpha_c: F(c) \rightarrow G(c)$ is an isomorphism for each object $c \in \text{Ob}(\mathcal{C})$.*

Proof. First suppose that α is an isomorphism with inverse $\beta: G \rightarrow F$, and let $\beta_c: G(c) \rightarrow F(c)$ denote its c component. We know that $\alpha \circ \beta = \text{id}_G$ and $\beta \circ \alpha = \text{id}_F$. Using the definitions of composition and identity given in Proposition 4.3.2.2, this means that for every $c \in \text{Ob}(\mathcal{C})$ we have $\alpha_c \circ \beta_c = \text{id}_{G(c)}$ and $\beta_c \circ \alpha_c = \text{id}_{F(c)}$; in other words α_c is an isomorphism.

Second suppose that each α_c is an isomorphism with inverse $\beta_c: G(c) \rightarrow F(c)$. We need to see that these components assemble into a natural transformation; i.e. for every morphism $f: c \rightarrow c'$ in $\mathcal{C}$ the right-hand square

$$\begin{array}{ccc} F(c) & \xrightarrow{\alpha_c} & G(c) \\ F(h) \downarrow & \checkmark & \downarrow G(h) \\ F(c') & \xrightarrow{\alpha_{c'}} & G(c') \end{array} \quad \begin{array}{ccc} G(c) & \xrightarrow{\beta_c} & F(c) \\ G(h) \downarrow & ? & \downarrow F(h) \\ G(c') & \xrightarrow{\beta_{c'}} & F(c') \end{array}$$

commutes. We know that the left-hand square commutes because α is a natural transformation; we have labeled each square with a ? or a $\checkmark$ accordingly. In the following diagram we want to show that the left-hand square commutes. We know that the middle square commutes.

$$\begin{array}{ccccccc} & & \text{id}_{G(c)} & & & & \\ & \nearrow & & \searrow & & \nearrow & \\ G(c) & \xrightarrow{\beta_c} & F(c) & \xrightarrow{\alpha_c} & G(c) & \xrightarrow{\beta_c} & F(c) \\ G(h) \downarrow & & ? & \downarrow F(h) & \checkmark & \downarrow G(h) & ? & \downarrow F(h) \\ G(c') & \xrightarrow{\beta_{c'}} & F(c') & \xrightarrow{\alpha_{c'}} & G(c') & \xrightarrow{\beta_{c'}} & F(c') \\ & & & \searrow & \nearrow & & \\ & & & \text{id}_{F(c')} & & & \end{array}$$

To complete the proof we need only to show that $F(h) \circ \beta_c = \beta_{c'} \circ G(h)$. This can be shown by a “diagram chase.” We go through it symbolically, for demonstration.

$$F(h) \circ \beta_c = \beta_{c'} \circ \alpha_{c'} \circ F(h) \circ \beta_c = \beta_{c'} \circ G(h) \circ \alpha_c \circ \beta_c = \beta_{c'} \circ G(h).$$

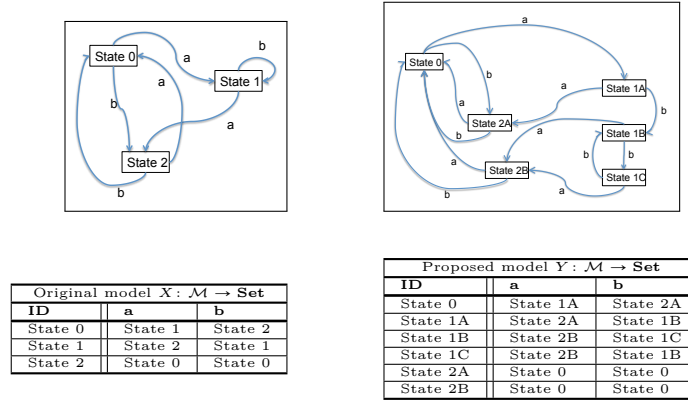
□

Exercise 4.3.2.13. Recall from Application 4.3.1.2 that a finite state machine on alphabet Σ can be understood as a functor $\mathcal{M} \rightarrow \mathbf{Set}$, where $\mathcal{M} = \text{List}(\Sigma)$ is the free monoid generated by Σ . In that example we also discussed how natural transformations provide a nice language for changing state machines. Describe what kinds of changes are made by natural isomorphisms. $\diamond$

4.3.2.14 Horizontal composition of natural transformations

Example 4.3.2.15. Suppose that $\mathcal{M} = \text{List}(a, b)$ and $\mathcal{M}' = \text{List}(m, n, p)$ are free monoids, and let $F: \mathcal{M}' \rightarrow \mathcal{M}$ be given by sending $[m] \mapsto [a]$, $[n] \mapsto [b]$, and $[p] \mapsto [b, a, a]$. An application of this might be if the sequence $[b, a, a]$ was commonly used in practice and one wanted to add a new button just for that sequence.

Recall Application 4.3.1.2. Let $X: \mathcal{M} \rightarrow \mathbf{Set}$ and $Y: \mathcal{M} \rightarrow \mathbf{Set}$ be the functors, and let $\alpha: Y \rightarrow X$ be the natural transformation found there. We reproduce them here:



We can compose X and Y with F as in the diagram below

$$\mathcal{M}' \xrightarrow{F} \mathcal{M} \begin{array}{c} \xrightarrow{Y} \\ \alpha \Downarrow \\ \xrightarrow{X} \end{array} \mathbf{Set}$$

to get functors $Y \circ F$ and $X \circ F$, both of type $\mathcal{M}' \rightarrow \mathbf{Set}$. What would these be? ¹³

$X \circ F$			
ID	m	n	p
State 0	State 1	State 2	State 1
State 1	State 2	State 1	State 0
State 2	State 0	State 0	State 2

$Y \circ F$			
ID	m	n	p
State 0	State 1A	State 2A	State 1A
State 1A	State 2A	State 1B	State 0
State 1B	State 2B	State 1C	State 0
State 1C	State 2B	State 1B	State 0
State 2A	State 0	State 0	State 2A
State 2B	State 0	State 0	State 2A

The map α is what sent both State 1A and State 1B in Y to State 1 in X , and so on. We can see that “the same α works now:” the p column of the table respects that mapping. But α was a natural transformation $Y \rightarrow X$ whereas we need a natural transformation $Y \circ F \rightarrow X \circ F$. This is called *whiskering*. It is a kind of horizontal composition of natural transformation.

¹³The p -column comes from applying b then a then a , as specified above by F .

Definition 4.3.2.16. Let $\mathcal{B}, \mathcal{C}, \mathcal{D}$, and $\mathcal{E}$ be categories, let $G_1, G_2: \mathcal{C} \rightarrow \mathcal{D}$ be functors, and let $\alpha: G_1 \rightarrow G_2$ a natural transformation. Suppose that $F: \mathcal{B} \rightarrow \mathcal{C}$ (respectively $H: \mathcal{C} \rightarrow \mathcal{D}$) is a functor, depicted below:

$$\mathcal{B} \xrightarrow{F} \mathcal{C} \begin{array}{c} \xrightarrow{G_1} \\ \alpha \Downarrow \\ \xrightarrow{G_2} \end{array} \mathcal{D} \quad \left(\text{respectively,} \quad \mathcal{C} \begin{array}{c} \xrightarrow{G_1} \\ \alpha \Downarrow \\ \xrightarrow{G_2} \end{array} \mathcal{D} \xrightarrow{H} \mathcal{E} \right),$$

Then the *pre-whiskering* of α by F , denoted $\alpha \diamond F: G_1 \circ F \rightarrow G_2 \circ F$ (respectively, the *post-whiskering* of α by H , denoted $H \diamond \alpha: H \circ G_1 \rightarrow H \circ G_2$) is defined as follows.

For each $b \in \text{Ob}(\mathcal{B})$ the component $(\alpha \diamond F)_b: G_1 \circ F(b) \rightarrow G_2 \circ F(b)$ is defined to be $\alpha_{F(b)}$. (Respectively, for each $c \in \text{Ob}(\mathcal{C})$ the component $(H \diamond \alpha)_c: H \circ G_1(c) \rightarrow H \circ G_2(c)$ is defined to be $H(\alpha_c)$.) Checking that the naturality squares (in each case) is straightforward.

The rest of this section can safely be skipped; I include it only for my own sense of completeness.

Definition 4.3.2.17 (Horizontal composition of natural transformations). Let $\mathcal{B}, \mathcal{C}$, and $\mathcal{D}$ be categories, let $F_1, F_2: \mathcal{B} \rightarrow \mathcal{C}$ and $G_1, G_2: \mathcal{C} \rightarrow \mathcal{D}$ be functors, and let $\alpha: F_1 \rightarrow F_2$ and $\beta: G_1 \rightarrow G_2$ be natural transformations, as depicted below:

$$\mathcal{B} \begin{array}{c} \xrightarrow{F_1} \\ \alpha \Downarrow \\ \xrightarrow{F_2} \end{array} \mathcal{C} \begin{array}{c} \xrightarrow{G_1} \\ \beta \Downarrow \\ \xrightarrow{G_2} \end{array} \mathcal{D}$$

By pre- and post-whiskering in one order or the other we get the following diagram

$$\begin{array}{ccc} G_1 \circ F_1 & \xrightarrow{G_1 \diamond \alpha} & G_1 \circ F_2 \\ \beta \diamond F_1 \downarrow & & \downarrow \beta \diamond F_2 \\ G_2 \circ F_1 & \xrightarrow{G_2 \diamond \alpha} & G_2 \circ F_2 \end{array}$$

It is straightforward to show that this diagram commutes, so we can take the composition to be our definition of the horizontal composition

$$\beta \diamond \alpha: G_1 \circ F_1 \rightarrow G_2 \circ F_2.$$

Remark 4.3.2.18. All of the above is somehow similar to the world of paths inside a database schema, as seen in Definition 3.5.2.3. The fact that our CPER is an equivalence relation is analogous to the fact that natural transformations are closed under vertical composition. Properties (3) and (4) in Definition 3.5.2.3 are analogous to pre- and post whiskering. See also Lemma 3.5.2.5.

Theorem 4.3.2.19.

$$\mathcal{C} \begin{array}{c} \xrightarrow{F_1} \\ \alpha_1 \Downarrow \\ \xrightarrow{F_2} \\ \alpha_2 \Downarrow \\ \xrightarrow{F_3} \end{array} \mathcal{D} \begin{array}{c} \xrightarrow{G_1} \\ \beta_1 \Downarrow \\ \xrightarrow{G_2} \\ \beta_2 \Downarrow \\ \xrightarrow{G_3} \end{array} \mathcal{E}$$

Given a setup of categories, functors, and natural transformations as above, we have

$$(\beta_2 \circ \beta_1) \diamond (\alpha_2 \circ \alpha_1) = (\beta_2 \diamond \alpha_2) \circ (\beta_1 \diamond \alpha_1).$$

Proof. One need only observe that each square in the following diagram commutes, so following the outer path $(\beta_2 \circ \beta_1) \diamond (\alpha_2 \circ \alpha_1)$ yields the same morphism as following the diagonal path $(\beta_2 \diamond \alpha_2) \circ (\beta_1 \diamond \alpha_1)$:

$$\begin{array}{ccccc}
 G_1 F_1 & \xrightarrow{G_1 \diamond \alpha_1} & G_1 F_2 & \xrightarrow{G_1 \diamond \alpha_2} & G_1 F_3 \\
 \beta_1 \diamond F_1 \downarrow & & \beta_1 \diamond F_2 \downarrow & & \beta_1 \diamond F_3 \downarrow \\
 G_2 F_1 & \xrightarrow{G_2 \diamond \alpha_1} & G_2 F_2 & \xrightarrow{G_2 \diamond \alpha_2} & G_2 F_3 \\
 \beta_2 \diamond F_1 \downarrow & & \beta_2 \diamond F_2 \downarrow & & \beta_2 \diamond F_3 \downarrow \\
 G_3 F_1 & \xrightarrow{G_3 \diamond \alpha_1} & G_3 F_2 & \xrightarrow{G_3 \diamond \alpha_2} & G_3 F_3
 \end{array}$$

□

4.3.3 The category of instances on a database schema

In Section 4.2.2 we showed that schemas are presentations of categories, and we will show in Section 4.4 that in fact the category of schemas is equivalent to the category of categories. In this section we therefore take license to blur the distinction between schemas and categories.

If $\mathcal{C}$ is a schema, i.e. a category, then as we discussed in Section 4.2.2.5, an instance on $\mathcal{C}$ is a functor $I: \mathcal{C} \rightarrow \mathbf{Set}$. But now we have a notion beyond categories and functors, namely that of natural transformations. So we make the following definition.

Definition 4.3.3.1. Let $\mathcal{C}$ be a schema (or category). The *category of instances on $\mathcal{C}$* , denoted $\mathcal{C}\text{-Set}$, is $\text{Fun}(\mathcal{C}, \mathbf{Set})$. Its objects are $\mathcal{C}$ -instances (i.e. functors $\mathcal{C} \rightarrow \mathbf{Set}$) and its morphisms are natural transformations.

Natural transformations should serve as some kind of morphism between instances on the same schema. How are we to interpret a natural transformation $\alpha: I \rightarrow J$ between database instances $I, J: \mathcal{C} \rightarrow \mathbf{Set}$?

Our first clue comes from Application 4.3.1.2. There we considered the case of a monoid $\mathcal{M}$, and we thought about a natural transformation between two functors $X, Y: \mathcal{M} \rightarrow \mathbf{Set}$, considered as different finite state machines. The notion of natural transformation captured the idea of one model being a refinement of another. This same kind of idea works for databases with more than one table (categories with more than one object), but the whole thing is a bit opaque. Let's work it through slowly.

Example 4.3.3.2. Let us consider the terminal schema, $\underline{1} \cong \boxed{\bullet \text{Grapes}}$. An instance is a functor $\underline{1} \rightarrow \mathbf{Set}$ and it is easy to see that this is the same thing as just a set. A natural transformation $\alpha: I \rightarrow J$ is a function from set I to set J . In the standard table view, we might have I and J as below:

Grapes (I)	
ID	
Grape 1	
Grape 3	
Grape 4	

Grapes (J)	
ID	
Jan1-01	
Jan1-02	
Jan1-03	
Jan1-04	
Jan3-01	
Jan4-01	
Jan4-02	

There are 343 natural transformations $I \rightarrow J$. Perhaps some of them make more sense than others; e.g. we could hope that the numbers in I corresponded to the numbers after the dash in J , or perhaps to what seems to be the date in January. But it could be that the rows in J correspond to batches, and all three grapes in I are part of the first batch on Jan-1. The notion of natural transformation is a mathematical one.

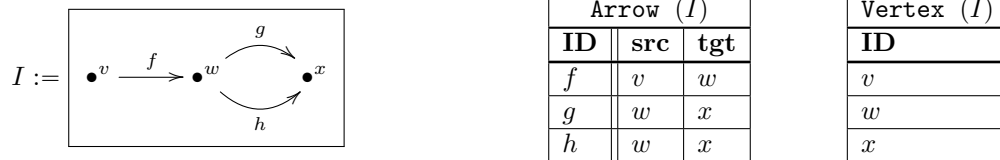
Exercise 4.3.3.3. Recall the notion of set-indexed sets from Definition 2.7.6.12. Let A be a set, and come up with a schema $\mathcal{A}$ such that instances on $\mathcal{A}$ are A -indexed sets. Is our current notion of morphism between instances (i.e. natural transformations) well-aligned with the above definition of “mapping of A -indexed sets”? $\diamond$

For a general schema (or category) $\mathcal{C}$, let us think through what a morphism $\alpha: I \rightarrow J$ between instances $I, J: \mathcal{C} \rightarrow \mathbf{Set}$ is. For each object $c \in \text{Ob}(\mathcal{C})$ there is a component $\alpha_c: I(c) \rightarrow J(c)$. This means that just like in Example 4.3.3.2, there is for each table c a function from the rows in I 's manifestation of c to the rows in J 's manifestation of c . So to make a natural transformation, such a function has to be specified table by table. But then we have to contend with naturality squares, one for every arrow in $\mathcal{C}$. Arrows in $\mathcal{C}$ correspond to foreign key columns in the database. The naturality requirement was already covered in Application 4.3.1.2 (and see especially how (4.10) is checked in (4.11) and (4.12)).

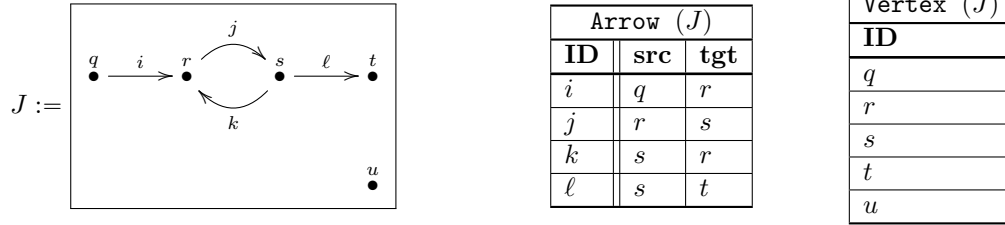
Example 4.3.3.4. We saw in Section 4.2.1.19 that graphs can be regarded as functors $\mathcal{G} \rightarrow \mathbf{Set}$, where $\mathcal{G}$ is the “schema for graphs” shown here:

$$\mathcal{G} := \begin{array}{ccc} \text{Arrow} & \xrightarrow{\text{src}} & \text{Vertex} \\ \bullet & \xRightarrow{\quad} & \bullet \\ & \text{tgt} & \end{array}$$

A database instance $I: \mathcal{G} \rightarrow \mathbf{Set}$ on $\mathcal{G}$ consists of two tables. Here is an example instance:



To discuss natural transformations, we need two instances. Here is another, $J: \mathcal{G} \rightarrow \mathbf{Set}$,



To give a natural transformation $\alpha: I \rightarrow J$, we give two components: one for arrows and one for vertices. We need to say where each vertex in I goes in J and we need to say where each arrow in I goes in J . The naturality squares insist that if we specify that $g \mapsto j$, for example, then we better specify that $w \mapsto r$ and that $x \mapsto s$. What a computer is very good at, but a human is fairly slow at, is checking that a given pair of components (arrows and vertices) really is natural.

There are 8000 ways to come up with component functions α_{Arrow} and α_{Vertex} , but precisely four natural transformations, i.e. four graph homomorphisms, $I \rightarrow J$; the other 7996 are haphazard flingings of arrows to arrows and vertices to vertices without any regard to sources and targets. We briefly describe the four now.

First off, nothing can be sent to u because arrows must go to arrows and u touches no arrows. If we send $v \mapsto q$ then f must map to i , and w must map to r , and both g and h must map to j , and x must map to s . If we send $v \mapsto r$ then there are two choices for g and h . If we send $v \mapsto s$ then there's one way to obtain a graph morphism. If we try to send $v \mapsto t$, we fail. All of this can be seen by staring at the tables rather than at the pictorial representations of the graphs; the human eye understands these pictures better, but the computer understands the tables better.

Exercise 4.3.3.5. If $I, J: \mathcal{G} \rightarrow \mathbf{Set}$ are as in Example 4.3.3.4, how many natural transformations are there $J \rightarrow I$? $\diamond$

Exercise 4.3.3.6. Let $Y_A: \mathcal{G} \rightarrow \mathbf{Set}$ denote the instance below:

ID	src	tgt
a	v_0	v_1

ID
v_0
v_1

Let $I: \mathcal{G} \rightarrow \mathbf{Set}$ be as in Example 4.3.3.4.

- How many natural transformations are there $Y_A \rightarrow I$?
- With J as above, how many natural transformations are there $Y_A \rightarrow J$?
- Do you have any conjecture about the way natural transformations $Y_A \rightarrow X$ behave for arbitrary graphs $X: \mathcal{G} \rightarrow \mathbf{Set}$? $\diamond$

In terms of databases, this notion of instance morphism $I \rightarrow J$ is fairly benign. For every table its a mapping from the set of rows in I 's version of the table to J 's version of the table, such that all the foreign keys are respected. We will see that this notion of morphism has excellent formal properties, so that projections, unions, and joins of tables

(the typical database operations) would be predicted to be “obviously interesting” by a category theorist who had no idea what a database was.¹⁴

However, something is also missing from the natural transformation picture. A very important occurrence in the world of databases is the update. Everyone can understand this: a person makes a change in one of the tables, like changing your address from Cambridge, MA to Hereford, UK. Most such arbitrary changes of database instance are not “natural”, in that the new linking pattern is incompatible with the old.

It is interesting to consider how updates of $\mathcal{C}$ -instances should be understood category theoretically. We might want a category $Upd_{\mathcal{C}}$ whose objects are $\mathcal{C}$ -instances and whose morphisms are updates. But then what is the composition formula? Is there a unique morphism $I \rightarrow J$ whenever J can be obtained as an update on I ? Because in that case, we would be defining $Upd_{\mathcal{C}}$ to be the indiscrete category on the set of $\mathcal{C}$ -instances (see Example 4.3.4.3).

Exercise 4.3.3.7. Research project: Can you come up with a satisfactory way to model database updates category-theoretically? Let $\mathbb{N}$ be the category

$$[\mathbb{N}] := \bullet^0 \longrightarrow \bullet^1 \longrightarrow \bullet^2 \longrightarrow \dots$$

representing a discrete timeline. A place to start might be to use something like the slice category $\mathbf{Cat}_{/[\mathbb{N}]}$ where the fiber over each object in $\mathbb{N}$ is a snapshot of the database in time. Can you make this work? $\diamond$

4.3.4 Equivalence of categories

We have a category $\mathbf{Cat}$ of categories, and in every category there is a notion of isomorphism between objects: one morphism each way, such that each round-trip composition is the identity. An isomorphism in $\mathbf{Cat}$, therefore, takes place between two categories, say $\mathcal{C}$ and $\mathcal{D}$: it is a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ and a functor $G: \mathcal{D} \rightarrow \mathcal{C}$ such that $G \circ F = \text{id}_{\mathcal{C}}$ and $F \circ G = \text{id}_{\mathcal{D}}$.

It turns out that categories are often similar enough to be considered equivalent without being isomorphic. For this reason, the notion of isomorphism is considered “too strong” to be useful for categories. The feeling to a category theorist might be akin to saying that two material samples are the same if there is an atom-by-atom matching, or that two words are the same if they are written in the same font, of the same size, by the same person, in the same state of mind.

As reasonable as isomorphism is as a notion *in* most categories, it fails to be the “right notion” *about* categories. The reason is that *in* categories there are objects and morphisms, whereas when we talk *about* categories, we have categories and functors, plus natural transformations. These serve as mappings between mappings, and this is not part of the structure of an ordinary category. In cases where a category $\mathcal{C}$ does have such mappings between mappings, it is often a “better notion” if we take that extra structure into account, like we will for categories. This whole subject leads us to the study of 2-categories (or n -categories, or ∞ -categories), which we do discuss in this book. See, for example, [Le1] for an introduction.

Regardless, our purpose now is to explain this “good notion” of sameness for categories, namely *equivalences of categories*, which appropriately take natural transformations into account. Instead of “functors going both ways with round trips equal to

¹⁴More precisely, given a functor between schemas $F: \mathcal{C} \rightarrow \mathcal{D}$, the pullback $\Delta_F: \mathcal{D}\text{-Set} \rightarrow \mathcal{C}\text{-Set}$, its left Σ_F and its right adjoint Π_F constitute these important queries. See Section 5.1.4.

identity”, which is required in order to be an isomorphism of categories, equivalence of categories demands “functors going both ways with round trips *isomorphic* to identity”.

Definition 4.3.4.1. Let $\mathcal{C}$ and $\mathcal{C}'$ be categories. A functor $F: \mathcal{C} \rightarrow \mathcal{C}'$ is called an *equivalence of categories*, and denoted $F: \mathcal{C} \xrightarrow{\simeq} \mathcal{C}'$,¹⁵ if there exists a functor $F': \mathcal{C}' \rightarrow \mathcal{C}$ and natural isomorphisms $\alpha: \text{id}_{\mathcal{C}} \xrightarrow{\simeq} F' \circ F$ and $\alpha': \text{id}_{\mathcal{C}'} \xrightarrow{\simeq} F \circ F'$. In this case we say that F and F' are *mutually inverse equivalences*.

Unpacking a bit, suppose we are given functors $F: \mathcal{C} \rightarrow \mathcal{C}'$ and $F': \mathcal{C}' \rightarrow \mathcal{C}$. We want to know something about the roundtrips on $\mathcal{C}$ and on $\mathcal{C}'$; we want to know the same kind of information about each roundtrip, so let’s concentrate on the $\mathcal{C}$ side. We want to know something about $F' \circ F: \mathcal{C} \rightarrow \mathcal{C}$, so let’s name it $i: \mathcal{C} \rightarrow \mathcal{C}$; we want to know that i is a natural isomorphism. That is, for every $c \in \text{Ob}(\mathcal{C})$ we want an isomorphism $\alpha_c: c \xrightarrow{\simeq} i(c)$, and we want to know that these isomorphisms are picked carefully enough that given $g: c \rightarrow c'$ in $\mathcal{C}$, the choice of isomorphisms for c and c' are compatible,

$$\begin{array}{ccc} c & \xrightarrow{\alpha_c} & i(c) \\ g \downarrow & & \downarrow i(g) \\ c' & \xrightarrow{\alpha_{c'}} & i(c'). \end{array}$$

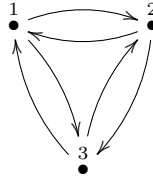
To be an equivalence, the same has to hold for the other roundtrip, $i' = F \circ F': \mathcal{C}' \rightarrow \mathcal{C}'$.

Exercise 4.3.4.2. Let $\mathcal{C}$ and $\mathcal{C}'$ be categories. Suppose that $F: \mathcal{C} \rightarrow \mathcal{C}'$ is an isomorphism of categories.

- a.) Is it an equivalence of categories?
- b.) What are the components of α and α' (with notation as in Definition 4.3.4.1)?

◇

Example 4.3.4.3. Let S be a set and let $S \times S \subseteq S \times S$ be the complete relation on S , which is a preorder K_S . Recall from Proposition 4.2.1.17 that we have a functor $i: \mathbf{Pro} \rightarrow \mathbf{Cat}$, and the resulting category $i(K_S)$ is called the *indiscrete category on S* ; it has objects S and a single morphism between every pair of objects. Here is a picture of $K_{\{1,2,3\}}$:



It is easy check that K_1 , the indiscrete category on one element, is isomorphic to $\underline{1}$, the discrete category on one object, also known as the terminal category (see Exercise 4.1.2.34). The category $\underline{1}$ consists of one object, its identity morphism, and nothing else.

¹⁵The notation $\simeq$ has already been used for equivalences of paths in a schema. We do not mean to equate these ideas; we are just reusing the symbol. Hopefully no confusion will arise.

The only way that K_S can be isomorphic to $\underline{1}$ is if S has one element.¹⁶ On the other hand, there is an equivalence of categories

$$K_S \simeq \underline{1}$$

for every set $S \neq \emptyset$.

In fact, there are many such equivalences, one for each element of S . To see this, let S be a nonempty set and choose an element $s_0 \in S$. For every $s \in S$, there is a unique isomorphism $k_s: s \xrightarrow{\cong} s_0$ in K_S . Let $F: K_S \rightarrow \underline{1}$ be the only possible functor (see Exercise 4.1.2.34), and let $F': \underline{1} \rightarrow K_S$ send the unique object in $\underline{1}$ to the object s_0 .

Note that $F' \circ F = \text{id}_{\underline{1}}: \underline{1} \rightarrow \underline{1}$ is the identity, but that $F \circ F': K_S \rightarrow K_S$ sends everything to s_0 . Let $\alpha = \text{id}_{\underline{1}}$ and define $\alpha': \text{id}_{K_S} \rightarrow F \circ F'$ by $\alpha'_s = k_s$. Note that α'_s is an isomorphism for each $s \in \text{Ob}(K_S)$, and note that α' is a natural transformation (hence natural isomorphism) because every possible square commutes in K_S . This completes the proof, initiated in the paragraph above, that the category K_S is equivalent to $\underline{1}$ for every nonempty set S , and that this fact can be witnessed by any element $s_0 \in S$.

Example 4.3.4.4. Consider the category **FLin**, described in Exercise 4.1.1.8, of finite nonempty linear orders. For every natural number $n \in \mathbb{N}$, let $[n] \in \text{Ob}(\mathbf{FLin})$ denote the linear order shown in Example 3.4.1.6. Define a category Δ whose objects are given by $\text{Ob}(\Delta) = \{[n] \mid n \in \mathbb{N}\}$ and with $\text{Hom}_{\Delta}([m], [n]) = \text{Hom}_{\mathbf{FLin}}([m], [n])$. The difference between **FLin** and Δ is only that objects in **FLin** may have “funny labels”, e.g.

$$\underset{\bullet}{5} \longrightarrow \underset{\bullet}{x} \longrightarrow \underset{\bullet}{\text{“Sam”}}$$

whereas objects in Δ all have standard labels, e.g.

$$\underset{\bullet}{0} \longrightarrow \underset{\bullet}{1} \longrightarrow \underset{\bullet}{2}$$

Clearly **FLin** is a much larger category, and yet feels like it is “pretty much the same as” Δ . Justly, they are equivalent, $\mathbf{FLin} \simeq \Delta$.

The functor $F': \Delta \rightarrow \mathbf{FLin}$ is the inclusion; the functor $F: \mathbf{FLin} \rightarrow \Delta$ sends every finite nonempty linear order $X \in \text{Ob}(\mathbf{FLin})$ to the object $F(X) := [n] \in \Delta$, where $\text{Ob}(X) \cong \{1, 2, \dots, n\}$. For each such X there is a unique isomorphism $\alpha_X: X \xrightarrow{\cong} [n]$, and these fit together into¹⁷ the required natural isomorphism $\text{id}_{\mathbf{FLin}} \rightarrow F' \circ F$. The other natural isomorphism $\alpha': \text{id}_{\Delta} \rightarrow F \circ F'$ is the identity.

Exercise 4.3.4.5. Recall from Definition 2.1.2.15 that a set X is called finite if there exists a natural number $n \in \mathbb{N}$ and an isomorphism of sets $X \rightarrow \underline{n}$. Let **Fin** denote the category whose objects are the finite sets and whose morphisms are the functions. Let $\mathcal{S}$ denote the category whose objects are the sets $\underline{n}$ and whose morphisms are again the functions. For every object $X \in \text{Ob}(\mathbf{Fin})$ there exists an isomorphism $p_X: X \rightarrow \underline{n}$ for some unique object $\underline{n} \in \text{Ob}(\mathcal{S})$. Find an equivalence of categories $\mathbf{Fin} \xrightarrow{\cong} \mathcal{S}$. $\diamond$

Exercise 4.3.4.6. We say that two categories $\mathcal{C}$ and $\mathcal{D}$ are equivalent if there exists an equivalence of categories between them. Show that the relation of “being equivalent” is an equivalence relation on $\text{Ob}(\mathbf{Cat})$. $\diamond$

¹⁶One way to see this is that by Exercise 4.1.2.35, we have a functor $\text{Ob}: \mathbf{Cat} \rightarrow \mathbf{Set}$, and we know by Exercise 4.1.2.21 that functors preserve isomorphisms, so an isomorphism between categories must restrict to an isomorphism between their sets of objects. The only sets that are isomorphic to $\underline{1}$ have one element.

¹⁷The phrase “these fit together into” is suggestive shorthand for, and thus can be replaced with, the phrase “the naturality squares commute for these components, so together they constitute”.

Example 4.3.4.7. Consider the group $\mathbb{Z}_2 := (\{0, 1\}, 0, +)$, where $1 + 1 = 0$. As a category, $\mathbb{Z}_2$ has one object $\blacktriangle$ and two morphisms, namely $0, 1$, such that 0 is the identity. Since $\mathbb{Z}_2$ is a group, the morphism $1: \blacktriangle \rightarrow \blacktriangle$ must have an inverse x , meaning $1 + x = 0$, and $x = 1$ is the only solution.

The point is that the morphism 1 in $\mathbb{Z}_2$ is an isomorphism. Let $\mathcal{C} = \underline{1}$ be the terminal category as in Exercise 4.1.2.34. One might accidentally believe that $\mathcal{C}$ is equivalent to $\mathbb{Z}_2$, but this is not the case! The argument in favor of the accidental belief is that we have unique functors $F: \mathbb{Z}_2 \rightarrow \mathcal{C}$ and $F': \mathcal{C} \rightarrow \mathbb{Z}_2$ (and this is true); the roundtrip $F \circ F': \mathcal{C} \rightarrow \mathcal{C}$ is the identity (and this is true); and for the roundtrip $F' \circ F: \mathbb{Z}_2 \rightarrow \mathbb{Z}_2$ both morphisms in $\mathbb{Z}_2$ are isomorphisms, so any choice of morphism $\alpha_\blacktriangle: \blacktriangle \rightarrow F' \circ F(\blacktriangle)$ will be an isomorphism (and this is true). The problem is that no such $\alpha_\blacktriangle$ will be a natural transformation.

When we roundtrip $F' \circ F: \mathbb{Z}_2 \rightarrow \mathbb{Z}_2$, the image of $1: \blacktriangle \rightarrow \blacktriangle$ is $F' \circ F(1) = 0 = \text{id}_\blacktriangle$. So the naturality square for the morphism 1 looks like this:

$$\begin{array}{ccc} \blacktriangle & \xrightarrow{\alpha_\blacktriangle} & \blacktriangle \\ 1 \downarrow & & \downarrow 0 = F' \circ F(1) \\ \blacktriangle & \xrightarrow{\alpha_\blacktriangle} & \blacktriangle \end{array}$$

where we still haven't decided whether we want $\alpha_\blacktriangle$ to be 0 or 1 . Unfortunately, neither choice works (i.e. for neither choice will the diagram commute) because $x + 1 \neq x + 0$ in $\mathbb{Z}_2$.

Definition 4.3.4.8 (Skeleton). Let $\mathcal{C}$ be a category. We saw in Lemma 4.1.1.16 that the relation of “being isomorphic” is an equivalence relation $\cong$ on $\text{Ob}(\mathcal{C})$. An *election* in $\mathcal{C}$ is a choice E of the following sort:

- for each $\cong$ -equivalence class $S \subseteq \text{Ob}(\mathcal{C})$ a choice of object $s_E \in S$, called the *elected object* for S , and
- for each object $c \in \text{Ob}(\mathcal{C})$ a choice of isomorphism $i_c: s_E \rightarrow c$ and $j_c: c \rightarrow s_E$ with $i_c \circ j_c = \text{id}_c$ and $j_c \circ i_c = \text{id}_{s_E}$, where s_E is an elected object.

Given an election E in $\mathcal{C}$, there is a category called the *E -elected skeleton* of $\mathcal{C}$, denoted $\text{Skel}_E(\mathcal{C})$, whose objects are the elected objects and whose morphisms $s \rightarrow t$ for any elected objects $s, t \in \text{Ob}(\mathcal{C})$ are given by $\text{Hom}_{\text{Skel}_E(\mathcal{C})}(s, t) = \text{Hom}_{\mathcal{C}}(s, t)$. Any object $c \in \text{Ob}(\mathcal{C})$ is isomorphic to a unique elected object s_E ; we refer to s_E as the *elected representative* of c ; we refer to the isomorphisms i_c and j_c as the *representing isomorphisms* for c .

Proposition 4.3.4.9. *Let $\mathcal{C}$ be a category and let E be an election in $\mathcal{C}$. There is an equivalence of categories*

$$\text{Skel}_E(\mathcal{C}) \simeq \mathcal{C}.$$

Proof. The functor $F': \text{Skel}_E(\mathcal{C}) \rightarrow \mathcal{C}$ is the inclusion. The functor $F: \mathcal{C} \rightarrow \text{Skel}_E(\mathcal{C})$ sends each object in $\mathcal{C}$ to its elected representative. Given objects $c, c' \in \text{Ob}(\mathcal{C})$ with elected representatives s, t respectively, and given a morphism $g: c \rightarrow c'$ in $\mathcal{C}$, let $i_c, j_c, i_{c'},$ and $j_{c'}$ be the representing isomorphisms, and define $F(g): s \rightarrow t$ to be the composite

$$s \xrightarrow{i_c} c \xrightarrow{g} c' \xrightarrow{j_{c'}} t.$$

This is functorial because it sends the identity to the identity and $F(g \circ g') = F(g) \circ F(g')$.

The composite $F \circ F': \text{Skel}_E(\mathcal{C}) \rightarrow \text{Skel}_E(\mathcal{C})$ is the identity. For each $c \in \text{Ob}(\mathcal{C})$ define $\alpha_c: c \xrightarrow{\cong} F' \circ F(c)$ by $\alpha_c := j_c$. Given $g: c \rightarrow c'$ the required naturality square is shown to the left below:

$$\begin{array}{ccccc}
 c & \xrightarrow{j_c} & s & \xrightarrow{i_c} & c \\
 \downarrow g & & \downarrow F' \circ F(g) & & \downarrow g \\
 c' & \xrightarrow{j'_c} & t & \xleftarrow{j'_c} & c'
 \end{array}$$

The right-hand part commutes by definition of F and F' ; i.e. $j'_c \circ g \circ i_c = F' \circ F(g)$. The left-hand square commutes because $i_c \circ j_c = \text{id}_c$.

□

Definition 4.3.4.10. A *skeleton* of $\mathcal{C}$ is a category $\mathcal{S}$, equivalent to $\mathcal{C}$, such that for any two objects $s, s' \in \text{Ob}(\mathcal{S})$, if $s \cong s'$ then $s = s'$.

Exercise 4.3.4.11. Let $\mathcal{P}$ be a preorder (considered as a category).

- a.) If $\mathcal{P}'$ is a skeleton of $\mathcal{P}$, is it a partial order?
- b.) Is every partial order the skeleton of some preorder?

◇

Definition 4.3.4.12. Let $\mathcal{C}$ and $\mathcal{D}$ be categories, and let $F: \mathcal{C} \rightarrow \mathcal{D}$ be a functor. For any two objects $c, c' \in \text{Ob}(\mathcal{C})$, we have a function $\text{Hom}_F(c, c'): \text{Hom}_{\mathcal{C}}(c, c') \rightarrow \text{Hom}_{\mathcal{D}}(F(c), F(c'))$ guaranteed by the definition of functor. We say that F is a *full functor* if $\text{Hom}_F(c, c')$ is surjective for every c, c' . We say that F is a *faithful functor* if $\text{Hom}_F(c, c')$ is injective for every c, c' . We say that F is a *fully faithful functor* if $\text{Hom}_F(c, c')$ is bijective for every c, c' .

Exercise 4.3.4.13. Let $\underline{1}$ and $\underline{2}$ be the discrete categories on one and two objects, respectively. There is only one functor $\underline{2} \rightarrow \underline{1}$.

- a.) Is it full?
- b.) Is it faithful?

◇

Exercise 4.3.4.14. Let $\underline{0}$ denote the empty category, and let $\mathcal{C}$ be any category. There is a unique functor $F: \underline{0} \rightarrow \mathcal{C}$.

- a.) For general $\mathcal{C}$ will F be full?
- b.) For general $\mathcal{C}$ will F be faithful?
- c.) For general $\mathcal{C}$ will F be an equivalence of categories?

◇

Proposition 4.3.4.15. Let $\mathcal{C}$ and $\mathcal{C}'$ be categories and let $F: \mathcal{C} \rightarrow \mathcal{C}'$ be an equivalence of categories. Then F is fully faithful.

Proof. Suppose F is an equivalence, so we can find a functor $F': \mathcal{C}' \rightarrow \mathcal{C}$ and natural isomorphisms $\alpha: \text{id}_{\mathcal{C}} \xrightarrow{\cong} F' \circ F$ and $\alpha': \text{id}_{\mathcal{C}'} \xrightarrow{\cong} F \circ F'$. We need to know that for any objects $c, d \in \text{Ob}(\mathcal{C})$, the map

$$\text{Hom}_F(c, d): \text{Hom}_{\mathcal{C}}(c, d) \rightarrow \text{Hom}_{\mathcal{C}'}(Fc, Fd)$$

is bijective. Consider the following diagram

$$\begin{array}{ccc} \text{Hom}_{\mathcal{C}}(c, d) & \xrightarrow{\text{Hom}_F(c, d)} & \text{Hom}_{\mathcal{C}'}(Fc, Fd) \\ & \searrow \alpha & \downarrow \text{Hom}_{F'}(Fc, Fd) \\ & & \text{Hom}_{\mathcal{C}}(F'Fc, F'Fd) \\ & & \xrightarrow{\text{Hom}_F(F'Fc, F'Fd)} \text{Hom}_{\mathcal{C}'}(FF'Fc, FF'Fd) \\ & & \nearrow \alpha' \end{array}$$

The fact that α is bijective implies that the vertical function is surjective. The fact that α' is bijective implies that the vertical function is injective, so it is bijective. This implies that $\text{Hom}_F(c, d)$ is bijective as well. $\square$

Exercise 4.3.4.16. Let $\mathbb{Z}_2$ be the group (as category) from Example 4.3.4.7. Are there any fully faithful functors $\mathbb{Z}_2 \rightarrow \underline{1}$? $\diamond$

4.4 Categories and schemas are equivalent, $\text{Cat} \simeq \text{Sch}$

Perhaps it is intuitively clear that schemas are somehow equivalent to categories, and in this section we make that precise. The basic idea was already laid out in Section 4.2.2.

4.4.1 The category Sch of schemas

Recall from Definition 3.5.2.6 that a schema consists of a pair $\mathcal{C} := (G, \simeq)$, where $G = (V, A, \text{src}, \text{tgt})$ is a graph and $\simeq$ is a categorical path equivalence relation (or CPER), meaning a kind of equivalence relation on the paths in G (see Definition 3.5.2.3). If we think of a schema as being analogous to a category, what should fulfill the role of functors? That is, what are to be the morphisms in Sch ?

Unfortunately, one's first guess may give the wrong notion if we want an equivalence $\text{Sch} \simeq \text{Cat}$. Since objects in Sch are graphs with additional structure, one might imagine that a morphism $\mathcal{C} \rightarrow \mathcal{C}'$ in Sch should be a graph homomorphism (as in Definition 3.3.3.1) that preserves said structure. But graph homomorphisms require that arrows be sent to arrows, whereas we are more interested in paths than in individual arrows—the arrows are merely useful for presentation.

If instead we define morphisms between schemas to be maps that send paths in $\mathcal{C}$ to paths in $\mathcal{C}'$, subject to the requirements that path endpoints, path concatenations, and path equivalences are preserved, this will turn out to give the correct notion. And since a path is a concatenation of its arrows, it suffices to give a function F from the arrows of $\mathcal{C}$ to the paths of $\mathcal{C}'$, which automatically takes care of the first two requirements above; we must only take care that F preserves path equivalences.

Recall from Examples 4.1.2.19 and 4.3.1.6 the paths-graph functor $\mathbf{Paths}: \mathbf{Grph} \rightarrow \mathbf{Grph}$, the paths of paths functor $\mathbf{Paths} \circ \mathbf{Paths}: \mathbf{Grph} \rightarrow \mathbf{Grph}$, and the natural transformations for any graph G ,

$$\eta_G: G \rightarrow \mathbf{Paths}(G) \quad \text{and} \quad \mu_G: \mathbf{Paths}(\mathbf{Paths}(G)) \rightarrow \mathbf{Paths}(G). \quad (4.13)$$

The function η_G spells out the fact that every arrow in G counts as a path in G , and the function μ_G spells out the fact that a head-to-tail sequence of paths (a path of paths) in G can be concatenated to a single path in G .

Exercise 4.4.1.1. Let $[2]$ denote the linear order graph of length 2 (see Exercise 4.1.1.8 and Proposition 4.1.2.6), and let $\mathcal{Loop}$ denote the unique graph having one vertex and one arrow (pictured in Diagram (3.15)).

- a.) Find an injective graph homomorphism $f: [2] \rightarrow \mathbf{Paths}(\mathcal{Loop})$.
- b.) The graph $[2]$ has 6 paths, so $\mathbf{Paths}([2])$ has 6 arrows. What are the images of these arrows under the graph homomorphism $\mathbf{Paths}(f): \mathbf{Paths}([2]) \rightarrow \mathbf{Paths}(\mathbf{Paths}(\mathcal{Loop}))$?

◇

We are almost ready to give the definition of schema morphism, but before we do, let's return to our original idea. Given graphs G, G' (underlying schemas $\mathcal{C}, \mathcal{C}'$) we originally wanted a function from the paths in G to the paths in G' , but we realized it was more concise to speak of a function from arrows in G to paths in G' . How do we get back what we originally wanted from the concise version? Given a graph homomorphism $f: G \rightarrow \mathbf{Paths}(G')$, we use (4.13) to form the following composition, which we denote simply by $\mathbf{Paths}_f: \mathbf{Paths}(G) \rightarrow \mathbf{Paths}(G')$:

$$\mathbf{Paths}(G) \xrightarrow{\mathbf{Paths}(f)} \mathbf{Paths}(\mathbf{Paths}(G')) \xrightarrow{\mu_{G'}} \mathbf{Paths}(G') \quad (4.14)$$

This says that given a function from arrows in G to paths in G' , a path in G becomes a path of paths in G' , which can be concatenated to a path in G' . This simply and precisely spells out our intuition.

Definition 4.4.1.2. Let $G = (V, A, \text{src}, \text{tgt})$ and $G' = (V', A', \text{src}', \text{tgt}')$ be graphs, and let $\mathcal{C} = (G, \simeq_G)$ and $\mathcal{C}' = (G', \simeq_{G'})$ be schemas. A *schema morphism* F from $\mathcal{C}$ to $\mathcal{D}$, denoted $F: \mathcal{C} \rightarrow \mathcal{D}$ is a graph homomorphism¹⁸

$$F: G \rightarrow \mathbf{Paths}(G')$$

that satisfies the following condition for any paths p and q in G :

$$\text{if } p \simeq_G q \text{ then } \mathbf{Paths}_f(p) \simeq_{G'} \mathbf{Paths}_f(q). \quad (4.15)$$

Two schema morphisms $E, F: \mathcal{C} \rightarrow \mathcal{C}'$ are considered identical if they agree on vertices (i.e. $E_0 = F_0$) and if, for every arrow f in G , there is a path equivalence in G'

$$E_1(f) \simeq_{G'} F_1(f).$$

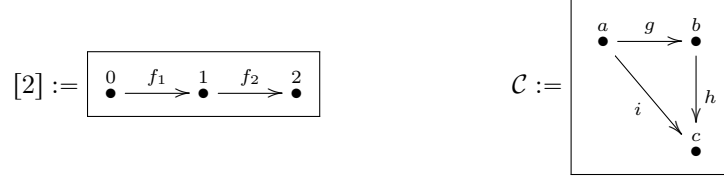
We now define the *category of schemas*, denoted $\mathbf{Sch}$, to be the category whose objects are schemas as in Definition 3.5.2.6 and whose morphisms are schema morphisms defined as above. The identity morphism on schema $\mathcal{C} = (G, \simeq_G)$ is the schema morphism $\text{id}_{\mathcal{C}} := \eta_G: G \rightarrow \mathbf{Paths}(G)$ as defined in Equation (4.13).

¹⁸By Definition 3.3.3.1, a graph homomorphism $F: G \rightarrow \mathbf{Paths}(G')$ will consist of a vertex part $F_0: V \rightarrow V'$ and an arrows part $F_1: E \rightarrow \mathbf{Path}(G')$. See also Definition 3.3.2.1.

Slogan 4.4.1.3.

“ A schema morphism sends vertices to vertices, arrows to paths, and path equivalences to path equivalences. ”

Example 4.4.1.4. Let $[2]$ be the linear order graph of length 2, pictured to the left, and let $\mathcal{C}$ denote the schema pictured to the right below:



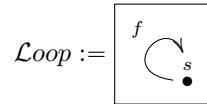
We impose on $\mathcal{C}$ the path equivalence declaration $[g, h] \simeq [i]$ and show that in this case $\mathcal{C}$ and $[2]$ are isomorphic in **Sch**. We have a schema morphism $F: [2] \rightarrow \mathcal{C}$ sending $0 \mapsto a, 1 \mapsto b, 2 \mapsto c$, and sending each arrow in $[2]$ to an arrow in $\mathcal{C}$. And we have a schema morphism $F': \mathcal{C} \rightarrow [2]$ which reverses this mapping on vertices; note that F' must send the arrow i in $\mathcal{C}$ to the path $[f_1, f_2]$ in $[2]$, which is ok! The roundtrip $F' \circ F: [2] \rightarrow [2]$ is identity. The roundtrip $F \circ F': \mathcal{C} \rightarrow \mathcal{C}$ may look like it's not the identity; indeed it sends vertices to themselves but it sends i to the path $[g, h]$. But according to Definition 4.4.1.2, this schema morphism is considered identical to $\text{id}_{\mathcal{C}}$ because there is a path equivalence $\text{id}_{\mathcal{C}}(i) = [i] \simeq [g, h] = F \circ F'(i)$.

Exercise 4.4.1.5. Consider the schema $[2]$ and the schema $\mathcal{C}$ pictured above, except where this time we *do not* impose any path equivalence declarations on $\mathcal{C}$, so $[g, h] \neq [i]$ in our current version of $\mathcal{C}$.

- How many schema morphisms are there $[2] \rightarrow \mathcal{C}$ that send 0 to a ?
- How many schema morphisms are there $\mathcal{C} \rightarrow [2]$ that send a to 0?

◇

Exercise 4.4.1.6. Consider the graph $\mathcal{Loop}$ pictured below



and for any natural number n , let $\mathcal{L}_n$ denote the schema $(\mathcal{Loop}, \simeq_n)$ where $\simeq_n$ is the PED $f^{n+1} \simeq f^n$. This is the “finite hierarchy” schema of Example 3.5.2.11. Let $\underline{1}$ denote the graph with one vertex and no arrows; consider it as a schema.

- Is $\underline{1}$ isomorphic to $\mathcal{L}_1$ in **Sch**?
- Is it isomorphic to any (other) $\mathcal{L}_n$?

◇

Exercise 4.4.1.7. Let $\mathcal{Loop}$ and $\mathcal{L}_n$ be the schemas defined in Exercise 4.4.1.6.

- What is the cardinality of the set $\text{Hom}_{\mathbf{Sch}}(\mathcal{L}_3, \mathcal{L}_5)$?
- What is the cardinality of the set $\text{Hom}_{\mathbf{Sch}}(\mathcal{L}_5, \mathcal{L}_3)$? Hint: the cardinality of the set $\text{Hom}_{\mathbf{Sch}}(\mathcal{L}_4, \mathcal{L}_9)$ is 8.

◇

4.4.2 Proving the equivalence

Construction 4.4.2.1 (From schema to category). We will define a functor $L: \mathbf{Sch} \rightarrow \mathbf{Cat}$. Let $\mathcal{C} = (G, \simeq)$ be a categorical schema, where $G = (V, A, \text{src}, \text{tgt})$. Define $L(\mathcal{C})$ to be the category with $\text{Ob}(L(\mathcal{C})) = V$, and with $\text{Hom}_{L(\mathcal{C})}(v_1, v_2) := \text{Path}_G(v, w) / \simeq$, i.e. the set of paths in G , modulo the path equivalence relation for $\mathcal{C}$. The composition of morphisms is defined by concatenation of paths, and Lemma 3.5.2.5 ensures that such composition is well-defined. We have thus defined L on objects of $\mathbf{Sch}$.

Given a schema morphism $F: \mathcal{C} \rightarrow \mathcal{C}'$, where $\mathcal{C}' = (G', \simeq')$, we need to produce a functor $L(F): L(\mathcal{C}) \rightarrow L(\mathcal{C}')$. The objects of $L(\mathcal{C})$ and $L(\mathcal{C}')$ are the vertices of G and G' respectively, and F provides the necessary function on objects. Diagram (4.14) provides a function $\text{Paths}_F: \text{Paths}(G) \rightarrow \text{Paths}(G')$ will provide the requisite function for morphisms.

A morphism in $L(\mathcal{C})$ is an equivalence class of paths in $\mathcal{C}$. For any representative path $p \in \text{Paths}(G)$, we have $\text{Paths}_F(p) \in \text{Paths}(G')$, and if $p \simeq q$ then $\text{Paths}_F(p) \simeq' \text{Paths}_F(q)$ by condition 4.15. Thus Paths_F indeed provides us with a function $\text{Hom}_{L(\mathcal{C})} \rightarrow \text{Hom}_{L(\mathcal{C}')}$. This defines L on morphisms in $\mathbf{Sch}$. It is clear that L preserves composition and identities, so it is a functor.

Construction 4.4.2.2 (From category to schema). We will define a functor $R: \mathbf{Cat} \rightarrow \mathbf{Sch}$. Let $\mathcal{C} = (\text{Ob}(\mathcal{C}), \text{Hom}_{\mathcal{C}}, \text{dom}, \text{cod}, \text{id}, \circ)$ be a category (see Exercise 4.1.1.18). Let $R(\mathcal{C}) = (G, \simeq)$ where G is the graph

$$G = (\text{Ob}(\mathcal{C}), \text{Hom}_{\mathcal{C}}, \text{dom}, \text{cod}),$$

and with $\simeq$ defined as the CPER generated by the following path equivalence declarations: for any composable sequence of morphisms $f_1, f_2, \dots, f_n$ (with $\text{dom}(f_{i+1}) = \text{cod}(f_i)$ for each $1 \leq i \leq n-1$) we put

$$[f_1, f_2, \dots, f_n] \simeq [f_n \circ \dots \circ f_2 \circ f_1]. \quad (4.16)$$

This defines R on objects of $\mathbf{Cat}$.

A functor $F: \mathcal{C} \rightarrow \mathcal{D}$ induces a schema morphism $R(F): R(\mathcal{C}) \rightarrow R(\mathcal{D})$, because vertices are sent to vertices, arrows are sent to arrows (as paths of length 1), and path equivalence is preserved by (4.16) and the fact that F preserves the composition formula. This defines R on morphisms in $\mathbf{Cat}$. It is clear that R preserves compositions, so it is a functor.

Theorem 4.4.2.3. *The functors*

$$L: \mathbf{Sch} \rightleftarrows \mathbf{Cat} : R$$

are mutually inverse equivalences of categories.

Sketch of proof. It is clear that there is a natural isomorphism $\alpha: \text{id}_{\mathbf{Cat}} \xrightarrow{\cong} L \circ R$; i.e. for any category $\mathcal{C}$, there is an isomorphism $\mathcal{C} \cong L(R(\mathcal{C}))$.

Before giving an isomorphism $\beta: \text{id}_{\mathbf{Sch}} \xrightarrow{\cong} R \circ L$, we briefly describe $R(L(\mathcal{S})) =: (G', \simeq')$ for a schema $\mathcal{S} = (G, \simeq)$. Write $G = (V, A, \text{src}, \text{tgt})$ and $G' = (V', A', \text{src}', \text{tgt}')$. On vertices we have $V = V'$. On arrows we have $A' = \text{Path}_G / \simeq$. The CPER $\simeq'$ for $R(L(\mathcal{S}))$ is imposed in (4.16). Under $\simeq'$, every path of paths in G is made equivalent to its concatenation, considered as a path of length 1 in G' .

There is a natural transformation $\beta: \text{id}_{\mathbf{Sch}} \rightarrow R \circ L$ whose $\mathcal{S}$ -component sends each arrow in G to a certain path of length 1 in G' . We need to see that $\beta_{\mathcal{S}}$ has an inverse.

But this is straightforward: every arrow f in $R \circ L(\mathcal{S})$ is an equivalence class of paths in $\mathcal{S}$; choose any one and send f there; by Definition 4.4.1.2 any other choice will give the identical morphism of schemas. It is easy to show that the roundtrips are identities (again up to the notion of identity given in Definition 4.4.1.2). $\square$

4.5 Limits and colimits

Limits and colimits are universal constructions, meaning they represent certain ideals of behavior in a category. When it comes to sets that map to A and B , the $(A \times B)$ -grid is ideal—it projects on to both A and B as straightforwardly as possible. When it comes to sets that can interpret the elements of both A and B , the disjoint union $A \sqcup B$ is ideal—it includes both A and B without confusion or superfluity. These are limits and colimits in **Set**. Limits and colimits exist in other categories as well.

Limits in a preorder are meets, colimits in a preorder are joins. Limits and colimits also exist for database instances and monoid actions, allowing us to discuss for example the product or union of different state machines. Limits and colimits exist for spaces, giving rise to products and unions, as well as quotients.

Limits and colimits do not exist in every category; when $\mathcal{C}$ is complete with respect to limits (or colimits), these limits always seem to mean something valuable to human intuition. For example, when a subject has already been studied for a long time before category theory came around, it often turns out that classically interesting constructions in the subject correspond to limits and colimits in $\mathcal{C}$. For example products, unions, equivalence relations, etc. are classical ideas in set theory that are naturally captured by limits and colimits in **Set**.

4.5.1 Products and coproducts in a category

In Sections 2.4, we discussed products and coproducts in the category **Set** of sets. Now we discuss the same notions in an arbitrary category. For both products and coproducts we will begin with examples and then write down the general concept, but we'll work on products first.

4.5.1.1 Products

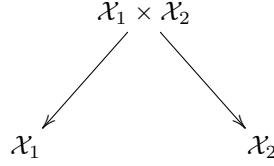
The product of two sets is a grid, which projects down onto each of the two sets. This is good intuition for products in general.

Example 4.5.1.2. Given two preorders, $\mathcal{X}_1 := (X_1, \leq_1)$ and $\mathcal{X}_2 := (X_2, \leq_2)$, we can take their product and get a new preorder $\mathcal{X}_1 \times \mathcal{X}_2$. Both $\mathcal{X}_1$ and $\mathcal{X}_2$ have underlying sets (namely X_1 and X_2), so we might hope that the underlying set of $\mathcal{X}_1 \times \mathcal{X}_2$ is the set $X_1 \times X_2$ of ordered pairs, and this turns out to be true. We have a notion of less-than on $\mathcal{X}_1$ and we have a notion of less-than on $\mathcal{X}_2$; we need to construct a notion of less-than on $\mathcal{X}_1 \times \mathcal{X}_2$. So, given two ordered pairs (x_1, x_2) and (x'_1, x'_2) , when should we say that $(x_1, x_2) \leq_{1,2} (x'_1, x'_2)$ holds? The obvious guess is to say that it holds iff both $x_1 \leq_1 x'_1$ and $x_2 \leq_2 x'_2$ hold, and this works:

$$\mathcal{X}_1 \times \mathcal{X}_2 := (X_1 \times X_2, \leq_{1,2})$$

Note that the projection functions $X_1 \times X_2 \rightarrow X_1$ and $X_1 \times X_2 \rightarrow X_2$ induce morphisms of preorders. That is, if $(x_1, x_2) \leq_{1,2} (x'_1, x'_2)$ then in particular $x_1 \leq_1 x'_1$. So

we have preorder morphisms



Exercise 4.5.1.3. Suppose that you have a partial order $(S, \leq_S)$ on songs (so you know some songs are preferable to others but sometimes you can't compare). And suppose you have a partial order $(A, \leq_A)$ on pieces of art. You're about to be given a pair (s, a) including a song and a piece of art. Does the product partial order $\mathcal{S} \times \mathcal{A}$ provide a reasonable guess for your preferences on pairs? $\diamond$

Exercise 4.5.1.4. Consider the partial order $\leq$ on $\mathbb{N}$ given by standard “less-than-or-equal-to”, so $5 \leq 9$ etc. And consider another partial order, **divides** on $\mathbb{N}$, where a **divides** b if “ a goes into b evenly”, i.e. if there exists $n \in \mathbb{N}$ such that $a * n = b$, so 5 **divides** 35. If we call the product order $(X, \leq) := (\mathbb{N}, \leq) \times (\mathbb{N}, \text{divides})$, which of the following are true:

$$(2, 4) \leq (3, 4)? \quad (2, 4) \leq (3, 5)? \quad (2, 4) \leq (8, 0)? \quad (2, 4) \leq (0, 0)?$$

$\diamond$

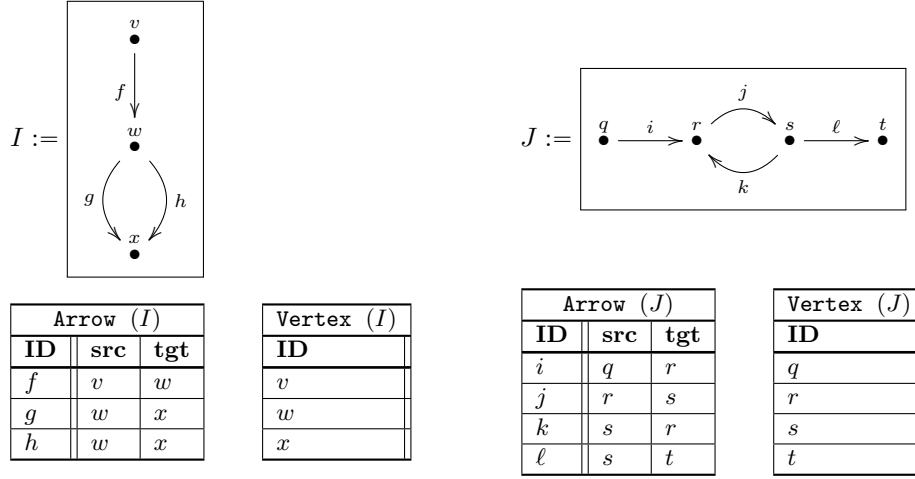
Example 4.5.1.5. Given two graphs $G_1 = (V_1, A_1, \text{src}_1, \text{tgt}_1)$ and $G_2 = (V_2, A_2, \text{src}_2, \text{tgt}_2)$, we can take their product and get a new graph $G_1 \times G_2$. The vertices will be the grid of vertices $V_1 \times V_2$, so each vertex in $G_1 \times G_2$ is labeled by a pair of vertices, one from G_1 and one from G_2 . When should an arrow connect (v_1, v_2) to (v'_1, v'_2) ? Whenever we can find an arrow in G_1 connecting v_1 to v'_1 and we can find an arrow in G_2 connecting v_2 to v'_2 . It turns out there is a simple formula for the set of arrows in $G_1 \times G_2$, namely $A_1 \times A_2$.

Let's write $G := G_1 \times G_2$ and say $G = (V, A, \text{src}, \text{tgt})$. We now know that $V = V_1 \times V_2$ and $A = A_1 \times A_2$. What should the source and target functions $A \rightarrow V$ be? Given a function $\text{src}_1: A_1 \rightarrow V_1$ and a function $\text{src}_2: A_2 \rightarrow V_2$, the universal property of products in **Set** (Lemma 2.4.1.10) provides a unique function

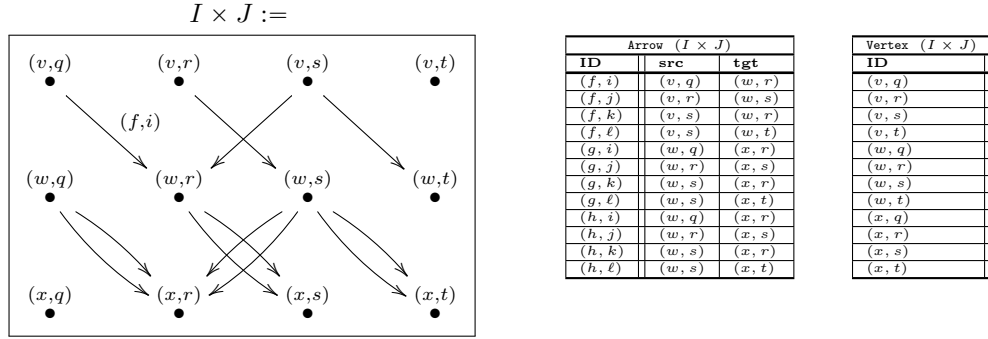
$$\text{src} := \text{src}_1 \times \text{src}_2: A_1 \times A_2 \rightarrow V_1 \times V_2$$

Namely the source of arrow (a_1, a_2) will be the vertex $(\text{src}_1(a_1), \text{src}_2(a_2))$. Similarly we have a ready-made choice of target function $\text{tgt} = \text{tgt}_1 \times \text{tgt}_2$. We have now defined the product graph.

Here's a concrete example. Let I and J be as drawn below:



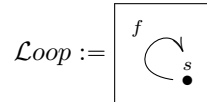
The product $I \times J$ drawn below has, as expected $3 \times 4 = 12$ vertices and $3 \times 4 = 12$ arrows:



Here is the most important thing to notice. Look at the **Arrow** table for $I \times J$, and for each ordered pair, look only at the second entry in all three columns; you will see something that matches with the **Arrow** table for J . Do the same for I , and again you'll see a perfect match. These “matchings” are readily-visible graph homomorphisms $I \times J \rightarrow I$ and $I \times J \rightarrow J$ in **Grph**.

Exercise 4.5.1.6. Let $[1] = \begin{array}{c} 0 \xrightarrow{f} 1 \\ \bullet \end{array}$ be the linear order graph of length 1 and let $P = \text{Paths}([1])$ be its paths-graph, as in Example 4.1.2.19 (so P should have three arrows and two vertices). Draw the graph $P \times P$. ◇

Exercise 4.5.1.7. Recall from Example 3.5.2.9 that a discrete dynamical system (DDS) is a set s together with a function $f: s \rightarrow s$. By now it should be clear that if



is the loop schema, then a DDS is simply an instance (a functor) $I: \mathcal{Loop} \rightarrow \mathbf{Set}$. We have not yet discussed products of DDS's, but perhaps you can guess how they should

work. For example, consider the instances $I, J: \mathcal{L}oop \rightarrow \mathbf{Set}$ tabulated below:

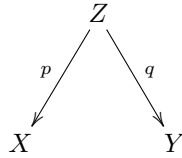
s (I)		s (J)	
ID	f	ID	f
A	C	x	y
B	C	y	x
C	C	z	z

- a.) Make a guess and tabulate $I \times J$. Then draw it.¹⁹
- b.) Recall the notion of natural transformations between functors (see Example 4.3.3.4), which in the case of functors $\mathcal{L}oop \rightarrow \mathbf{Set}$ are the morphisms of instances. Do you see clearly that there is a morphism of instances $I \times J \rightarrow I$ and $I \times J \rightarrow J$? Just check that if you look only at the left-hand coordinates in your $I \times J$, you see something compatible with I .

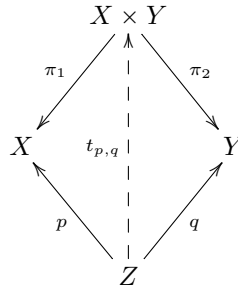
◇

In every case above, what's most important to recognize is that there are projection maps $I \times J \rightarrow I$ and $I \times J \rightarrow J$, and that the construction of $I \times J$ seems as straightforward as possible, subject to having these projections. It is time to give the definition.

Definition 4.5.1.8. Let $\mathcal{C}$ be a category and let $X, Y \in \text{Ob}(\mathcal{C})$ be objects. A *span on X and Y* consists of three constituents (Z, p, q) , where $Z \in \text{Ob}(\mathcal{C})$ is an object, and where $p: Z \rightarrow X$ and $q: Z \rightarrow Y$ are morphisms in $\mathcal{C}$.



A *product of X and Y* is a span $X \xleftarrow{\pi_1} X \times Y \xrightarrow{\pi_2} Y$,²⁰ such that for any other span $X \xleftarrow{p} Z \xrightarrow{q} Y$ there exists a unique morphism $t_{p,q}: Z \rightarrow X \times Y$ such that the diagram below commutes:



Remark 4.5.1.9. Definition 4.5.1.8 endows the product of two objects with something known as a *universal property*. It says that a product of two objects X and Y maps to those two objects, and serves as a gateway for all who do the same. “None shall map to X and Y except through me!” This grandiose property is held by products in all the various categories we have discussed so far. It is what I meant when I said things like

¹⁹The result is not necessarily inspiring, but at least computing it is straightforward.

²⁰The names $X \times Y$ and π_1, π_2 are not mathematically important, they are pedagogically suggestive.

“ $X \times Y$ maps to both X and Y and does so as straightforwardly as possible”. The grid of dots obtained as the product of two sets has such a property, as was shown in Example 2.4.1.11.

Example 4.5.1.10. In Example 4.5.1.2 we discussed products of preorders. In this example we will discuss products in an individual preorder. That is, by Proposition 4.2.1.17, there is a functor $\mathbf{PrO} \rightarrow \mathbf{Cat}$ that realizes every preorder as a category. If $\mathcal{P} = (P, \leq)$ is a preorder, what are products in $\mathcal{P}$? Given two objects $a, b \in \text{Ob}(\mathcal{P})$ we first consider spans on a and b , i.e. $a \leftarrow z \rightarrow b$. That would be some z such that $z \leq a$ and $z \leq b$. The product will be such a span $a \geq a \times b \leq b$, but such that every other spanning object z is less than or equal to $a \times b$. In other words $a \times b$ is as big as possible subject to the condition of being less than a and less than b . This is precisely the meet of a and b (see Definition 3.4.2.1).

Example 4.5.1.11. Note that the product of two objects in a category $\mathcal{C}$ may not exist. Let’s return to preorders to see this phenomenon.

Consider the set $\mathbb{R}^2$, and say that $(x_1, y_1) \leq (x_2, y_2)$ if there exists $\ell \geq 1$ such that $x_1 \ell = x_2$ and $y_1 \ell = y_2$; in other words, point p is less than point q if, in order to travel from q to the origin along a straight line, one must pass through p along the way.²¹ We have given a perfectly good partial order, but $p := (1, 0)$ and $q := (0, 1)$ do not have a product. Indeed, it would have to be a non-zero point that was on the same line-through-the origin as p and the same line-through-the-origin as q , of which there are none.

Example 4.5.1.12. Note that there can be more than one product of two objects in a category $\mathcal{C}$, but that any two choices will be canonically isomorphic. Let’s return once more to preorders to see this phenomenon.

Consider the set $\mathbb{R}^2$ and say that $(x_1, y_1) \leq (x_2, y_2)$ if $x_1^2 + y_1^2 \leq x_2^2 + y_2^2$, in other words if the former is on a smaller 0-circle (by which I mean “circle centered at the origin”) than the latter is.

For any two points p, q there will be lots of points that serve as products: anything on the smaller of their two 0-circles will suffice. Given any two points p, q on this smaller circle, we will have a unique isomorphism $p \cong q$ because $p \leq q$ and $q \leq p$ and all morphisms are unique in a preorder.

Exercise 4.5.1.13. Consider the preorder $\mathcal{P}$ of cards in a deck, shown in Example 3.4.1.3; it is not the entire story of cards in a deck, but take it to be so. In other words, be like a computer and take what’s there at face value. Consider the preorder $\mathcal{P}$ as a category (by way of the functor $\mathbf{PrO} \rightarrow \mathbf{Cat}$).

a.) For each of the following pairs, what is their product in $\mathcal{P}$ (if it exists)?

‘a diamond’ $\times$ ‘a heart’ ?

‘a queen’ $\times$ ‘a black card’ ?

‘a card’ $\times$ ‘a red card’ ?

‘a face card’ $\times$ ‘a black card’ ?

b.) How would these answers differ if $\mathcal{P}$ was completed to the “whole story” partial order classifying cards in a deck?

◇

Exercise 4.5.1.14. Let X be a set, and consider it as a discrete category. Given two objects $x, y \in \text{Ob}(X)$, under what conditions will there exist a product $x \times y$? ◇

²¹Note that $(0, 0)$ is not related to anything else.

Exercise 4.5.1.15. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ be a function, like you would see in 6th grade (maybe $f(x) = x + 7$). A typical thing to do is to graph f as a curve running through the plane $\mathbb{R}^2 := \mathbb{R} \times \mathbb{R}$. This curve can be understood as a function $F: \mathbb{R} \rightarrow \mathbb{R}^2$.

- a.) Given some $x \in \mathbb{R}$, what are the coordinates of $F(x) \in \mathbb{R}^2$?
- b.) Obtain $F: \mathbb{R} \rightarrow \mathbb{R}^2$ using the universal property given in Definition 4.5.1.8.

◇

Exercise 4.5.1.16. Consider the preorder $(\mathbb{N}, \text{divides})$, discussed in Exercise 4.5.1.4, where e.g. $5 \leq 15$ but $5 \not\leq 6$.

- a.) What is the product of 9 and 12 in this category?
- b.) Is there a standard name for products in this category?

◇

Example 4.5.1.17. All products exist in the category **Cat**. Given two categories $\mathcal{C}$ and $\mathcal{D}$, there is a product category $\mathcal{C} \times \mathcal{D}$. We have $\text{Ob}(\mathcal{C} \times \mathcal{D}) = \text{Ob}(\mathcal{C}) \times \text{Ob}(\mathcal{D})$ and for any two objects (c, d) and (c', d') , we have

$$\text{Hom}_{\mathcal{C} \times \mathcal{D}}((c, d), (c', d')) = \text{Hom}_{\mathcal{C}}(c, c') \times \text{Hom}_{\mathcal{D}}(d, d').$$

The composition formula is “obvious”.

Let $[1] \in \text{Ob}(\mathbf{Cat})$ denote the linear order category of length 1, drawn

$$[1] := \boxed{\begin{array}{ccc} 0 & \xrightarrow{f} & 1 \\ \bullet & & \bullet \end{array}}$$

As a schema it has one arrow, but as a category it has three morphisms. So we expect $[1] \times [1]$ to have 9 morphisms, and that’s true. In fact, $[1] \times [1]$ looks like a commutative square:

$$\begin{array}{ccc} (0,0) & \xrightarrow{\text{id}_0 \times f} & (0,1) \\ \downarrow f \times \text{id}_0 & & \downarrow f \times \text{id}_1 \\ (1,0) & \xrightarrow{\text{id}_1 \times f} & (1,1) \end{array}$$

We see only four morphisms here, but there are also four identities and one morphism $(0,0) \rightarrow (1,1)$ given by composition of either direction. It is a minor miracle that the categorical product somehow “knows” that this square should commute; however, this is not the mere preference of man but instead the dictate of God! By which I mean, this follows rigorously from the definitions we already gave of **Cat** and products.

4.5.1.18 Coproducts

The coproduct of two sets is their disjoint union, which includes non-overlapping copies of each of the two sets. This is good intuition for coproducts in general.

Example 4.5.1.19. Given two preorders, $\mathcal{X}_1 := (X_1, \leq_1)$ and $\mathcal{X}_2 := (X_2, \leq_2)$, we can take their coproduct and get a new preorder $\mathcal{X}_1 \sqcup \mathcal{X}_2$. Both $\mathcal{X}_1$ and $\mathcal{X}_2$ have underlying sets

(namely X_1 and X_2), so we might hope that the underlying set of $\mathcal{X}_1 \times \mathcal{X}_2$ is the set $X_1 \sqcup X_2$ consisting of their disjoint union, and that turns out to be true. We have a notion of less-than on $\mathcal{X}_1$ and we have a notion of less-than on $\mathcal{X}_2$; given an element $x \in X_1 \sqcup X_2$ and an element $x' \in X_1 \sqcup X_2$, when should we say that $x \leq_{1 \sqcup 2} x'$ holds? The obvious guess is to say that it holds iff somebody says it does; that is, either both x and x' are from the same home (i.e. $x \in X_1$ and $x' \in X_1$ OR $x \in X_2$ and $x' \in X_2$) or they are from different homes (i.e. $x \in X_1$ and $x' \in X_2$ OR $x \in X_2$ and $x' \in X_1$), and this is correct:

$$\mathcal{X}_1 \sqcup \mathcal{X}_2 := (X_1 \sqcup X_2, \leq_{1 \sqcup 2})$$

Note that the inclusion functions $X_1 \rightarrow X_1 \sqcup X_2$ and $X_2 \rightarrow X_1 \sqcup X_2$ induce morphisms of preorders. That is, if $x, x' \in X_1$ are elements such that $x \leq_1 x'$ in $\mathcal{X}_1$ then the same will hold in $\mathcal{X}_1 \sqcup \mathcal{X}_2$. So we have preorder morphisms

$$\begin{array}{ccc} & \mathcal{X}_1 \sqcup \mathcal{X}_2 & \\ \nearrow & & \nwarrow \\ \mathcal{X}_1 & & \mathcal{X}_2 \end{array}$$

Exercise 4.5.1.20. Suppose that you have a partial order $\mathcal{A} := (A, \leq_A)$ on apples (so you know some apples are preferable to others but sometimes you can't compare). And suppose you have a partial order $\mathcal{O} := (O, \leq_O)$ on oranges. You're about to be given two pieces of fruit from a basket of apples and oranges. Is the coproduct partial order $\mathcal{A} \sqcup \mathcal{O}$ a reasonable guess for your preferences, or does it seem biased? $\diamond$

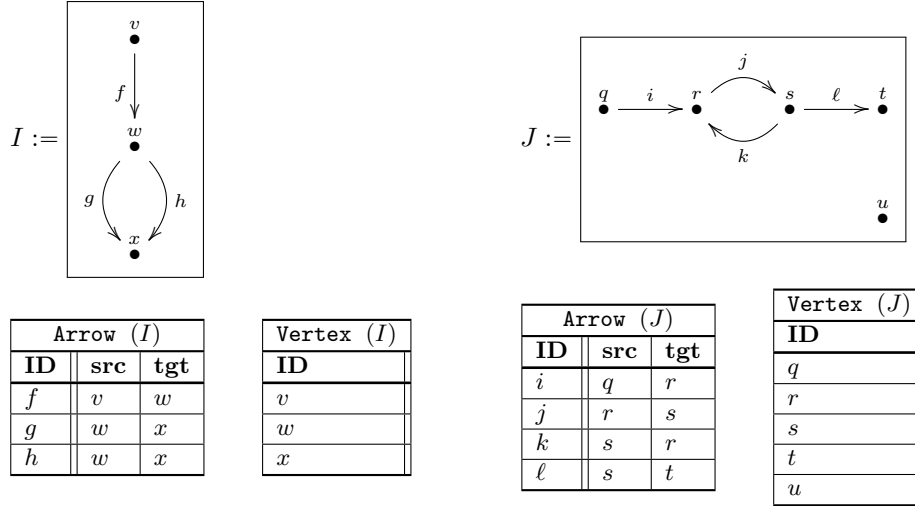
Example 4.5.1.21. Given two graphs $G_1 = (V_1, A_1, src_1, tgt_1)$ and $G_2 = (V_2, A_2, src_2, tgt_2)$, we can take their coproduct and get a new graph $G_1 \sqcup G_2$. The vertices will be the disjoint union of vertices $V_1 \sqcup V_2$, so each vertex in $G_1 \times G_2$ is labeled either by a vertex in G_1 or by one in G_2 (and if any labels are shared, then something must be done to differentiate them). When should an arrow connect v to v' ? Whenever both are from the same component (i.e. either $v, v' \in V_1$ or $v, v' \in V_2$) and we can find an arrow connecting them in that component. It turns out there is a simple formula for the set of arrows in $G_1 \sqcup G_2$, namely $A_1 \sqcup A_2$.

Let's write $G := G_1 \sqcup G_2$ and say $G = (V, A, src, tgt)$. We now know that $V = V_1 \sqcup V_2$ and $A = A_1 \sqcup A_2$. What should the source and target functions $A \rightarrow V$ be? Given a function $src_1: A_1 \rightarrow V_1$ and a function $src_2: A_2 \rightarrow V_2$, the universal property of coproducts in **Set** (Lemma 2.4.2.7) provides a unique function

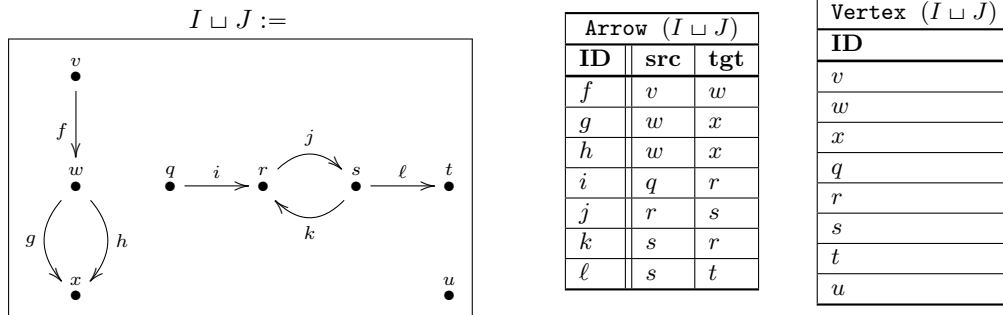
$$src := src_1 \sqcup src_2: A_1 \sqcup A_2 \rightarrow V_1 \sqcup V_2.$$

Namely for any arrow $a \in A$, we know either $a \in A_1$ or $a \in A_2$ (and not both), so the source of a will be the vertex $src_1(a)$ if $a \in A_1$ and $src_2(a)$ if $a \in A_2$. Similarly we have a ready-made choice of target function $tgt = tgt_1 \sqcup tgt_2$. We have now defined the coproduct graph.

Here's a real example. Let I and J be as in Example 4.3.3.4, drawn below:

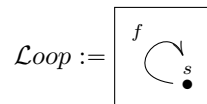


The coproduct $I \sqcup J$ drawn below has, as expected $3 + 5 = 8$ vertices and $3 + 4 = 7$ arrows:



Here is the most important thing to notice. Look at the **Arrow** table I and notice that there is a way to send each row to a row in $I \sqcup J$, such that all the foreign keys match. Similarly in the arrow table and the two vertex tables for J . These “matchings” are readily-visible graph homomorphisms $I \rightarrow I \sqcup J$ and $J \rightarrow I \sqcup J$ in **Grph**.

Exercise 4.5.1.22. Recall from Example 3.5.2.9 that a discrete dynamical system (DDS) is a set s together with a function $f: s \rightarrow s$; if



is the loop schema, then a DDS is simply an instance (a functor) $I: \mathcal{L}oop \rightarrow \mathbf{Set}$. We have not yet discussed coproducts of DDS's, but perhaps you can guess how they should

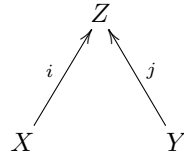
work. For example, consider the instances $I, J: \mathcal{L}oop \rightarrow \mathbf{Set}$ tabulated below:

s (I)		s (J)	
ID	f	ID	f
A	C	x	y
B	C	y	x
C	C	z	z

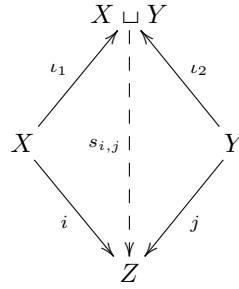
Make a guess and tabulate $I \sqcup J$. Then draw it. $\diamond$

In every case above (preorders, graphs, DDSs), what's most important to recognize is that there are inclusion maps $I \rightarrow I \sqcup J$ and $J \rightarrow I \sqcup J$, and that the construction of $I \sqcup J$ seems as straightforward as possible, subject to having these inclusions. It is time to give the definition.

Definition 4.5.1.23. Let $\mathcal{C}$ be a category and let $X, Y \in \text{Ob}(\mathcal{C})$ be objects. A *cospan* on X and Y consists of three constituents (Z, i, j) , where $Z \in \text{Ob}(\mathcal{C})$ is an object, and where $i: X \rightarrow Z$ and $j: Y \rightarrow Z$ are morphisms in $\mathcal{C}$.



A *coproduct* of X and Y is a cospan $X \xrightarrow{\iota_1} X \sqcup Y \xleftarrow{\iota_2} Y$,²² such that for any other cospan $X \xrightarrow{i} Z \xleftarrow{j} Y$ there *exists a unique* morphism $s_{i,j}: X \sqcup Y \rightarrow Z$ such that the diagram below commutes:



Remark 4.5.1.24. Definition 4.5.1.8 endows the coproduct of two objects with a *universal property*. It says that a coproduct of two objects X and Y receives maps from those two objects, and serves as a gateway for all who do the same. “None shall receive maps from X and Y except through me!” This grandiose property is held by all the coproducts we have discussed so far. It is what I meant when I said things like “ $X \sqcup Y$ receives maps from both X and Y and does so as straightforwardly as possible”. The disjoint union of dots obtained as the coproduct of two sets has such a property, as can be seen by thinking about Example 2.4.2.5.

Example 4.5.1.25. By Proposition 4.2.1.17, there is a functor $\mathbf{PrO} \rightarrow \mathbf{Cat}$ that realizes every preorder as a category. If $\mathcal{P} = (P, \leq)$ is a preorder, what are coproducts in $\mathcal{P}$? Given two objects $a, b \in \text{Ob}(\mathcal{P})$ we first consider cospans on a and b , i.e. $a \rightarrow z \leftarrow b$.

²²The names $X \sqcup Y$ and ι_1, ι_2 are not mathematically important, they are pedagogically suggestive.

A cospan of a and b is any z such that $a \leq z$ and $b \leq z$. The coproduct will be such a cospan $a \leq a \sqcup b \leq b$, but such that every other cospanning object z is greater than or equal to $a \sqcup b$. In other words $a \sqcup b$ is as small as possible subject to the condition of being bigger than a and bigger than b . This is precisely the join of a and b (see Definition 3.4.2.1).

Just as for products, the coproduct of two objects in a category $\mathcal{C}$ may not exist, or it may not be unique. The non-uniqueness is much less “bad” because given two candidate coproducts, they will be canonically isomorphic. They may not be equal, but they are isomorphic. But coproducts might not exist at all in certain categories. We will explore that a bit below.

Example 4.5.1.26. Consider the set $\mathbb{R}^2$ and partial order from Example 4.5.1.11 where $(x_1, y_1) \leq (x_2, y_2)$ if there exists $\ell \geq 1$ such that $x_1 \ell = x_2$ and $y_1 \ell = y_2$. Again the points $p := (1, 0)$ and $q := (0, 1)$ do not have a coproduct. Indeed, it would have to be a non-zero point that was on the same line-through-the origin as p and the same line-through-the-origin as q , of which there are none.

Exercise 4.5.1.27. Consider the preorder $\mathcal{P}$ of cards in a deck, shown in Example 3.4.1.3; it is not the entire story of cards in a deck, but take it to be so. In other words, be like a computer and take what’s there at face value. Consider the preorder $\mathcal{P}$ as a category (by way of the functor $\mathbf{Pro} \rightarrow \mathbf{Cat}$). For each of the following pairs, what is their coproduct in $\mathcal{P}$ (if it exists)?

‘a diamond’ $\sqcup$ ‘a heart’ ?

‘a queen’ $\sqcup$ ‘a black card’ ?

‘a card’ $\sqcup$ ‘a red card’ ?

‘a face card’ $\sqcup$ ‘a black card’ ?

How would these answers differ if $\mathcal{P}$ was completed to the “whole story” partial order classifying cards in a deck? $\diamond$

Exercise 4.5.1.28. Let X be a set, and consider it as a discrete category. Given two objects $x, y \in \text{Ob}(X)$, under what conditions will there exist a coproduct $x \sqcup y$? $\diamond$

Exercise 4.5.1.29. Consider the preorder $(\mathbb{N}, \text{divides})$, discussed in Exercise 4.5.1.4, where e.g. $5 \leq 15$ but $5 \not\leq 6$.

a.) What is the coproduct of 9 and 12 in that category?

b.) Is there a standard name for coproducts in that category?

$\diamond$

4.5.2 Diagrams in a category

We have been drawing diagrams since the beginning of the book. What is it that we have been drawing pictures *of*? The answer is that we have been drawing functors.

Definition 4.5.2.1. Let $\mathcal{C}$ and I be categories. ²³ An *I-shaped diagram in $\mathcal{C}$* is simply a functor $d: I \rightarrow \mathcal{C}$. In this case I is called the *indexing category* for the diagram.

²³In fact, the indexing category I is usually assumed to be small in the sense of Remark 4.1.1.2, meaning that its collection of objects is a set.

Example 4.5.2.2. Consider the commutative diagram in **Set** drawn below:

$$\begin{array}{ccc} \mathbb{N} & \xrightarrow{+1} & \mathbb{N} \\ *2 \downarrow & & \downarrow *2 \\ \mathbb{N} & \xrightarrow{+2} & \mathbb{Z} \end{array} \quad (4.17)$$

This is the drawing of a functor $d: [1] \times [1] \rightarrow \mathbf{Set}$. With notation for the objects and morphisms of $[1] \times [1]$ as shown in Example 4.5.1.17, we have $d(0, 0) = d(0, 1) = d(1, 0) = \mathbb{N}$ and $d(1, 1) = \mathbb{Z}$ (for some reason..) and $d(\text{id}_0, f): \mathbb{N} \rightarrow \mathbb{N}$ given by $n \mapsto n + 1$, etc.

The fact that d is a functor means it must respect composition formulas, which implies that this diagram commutes. Recall from Section 2.2 that not all diagrams one can draw will commute; one must specify that a given diagram commutes if he or she wishes to communicate this fact. But then how is a non-commuting diagram to be understood as a functor?

Let $G \in \text{Ob}(\mathbf{Grph})$ denote the following graph

$$\begin{array}{ccc} (0,0) & \xrightarrow{f} & (0,1) \\ \downarrow h & & \downarrow g \\ (1,0) & \xrightarrow{i} & (1,1) \end{array}$$

Recall the free category functor $F: \mathbf{Grph} \rightarrow \mathbf{Cat}$ from Example 4.1.2.27. The free category $F(G) \in \text{Ob}(\mathbf{Cat})$ on G looks almost like $[1] \times [1]$ except that since $[f, g]$ is a different path in G than is $[h, i]$, they become different morphisms in $F(G)$. A functor $F(G) \rightarrow \mathbf{Set}$ might be drawn the same way that (4.17) is, but it would a diagram that would not be said to commute.

We call $[1] \times [1]$ the *commutative square indexing category*.²⁴

Exercise 4.5.2.3. Consider $[2]$, the linear order category of length 2.

- Is $[2]$ the appropriate indexing category for commutative triangles.
- If not, what is?

◇

Example 4.5.2.4. Recall that an equalizer in **Set** was a diagram of sets that looked like this:

$$\begin{array}{ccccc} E & \xrightarrow{f} & A & \xrightarrow[g_2]{g_1} & B \end{array} \quad (4.18)$$

where $g_1 \circ f = g_2 \circ f$. What is the indexing category for such a diagram? It is the schema (4.18) with the PED $[f, g_1] \simeq [f, g_2]$. That is, in some sense you're seeing the indexing category, but the PED needs to be declared.

Exercise 4.5.2.5. Let $\mathcal{C}$ be a category, $A \in \text{Ob}(\mathcal{C})$ an object, and $f: A \rightarrow A$ a morphism in $\mathcal{C}$. Consider the two diagrams in $\mathcal{C}$ drawn below:

$$\boxed{\begin{array}{c} A \xrightarrow{f} A \xrightarrow{f} A \xrightarrow{f} \dots \end{array}} \quad \boxed{\begin{array}{c} A \\ \downarrow f \\ A \end{array}}$$

²⁴We might call what is here denoted by $F(G)$ the *noncommutative square indexing category*.

- a.) Should these two diagrams have the same indexing category?
- b.) If they should have the same indexing category, what is causing or allowing the pictures to appear different?
- c.) If they should not have the same indexing category, what coincidence makes the two pictures have so much in common?

◇

Definition 4.5.2.6. Let $I \in \text{Ob}(\mathbf{Cat})$ be a category. The *left cone on I* , denoted $I^\triangleleft$, is the category defined as follows. On objects we put $\text{Ob}(I^\triangleleft) = \{-\infty\} \sqcup \text{Ob}(I)$, and we call the new object $-\infty$ the *cone point of $I^\triangleleft$* . On morphisms we add a single new morphism $s_b: -\infty \rightarrow b$ for every object $b \in \text{Ob}(I)$; more precisely,

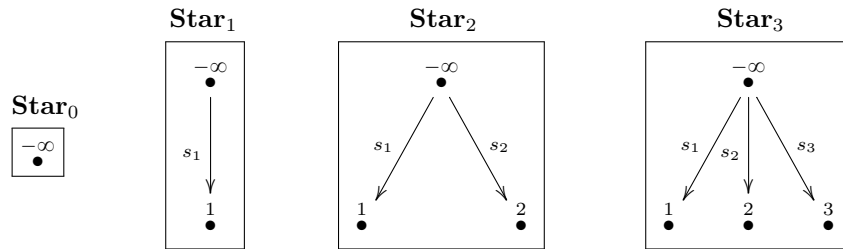
$$\text{Hom}_{I^\triangleleft}(a, b) = \begin{cases} \text{Hom}_I(a, b) & \text{if } a, b \in \text{Ob}(I) \\ \{s_b\} & \text{if } a = -\infty, b \in \text{Ob}(I) \\ \{\text{id}_{-\infty}\} & \text{if } a = b = -\infty \\ \emptyset & \text{if } a \in \text{Ob}(I), b = -\infty. \end{cases}$$

The composition formula is in some sense obvious. To compose two morphisms both in I , compose as dictated by I ; if one has $-\infty$ as source then there will be a unique choice of composite.

There is an obvious inclusion of categories $I \rightarrow I^\triangleleft$.

Remark 4.5.2.7. Note that the specification of $I^\triangleleft$ given in Definition 4.5.2.6 works just as well if I is considered a schema and we are constructing a schema $I^\triangleleft$: add the new object $-\infty$ and the new arrows $s_b: -\infty \rightarrow b$ for each $b \in \text{Ob}(I)$, and for every morphism $f: b \rightarrow b'$ in I add a PED $[s_{b'}] \simeq [s_b, f]$. We generally will not distinguish between categories and schemas, since they are equivalent.

Example 4.5.2.8. For a natural number $n \in \mathbb{N}$, we define the *n -leaf star schema*, denoted $\mathbf{Star}_n$, to be the category (or schema, see Remark 4.5.2.7) $\underline{n}^\triangleleft$, where $\underline{n}$ is the discrete category on n objects. Below we draw $\mathbf{Star}_0, \mathbf{Star}_1, \mathbf{Star}_2$, and $\mathbf{Star}_3$.



Exercise 4.5.2.9. Let $\mathcal{C}_0 := \underline{0}$ denote the empty category and for any natural number $n \in \mathbb{N}$, let $\mathcal{C}_{n+1} = (\mathcal{C}_n)^\triangleleft$. Draw $\mathcal{C}_4$. ◇

Exercise 4.5.2.10. Let $\mathcal{C}$ be the graph indexing schema as in (4.7). What is $\mathcal{C}^\triangleleft$ and how does it compare to (4.18)? ◇

Definition 4.5.2.11. Let $I \in \text{Ob}(\mathbf{Cat})$ be a category. The *right cone on I* , denoted $I^\triangleright$, is the category defined as follows. On objects we put $\text{Ob}(I^\triangleright) = \text{Ob}(I) \sqcup \{\infty\}$, and we call

the new object ∞ the *cone point* of $I^\triangleright$. On morphisms we add a single new morphism $t_b: b \rightarrow \infty$ for every object $b \in \text{Ob}(I)$; more precisely,

$$\text{Hom}_{I^\triangleright}(a, b) = \begin{cases} \text{Hom}_I(a, b) & \text{if } a, b \in \text{Ob}(I) \\ \{t_b\} & \text{if } a \in \text{Ob}(I), b = \infty \\ \{\text{id}_\infty\} & \text{if } a = b = \infty \\ \emptyset & \text{if } a = \infty, b \in \text{Ob}(I). \end{cases}$$

The composition formula is in some sense obvious. To compose two morphisms both in I , compose as dictated by I ; if one has ∞ as target then there will be a unique choice of composite.

There is an obvious inclusion of categories $I \rightarrow I^\triangleright$.

Exercise 4.5.2.12. Let $\mathcal{C}$ be the category $(\underline{2}^\triangleleft)^\triangleright$, where $\underline{2}$ is the discrete category on two objects. Then $\mathcal{C}$ is somehow square-shaped, but what category is it exactly? Looking at Example 4.5.2.2, is $\mathcal{C}$ the commutative diagram indexing category $[1] \times [1]$, is it the non-commutative diagram indexing category $F(G)$, or is it something else? $\diamond$

4.5.3 Limits and colimits in a category

Let $\mathcal{C}$ be a category, I be an indexing category (which just means a category that we're about to use as the indexing category for a diagram), and $D: I \rightarrow \mathcal{C}$ an I -shaped diagram (which just means a functor). It is in relation to this setup that we can discuss the limit or colimit. In general the limit of a diagram $D: I \rightarrow \mathcal{C}$ will be a $I^\triangleleft$ shaped diagram $\lim D: I^\triangleleft \rightarrow \mathcal{C}$. In the case of products $I = \underline{2}$ and $I^\triangleleft = \mathbf{Star}_2$ looks like a span (see Example 4.5.2.8). But out of all the $I^\triangleleft$ -shaped diagrams, which is the product of D ? Answer: the one with the universal “gateway” property, see Remark 4.5.1.9.

4.5.3.1 Universal objects

Definition 4.5.3.2. Let $\mathcal{C}$ be a category. An object $a \in \text{Ob}(\mathcal{C})$ is called *initial* if, for all objects $c \in \text{Ob}(\mathcal{C})$ there exists a unique morphism $a \rightarrow c$, i.e. $|\text{Hom}_{\mathcal{C}}(a, c)| = 1$. An object $z \in \text{Ob}(\mathcal{C})$ is called *terminal* if, for all objects $c \in \text{Ob}(\mathcal{C})$ there is exists a unique morphism $c \rightarrow z$, i.e. $|\text{Hom}_{\mathcal{C}}(c, z)| = 1$.

An object in a category is called *universal* if it is either initial or terminal, but we rarely use that term in practice, preferring to be specific about whether the object is initial or terminal. The word *final* is synonymous with the word terminal, but we'll try to constantly use terminal.

Colimits will end up being initial things of a certain sort, and limits will end up being terminal things of a certain sort. But we will get to that in Section 4.5.3.14.

Example 4.5.3.3. The initial object in **Set** is the set a for which there is always one way to map from a to anything else. Given $c \in \text{Ob}(\mathbf{Set})$ there is exactly one function $\emptyset \rightarrow c$, because there are no choices to be made, so the empty set $\emptyset$ is the initial object in **Set**.

The terminal object in **Set** is the set z for which there is always one way to map to z from anything else. Given $c \in \text{Ob}(\mathbf{Set})$ there is exactly one function $c \rightarrow \{\odot\}$, where $\{\odot\}$ is any set with one element, because there are no choices to be made: everything in c must be sent to the single element in $\{\odot\}$. There are lots of terminal objects in **Set**, and they are all isomorphic to $\underline{1}$.

Example 4.5.3.4. The initial object in **Grph** is the graph a for which there is always one way to map from a to anything else. Given $c \in \text{Ob}(\mathbf{Grph})$, there is exactly one function $\emptyset \rightarrow c$, where $\emptyset \in \mathbf{Grph}$ is the empty graph; so $\emptyset$ is the initial object.

The terminal object in **Grph** is more interesting. It is $\mathcal{L}oop$, the graph with one vertex and one arrow. In fact there are infinitely many terminal objects in **Grph**, but all of them are isomorphic to $\mathcal{L}oop$.

Proposition 4.5.3.5. *Let $\mathcal{C}$ be a category and let $a_1, a_2 \in \text{Ob}(\mathcal{C})$ both be initial objects. Then there is a unique isomorphism $a_1 \xrightarrow{\cong} a_2$. (Similarly, for any two terminal objects in $\mathcal{C}$ there is a unique isomorphism between them.)*

Proof. Suppose a_1 and a_2 are initial. Since a_1 is initial there is a unique morphism $f: a_1 \rightarrow a_2$; there is also a unique morphism $a_1 \rightarrow a_1$, which must be id_{a_1} . Since a_2 is initial there is a unique morphism $g: a_2 \rightarrow a_1$; there is also a unique morphism $a_2 \rightarrow a_2$, which must be id_{a_2} . So $g \circ f = \text{id}_{a_1}$ and $f \circ g = \text{id}_{a_2}$, which means that f is the desired (unique) isomorphism.

The proof for terminal objects is appropriately “dual”.

□

Exercise 4.5.3.6. Let X be a set, let $\mathbb{P}(X)$ be the set of subsets of X (see Definition 2.7.4.9). We can regard $\mathbb{P}(X)$ as a preorder under inclusion of subsets (see for example Section 3.4.2). And we can regard preorders as categories using a functor **PrO** → **Cat** (see Proposition 4.2.1.17).

- a.) What is the initial object in $\mathbb{P}(X)$?
- b.) What is the terminal object in $\mathbb{P}(X)$?

◇

Example 4.5.3.7. The initial object in the category **Mon** of monoids is the trivial monoid, $\underline{1}$. For any monoid M , a morphism of monoids $\underline{1} \rightarrow M$ is a functor between 1-object categories and these are determined by where they send morphisms. Since $\underline{1}$ has only the identity morphism and functors must preserve identities, there is no choice involved in finding a monoid morphism $\underline{1} \rightarrow M$.

Similarly, the terminal object in **Mon** is also the trivial monoid, $\underline{1}$. For any monoid M , a morphism of monoids $M \rightarrow \underline{1}$ sends everything to the identity; there is no choice.

Exercise 4.5.3.8. a.) What is the initial object in **Grp**, the category of groups?

- b.) What is the terminal object in **Grp**?

◇

Example 4.5.3.9. Recall the preorder **Prop** of logical propositions from Section 4.2.4.1. The initial object is a proposition that implies all others. It turns out that “FALSE” is such a proposition. The proposition “FALSE” is like “ $1 \neq 1$ ”; in logical formalism it can be shown that if “FALSE” is true then everything is true.

The terminal object in **Prop** is a proposition that is implied by all others. It turns out that “TRUE” is such a proposition. In logical formalism, everything implies that “TRUE” is true.

Example 4.5.3.10. The discrete category $\underline{2}$ has no initial object and no terminal object. The reason is that it has two objects 1, 2, but no maps from one to the other, so when $\text{Hom}_{\underline{2}}(1, 2) = \text{Hom}_{\underline{2}}(2, 1) = \emptyset$.

Exercise 4.5.3.11. Recall the **divides** preorder from Exercise 4.5.1.4, where 5 divides 15.

- a.) Considering this preorder as a category, does it have an initial object?
 b.) Does it have a terminal object?

◇

Exercise 4.5.3.12. Let $\mathcal{M} = (\text{List}(\{a, b\}), [], ++)$ denote the free monoid on $\{a, b\}$ (see Definition 3.1.1.12).

- a.) Does it have an initial object?
 b.) Does it have a terminal object?

◇

Exercise 4.5.3.13. Let S be a set and consider the indiscrete category $K_S \in \text{Ob}(\mathbf{Cat})$ on S objects (see Example 4.3.4.3).

- a.) Does K_S have an initial object?
 b.) Does it have a terminal object?

◇

4.5.3.14 Examples of limits

Let $\mathcal{C}$ be a category and let $X, Y \in \text{Ob}(\mathcal{C})$ be objects. Definition 4.5.1.8 defines a product of X and Y to be a span $X \xleftarrow{\pi_1} X \times Y \xrightarrow{\pi_2} Y$ such that for every other span $X \xleftarrow{p} Z \xrightarrow{q} Y$ there exists a unique morphism $Z \rightarrow X \times Y$ making the triangles commute. It turns out that we can enunciate this in our newly formed language of universal objects by saying that the span $X \xleftarrow{\pi_1} X \times Y \xrightarrow{\pi_2} Y$ is itself a terminal object in the category of spans on X and Y . Phrasing the definition of products in this way will be generalizable to defining arbitrary limits.

Construction 4.5.3.15 (Products). Let $\mathcal{C}$ be a category and let X_1, X_2 be objects. We can consider this setup as a diagram $X: \underline{2} \rightarrow \mathcal{C}$, where $X(1) = X_1$ and $X(2) = X_2$. Consider the category $\underline{2}^\triangleleft = \mathbf{Star}_2$, which is drawn in Example 4.5.2.8, and the category of functors $\text{Fun}(\underline{2}^\triangleleft, \mathcal{C})$. The objects in $\text{Fun}(\underline{2}^\triangleleft, \mathcal{C})$ are spans in $\mathcal{C}$ and the morphisms are natural transformations between them. Given a functor $S: \underline{2}^\triangleleft \rightarrow \mathcal{C}$ we can compose with $i: \underline{2} \rightarrow \underline{2}^\triangleleft$ to get a functor $\underline{2} \rightarrow \mathcal{C}$. We want that to be X .

$$\begin{array}{ccc} \underline{2} & \xrightarrow{X} & \mathcal{C} \\ i \downarrow & \nearrow S & \\ \underline{2}^\triangleleft & & \end{array}$$

So we are ready to define the category of spans on X_1 and X_2 .

Define the *category of spans on X* , denoted $\mathcal{C}_{/X}$, to be the category whose objects and morphisms are as follows:

$$\begin{aligned} \text{Ob}(\mathcal{C}_{/X}) &= \{S: \underline{2}^\triangleleft \rightarrow \mathcal{C} \mid S \circ i = X\} \\ \text{Hom}_{\mathcal{C}_{/X}}(S, S') &= \{\alpha: S \rightarrow S' \mid \alpha \circ i = \text{id}_X\}. \end{aligned} \tag{4.19}$$

The product of X_1 and X_2 is the terminal object in $\mathcal{C}_{/X}$.

To bring this down to earth, an object in $\mathcal{C}_{/X}$ looks like any diagram in $\mathcal{C}$ that looks like this:

$$\begin{array}{ccc} & Z & \\ p \swarrow & & \searrow q \\ X_1 & & X_2 \end{array}$$

In other words, the objects of $\mathcal{C}_{/X}$ are spans, each of which we might write in-line as $X_1 \xleftarrow{p} Z \xrightarrow{q} X_2$. A morphism in $\mathcal{C}_{/X}$ from object $X_1 \xleftarrow{p} Z \xrightarrow{q} X_2$ to object $X_1 \xleftarrow{p'} Z' \xrightarrow{q'} X_2$ consists of a morphism $\ell: Z \rightarrow Z'$, such that $p' \circ \ell = p$ and $q' \circ \ell = q$. So morphisms in $\mathcal{C}_{/X}$ look like diagrams as to the right: ²⁵

$$\mathrm{Hom}_{\mathcal{C}_{/X}} \left(\begin{array}{ccc} & Z & \\ p \swarrow & & \searrow q \\ X_1 & & X_2 \end{array}, \begin{array}{ccc} & Z' & \\ p' \swarrow & & \searrow q' \\ X_1 & & X_2 \end{array} \right) = \left\{ \begin{array}{ccc} & Z & \\ p \swarrow & & \searrow q \\ X_1 & \ell & X_2 \\ p' \swarrow & & \searrow q' \\ & Z' & \end{array} \right\} \quad (4.20)$$

Each object in $\mathcal{C}_{/X}$ is a span on X_1 and X_2 , and each morphism in $\mathcal{C}_{/X}$ is a “morphism of cone points in $\mathcal{C}$ making everything in sight commute”. The terminal object in $\mathcal{C}_{/X}$ is the product of X_1 and X_2 ; see Definition 4.5.1.8.

It may be strange to have a category in which the objects are spans in another category. But once you admit this possibility, the notion of morphism between spans is totally sensible. Or if it isn't, then stare at (4.20) for 30 seconds and say to yourself “When in Rome..!” These are the aqueducts of category theory, and they work wonders.

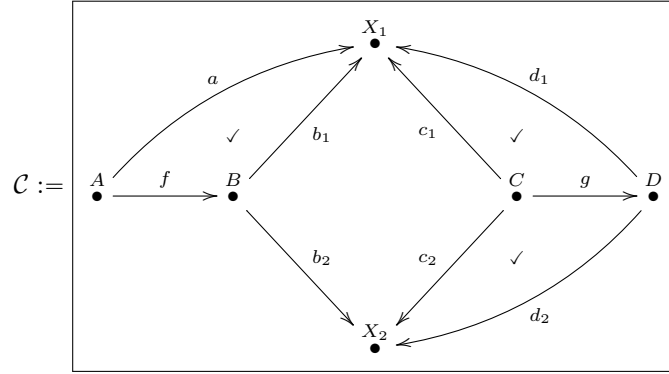
Example 4.5.3.16. Consider the 6-object category $\mathcal{C}$ drawn below, in which the three

²⁵To be completely pedantic, according to (4.19), the morphisms in $\mathcal{C}_{/X}$ should be drawn like this:

$$\mathrm{Hom}_{\mathcal{C}_{/X}} \left(\begin{array}{ccc} & Z & \\ p \swarrow & & \searrow q \\ X_1 & & X_2 \end{array}, \begin{array}{ccc} & Z' & \\ p' \swarrow & & \searrow q' \\ X_1 & & X_2 \end{array} \right) = \left\{ \begin{array}{ccc} & Z & \\ p \swarrow & & \searrow q \\ X_1 & \alpha_1 \parallel & X_2 \\ X_1 & \alpha_{-\infty} & X_2 \\ p' \swarrow & & \searrow q' \\ & Z' & \end{array} \right\}$$

But this is going a bit overboard.

diagrams that can commute do:



Let $X: \underline{2} \rightarrow \mathcal{C}$ be given by $X(1) = X_1$ and $X(2) = X_2$. Then the category of spans on X might be drawn

$$\mathcal{C}_{/X} \cong \boxed{\begin{array}{ccc} (B, b_1, b_2) & & (C, c_1, c_2) \xrightarrow{g} (D, d_1, d_2) \end{array}}$$

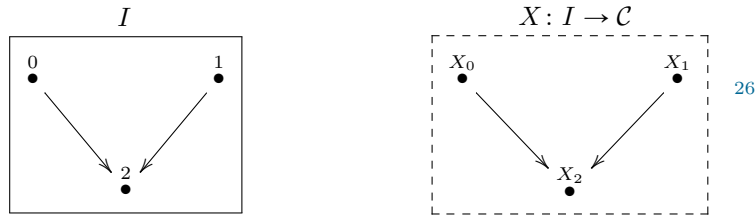
4.5.3.17 Definition of limit

Definition 4.5.3.18. Let $\mathcal{C}$ be a category, let I be a category; let $I^\triangleleft$ be the left cone on I , and let $i: I \rightarrow I^\triangleleft$ be the inclusion. Suppose that $X: I \rightarrow \mathcal{C}$ is an I -shaped diagram in $\mathcal{C}$. The *slice category of $\mathcal{C}$ over X* denoted $\mathcal{C}_{/X}$ is the category whose objects and morphisms are as follows:

$$\begin{aligned} \text{Ob}(\mathcal{C}_{/X}) &= \{S: I^\triangleleft \rightarrow \mathcal{C} \mid S \circ i = X\} \\ \text{Hom}_{\mathcal{C}_{/X}}(S, S') &= \{\alpha: S \rightarrow S' \mid \alpha \circ i = \text{id}_X\}. \end{aligned}$$

A *limit of X* , denoted $\lim_I X$ or $\lim X$, is a terminal object in $\mathcal{C}_{/X}$.

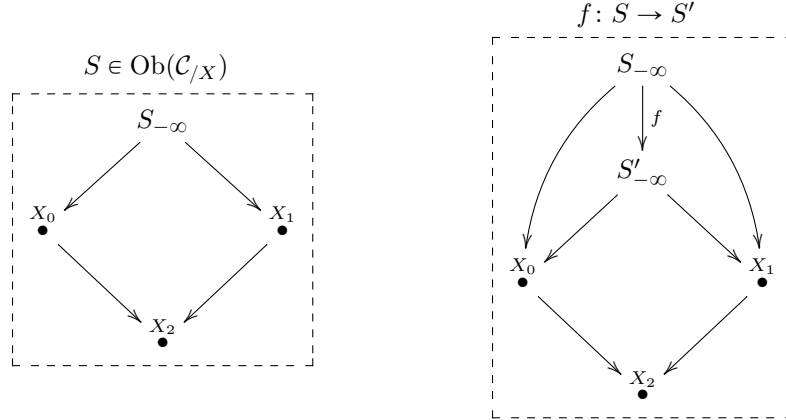
Pullbacks The relevant indexing category for pullbacks is the cospan, $I = \underline{2}^\triangleright$ drawn as to the left below:



A I -shaped diagram in $\mathcal{C}$ is a functor $X: I \rightarrow \mathcal{C}$, which we might draw as to the right above (e.g. $X_0 \in \text{Ob}(\mathcal{C})$).

²⁶We use a dash box here because we're not drawing the whole category but is merely a diagram existing inside $\mathcal{C}$.

An object S in the slice category $\mathcal{C}_{/X}$ is a commutative diagram $S: I^{\triangleleft} \rightarrow \mathcal{C}$ over X , which looks like the box to the left below:



A morphism in $\mathcal{C}_{/X}$ is drawn in the dashbox to the right above. A terminal object in $\mathcal{C}_{/X}$ is precisely the “gateway” we want, i.e. the limit of X is the pullback $X_0 \times_{X_2} X_1$.

Exercise 4.5.3.19. Let I be the graph indexing category (see 4.7).

- What is $I^{\triangleleft}$?
- Now let $G: I \rightarrow \mathbf{Set}$ be the graph from Example 3.3.1.2. Give an example of an object in $\mathbf{Set}_{/G}$.
- We have already given a name to the limit of $G: I \rightarrow \mathbf{Set}$; what is it?

◇

Exercise 4.5.3.20. Let $\mathcal{C}$ be a category and let $I = \emptyset$ be the empty category. There is a unique functor $X: \emptyset \rightarrow \mathcal{C}$.

- What is the slice category $\mathcal{C}_{/X}$?
- What is the limit of X ?

◇

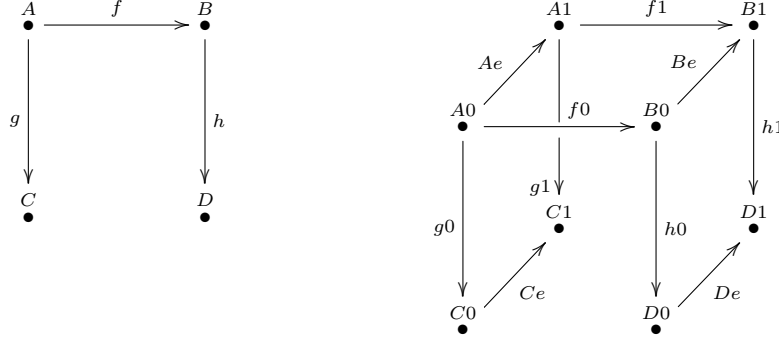
Example 4.5.3.21. Often one wants to take the limit of some strange diagram. We have now constructed the limit for any shape diagram. For example, if we want to take the product of more than two, say n , objects, we could use the diagram shape $I = \underline{n}$ whose cone is $\mathbf{Star}_n$ from Example 4.5.2.8.

Example 4.5.3.22. We have now defined limits in any category, so we have defined limits in $\mathbf{Cat}$. Let $[1]$ denote the category depicted

$$\bullet \xrightarrow{e} \bullet$$

and let $\mathcal{C}$ be a category. Naming two categories is the same thing as naming a functor $X: \underline{2} \rightarrow \mathbf{Cat}$, so we now have such a functor. Its limit is denoted $[1] \times \mathcal{C}$. It turns out that $[1] \times \mathcal{C}$ looks like a “ $\mathcal{C}$ -shaped prism”. It consists of two panes, front and back say, each having the precise shape as $\mathcal{C}$ (same objects, same arrows, same composition), and morphisms from the front pane to the back pane making all front-to-back squares

commute. For example, if $\mathcal{C}$ looked was the category generated by the schema to the left below, then $\mathcal{C} \times [1]$ would be the category generated by the schema to the right below:



It turns out that a natural transformation $\alpha: F \rightarrow G$ between functors $F, G: \mathcal{C} \rightarrow \mathcal{D}$ is the same thing as a functor $\mathcal{C} \times [1] \rightarrow \mathcal{D}$ such that the front pane is sent via F and the back pane is sent via G . The components are captured by the front-to-back morphisms, and the naturality is captured by the commutativity of the front-to-back squares in $\mathcal{C} \times [1]$.

Remark 4.5.3.23. Recall in Section 2.7.6.6 we described relative sets. In fact, Definition 2.7.6.7 basically defines a category of relative sets over any fixed set B . Let $\underline{1}$ denote the discrete category on one object, and note that providing a functor $\underline{1} \rightarrow \mathbf{Set}$ is the same as simply providing a set, so consider $B: \underline{1} \rightarrow \mathbf{Set}$. Then the slice category $\mathbf{Set}_{/B}$, as defined in Definition 4.5.3.18 is precisely the category of relative sets over B : it has the same objects and morphisms as was described in Definition 2.7.6.7.

4.5.3.24 Definition of colimit

The definition of colimits is appropriately “dual” to the definition of limits. Instead of looking at left cones, we look at right cones; instead of being interested in terminal objects, we are interested in initial objects.

Definition 4.5.3.25. Let $\mathcal{C}$ be a category, let I be a category; let $I^\triangleright$ be the right cone on I , and let $i: I \rightarrow I^\triangleright$ be the inclusion. Suppose that $X: I \rightarrow \mathcal{C}$ is an I -shaped diagram in $\mathcal{C}$. The *coslice category of $\mathcal{C}$ over X* denoted $\mathcal{C}_{X/}$ is the category whose objects and morphisms are as follows:

$$\begin{aligned} \text{Ob}(\mathcal{C}_{X/}) &= \{S: I^\triangleright \rightarrow \mathcal{C} \mid S \circ i = X\} \\ \text{Hom}_{\mathcal{C}_{X/}}(S, S') &= \{\alpha: S \rightarrow S' \mid \alpha \circ i = \text{id}_X\}. \end{aligned}$$

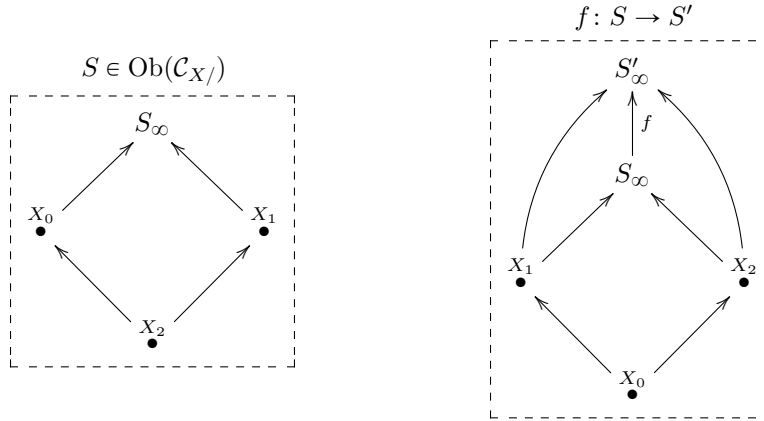
A *colimit of X* , denoted $\text{colim}_I X$ or $\text{colim } X$, is an initial object in $\mathcal{C}_{X/}$.

Pushouts The relevant indexing category for pushouts is the span, $I = \underline{2}^{\triangleleft}$ drawn as to the left below:



An I -shaped diagram in $\mathcal{C}$ is a functor $X: I \rightarrow \mathcal{C}$, which we might draw as to the right above (e.g. $X_0 \in \text{Ob}(\mathcal{C})$).

An object S in the coslice category $\mathcal{C}_{X/}$ is a commutative diagram $S: I^{\triangleright} \rightarrow \mathcal{C}$ over X , which looks like the box to the left below:



A morphism in $\mathcal{C}_{X/}$ is drawn in the dashbox to the right above. An initial object in $\mathcal{C}_{X/}$ is precisely the “gateway” we want; i.e. the colimit of X is the pushout, $X_1 \sqcup_{X_0} X_2$.

Exercise 4.5.3.26. Let I be the graph indexing category (see 4.7).

- What is $I^{\triangleright}$?
- Now let $G: I \rightarrow \mathbf{Set}$ be the graph from Example 3.3.1.2. Give an example of an object in $\mathbf{Set}_{G/}$.
- We have already given a name to the colimit of $G: I \rightarrow \mathbf{Set}$; what is it?

◇

Exercise 4.5.3.27. Let $\mathcal{C}$ be a category and let $I = \emptyset$ be the empty category. There is a unique functor $X: \emptyset \rightarrow \mathcal{C}$.

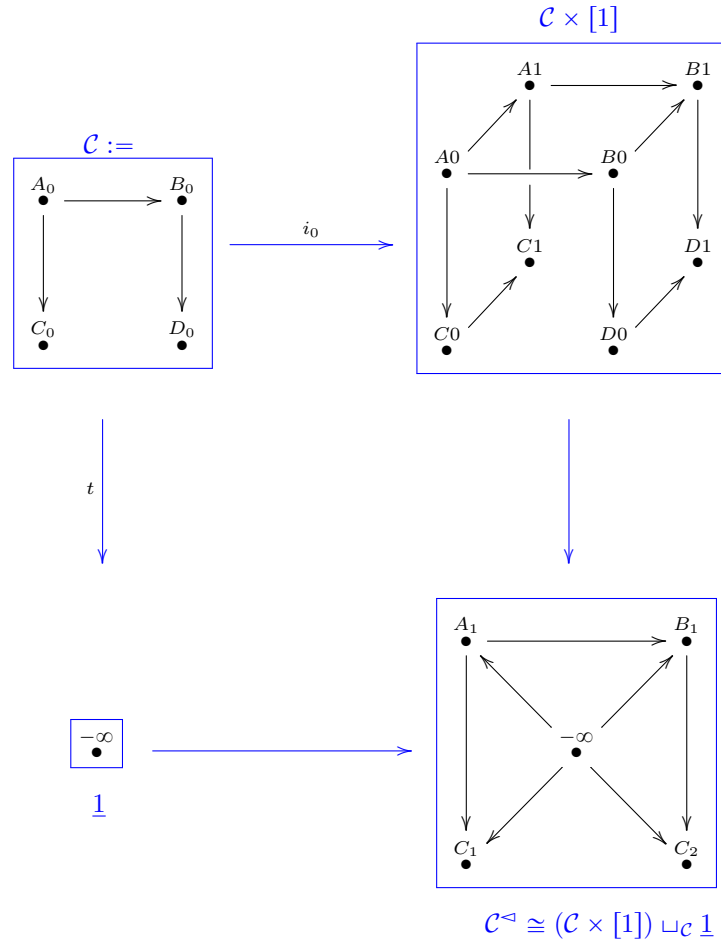
- What is the coslice category $\mathcal{C}_{X/}$?
- What is the colimit of X ?

◇

Example 4.5.3.28 (Cone as colimit). We have now defined colimits in any category, so we have defined colimits in **Cat**. Let $\mathcal{C}$ be a category and recall from Example 4.5.3.22 the category $\mathcal{C} \times [1]$. The inclusion of the front pane is a functor $i_0: \mathcal{C} \rightarrow \mathcal{C} \times [1]$ (similarly, the inclusion of the back pane is a functor $i_1: \mathcal{C} \rightarrow \mathcal{C} \times [1]$). Finally let $t: \mathcal{C} \rightarrow \underline{1}$ be the unique functor to the terminal category (see Exercise 4.1.2.34). We now have a diagram in **Cat** of the form

$$\begin{array}{ccc} \mathcal{C} & \xrightarrow{i_0} & \mathcal{C} \times [1] \\ t \downarrow & & \\ \underline{1} & & \end{array}$$

The colimit (i.e. the pushout) of this diagram in **Cat** slurps down the entire front pane of $\mathcal{C} \times [1]$ to a point, and the resulting category is isomorphic to $\mathcal{C}^{\triangleleft}$. Here is a drawing of this phenomenon



(Similarly, the pushout of the analogous diagram for i_1 would give $\mathcal{C}^{\triangleright}$.)

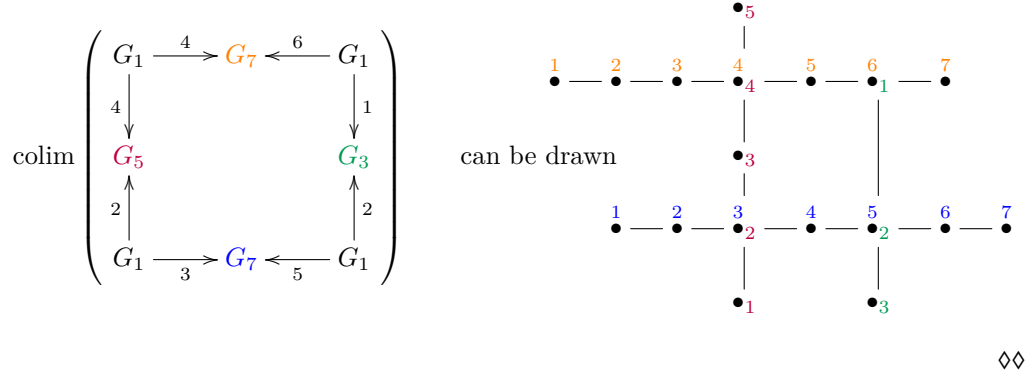
Example 4.5.3.29. Consider the category **Top** of topological spaces. The (empty) circle is a topological space which people often denote S^1 (for “1-dimensional sphere”). The

filled-in circle, also called a 2-dimensional disk, is denoted D^2 . The inclusion of the circle into the disk is continuous so we have a morphism in **Top** of the form $i: S^1 \rightarrow D^2$. The terminal object in **Top** is the one-point space $\{\odot\}$, and so there is a unique morphism $t: S^1 \rightarrow \{\odot\}$. The pushout of the diagram $D^2 \xleftarrow{i} S^1 \xrightarrow{t} \{\odot\}$ is isomorphic to the 2-dimensional sphere (the exterior of a tennis ball), S^2 . The reason is that we have slurped the entire bounding circle to a point, and the category of topological spaces has the right morphisms to ensure that the resulting space really is a sphere.

Application 4.5.3.30. Consider the symmetric graph G_n consisting of a chain of n vertices,

$$\bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } \bullet$$

Think of this as modeling a subway line. There are n -many graph homomorphisms $G_1 \rightarrow G_n$ given by the various vertices. One can create **transit maps** using colimits. For example, the colimit of the diagram to the left is the symmetric graph drawn to the right below.



4.6 Other notions in Cat

In this section we discuss some leftover notions about categories. For example in Section 4.6.1 we explain a kind of duality for categories, in which arrows are flipped. For example reversing the order in a preorder is an example of this duality, as is the similarity between limits and colimits. In Section 4.6.2 we discuss the so-called Grothendieck construction which in some sense graphs functors, and we show that it is useful for transforming databases into the kind of format (RDF) used in scraping data off webpages. We define a general construction for creating categories in Section 4.6.4. Finally, in Section 4.6.5 we show that precisely the same arithmetic statements that held for sets in Section 2.7.3 hold for categories.

4.6.1 Opposite categories

People used to discuss two different kinds of functors between categories: the so-called *covariant functors* and the so-called *contravariant functors*. Covariant functors are what we have been calling functors. The reader may have come across the idea of contravariance when considering Exercise 4.2.3.2.²⁷ There we saw that a continuous mapping of

²⁷Similarly, see Exercise 4.2.4.4).

topological spaces $f: X \rightarrow Y$ does not induce a morphism of orders on their open sets $\text{Op}(X) \rightarrow \text{Op}(Y)$; that is not required by the notion of continuity. Instead, a morphism of topological spaces $f: X \rightarrow Y$ induces a morphism of orders $\text{Op}(Y) \rightarrow \text{Op}(X)$, going backwards. So we do not have a functor $\mathbf{Top} \rightarrow \mathbf{PrO}$ in this way, but it's quite close. One used to say that Op is a *contravariant functor* $\mathbf{Top} \rightarrow \mathbf{PrO}$.

As important and common as contravariance is, people found that keeping track of which functors were covariant and which were contravariant was a big hassle. Luckily, there is a simple work-around, which simplifies everything: the notion of opposite categories.

Definition 4.6.1.1. Let $\mathcal{C}$ be a category. The *opposite category* of $\mathcal{C}$, denoted $\mathcal{C}^{\text{op}}$, has the same objects as $\mathcal{C}$, i.e. $\text{Ob}(\mathcal{C}^{\text{op}}) = \text{Ob}(\mathcal{C})$, and for any two objects c, c' , one defines

$$\text{Hom}_{\mathcal{C}^{\text{op}}}(c, c') := \text{Hom}_{\mathcal{C}}(c', c).$$

Example 4.6.1.2. If $n \in \mathbb{N}$ is a natural number and $\underline{n}$ the corresponding discrete category, then $\underline{n}^{\text{op}} = \underline{n}$. Recall the span category $I = \underline{2}^{\triangleleft}$ from Definition 4.5.1.8. Its opposite is the cospan category $I^{\text{op}} = \underline{2}^{\triangleright}$, from Definition 4.5.1.23.

Exercise 4.6.1.3. Let $\mathcal{C}$ be the category from Example 4.5.3.16. Draw $\mathcal{C}^{\text{op}}$. ◇

Lemma 4.6.1.4. Let $\mathcal{C}$ and $\mathcal{D}$ be categories. One has $(\mathcal{C}^{\text{op}})^{\text{op}} = \mathcal{C}$. Also we have $\text{Fun}(\mathcal{C}, \mathcal{D}) \cong \text{Fun}(\mathcal{C}^{\text{op}}, \mathcal{D}^{\text{op}})$. This implies that a functor $\mathcal{C}^{\text{op}} \rightarrow \mathcal{D}$ can be identified with a functor $\mathcal{C} \rightarrow \mathcal{D}^{\text{op}}$.

Proof. This follows straightforwardly from the definitions. □

Exercise 4.6.1.5. In Exercises 4.2.3.2, 4.2.4.3, and 4.2.4.4 there were questions about whether a certain function $\text{Ob}(\mathcal{C}) \rightarrow \text{Ob}(\mathcal{D})$ extended to a functor $\mathcal{C} \rightarrow \mathcal{D}$. In each case, see if the proposed function would extend to a “contravariant functor” i.e. to a functor $\mathcal{C}^{\text{op}} \rightarrow \mathcal{D}$. ◇

Example 4.6.1.6 (Simplicial sets). Recall from Example 4.3.4.4 the category Δ of linear orders $[n]$. For example, $[1]$ is the linear order $0 \leq 1$ and $[2]$ is the linear order $0 \leq 1 \leq 2$. Both $[1]$ and $[2]$ are objects of Δ . There are 6 morphisms from $[1]$ to $[2]$, which we could denote

$$\text{Hom}_{\Delta}([1], [2]) = \{(0, 0), (0, 1), (0, 2), (1, 1), (1, 2), (2, 2)\}.$$

It may seem strange, but the category Δ^{op} turns out to be quite useful in algebraic topology. It is the indexing category for a combinatorial approach to the homotopy theory of spaces. That is, we can represent something like the category of spaces and continuous maps using the functor category $\mathbf{sSet} := \text{Fun}(\Delta^{\text{op}}, \mathbf{Set})$, which is called the *category of simplicial sets*.

This may seem very complicated compared to something we did earlier, namely simplicial complexes. But simplicial sets have excellent formal properties that simplicial complexes do not. We will not go further with this here, but through the work of Dan Kan, André Joyal, Jacob Lurie, and many others, simplicial sets have allowed category theory to pierce deeply into the realm of topology and vice versa.

4.6.2 Grothendieck construction

Let $\mathcal{C}$ be a database schema (or category) and let $I: \mathcal{C} \rightarrow \mathbf{Set}$ be an instance. We have been drawing this in table form, but there is another standard way of laying out the data in I , called the *resource descriptive framework* or RDF. Developed for the web, RDF is a useful format when one does not have a schema in hand, e.g. when scraping information off of a website, one does not know what schema will be best. In these cases, information is stored in so-called RDF triples, which are of the form

$$\langle \text{Subject, Predicate, Object} \rangle$$

For example, one might see something like

Subject	Predicate	Object
A01	occuredOn	D13114
A01	performedBy	P44
A01	actionDescription	Told congress to raise debt ceiling
D13114	hasYear	2013
D13114	hasMonth	January
D13114	hasDay	14
P44	FirstName	Barack
P44	LastName	Obama

(4.21)

Category-theoretically, it is quite simple to convert a database instance $I: \mathcal{C} \rightarrow \mathbf{Set}$ into an RDF triple store. To do so, we use the *Grothendieck construction*, which is more aptly named the category of elements construction, defined below.²⁸

Definition 4.6.2.1. Let $\mathcal{C}$ be a category and let $I: \mathcal{C} \rightarrow \mathbf{Set}$ be a functor. The *category of elements of I* , denoted $\int_{\mathcal{C}} I$, is defined as follows:

$$\begin{aligned} \text{Ob}(\int_{\mathcal{C}} I) &:= \{(C, x) \mid C \in \text{Ob}(\mathcal{C}), x \in I(C)\}. \\ \text{Hom}_{\int_{\mathcal{C}} I}((C, x), (C', x')) &:= \{f: C \rightarrow C' \mid I(f)(x) = x'\}. \end{aligned}$$

There is a natural functor $\pi_I: \int_{\mathcal{C}} I \rightarrow \mathcal{C}$. It sends each object $(C, x) \in \text{Ob}(\int_{\mathcal{C}} I)$ to the object $C \in \text{Ob}(\mathcal{C})$. And it sends each morphism $f: (C, x) \rightarrow (C', x')$ to the morphism $f: C \rightarrow C'$.

Example 4.6.2.2. Let A be a set, and consider it as a discrete category. We saw in Exercise 4.3.3.3 that a functor $S: A \rightarrow \mathbf{Set}$ is the same thing as an A -indexed set, as discussed in Section 2.7.6.10. We will follow Definition 2.7.6.12 and for each $a \in A$ write $S_a := S(a)$.

What is the category of elements of a functor $S: A \rightarrow \mathbf{Set}$? The objects of $\int_A S$ are pairs (a, s) where $a \in A$ and $s \in S(a)$. Since A has nothing but identity morphisms, $\int_A S$

²⁸Apparently, Alexander Grothendieck did not invent this construction, it was discussed prior to Grothendieck's use of it, e.g. by Mac Lane. But more to the point, the term Grothendieck construction is not grammatically suited in the sense that both the following are awkward in English: "the Grothendieck construction of I is ..." (awkward because I is not being constructed but used in a construction) and "the Grothendieck construct for I is..." (awkward because it just is). The term *category of elements* is more descriptive and easier to use grammatically.

has nothing but identity morphisms; i.e. it is the discrete category on a set. In fact that set is the disjoint union

$$\int_A S = \sqcup_{a \in A} S_a.$$

The functor $\pi_S: \int_A S \rightarrow A$ sends each element in S_a to the element $a \in A$.

One can see this as a kind of histogram. For example, let $A = \{\text{BOS}, \text{NYC}, \text{LA}, \text{DC}\}$ and let $S: A \rightarrow \mathbf{Set}$ assign

$$\begin{aligned} S_{\text{BOS}} &= \{\text{Abby}, \text{Bob}, \text{Casandra}\}, \\ S_{\text{NYC}} &= \emptyset, \\ S_{\text{LA}} &= \{\text{John}, \text{Jim}\}, \text{ and} \\ S_{\text{DC}} &= \{\text{Abby}, \text{Carla}\}. \end{aligned}$$

Then the category of elements of S would look like the (discrete) category at the top:

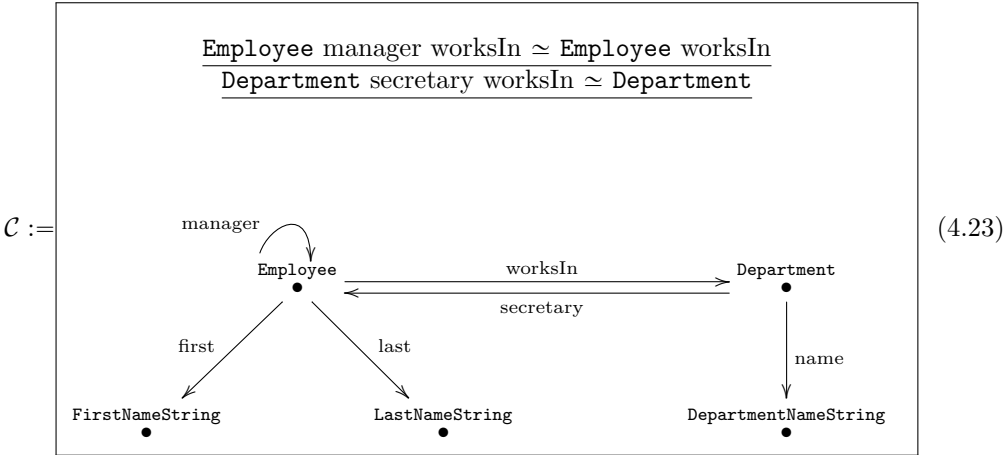
$$\begin{array}{c} \int_A S = \begin{array}{|c|} \hline \begin{array}{ccc} (\text{BOS}, \text{Abby}) & & \\ \bullet & & \\ (\text{BOS}, \text{Bob}) & (\text{LA}, \text{John}) & (\text{DC}, \text{Abby}) \\ \bullet & \bullet & \bullet \\ (\text{BOS}, \text{Casandra}) & (\text{LA}, \text{Jim}) & (\text{DC}, \text{Carla}) \\ \bullet & \bullet & \bullet \end{array} \\ \hline \end{array} \quad (4.22) \\ \\ \pi_S \downarrow \\ A = \begin{array}{|c|} \hline \begin{array}{cccc} \text{BOS} & \text{NYC} & \text{LA} & \text{DC} \\ \bullet & \bullet & \bullet & \bullet \end{array} \\ \hline \end{array} \end{array}$$

We also see that the category of elements construction has converted an A -indexed set into a relative set over A , as in Definition 2.7.6.7.

The above example does not show at all how the Grothendieck construction transforms a database instance into an RDF triple store. The reason is that our database schema was A , a discrete category that specifies no connections between data (it simply collects the data into bins). So let's examine a more interesting database schema and instance. This is taken from [Sp2].

Application 4.6.2.3. Consider the schema below, which we first encountered in Example

3.5.2.1:



And consider the instance $I: \mathcal{C} \rightarrow \mathbf{Set}$, which we first encountered in (3.11) and (3.13)

Employee				
ID	first	last	manager	worksIn
101	David	Hilbert	103	q10
102	Bertrand	Russell	102	x02
103	Emmy	Noether	103	q10

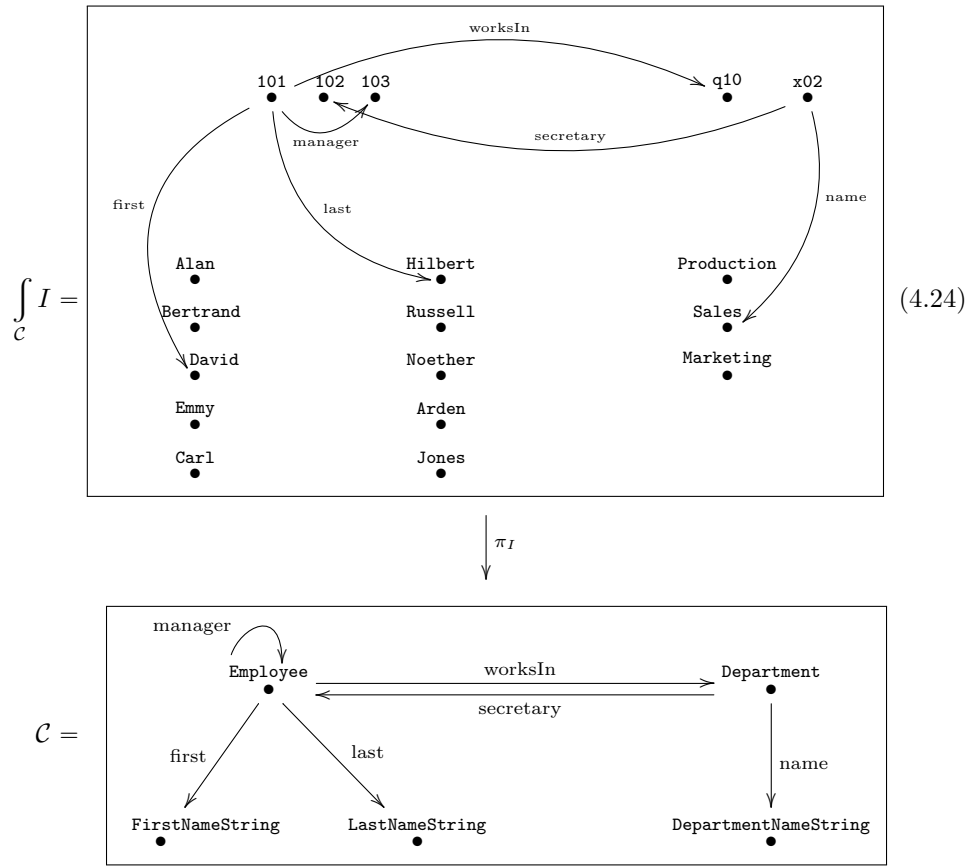
Department		
ID	name	secretary
q10	Sales	101
x02	Production	102

FirstNameString	
ID	
Alan	
Bertrand	
Carl	
David	
Emmy	

LastNameString	
ID	
Arden	
Hilbert	
Jones	
Noether	
Russell	

DepartmentNameString	
ID	
Marketing	
Production	
Sales	

The category of elements of $I: \mathcal{C} \rightarrow \mathbf{Set}$ looks like this:



In the above drawing (4.24) of $\int_C I$, we left out 10 arrows for ease of readability, for example, we left out an arrow $102 \xrightarrow{\text{first}} \text{Bertrand}$.

For the punchline, how do we see the category of elements $\int_C I$ as an RDF triple store? For each arrow in $\int_C I$, we take the triple consisting of the source vertex, the arrow name, and the target vertex. So our triple store would include triples such as $\langle 102 \text{ first } \text{Bertrand} \rangle$ and $\langle 101 \text{ manager } 103 \rangle$.

◇◇

Exercise 4.6.2.4. Come up with a schema and instance whose category of elements contains (at least) the data from (4.21). ◇

Exercise 4.6.2.5. Recall from Section 3.1.2.10 that a finite state machine is a free monoid $(\text{List}(\Sigma), [], ++)$ acting on a set X . Recall also that we can consider a monoid as a category $\mathcal{M}$ with one object and a monoid action as a set-valued functor $F: \mathcal{M} \rightarrow \mathbf{Set}$, (see Section 4.2.1.1). In the case of Figure 3.1 the monoid in question is $\text{List}(a, b)$, which can be drawn as the schema



and the functor $F: \mathcal{M} \rightarrow \mathbf{Set}$ is recorded in an action table in Example 3.1.3.1. What is $\int_{\mathcal{M}} F$? How does it relate to the picture in Figure 3.1? ◇

4.6.3 Full subcategory

Definition 4.6.3.1. Let $\mathcal{C}$ be a category and let $X \subseteq \text{Ob}(\mathcal{C})$ be a set of objects in $\mathcal{C}$. The *full subcategory of $\mathcal{C}$ spanned by X* is the category, which we denote by $\mathcal{C}_{\text{Ob}=X}$, with objects $\text{Ob}(\mathcal{C}_{\text{Ob}=X}) := X$ and with morphisms $\text{Hom}_{\mathcal{C}_{\text{Ob}=X}}(x, x') := \text{Hom}_{\mathcal{C}}(x, x')$.

Example 4.6.3.2. The following are examples of full subcategories. We will name them in the form “ X inside of Y ”, and each time we mean that X and Y are names of categories, the category X can be considered as a subcategory of the category Y in some sense, and it is full. In other words, all morphisms in Y “count” as morphisms in X .

- Finite sets inside of sets, **Fin** $\subseteq$ **Set**;
- Finite sets of the form $\underline{n}$ inside of **Fin**;
- Linear orders of the form $[n]$ inside of all finite linear orders, **Δ** $\subseteq$ **FLin**;
- Groups inside of monoids, **Grp** $\subseteq$ **Mon**;
- Monoids inside of categories, **Mon** $\subseteq$ **Cat**;
- Sets inside of graphs, **Set** $\subseteq$ **Grph**;
- Partial orders (resp. linear orders) inside of **Pro**;
- Discrete categories (resp. indiscrete categories) inside of **Cat**;

Remark 4.6.3.3. A subcategory $\mathcal{C} \subseteq \mathcal{D}$ is just a functor $i: \mathcal{C} \rightarrow \mathcal{D}$ that happens to be injective on objects and arrows. The subcategory is full if and only if i is a full functor in the sense of Definition 4.3.4.12.

Example 4.6.3.4. Let $\mathcal{C}$ be a category, let $X \subseteq \text{Ob}(\mathcal{C})$ be a set of objects, and let $\mathcal{C}_{\text{Ob}=X}$ denote the full subcategory of $\mathcal{C}$ spanned by X . We can realize this as a fiber product of categories. Indeed, recall that for any set, we can form the indiscrete category on that set; see Example 4.3.4.3. In fact, we have a functor $\text{Ind}: \mathbf{Set} \rightarrow \mathbf{Cat}$. Thus our function $X \rightarrow \text{Ob}(\mathcal{C})$ can be converted into a functor between indiscrete categories $\text{Ind}(X) \rightarrow \text{Ind}(\text{Ob}(\mathcal{C}))$. There is also a functor $\mathcal{C} \rightarrow \text{Ind}(\text{Ob}(\mathcal{C}))$ sending each object to itself. Then the full subcategory of $\mathcal{C}$ spanned by X is the fiber product of categories,

$$\begin{array}{ccc} \mathcal{C}_{\text{Ob}=X} & \longrightarrow & \mathcal{C} \\ \downarrow & & \downarrow \\ \text{Ind}(X) & \longrightarrow & \text{Ind}(\text{Ob}(\mathcal{C})) \end{array}$$

4.6.4 Comma categories

Category theory includes a highly developed and interoperable catalogue of materials and production techniques. One such is the comma category.

Definition 4.6.4.1. Let $\mathcal{A}, \mathcal{B}$, and $\mathcal{C}$ be categories and let $F: \mathcal{A} \rightarrow \mathcal{C}$ and $G: \mathcal{B} \rightarrow \mathcal{C}$ be functors. The *comma category of $\mathcal{C}$ morphisms from F to G* , denoted $(F \downarrow_{\mathcal{C}} G)$ or simply $(F \downarrow G)$, is the category with objects

$$\text{Ob}(F \downarrow G) = \{(a, b, f) \mid a \in \text{Ob}(\mathcal{A}), b \in \text{Ob}(\mathcal{B}), f: F(a) \rightarrow G(b) \text{ in } \mathcal{C}\}$$

$$\begin{array}{ccccc}
a & & F(a) & \xrightarrow{f} & G(b) & & b \\
\downarrow q & & \downarrow F(q) & & \downarrow G(r) & & \downarrow r \\
a' & & F(a') & \xrightarrow{f'} & G(b') & & b'
\end{array}$$

Figure 4.25: A morphism $(q, r): (a, b, f) \longrightarrow (a', b', f')$ in $(F \downarrow G)$

and for any two objects (a, b, f) and (a', b', f') the set $\text{Hom}_{(F \downarrow G)}((a, b, f), (a', b', f'))$ of morphisms $(a, b, f) \longrightarrow (a', b', f')$ is

$$\{(q, r) \mid q: a \rightarrow a' \text{ in } \mathcal{A}, \ r: b \rightarrow b' \text{ in } \mathcal{B}, \text{ such that } f' \circ F(q) = G(r) \circ f\}.$$

Example 4.6.4.2. Let $\mathcal{C}$ be a category and $I: \mathcal{C} \rightarrow \mathbf{Set}$ a functor. In this example we show that the comma category construction captures the notion of taking the category of elements $\int_{\mathcal{C}} I$; see Definition 4.6.2.1.

Consider the set $\underline{1}$, the category $\text{Disc}(\underline{1})$, and the functor $F: \text{Disc}(\underline{1}) \rightarrow \mathbf{Set}$ sending the unique object to the set $\underline{1}$. We use the comma category setup $\underline{1} \xrightarrow{F} \mathbf{Set} \xleftarrow{I} \mathcal{C}$. There is an isomorphism of categories

$$\int_{\mathcal{C}} I \cong (F \downarrow I).$$

Indeed, an object in $(F \downarrow I)$ is a triple (a, b, f) where $a \in \text{Ob}(\underline{1})$, $b \in \text{Ob}(\mathcal{C})$, and $f: F(a) \rightarrow I(b)$ is a morphism in $\mathbf{Set}$. There is only one object in $\underline{1}$, so this reduces to a pair (b, f) where $b \in \text{Ob}(\mathcal{C})$ and $f: \{\odot\} \rightarrow I(b)$. The set of functions $\{\odot\} \rightarrow I(b)$ is isomorphic to $I(b)$, as we saw in Exercise 2.1.2.13. So we have reduced $\text{Ob}(F \downarrow I)$ to the set of pairs (b, x) where $b \in \text{Ob}(\mathcal{C})$ and $x \in I(b)$; this is $\text{Ob}(\int_{\mathcal{C}} I)$. Because there is only one function $\underline{1} \rightarrow \underline{1}$, a morphism $(b, x) \rightarrow (b', x')$ in $(F \downarrow I)$ boils down to a morphism $r: b \rightarrow b'$ such that the diagram

$$\begin{array}{ccc}
\underline{1} & \xrightarrow{x} & I(b) \\
\parallel & & \downarrow r \\
\underline{1} & \xrightarrow{x'} & I(b')
\end{array}$$

But such diagrams are in one-to-one correspondence with the diagrams needed for morphisms in $\int_{\mathcal{C}} I$.

Exercise 4.6.4.3. Let $\mathcal{C}$ be a category and let $c, c' \in \text{Ob}(\mathcal{C})$ be objects. Consider them as functors $c, c': \underline{1} \rightarrow \mathcal{C}$, and consider the setup $\underline{1} \xrightarrow{c} \mathcal{C} \xleftarrow{c'} \underline{1}$. What is the comma category $(c \downarrow c')$? $\diamond$

4.6.5 Arithmetic of categories

In Section 2.7.3, we summarized some of the properties of products, coproducts, and exponentials for sets, attempting to show that they lined up precisely with familiar arithmetic properties of natural numbers. Astoundingly, we can do the same for categories.

In the following proposition, we denote the coproduct of two categories $\mathcal{A}$ and $\mathcal{B}$ by the notation $\mathcal{A} + \mathcal{B}$ rather than $\mathcal{A} \sqcup \mathcal{B}$. We also denote the functor category $\text{Fun}(\mathcal{A}, \mathcal{B})$ by $\mathcal{B}^{\mathcal{A}}$. Finally, we use $\underline{0}$ and $\underline{1}$ to refer to the discrete category on 0 and on 1 object, respectively.

Proposition 4.6.5.1. *The following isomorphisms exist for any small categories $\mathcal{A}, \mathcal{B}$, and $\mathcal{C}$.*

- $\mathcal{A} + \underline{0} \cong \mathcal{A}$
- $\mathcal{A} + \mathcal{B} \cong \mathcal{B} + \mathcal{A}$
- $(\mathcal{A} + \mathcal{B}) + \mathcal{C} \cong \mathcal{A} + (\mathcal{B} + \mathcal{C})$
- $\mathcal{A} \times \underline{0} \cong \underline{0}$
- $\mathcal{A} \times \underline{1} \cong \mathcal{A}$
- $\mathcal{A} \times \mathcal{B} \cong \mathcal{B} \times \mathcal{A}$
- $(\mathcal{A} \times \mathcal{B}) \times \mathcal{C} \cong \mathcal{A} \times (\mathcal{B} \times \mathcal{C})$
- $\mathcal{A} \times (\mathcal{B} + \mathcal{C}) \cong (\mathcal{A} \times \mathcal{B}) + (\mathcal{A} \times \mathcal{C})$
- $\mathcal{A}^{\underline{0}} \cong \underline{1}$
- $\mathcal{A}^{\underline{1}} \cong \mathcal{A}$
- $\underline{0}^{\mathcal{A}} \cong \underline{0}$, if $\mathcal{A} \neq \underline{0}$
- $\underline{1}^{\mathcal{A}} \cong \underline{1}$
- $\mathcal{A}^{\mathcal{B}+\mathcal{C}} \cong \mathcal{A}^{\mathcal{B}} \times \mathcal{A}^{\mathcal{C}}$
- $(\mathcal{A}^{\mathcal{B}})^{\mathcal{C}} \cong \mathcal{A}^{\mathcal{B} \times \mathcal{C}}$

Proof. These are standard results; see [Mac].

□

Chapter 5

Categories at work

We have now set up an understanding of the basic notions of category theory: categories, functors, natural transformations, and universal properties. We have discussed many sources of examples: orders, graphs, monoids, and databases. We begin this chapter with the notion of adjoint functors, which are like dictionaries that translate back and forth between different categories.

5.1 Adjunctions

In the introduction to this chapter I said that adjoint functors are like dictionaries that translate back and forth between different categories. In the common understanding of dictionaries, we assume that the two languages (say French and English) are equally expressive, and that a good dictionary will be an even exchange of ideas. But in category theory we often have two categories that are not on the same conceptual level. This is most clear in the case of so-called *free-forgetful adjunctions*.

5.1.1 Discussion and definition

Consider the category of monoids and the category of sets. A monoid $(M, \text{id}, \star)$ is a set with an identity element and a multiplication formula that is associative. A set is just a set. A dictionary between **Mon** and **Set** should not have to set up an even exchange, but instead an exchange that is appropriate to the structures at hand. It will be in the form of two functors, one we'll denote by $L: \mathbf{Set} \rightarrow \mathbf{Mon}$, and one we'll denote by $R: \mathbf{Mon} \rightarrow \mathbf{Set}$. But to say what "appropriate" means requires more work.

Let's bring it down to earth. A one-year-old can make repeatable noises and an adult can make repeatable noises. One might say "after all, talking is nothing but making repeatable noises." But the adult's repeatable noises are called words, they form sentences, and these sentences can cause nuclear wars. There is a difference. A tennis match can be viewed as physics, but you won't see the match. So we have two categories here: ((repeated noises)) and ((meaningful words)). We need adjoint functors going back and forth.

To translate baby talk into adult language we would make every repeated noise a kind of word, thereby granting it meaning. We don't know what it means, but we give it a slot in our lexicon, always pondering "I wonder what she means by Ronnon.." To

translate from meaningful words to repeatable noises is easy. We just hear the word as a repeated noise, which is how the baby hears it.

Adjoint functors often come in the form of “free” and “forgetful”. Here we freely add *Ronnon* to our lexicon without having any idea how it adheres to the rest of the child’s noises or feelings. But it doesn’t act like a sound to us, it acts like a word; we don’t know what it means but we figure it means something. Conversely, the translation going the other way is “forgetful”, forgetting the meaning of our words and just hearing them as sounds. The baby hears our words and accepts them as mere sounds, not knowing that there is anything extra to get.

Back to sets and monoids, the sets are like the babies from our story: they are simple objects full of unconnected dots. The monoids are like adults, forming words and performing actions. Each element means something and combines with other elements in some way. We have lots of different sets and lots of different monoids, just as for babies and adults, but there are patterns to the behavior of each kind and we put them in different categories.

Applying free functor $L: \mathbf{Set} \rightarrow \mathbf{Mon}$ to a set X makes every element $x \in X$ a word, but they combine freely into lists. We discussed the free functor in Section 3.1.1.9; there are no relations between the “words” in these lists. Applying the forgetful functor $R: \mathbf{Mon} \rightarrow \mathbf{Set}$ is merely forgetting that the elements of the monoid are anything but mere elements of a set. It sends a monoid $(M, 1, \star)$ to the set M .

But this is all just ideas. Let’s give a definition, then return to our sets, monoids, sounds, and words.

Definition 5.1.1.1. Let $\mathcal{A}$ and $\mathcal{B}$ be categories. An *adjunction between $\mathcal{A}$ and $\mathcal{B}$* is a pair of functors

$$L: \mathcal{B} \rightarrow \mathcal{A} \quad \text{and} \quad R: \mathcal{A} \rightarrow \mathcal{B}$$

together with a natural isomorphism for any objects $A \in \text{Ob}(\mathcal{A})$ and $B \in \text{Ob}(\mathcal{B})$:

$$\alpha_{B,A} \text{Hom}_{\mathcal{A}}(L(B), A) \xrightarrow{\cong} \text{Hom}_{\mathcal{B}}(B, R(A)).$$

This isomorphism is called the *mate isomorphism* for the (L, R) adjunction, and for any morphism $f: L(B) \rightarrow A$ in $\mathcal{A}$, we refer to $\alpha_{B,A}(f): B \rightarrow R(A)$ as *the mate of f* .¹

The functor L is called the *left adjoint* and the functor R is called the *right adjoint*. We may say that *L is the left adjoint of R* or that *R is the right adjoint of L* .² We often denote this setup by

$$L: \mathcal{B} \rightleftarrows \mathcal{A} : R$$

Proposition 5.1.1.2. Let $L: \mathbf{Set} \rightarrow \mathbf{Mon}$ be the functor sending $X \in \text{Ob}(\mathbf{Set})$ to the free monoid $L(X) := (\text{List}(X), [], ++)$, as in Definition 3.1.1.12. Let $R: \mathbf{Mon} \rightarrow \mathbf{Set}$ be the functor sending each monoid $\mathcal{M} := (M, 1, \star)$ to its underlying set $R(\mathcal{M}) := M$. Then L is left adjoint to R .

Proof. If we can find a natural isomorphism of sets

$$\alpha_{X,\mathcal{M}}: \text{Hom}_{\mathbf{Mon}}(L(X), \mathcal{M}) \rightarrow \text{Hom}_{\mathbf{Set}}(X, R(\mathcal{M}))$$

we will have succeeded in showing that these functors are adjoint.

¹Conversely, for any $g: B \rightarrow R(A)$ in $\mathcal{B}$ we refer to $\alpha_{B,A}^{-1}(g): L(B) \rightarrow A$ as *the mate of g* .

²The left adjoint does not have to be called L , nor does the right adjoint have to be called R , of course. This is suggestive.

Suppose given an element $f \in \text{Hom}_{\mathbf{Mon}}(L(X), \mathcal{M})$, i.e. a monoid homomorphism $f: \text{List}(X) \rightarrow M$ (sending $[]$ to 1 and list concatenation to $\star$). Then in particular we can apply f to the singleton list $[x]$ for any $x \in X$. This gives a function $X \rightarrow M$ by $x \mapsto f([x])$, and this is $\alpha_{X, \mathcal{M}}(f): X \rightarrow M = R(\mathcal{M})$. We need only to supply an inverse $\beta_{X, \mathcal{M}}: \text{Hom}_{\mathbf{Set}}(X, R(\mathcal{M})) \rightarrow \text{Hom}_{\mathbf{Mon}}(L(X), \mathcal{M})$.

Suppose given an element $g \in \text{Hom}_{\mathbf{Set}}(X, R(\mathcal{M}))$, i.e. a function $g: X \rightarrow M$. Then to any list $\ell = [x_1, x_2, \dots, x_n] \in \text{List}(X)$ we can assign $\beta_{X, \mathcal{M}}(\ell) := g(x_1) \star g(x_2) \star \dots \star g(x_n)$ (if $\ell = []$ is the empty list, assign $\beta_{X, \mathcal{M}}([]) := 1$). We now have a function $\text{List}(X) \rightarrow M$. It is a monoid homomorphism because it respects identity and composition. It is easy to check that β and α are mutually inverse, completing the proof. $\square$

Exercise 5.1.1.3. Let $X = \{a, b, c\}$ and let $\mathcal{M} = (\mathbb{N}, 1, *)$ be the multiplicative monoid of natural numbers (see Example 3.1.3.2). Let $f: X \rightarrow \mathbb{N}$ be the function given by $f(a) = 7, f(b) = 2, f(c) = 2$, and let $\beta_{X, \mathcal{M}}: \text{Hom}_{\mathbf{Set}}(X, R(\mathcal{M})) \rightarrow \text{Hom}_{\mathbf{Mon}}(L(X), \mathcal{M})$ be as in the proof of Proposition 5.1.1.2. What is $\beta_{X, \mathcal{M}}(f)([b, b, a, c])$? $\diamond$

Let us look once more at the adjunction between adults and babies. Using the notation of Definition 5.1.1.1 $\mathcal{A}$ is the “category” of meaningful words and $\mathcal{B}$ is the “category” of repeated noises. The left adjoint turns every repeated sound into a meaningful word and the right adjoint forgets the meaning of any word and considers it merely as a sound.

At the risk of taking this simple analogy too far, let’s have a go at the heart of the issue: how to conceive of the isomorphism of Hom’s. Once we have freely given a slot to each of baby’s repeated sounds, we try to find a mapping from the lexicon $L(B)$ of these new words to our own lexicon A of meaningful words; these are mappings in the adult category $\mathcal{A}$ of the form $L(B) \rightarrow A$. And the baby tries to find a mapping (which we might see as emulation) from the set B of repeatable sounds to the set $R(A)$ of the sounds the adult seem to repeat. If there was a global system for making these transformations that would be an adjunction.

Note that it makes little sense in this example to look for a mapping in the category of meaningful words from our lexicon to the wordifications of baby-sounds; there is unlikely to be a good candidate for most of our words. The next example shows that the “leftness” and “rightness” are not arbitrary.

Example 5.1.1.4. Let $L: \mathbf{Set} \rightarrow \mathbf{Mon}$ and $R: \mathbf{Mon} \rightarrow \mathbf{Set}$ be the free and forgetful functors from Proposition 5.1.1.2. We know that L is left adjoint to R ; however L is *not* right adjoint to R . In other words, we can show that the necessary natural isomorphism cannot exist.

Let $X = \{a, b\}$ and let $\mathcal{M} = (\{1\}, 1, !)$ be the trivial monoid. Then the necessary natural isomorphism would need to give us a bijection $\text{Hom}_{\mathbf{Mon}}(\mathcal{M}, L(X)) \cong \text{Hom}_{\mathbf{Set}}(\{1\}, X)$. But the left-hand side has one element, because $\mathcal{M}$ is the initial object in $\mathbf{Mon}$ (see Example 4.5.3.7), whereas the right-hand side has two elements. Therefore no isomorphism can exist.

Example 5.1.1.5. Preorders have underlying sets, giving rise to a functor $U: \mathbf{PrO} \rightarrow \mathbf{Set}$. The functor U has both a left adjoint and a right adjoint. The left adjoint of U is $D: \mathbf{Set} \rightarrow \mathbf{PrO}$, sending a set X to the discrete preorder on X (the preorder with underlying set X , having the fewest possible $\leq$ ’s). The right adjoint of U is $I: \mathbf{Set} \rightarrow \mathbf{PrO}$, sending a set X to the indiscrete preorder on X (the preorder with underlying set X , having the most possible $\leq$ ’s). See Example 3.4.3.5.

Exercise 5.1.1.6. Let $U: \mathbf{Grph} \rightarrow \mathbf{Set}$ denote the functor sending a graph to its underlying set of vertices. This functor has both a left and a right adjoint. What are they? $\diamond$

Example 5.1.1.7. Here are some other adjunctions:

- $\text{Ob}: \mathbf{Cat} \rightarrow \mathbf{Set}$ has a left adjoint $\mathbf{Set} \rightarrow \mathbf{Cat}$ given by the discrete category.
- $\text{Ob}: \mathbf{Cat} \rightarrow \mathbf{Set}$ has a right adjoint $\mathbf{Set} \rightarrow \mathbf{Cat}$ given by the indiscrete category.
- The underlying graph functor $\mathbf{Cat} \rightarrow \mathbf{Grph}$ has a left adjoint $\mathbf{Grph} \rightarrow \mathbf{Cat}$ given by the free category.
- The functor $\mathbf{Pro} \rightarrow \mathbf{Grph}$ given by drawing edges for $\leq$'s has a left adjoint given by existence of paths.
- The forgetful functor from posets to preorders has a left adjoint given by quotient by isomorphism relation.
- Given a set A , the functor $- \times A: \mathbf{Set} \rightarrow \mathbf{Set}$ has a right adjoint $\text{Hom}(A, -)$ (this is currying).

Exercise 5.1.1.8. Let $F: \mathcal{C} \rightarrow \mathcal{D}$ and $G: \mathcal{D} \rightarrow \mathcal{C}$ be mutually inverse equivalences of categories (see Definition 4.3.4.1). Are they adjoint in one direction or the other? $\diamond$

Exercise 5.1.1.9. The discrete category functor $\text{Disc}: \mathbf{Set} \rightarrow \mathbf{Cat}$ has a left adjoint $p: \mathbf{Cat} \rightarrow \mathbf{Set}$. Describe it. Hint: look at the mate isomorphism at the component corresponding to $[1] \in \text{Ob}(\mathbf{Cat})$ and $\underline{2} \in \text{Ob}(\mathbf{Set})$ $\diamond$

5.1.1.10 Quantifiers as adjoints

One of the simplest but neatest places that adjoints show up is between preimages and the logical quantifiers $\exists$ and $\forall$. The setting in which to discuss this is that of sets and their power preorders. That is, if X is a set then recall from Section 3.4.2 that the power set $\mathbb{P}(X)$ has a natural ordering by inclusion of subsets.

Given a function $f: X \rightarrow Y$ and a subset $V \subseteq Y$ the preimage is $f^{-1}(V) := \{x \in X \mid f(x) \in V\}$. If $V' \subseteq V$ then $f^{-1}(V') \subseteq f^{-1}(V)$, so in fact $f^{-1}: \mathbb{P}(Y) \rightarrow \mathbb{P}(X)$ can be considered a functor (where of course we are thinking of preorders as categories). The quantifiers appear as adjoints of f^{-1} .

Let's begin with the left adjoint of f^{-1} . It is a functor $L: \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$. Choose an object $U \subseteq X$ in $\mathbb{P}(X)$. It turns out that

$$L(U) = \{y \in Y \mid \exists x \in f^{-1}(y) \text{ such that } x \in U\}.$$

And the right adjoint $R: \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$, when applied to U is

$$R(U) = \{y \in Y \mid \forall x \in f^{-1}(y), x \in U\}.$$

5.1.2 Universal concepts in terms of adjoints

In this section we discuss how universal concepts, i.e. initial objects and terminal objects, colimits and limits, are easily phrased in the language of adjoint functors.

Example 5.1.2.1. Let $\mathcal{C}$ be a category and $t: \mathcal{C} \rightarrow \underline{1}$ the unique functor to the terminal category. Then t is a left adjoint if and only if $\mathcal{C}$ has a terminal object, and t is a right adjoint if and only if $\mathcal{C}$ has an initial object. The proofs are dual, so let's focus on the first.

The functor t has a right adjoint $R: \underline{1} \rightarrow \mathcal{C}$ if and only if there is an isomorphism

$$\mathrm{Hom}_{\mathcal{C}}(c, r) \cong \mathrm{Hom}_{\underline{1}}(t(c), 1),$$

where $r = R(1)$. But $\mathrm{Hom}_{\underline{1}}(t(c), 1)$ has one element. Thus t has a right adjoint iff there is a unique morphism $c \rightarrow r$ in $\mathcal{C}$. This is the definition of r being a terminal object.

When we defined colimits and limits in Definitions 4.5.3.25 and 4.5.3.18 we did so for individual I -shaped diagrams $X: I \rightarrow \mathcal{C}$. Using adjoints we can define the limit of every I -shaped diagram in $\mathcal{C}$ at once.

Let $t: \mathcal{C} \rightarrow \underline{1}$ denote the unique functor to the terminal category. Given an object $c \in \mathrm{Ob}(\mathcal{C})$, consider it as a functor $c: \underline{1} \rightarrow \mathcal{C}$. Then $c \circ t: I \rightarrow \mathcal{C}$ is the *constant functor at c* , sending each object in I to the same $\mathcal{C}$ -object c , and every morphism in I to id_c . This induces a functor that we denote by $\Delta_t: \mathcal{C} \rightarrow \mathrm{Fun}(I, \mathcal{C})$.

Suppose we want to take the colimit or limit of X . We are given an object X of $\mathrm{Fun}(I, \mathcal{C})$ and we want back an object of $\mathcal{C}$. We could hope, and it turns out to be true, that the adjoints of Δ_t are the limit and colimit. Indeed let $\Sigma_t: \mathrm{Fun}(I, \mathcal{C}) \rightarrow \mathcal{C}$ be the left adjoint of Δ_t , and let $\Pi_t: \mathrm{Fun}(I, \mathcal{C}) \rightarrow \mathcal{C}$ be the right adjoint of Δ_t . Then Σ_t is the functor that takes colimits, and Π_t is the functor that takes limits.

We will work with a generalization of colimits and limits in Section 5.1.4. But for now, let's bring this down to earth with a concrete example.

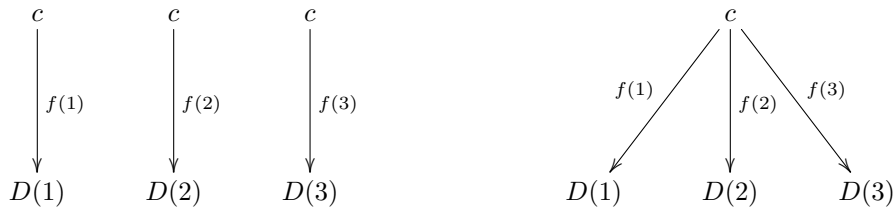
Example 5.1.2.2. Let $\mathcal{C} = \mathbf{Set}$, and let $I = \underline{3}$. The category $\mathrm{Fun}(I, \mathbf{Set})$ is the category of $\{1, 2, 3\}$ -indexed sets, e.g. $(\mathbb{Z}, \mathbb{N}, \mathbb{Z}) \in \mathrm{Ob}(\mathrm{Fun}(I, \mathbf{Set}))$ is an object of it. The functor $\Delta_t: \mathbf{Set} \rightarrow \mathrm{Fun}(I, \mathbf{Set})$ acts as follows. Given a set $c \in \mathrm{Ob}(\mathbf{Set})$, consider it as a functor $c: \underline{1} \rightarrow \mathbf{Set}$, and the composite $c \circ t: I \rightarrow \mathbf{Set}$ is the constant functor. That is, $\Delta_t(c): I \rightarrow \mathbf{Set}$ is the functor sending $1 \mapsto c, 2 \mapsto c$, and $3 \mapsto c$.

To say that Δ_t has a right adjoint called $\Pi_t: \mathrm{Fun}(I, \mathbf{Set}) \rightarrow \mathbf{Set}$ and that it “takes limits” should mean that if we look through the definition of right adjoint, we will see that the formula will somehow yield the appropriate limit. Fix a functor $D: I \rightarrow \mathbf{Set}$, so $D(1), D(2)$, and $D(3)$ are sets. The limit $\lim D$ of D is the product $D(1) \times D(2) \times D(3)$. How does this fact arise in the definition of adjoint?

The definition of Π_t being the right adjoint to Δ_t says that there is a natural isomorphism of sets,

$$\mathrm{Hom}_{\mathrm{Fun}(I, \mathbf{Set})}(\Delta(c), D) \cong \mathrm{Hom}_{\mathbf{Set}}(c, \Pi_t(D)). \quad (5.1)$$

The left-hand side has elements $f \in \mathrm{Hom}_{\mathrm{Fun}(I, \mathbf{Set})}(\Delta(c), D)$ that look like the left below, but having these three maps is equivalent to having the diagram to the right below:



The isomorphism in (5.1) says that choosing the three maps $f(1), f(2), f(3)$ is the same thing as choosing a function $c \rightarrow \Pi_t(D)$. But this is very close to the universal property of limits: there is a unique map $\ell: c \rightarrow D(1) \times D(2) \times D(3)$, so this product serves well as Π_t as we have said. We're not giving a formal proof here, but what is missing at this point is the fact that certain diagrams have to commute. This comes down to the naturality of the isomorphism (5.1). The map ℓ induces a naturality square

$$\begin{array}{ccc} \Delta_t(c) & \xrightarrow{\Delta_t(\ell)} & \Delta_t \Pi_t D \\ f \downarrow & & \downarrow \pi \\ D & \xlongequal{\quad} & D \end{array}$$

which says that the following diagram commutes:

$$\begin{array}{ccccc} & & c & & \\ & \swarrow f(1) & \downarrow f(2) & \searrow f(3) & \\ D(1) & & D(2) & & D(3) \\ & \nwarrow \pi_1 & \uparrow \pi_2 & \swarrow \pi_3 & \\ & & D(1) \times D(2) \times D(3) & & \end{array}$$

ℓ (curved arrow from c to the product)

5.1.3 Preservation of colimits or limits

One useful fact about adjunctions is that left adjoints preserve all colimits and right adjoints preserve all limits.

Proposition 5.1.3.1. *Let $L: \mathcal{B} \rightleftarrows \mathcal{A} : R$ be an adjunction. For any indexing category I and functor $D: I \rightarrow \mathcal{B}$, if D has a colimit in $\mathcal{B}$ then there is a unique isomorphism*

$$L(\operatorname{colim} D) \cong \operatorname{colim}(L \circ D).$$

Similarly, for any $I \in \operatorname{Ob}(\mathbf{Cat})$ and functor $D: I \rightarrow \mathcal{A}$, if D has a limit in $\mathcal{A}$ then there is a unique isomorphism

$$R(\operatorname{lim} D) \cong \operatorname{lim}(R \circ D).$$

Proof. The proof is simple if one knows the Yoneda lemma. I have decided to skip that to keep the book shorter. See $\square$.

$\square$

Example 5.1.3.2. Since $\operatorname{Ob}: \mathbf{Cat} \rightarrow \mathbf{Set}$ is both a left adjoint and a right adjoint, it must preserve both limits and colimits. This means that if you want to know the set of objects in the fiber product of some categories, you can simply take the fiber product of the set of objects in those categories,

$$\operatorname{Ob}(\mathcal{A} \times_{\mathcal{C}} \mathcal{B}) \cong \operatorname{Ob}(\mathcal{A}) \times_{\operatorname{Ob}(\mathcal{C})} \operatorname{Ob}(\mathcal{B}).$$

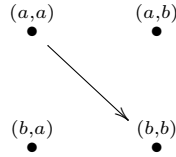
While the right-hand side might look daunting, it is just a fiber product in **Set** which is quite understandable.

This is greatly simplifying. If one thinks through what defines a limit in **Cat**, one is dragged through notions of slice categories and terminal objects in them. These slice categories are in **Cat** so they involve several categories and functors, and it gets hairy or even hopeless to a beginner. Knowing that the objects are given by a simple fiber product makes the search for limits in **Cat** much simpler.

For example, if $[n]$ is the linear order category of length n then $[n] \times [m]$ has $nm + n + m + 1$ objects because $[n]$ has $n + 1$ objects and $[m]$ has $m + 1$ objects.

Example 5.1.3.3. The “path poset” functor $L: \mathbf{Grph} \rightarrow \mathbf{PrO}$ given by existence of paths (see Exercise 4.1.2.9) is left adjoint to the functor $R: \mathbf{PrO} \rightarrow \mathbf{Grph}$ given by replacing $\leq$ ’s by arrows. This means that L preserves colimits. So taking the union of graphs G and H results in a graph whose path poset $L(G \sqcup H)$ is the union of the path posets of G and H . But this is not so for products.

Let $G = H = \begin{array}{ccc} & a & \xrightarrow{f} & b \\ & \bullet & & \bullet \end{array}$. Then $L(G) = L(H) = [1]$, the linear order of length 1. But the product $G \times H$ in **Grph** looks like the graph



Its preorder $L(G \times H)$ does not have $(a, a) \leq (a, b)$, whereas this is the case in $L(G) \times L(H)$.

Exercise 5.1.3.4. The distributive law for addition of natural numbers says $(A + B) \times C = A \times C + B \times C$. Show that this follows from the currying adjunction on **Set**. $\diamond$

5.1.4 Data migration

As we saw in Sections 4.2.2 and 4.2.2.5, a database schema is a category $\mathcal{C}$ and an instance is a functor $I: \mathcal{C} \rightarrow \mathbf{Set}$.

Notation 5.1.4.1. Let $\mathcal{C}$ be a category. Throughout this section we denote by $\mathcal{C}\text{-Set}$ the category $\mathbf{Fun}(\mathcal{C}, \mathbf{Set})$ of functors from $\mathcal{C}$ to **Set**, i.e. the category of instances on $\mathcal{C}$.

In this section we discuss what happens to the resulting instances when different schemas are connected by a functor, say $F: \mathcal{C} \rightarrow \mathcal{D}$. It turns out that three adjoint functors emerge: $\Delta_F: \mathcal{D}\text{-Set} \rightarrow \mathcal{C}\text{-Set}$, $\Sigma_F: \mathcal{C}\text{-Set} \rightarrow \mathcal{D}\text{-Set}$, and $\Pi_F: \mathcal{C}\text{-Set} \rightarrow \mathcal{D}\text{-Set}$, where Δ_F is adjoint to both,

$$\Sigma_F: \mathcal{C}\text{-Set} \rightleftarrows \mathcal{D}\text{-Set} : \Delta_F \qquad \Delta_F: \mathcal{D}\text{-Set} \rightleftarrows \mathcal{C}\text{-Set} : \Pi_F.$$

It turns out that almost all the basic database operations are captured by these three functors. For example, Δ_F handles the job of duplicating or deleting tables, as well as duplicating or deleting columns in a single table. The functor Σ_F handles taking unions, and the functor Π_F handles joining tables together, matching columns, or selecting the rows with certain properties (e.g. everyone whose first name is Mary).

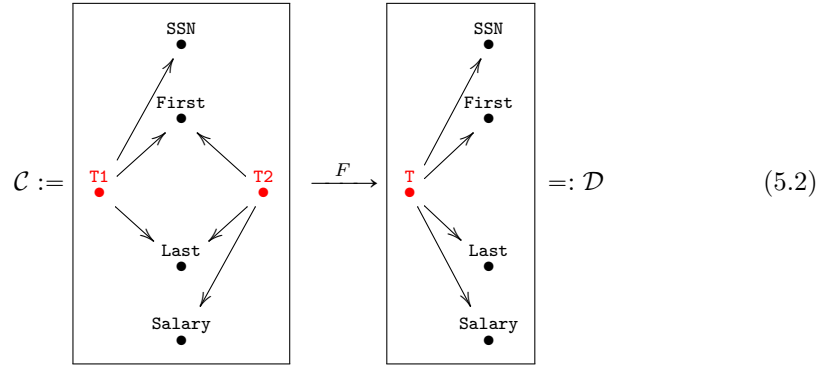
5.1.4.2 Pullback: Δ

Given a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ and a functor $I: \mathcal{D} \rightarrow \mathbf{Set}$, we can compose them to get a functor $I \circ F: \mathcal{C} \rightarrow \mathbf{Set}$. In other words, the presence of F provides a way to convert $\mathcal{D}$ -instances into $\mathcal{C}$ -instances. In fact this conversion is functorial, meaning that morphisms of $\mathcal{D}$ -instances are sent to morphisms of $\mathcal{C}$ -instances. We denote the resulting functor by $\Delta_F: \mathcal{D}\text{-Set} \rightarrow \mathcal{C}\text{-Set}$ and call it *pullback along F* .

We have seen an example of this before in Example 4.3.2.15, where we showed how a monoid homomorphism $F: \mathcal{M}' \rightarrow \mathcal{M}$ could add functionality to a finite state machine. More generally, we can use pullbacks to reorganize data, copying and deleting tables and columns.

Remark 5.1.4.3. Given a functor $F: \mathcal{C} \rightarrow \mathcal{D}$, which we think of as a schema translation, the functor $\Delta_F: \mathcal{D}\text{-Set} \rightarrow \mathcal{C}\text{-Set}$ “goes the opposite way”, which is often very strange to people at first. The rough idea is that if I get my information from you, then your information becomes my information.

Consider the following functor $F: \mathcal{C} \rightarrow \mathcal{D}$:³



Let’s spend a moment recalling how to “read” schemas. In schema $\mathcal{C}$ there are leaf tables **SSN**, **First**, **Last**, **Salary**, which represent different kinds of basic data. More interestingly, there are two *fact tables*. The first is called **T1** and it relates **SSN**, **First**, and **Last**. The second is called **T2** and it relates **First**, **Last**, and **Salary**.

The functor $F: \mathcal{C} \rightarrow \mathcal{D}$ relates $\mathcal{C}$ to a schema with a single fact table relating **SSN**, **First**, **Last**, and **Salary**. We are interested in $\Delta_F: \mathcal{D}\text{-Set} \rightarrow \mathcal{C}\text{-Set}$. Suppose given the following database instance $I: \mathcal{D} \rightarrow \mathbf{Set}$ on $\mathcal{D}$:

T				
ID	SSN	First	Last	Salary
XF667	115-234	Bob	Smith	\$250
XF891	122-988	Sue	Smith	\$300
XF221	198-877	Alice	Jones	\$100

³This example was taken from [Sp1].

SSN	First	Last	Salary
ID	ID	ID	ID
115-234	Adam	Jones	\$100
118-334	Alice	Miller	\$150
122-988	Bob	Pratt	\$200
198-877	Carl	Richards	\$250
342-164	Sam	Smith	\$300
	Sue		

How do you get the instance $\Delta_F(I): \mathcal{C} \rightarrow \mathbf{Set}$? The formula was given above: compose with F . In terms of tables, it feels like duplicating table T as T_1 and T_2 , but deleting a column from each in accordance with the definition of $\mathcal{C}$. Here is the result in table form:

T1			
ID	SSN	First	Last
XF667	115-234	Bob	Smith
XF891	122-988	Sue	Smith
XF221	198-877	Alice	Jones

T2			
ID	First	Last	Salary
XF221	Alice	Jones	\$100
XF667	Bob	Smith	\$250
XF891	Sue	Smith	\$300

SSN	First	Last	Salary
ID	ID	ID	ID
115-234	Adam	Jones	\$100
118-334	Alice	Miller	\$150
122-988	Bob	Pratt	\$200
198-877	Carl	Richards	\$250
342-164	Sam	Smith	\$300
	Sue		

Exercise 5.1.4.4. Let $\mathcal{C} = (G, \simeq)$ be a schema. A leaf table is an object $c \in \text{Ob}(\mathcal{C})$ with no outgoing arrows.

- Write this condition mathematically in three different languages: that of graphs (using symbols $V, A, \text{src}, \text{tgt}$), that of categories (using $\text{Hom}_{\mathcal{C}}$, etc.), and that of tables (in terms of columns, tables, rows, etc.).
- In the language of categories, is there a difference between a terminal object and a leaf table?

◇

Exercise 5.1.4.5. Consider the schemas

$$[1] = \begin{array}{|c|} \hline \begin{array}{ccc} 0 & \xrightarrow{f} & 1 \\ \bullet & & \bullet \end{array} \\ \hline \end{array} \quad \text{and} \quad [2] = \begin{array}{|c|} \hline \begin{array}{ccccc} 0 & \xrightarrow{g} & 1 & \xrightarrow{h} & 2 \\ \bullet & & \bullet & & \bullet \end{array} \\ \hline \end{array},$$

and the functor $F: [1] \rightarrow [2]$ given by sending $0 \mapsto 0$ and $1 \mapsto 2$.

- How many possibilities are there for $F(f)$?

b.) Now suppose $I: [2] \rightarrow \mathbf{Set}$ is given by the following tables.

0	
ID	g
Am	To be verb
Baltimore	Place
Carla	Person
Develop	Action verb
Edward	Person
Foolish	Adjective
Green	Adjective

1	
ID	h
Action verb	Verb
Adjective	Adjective
Place	Noun
Person	Noun
To be verb	Verb

2	
ID	
Adjective	
Noun	
Verb	

Write out the two tables associated to the $[1]$ -instance $\Delta_F(I): [1] \rightarrow \mathbf{Set}$.

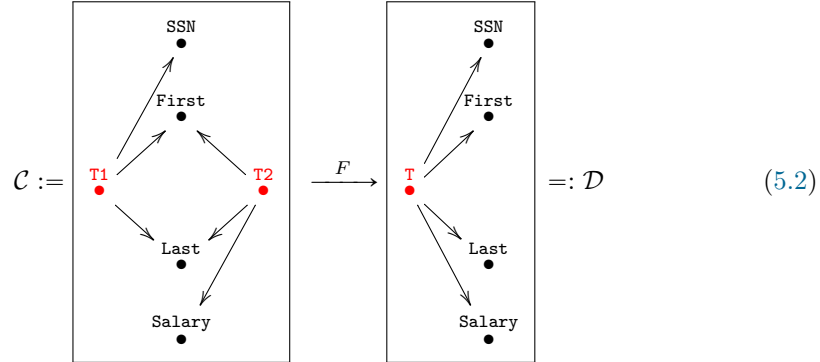
◇

5.1.4.6 Left pushforward: Σ

Let $F: \mathcal{C} \rightarrow \mathcal{D}$ be a functor. The functor $\Delta_F: \mathcal{D}\text{-}\mathbf{Set} \rightarrow \mathcal{C}\text{-}\mathbf{Set}$ has a left adjoint, $\Sigma_F: \mathcal{C}\text{-}\mathbf{Set} \rightarrow \mathcal{D}\text{-}\mathbf{Set}$. The rough idea is that Σ_F performs parameterized colimits. Given an instance $I: \mathcal{C} \rightarrow \mathbf{Set}$, we get an instance on $\mathcal{D}$ that acts as follows. For each object $d \in \text{Ob}(\mathcal{D})$, the set $\Sigma_F(I)(d)$ is the colimit (think, union) of some diagram back home in $\mathcal{C}$.

The precise description can be found in [Sp1]; here we just repeat some examples from that paper.

Example 5.1.4.7. We again use the functor $F: \mathcal{C} \rightarrow \mathcal{D}$ drawn below



We will be applying the left pushforward $\Sigma_F: \mathcal{C}\text{-}\mathbf{Set} \rightarrow \mathcal{D}\text{-}\mathbf{Set}$ to the following instance $I: \mathcal{C} \rightarrow \mathbf{Set}$:

T1			
ID	SSN	First	Last
T1-001	115-234	Bob	Smith
T1-002	122-988	Sue	Smith
T1-003	198-877	Alice	Jones

T2			
ID	First	Last	Salary
T2-001	Alice	Jones	\$100
T2-002	Sam	Miller	\$150
T2-004	Sue	Smith	\$300
T2-010	Carl	Pratt	\$200

SSN	First	Last	Salary
ID	ID	ID	ID
115-234	Adam	Jones	\$100
118-334	Alice	Miller	\$150
122-988	Bob	Pratt	\$200
198-877	Carl	Richards	\$250
342-164	Sam	Smith	\$300
	Sue		

The functor $F: \mathcal{C} \rightarrow \mathcal{D}$ sent both tables T1 and T2 to table T. Applying Σ_F will take the what was in T1 and T2 and put the union in T. The result $\Sigma_F I: \mathcal{D} \rightarrow \mathbf{Set}$ is as follows:

T				
ID	SSN	First	Last	Salary
T1-001	115-234	Bob	Smith	T1-001.Salary
T1-002	122-988	Sue	Smith	T1-002.Salary
T1-003	198-877	Alice	Jones	T1-003.Salary
T2-001	T2-A101.SSN	Alice	Jones	\$100
T2-002	T2-A102.SSN	Sam	Miller	\$150
T2-004	T2-004.SSN	Sue	Smith	\$300
T2-010	T2-A110.SSN	Carl	Pratt	\$200

SSN	First	Last	Salary
ID	ID	ID	ID
115-234	Adam	Jones	\$100
118-334	Alice	Miller	\$150
122-988	Bob	Pratt	\$200
198-877	Carl	Richards	\$250
342-164	Sam	Smith	\$300
T2-001.SSN	Sue		T1-001.Salary
T2-002.SSN			T1-002.Salary
T2-004.SSN			T1-003.Salary
T2-010.SSN			

As you can see, there was no set salary information for any data coming from table T1 nor any set SSN information for any data coming from table T2. But the definition of adjoint, given in Definition 5.1.1.1, yielded the universal response: freely add new variables that take the place of missing information. It turns out that this idea already has a name in logic, *Skolem variables*, and a name in database theory, *labeled nulls*.

Exercise 5.1.4.8. Consider the functor $F: \underline{3} \rightarrow \underline{2}$ sending $1 \mapsto 1, 2 \mapsto 2, 3 \mapsto 2$.

- Write down an instance $I: \mathcal{C} \rightarrow \mathbf{Set}$.
- Given the description that “ Σ_F performs a parameterized colimit”, make an educated guess about what $\Sigma_F(I)$ will be. Give your answer in the form of two sets that are made up from the three sets you already wrote down.

◇

5.1.4.9 Right pushforward: Π

Let $F: \mathcal{C} \rightarrow \mathcal{D}$ be a functor. We heard in Section 5.1.4.6 that the functor $\Delta_F: \mathcal{D}\text{-Set} \rightarrow \mathcal{C}\text{-Set}$ has a left adjoint. Here we explain that it has a right adjoint, $\Pi_F: \mathcal{C}\text{-Set} \rightarrow \mathcal{D}\text{-Set}$ as well. The rough idea is that Π_F performs parameterized limits. Given an instance $I: \mathcal{C} \rightarrow \mathbf{Set}$, we get an instance on $\mathcal{D}$ that acts as follows. For each object $d \in \text{Ob}(\mathcal{D})$, the set $\Pi_F(I)(d)$ is the limit (think, fiber product) of some diagram back home in $\mathcal{C}$.

The precise description can be found in [Sp1]; here we just repeat some examples from that paper.

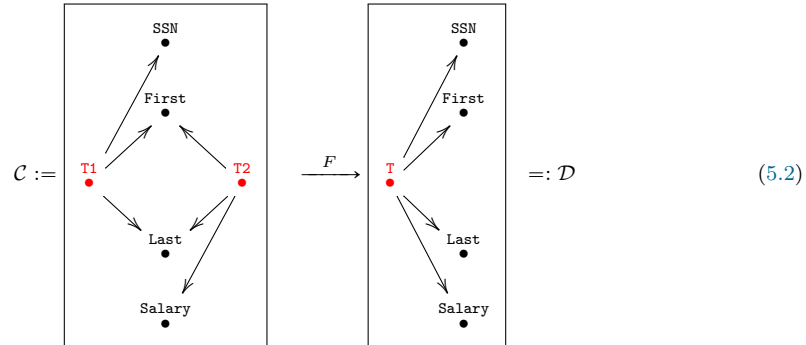
Example 5.1.4.10. We once again use the functor $F: \mathcal{C} \rightarrow \mathcal{D}$ from Example 5.1.4.7. We will apply the right pushforward Π_F to instance $I: \mathcal{C} \rightarrow \mathbf{Set}$ from that example.⁴

The instance $\Pi_F(I)$ will put data in all 5 tables in $\mathcal{D}$. In $\mathbf{T}$ it will put pairs (t_1, t_2) where t_1 is a row in $\mathbf{T1}$ and t_2 is a row in $\mathbf{T2}$ for which the first and last names agree. It will copy the leaf tables exactly, so we do not display them here; the following is the table $\mathbf{T}$ for $\Pi_F(I)$:

T				
ID	SSN	First	Last	Salary
T1-002T2-A104	122-988	Sue	Smith	\$300
T1-003T2-A101	198-877	Alice	Jones	\$100

Looking at $\mathbf{T1}$ and $\mathbf{T2}$, there were only two ways to match first and last names.

⁴To repeat for convenience,



$I: \mathcal{C} \rightarrow \mathbf{Set}$ is

T1			
ID	SSN	First	Last
T1-001	115-234	Bob	Smith
T1-002	122-988	Sue	Smith
T1-003	198-877	Alice	Jones

T2			
ID	First	Last	Salary
T2-001	Alice	Jones	\$100
T2-002	Sam	Miller	\$150
T2-004	Sue	Smith	\$300
T2-010	Carl	Pratt	\$200

SSN	
ID	
115-234	
118-334	
122-988	
198-877	
342-164	

First	
ID	
Adam	
Alice	
Bob	
Carl	
Sam	
Sue	

Last	
ID	
Jones	
Miller	
Pratt	
Richards	
Smith	

Salary	
ID	
\$100	
\$150	
\$200	
\$250	
\$300	

Exercise 5.1.4.11. Consider the functor $F: \underline{3} \rightarrow \underline{2}$ sending $1 \mapsto 1, 2 \mapsto 2, 3 \mapsto 2$.

- a.) Write down an instance $I: \mathcal{C} \rightarrow \mathbf{Set}$.
- b.) Given the description that “ Π_F performs a parameterized limit”, make an educated guess about what $\Pi_F(I)$ will be. Give your answer in the form of two sets that are made up from the three sets you already wrote down.

◇

5.2 Categories of functors

For any two categories $\mathcal{C}$ and $\mathcal{D}$,⁵ we discussed the category $\mathbf{Fun}(\mathcal{C}, \mathcal{D})$ of functors and natural transformations between them. In this section we discuss functor categories a bit more and give some important applications within mathematics (sheaves) that extend to the real world.

5.2.1 Set-valued functors

Let $\mathcal{C}$ be a category. Then we have been writing $\mathcal{C}\text{-}\mathbf{Set}$ to denote the functor category $\mathbf{Fun}(\mathcal{C}, \mathbf{Set})$. Here is a nice result about these categories.

Proposition 5.2.1.1. *Let $\mathcal{C}$ be a category. The category $\mathcal{C}\text{-}\mathbf{Set}$ is closed under colimits and limits.*

Sketch of proof. Let J be an indexing category and $D: J \rightarrow \mathcal{C}\text{-}\mathbf{Set}$ a functor. For each object $c \in \mathbf{Ob}(\mathcal{C})$, we have a functor $D_c: J \rightarrow \mathbf{Set}$ defined by $D_c(j) = D(j)(c)$. Define a functor $L: \mathcal{C} \rightarrow \mathbf{Set}$ by $L(c) = \lim_J D_c$, and note that for each $f: c \rightarrow c'$ in $\mathcal{C}$ there is an induced function $L(f): L(c) \rightarrow L(c')$. One can check using that L is a limit of J , because it satisfies the relevant universal property.

The dual proof holds for colimits.

□

Application 5.2.1.2. When taking in data about a scientific subject, one often finds that the way one thinks about the problem changes over time. We understand this phenomenon in the language of databases in terms of a **series of schemas** $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_n$, perhaps indexed chronologically. The problem is that old data is held in old schemas and we want to see it in our current understanding. The first step is to transfer all the old data to our new schema in the freest possible way, that is, making no assumptions about how to fill in the new fields. If one creates functors $F_i: \mathcal{C}_i \rightarrow \mathcal{C}_{i+1}$ from each of these schemas to the next, then we can push the data forward using Σ_{F_i} .

Doing this we will have n datasets on $\mathcal{D} := \mathcal{C}_n$, namely one for each “epoch of understanding”. Since the category $\mathcal{D}\text{-}\mathbf{Set}$ has all colimits, we can take the union of these datasets and get one. It will have many Skolem variables (see Example 5.1.4.7), and these need to be handled in a coherent way. However, the universality of left adjoints could be interpreted as saying that any reasonable formula for handling this old data can be applied to our results.

◇◇

⁵Technically $\mathcal{C}$ has to be small (see Remark 4.1.1.2), but as we said there, we are not worrying about that distinction in this book.

Proposition 5.2.1.1 says that we can add or multiply database states together. In fact, database states on $\mathcal{C}$ form what is called a *topos* which means that just about every consideration we made for sets holds for instances on any schema. Perhaps the simplest schema is $\mathcal{C} = \boxed{\bullet}$, on which the relevant topos is indeed **Set**. But schemas can be arbitrarily complex, and it is impressive that all of these considerations make sense in such generality. Here is a table that makes a comparison between these domains.

Dictionary between Set and $\mathcal{C}\text{-Set}$	
Concept in Set	Concept in $\mathcal{C}\text{-Set}$
Set	Object in $\mathcal{C}\text{-Set}$
Function	Morphism in $\mathcal{C}\text{-Set}$
Element	Representable functor
Empty set	Initial object
Natural numbers	Natural numbers object
Image	Image
(Co)limits	(Co)limits
Exponential objects	Exponential objects
“Familiar” arithmetic	“Familiar” arithmetic
Power sets 2^X	Power objects Ω^X
Characteristic functions	Characteristic functions
Surjections, injections	Epimorphisms, monomorphisms

In the above table we said that elements of a set are akin to representable functors, but we have not yet defined those; we do so now.

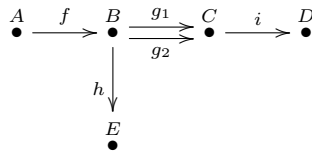
5.2.1.3 Representable functors

Definition 5.2.1.4. Let $\mathcal{C}$ be a category and let $c \in \text{Ob}(\mathcal{C})$ be an object. The functor $\text{Hom}_{\mathcal{C}}(c, -): \mathcal{C} \rightarrow \mathbf{Set}$, sending $d \in \text{Ob}(\mathcal{C})$ to the set $\text{Hom}_{\mathcal{C}}(c, d)$ and acting similarly on morphisms $d \rightarrow d'$, is said to be *represented by c* . If a functor $F: \mathcal{C} \rightarrow \mathbf{Set}$ is isomorphic to $\text{Hom}_{\mathcal{C}}(c, -)$, we say that F is a *representable functor*. We sometimes write $Y(c) := \text{Hom}_{\mathcal{C}}(c, -)$ for short.

Example 5.2.1.5. Given a category $\mathcal{C}$ and an object $c \in \text{Ob}(\mathcal{C})$, we get a representable functor. If we think of $\mathcal{C}$ as a database schema and c as a table, then what does the representable functor look like in databases? It turns out that the following procedure will generate it.

Begin by writing a new row, say “ $\odot$ ”, in the ID column of table c . For each foreign key column $f: c \rightarrow c'$, add a row in the ID column of table c' called “ $f(\odot)$ ” and record that result (i.e. “ $f(\odot)$ ”) in the f column of table c . Repeat as follows: for each table d , identify all rows r that have blank cell in column $g: d \rightarrow e$. Add a new row called “ $g(r)$ ” to table e and record that result in the (r, g) cell of table d .

Here is a concrete example. Let $\mathcal{C}$ be the following schema:



Then $y_B: \mathcal{C} \rightarrow \mathbf{Set}$ is the following instance

A		B				C	
ID	f	ID	g_1	g_2	h	ID	i
		$\ominus$	$g_1(\ominus)$	$g_2(\ominus)$	$h(\ominus)$	$g_1(\ominus)$	$i(g_1(\ominus))$
						$g_2(\ominus)$	$i(g_2(\ominus))$

D		E	
ID		ID	
$i(g_1(\ominus))$		$h(\ominus)$	
$i(g_2(\ominus))$			

We began with a single element in table B and followed the arrows, putting new entries wherever they were required. One might call this the *schematically implied reference spread* or *SIRS* of the element $\ominus$ in table B .

Representable functors $Y(c)$ yield database states that are as free as possible, subject to having the initial row $\ominus$ in table c . We have seen such things this before (by the name of Skolem variables) when studying the left pushforward Σ . Indeed, if $c \in \text{Ob}(\mathcal{C})$ is an object, we can consider it as a functor $c: \underline{1} \rightarrow \mathcal{C}$. A database instance on $\underline{1}$ is the same thing as a set X . The left pushforward $\Sigma_c(X)$ has the same kinds of Skolem variables. If $X = \{\ominus\}$ is a one element set, then we get the representable functor $\Sigma_c(\{\ominus\}) \cong Y(c)$.

5.2.1.6 Yoneda's lemma

One of the most powerful tools in category theory is Yoneda's lemma. It is often considered by new students to be quite abstract, but grounding it in databases may help.

The idea is this. Suppose that $I: \mathcal{C} \rightarrow \mathbf{Set}$ is a database instance, and let $c \in \text{Ob}(\mathcal{C})$ be an object. Because I is a functor, we know that for every row $r \in I(c)$ in table c a value has been recorded in the f -column, where $f: c \rightarrow c'$ is any outgoing arrow. The value in the (r, f) -cell refers to some row in table c' . What we're saying is that each row in table c induces SIRS throughout the database. They may not be "Skolem", or in any sense "freely generated", but they are there nonetheless. The point is that to each row in c there is a unique mapping $Y(c) \rightarrow I$.

Lemma 5.2.1.7 (Yoneda's lemma). *Let $\mathcal{C}$ be a category, $c \in \text{Ob}(\mathcal{C})$ an object, and $I: \mathcal{C} \rightarrow \mathbf{Set}$ an instance. There is a natural bijection*

$$\text{Hom}_{\mathcal{C}-\mathbf{Set}}(Y(c), I) \xrightarrow{\cong} I(c).$$

Proof. See [Mac].

□

5.2.2 Database instances in other categories

5.2.2.1 Representations of groups

The classical mathematical subject of *representation theory* is the study of $\text{Fun}(G, \mathbf{Vect})$ where G is a group and $\mathbf{Vect}$ is the category of vector spaces (over say $\mathbb{R}$). Every such functor $F: G \rightarrow \mathbf{Vect}$ is called a *representation of G* . Since G is a category with one object $\blacktriangle$, F consists of a single vector space $V = F(\blacktriangle)$ together with an action of G on it.

We can think of this in terms of databases if we have a presentation of G in terms of generators and relations. The schema corresponding to G has one table and this table

has a column for each generator. Giving a representation F is the same as giving an instance on our schema, with some properties that stem from the fact that our target category is **Vect** rather than **Set**. There are many possibilities for expressing ⁶ such data.

One possibility is if we could somehow draw V , say if V is 1-, 2-, or 3-dimensional. If so, let P be our chosen picture of V , e.g. P is the standard drawing of a Cartesian coordinate plane. Then every column of our table would consist entirely of the picture P instead of a set of rows. Drawing a point in the ID-column picture would result in a point being drawn in each other column's picture, in accordance with the G -action. Each column would of course respect addition and scalar multiplication.

Another possibility is to use the fact that there is a functor $U: \mathbf{Vect} \rightarrow \mathbf{Set}$, so our instance $F: G \rightarrow \mathbf{Vect}$ can be converted to an ordinary instance $U \circ F: G \rightarrow \mathbf{Set}$. We would have an ordinary set of rows. This set would generally be infinite, but it would be structured by addition and scalar multiplication. For example, assuming V is finite dimensional, one could find a few rows that generated the rest.

A third possibility is to use monads, which allow the table to have only as many rows as V has dimensions. This is a considerable savings of space. See Section 5.3.

5.2.2.2 Representations of quivers

Representation theory also studies representations of quivers. A *quiver* is just the free category (see Example 4.1.2.27) on a graph. If P is a graph with free category $\mathcal{P}$ then a representation of the quiver $\mathcal{P}$ is a functor $F: \mathcal{P} \rightarrow \mathbf{Set}$. Such a representation consists of a vector space at every vertex of P and a linear transformation for every arrow. All of the discussion from Section 5.2.2.1 works in this setting, except that there is more than one table.

5.2.2.3 Other target categories

One can imagine the value of using target categories other than **Set** or **Vect** for databases.

Application 5.2.2.4. Geographic data consists of maps of the earth together with various functions on it. For example for any point on the earth one may want to know the average temperature recorded in the past 10 years, or the precise temperature at this moment. Earth can be considered as a topological space, E . Similarly, temperatures on earth reside on a continuum, say the space T of real numbers $[-100, 200]$. Thus the temperature record is a function $E \rightarrow T$.

Other records such as precipitation, population density, elevation, etc. can all be considered as continuous functions from E to some space. Agencies like the US Geological Survey hold databases of such information. By modeling them on functors $\mathcal{C} \rightarrow \mathbf{Top}$, they may be able to employ mathematical tools such as persistent homology [WeS] to find interesting invariants of the data.

◇◇

Application 5.2.2.5. Many other scientific disciplines could use the same kind of tool. For example, in studying the *mechanics of materials*, one may want to consider the material as a topological space M and measure values such as energy as a continuous $M \rightarrow E$. Such observations could be modeled by databases with target category **Top** or **Vect** rather than **Set**.

⁶We would use the term “representing” or “presenting”, but they are both taken in the context of our narrative!

◇◇

5.2.3 Sheaves

Let X be a topological space (see Example 4.2.3.1), such as a sphere. In Section 5.2.2.3 we discussed continuous functions out of X , and their use in science (e.g. recording temperatures on the earth as a continuous map $X \rightarrow [-100, 200]$). Sheaves allow us to consider the local-global nature of such maps, taking into account reparable discrepancies in data gathering tools.

Application 5.2.3.1. Suppose that X is the topological space corresponding to the earth; by a *region* we mean an open subset $U \subseteq X$. Suppose that we cover X with 10,000 regions $U_1, U_2, \dots, U_{10000}$, such that there are overlaps between some of these regions (e.g. perhaps $U_5 \cap U_9 \neq \emptyset$). For each i, j let $U_{i,j} = U_i \cap U_j$.

For each region $U_i \subseteq X$ we have a temperature recording device, which gives a function $T_i: U_i \rightarrow [-100, 200]$. If $U_i \cap U_j \neq \emptyset$ then two different recording devices give us temperature data for the intersection $U_{i,j}$. Suppose we find that they do not give precisely the same data, but that there is a translation formula between their results. For example, T_i might register 3° warmer than T_j registers in that region.

A consistent system of translation formulas is called a *sheaf*. It does not demand a universal “true” temperature function, but only a consistent translation system between them.

◇◇

The following definitions makes the notion of sheaf precise, but we must go slowly (because it will already feel quick to the novice). For every region U , we can record the value of some function (say temperature) throughout U ; although this record might consist of a mountain of data (a temperature for each point in U !), we think of it as one thing. That is, it is one element in the set of value-assignments throughout U . A sheaf holds the set of possible values-assignments-throughout- U 's for all the different U 's, as well as how a value-assignment throughout U restricts to a value-assignment throughout V for any subset $V \subseteq U$.

Definition 5.2.3.2. Let X be a topological space, let $\text{Op}(X)$ denote its partial order of open sets, and let $\text{Op}(X)^{\text{op}}$ be the opposite category. A *presheaf on X* is a functor $\mathcal{O}: \text{Op}(X)^{\text{op}} \rightarrow \mathbf{Set}$. For every open set $U \subseteq X$ we refer to the set $\mathcal{O}(U)$ as the *set of values-assignments throughout U of $\mathcal{O}$* . If $V \subseteq U$ is an open subset, it corresponds to an arrow in $\text{Op}(X)$ and applying the functor $\mathcal{O}$ yields a function called the *restriction map from U to V* and denoted $\rho_{V,U}: \mathcal{O}(U) \rightarrow \mathcal{O}(V)$. Given $a \in \mathcal{O}(U)$, we may denote $\rho(V,U)(a)$ by $a|_V$; it is called *the restriction of a to V* .

The *category of presheaves on X* is simply $\text{Op}(X)^{\text{op}}\text{-Set}$; see Definition 4.3.3.1.

Exercise 5.2.3.3. a.) Come up with 4 overlapping open subsets that cover the square $X := [0, 3] \times [0, 3] \subseteq \mathbb{R}^2$. Write down a label for each open set as well as a label for each overlap (2-fold, 3-fold, etc.); you now have labeled n open sets. For each of these open sets, draw a dot with the appropriate label, and then draw an arrow from one dot to another when the first refers to an open subset of the second. This is a preorder; call it $\text{Op}(X)$. Now make up and write down formulas $R_1: X \rightarrow \mathbb{R}$ and $R_2: X \rightarrow \mathbb{R}$ with $R_1 \leq R_2$, expressing a range of temperatures $R_1(p) \leq x \leq R_2(p)$ that an imaginary experiment shows can exist at each point p in the square.

- b.) Suppose we now tried to make our presheaf $\mathcal{O}: \text{Op}(X)^{\text{op}} \rightarrow \mathbf{Set}$ as follows. For each of your open sets, say A , we could put

$$\mathcal{O}(A) := \{f: A \rightarrow \mathbb{R} \mid R_1(a) \leq f(a) \leq R_2(a)\}.$$

What are the restriction maps? Do you like the name “value-assignment throughout A ” for elements of $\mathcal{O}(A)$?

- c.) We can now make another presheaf $\mathcal{O}'$ given the same experiment. For each of your open sets, say A , we could put

$$\mathcal{O}'(A) := \{f: A \rightarrow \mathbb{R} \mid f \text{ is continuous, and } R_1(a) \leq f(a) \leq R_2(a)\}.$$

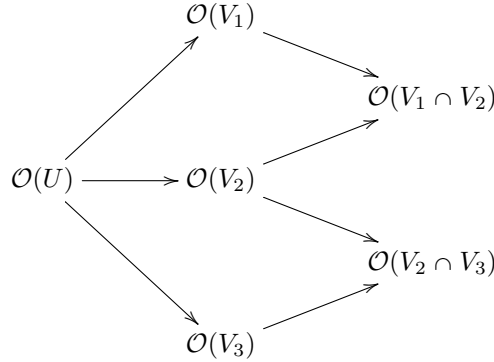
Are you comfortable with the idea that there is a morphism of presheaves $\mathcal{O}' \rightarrow \mathcal{O}$?

◇

Before we define sheaves, we need to clarify the notion of covering. If U is a region and $V_1, \dots, V_n$ are subregions (i.e. for each $1 \leq i \leq n$ we have $V_i \subseteq U$). Then we say that the V_i *cover* U if every point in U is in V_i for some i . Another way to say this is that the natural function $\sqcup_i V_i \rightarrow U$ is surjective.⁷

Example 5.2.3.4. Let $X = \mathbb{R}$ be the space of real numbers, and define the following open subsets: $U = (5, 10)$, $V_1 = (5, 7)$, $V_2 = (6, 9)$, $V_3 = (7, 10)$. Then V_1, V_2, V_3 is a cover of U . It has overlaps $V_{12} = V_1 \cap V_2 = (6, 7)$, $V_{13} = V_1 \cap V_3 = \emptyset$, $V_{23} = V_2 \cap V_3 = (7, 9)$.

Given a presheaf $\mathcal{O}: \text{Op}(X)^{\text{op}} \rightarrow \mathbf{Set}$, we have sets and functions as in the following (incomplete) diagram



A presheaf $\mathcal{O}$ on X tells us what value-assignments throughout U can exist for each U . Suppose we have a value-assignment $a \in \mathcal{O}(U)$ throughout U and another value-assignment $a' \in \mathcal{O}(U')$ throughout U' , and suppose that they agree as value-assignments throughout $U \cap U'$, i.e. $a|_{U \cap U'} = a'|_{U \cap U'}$. In this case we should have a unique value-assignment $b \in \mathcal{O}_{U \cup U'}$ throughout $U \cup U'$ that agrees on the U -part with a and agrees on the U' -part with a' ; i.e. $b|_U = a$ and $b|_{U'} = a'$. This is the sheaf condition.

Definition 5.2.3.5. Let X be a topological space, let $\text{Op}(X)$ be its partial order of open sets, and let $\mathcal{O}: \text{Op}(X)^{\text{op}} \rightarrow \mathbf{Set}$ be a presheaf. Given an open set $U \subseteq X$ and a cover $V_1, \dots, V_n$ of U , the following condition is called the *sheaf condition* for that cover.

⁷To be pedantic, we should say that $\sqcup_i V_i \rightarrow U$ is an epimorphism, which is the same thing as saying that on underlying points (i.e. via the “underlying points” functor $\mathbf{Top} \rightarrow \mathbf{Set}$) the map is a surjection.

Sheaf condition Given a sequence $a_1, \dots, a_n$ where each is a value-assignment $a_i \in \mathcal{O}(V_i)$ throughout V_i , and suppose that for all i, j we have $a_i|_{V_i \cap V_j} = a_j|_{V_i \cap V_j}$; then there is a unique value-assignment $b \in \mathcal{O}_U$ such that $b|_{V_i} = a_i$.

The presheaf $\mathcal{O}$ is called a *sheaf* if it satisfies the sheaf condition for every cover.

Example 5.2.3.6. Let $X = \mathbb{R}$ and let U, V_1, V_2, V_3 be the open cover given in Example 5.2.3.4. Given a measurement taken throughout V_1 , a measurement taken throughout V_2 , and a measurement taken throughout V_3 , we have elements $a_1 \in \mathcal{O}(V_1)$, $a_2 \in \mathcal{O}(V_2)$, and $a_3 \in \mathcal{O}(V_3)$. If they are in agreement on the overlap intervals, we can *glue* them to give a measurement throughout U .

Application 5.2.3.7. Consider outer space as a topological space X . Different astronomers record observations. Let $C = [390, 700]$ denote the set of wavelengths in the visible light spectrum (written in nanometers). Given an open subset $U \subseteq X$ let $\mathcal{O}(U)$ denote the set of functions $U \rightarrow C$. The presheaf $\mathcal{O}$ satisfies the sheaf condition; this is the taken-for-granted fact that we can patch together different **observations of space**.

Below are three views of the night sky. Given a telescope position to obtain the first view, one moves the telescope up and a little down to obtain the second and one moves it down and left to obtain the third.



These are value-assignments $a_1 \in \mathcal{O}(V_1)$, $a_2 \in \mathcal{O}(V_2)$, and $a_3 \in \mathcal{O}(V_3)$ throughout subsets $V_1, V_2, V_3 \subseteq X$ (respectively). These subsets V_1, V_2, V_3 cover some (strangely-shaped) subset $U \subseteq X$. The sheaf condition says that these three value-assignments glue together to form a single value-assignment throughout U :



◇◇

Exercise 5.2.3.8. Find an application of sheaves in your own domain of expertise. ◇

Application 5.2.3.9. Suppose we have a sheaf for temperatures on earth. For every region U we have a set of theoretically possible temperature-assignments throughout U . For example we may know that if it is warm in Texas, warm in Arkansas, and warm in Kansas, then it cannot be cold in Oklahoma. With such a sheaf $\mathcal{O}$ in hand, one can use facts about the temperature in one region U to predict the temperature in another region V .

The mathematics is as follows. Suppose given regions $U, V \subseteq X$ and a subset $A \subseteq \mathcal{O}(U)$ corresponding to what we know about the temperature assignment throughout U . We take the following fiber product

$$\begin{array}{ccc} (\rho_{U,X})^{-1}(A) & \longrightarrow & \mathcal{O}(X) \xrightarrow{\rho_{V,X}} \mathcal{O}(V) \\ \downarrow \text{ } \text{ } \text{ } \downarrow \rho_{U,X} & \text{ } \text{ } \text{ } \downarrow \rho_{U,X} & \\ A & \longrightarrow & \mathcal{O}(U) \end{array}$$

The image of the top map is a subset of $\mathcal{O}(V)$ telling us which temperature-assignments are possible throughout V given our knowledge A about the temperature throughout U .

We can imagine the same type of prediction systems for other domains as well, such as the energy of various parts of a material. ◇◇

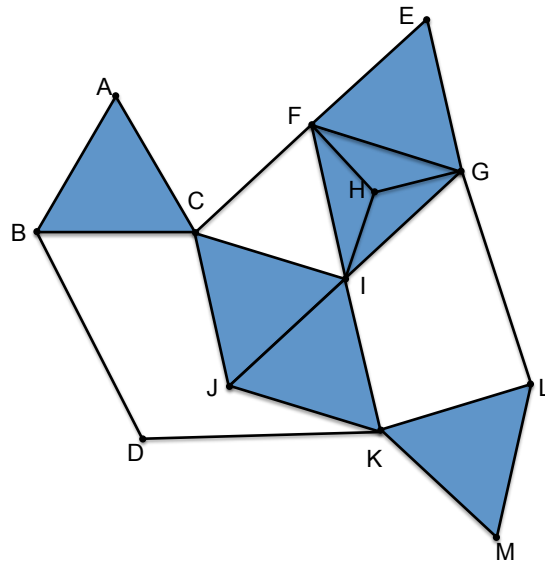
Example 5.2.3.10. In Exercises 4.2.4.3 and 4.2.4.4 we discussed the idea of laws being dictated or respected throughout a jurisdiction. If X is earth, to every jurisdiction $U \subseteq X$ we assign the set $\mathcal{O}(U)$ of laws that are dictated to hold throughout U . Given a law on U and a law on V , we can see if they amount to the same law on $U \cap V$. For

example, on U a law might say “no hunting near rivers” and on V a law might say “no hunting in public areas”. It just so happens that on $U \cap V$ all public areas are near rivers and vice versa, so the laws agree there. These laws patch together to form a single rule about hunting that is enforced throughout the union $U \cup V$, respected by all jurisdictions within it.

5.2.3.11 Sheaf of ologed concepts

Definition 5.2.3.5 defines what should be called a sheaf of sets. We can discuss sheaves of groups or even sheaves of categories. Here is an application of the latter.

Recall the notion of simplicial complexes discussed in Section 2.7.4.3. They look like this:



(5.3)

Given such a simplicial complex X , we can imagine each vertex $v \in X_0$ as an entity with a worldview (e.g. a person) and each simplex as the common worldview shared by its vertices. To model this, we will assign to each vertex $v \in X$ an olog $\mathcal{O}(v)$, corresponding to the worldview held by that entity, and to each simplex $u \in X_n$, we assign an olog $\mathcal{O}(u)$ corresponding to a *common ground* worldview.. Recall that X is a subset of $\mathbb{P}(X_0)$; it is a preorder and its elements (the simplices) are ordered by inclusion. If u, v are simplices with $u \subseteq v$ then we want a map of ologs (i.e. a schema morphism) $\mathcal{O}(v) \rightarrow \mathcal{O}(u)$ corresponding to how any idea that is shared among the people in v is shared among the people in u . Thus we have a functor $\mathcal{O}: X \rightarrow \mathbf{Sch}$ (where we are forgetting the distinction between ologs and databases for notational convenience).

To every simplicial complex (indeed every ordered set) one can associate a topological space; in fact we have a functor $Alx: \mathbf{Pro} \rightarrow \mathbf{Top}$, called the **Alexandrov** functor. Applying $Alx(X^{\text{op}})$ we have a space which we denote by $\mathcal{X}$. One can visualize $\mathcal{X}$ as X , but the open sets include unions of simplices. There is a unique sheaf of categories on $\mathcal{X}$ that behaves like X on simplices.

How does this work in the case of our sheaf $\mathcal{O}$ of worldviews? For simplices such as (A) or (CI) , the sheaf returns the olog corresponding to that person or shared worldview. But for open sets like the union of (CIJ) and (IJK) , what we get is the olog consisting

of the types shared by C, I , and J for which I and J affirm agreement with types shared by I, J , and K .

Example 5.2.3.12. Imagine two groups of people G_1 and G_2 each making observations about the world. Suppose that there is some overlap $H = G_1 \cap G_2$. Then it may happen that there is a conversation including G_1 and G_2 and both groups are talking about something and, although using different words, H says “you guys are talking about the same things, you just use different words.” In this case there is an object-assignment throughout $G_1 \cup G_2$ that agrees with both those on G_1 and those on G_2 .

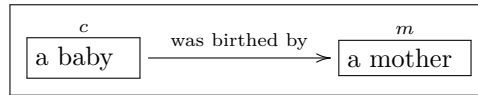
5.2.3.13 Time

One can use sheaves to model objects in time; Goguen gave an approach to this in [Gog]. For another approach, let $\mathcal{C}$ be a database schema. The lifespan of information about the world is generally finite; that is, what was true yesterday is not always the case today. Thus we can associate to each interval U of time the information that we deem to hold throughout U . This is sometimes called the *valid time* of the data.

If something is the case throughout U and we have a subset $V \subseteq U$ then of course it is the case throughout V . And the sheaf condition holds too: if some information holds throughout U and some other information holds throughout U' , and if these two things restrict to the same information on the overlap $U \cap V$, then they can be glued to information that holds throughout the union $U \cup V$.

So we can model information-change over time by using a sheaf of $\mathcal{C}$ -sets on the topological space $\mathbb{R}$. One way to think of this is simply as an instance on the schema $\mathcal{C} \times \text{Op}(\mathbb{R})^{\text{op}}$. The sheaf condition is just an added property that our instances have to obey.

Example 5.2.3.14. Consider a hospital in which babies are born. In our scenario, mothers enter the hospital, babies are born, mothers and babies leave the hospital. Let $\mathcal{C}$ be the schema



Consider the 8-hour intervals

$$\text{Shift}_1 := (\text{Jan } 1 - 00 : 00, \text{ Jan } 1 - 08 : 00),$$

$$\text{Shift}_2 := (\text{Jan } 1 - 04 : 00, \text{ Jan } 1 - 12 : 00),$$

$$\text{Shift}_3 := (\text{Jan } 1 - 8 : 00, \text{ Jan } 1 - 16 : 00).$$

The nurses take shifts of 8 hours, overlapping with their predecessors by 4 hours, and they record in the database only patients that were there throughout their shift or throughout any overlapping shift. A mother might be in the hospital throughout shift 1, arriving before the new year. A baby is born at 05:00 on Jan 1, and thus does not make it into the Shift_1 -table, but does make it into the $(\text{Shift}_1 \cap \text{Shift}_2)$ -table. The two are there until 17:00 on Jan 1, and so they are recorded in the Shift_2 and Shift_3 tables.

Whether or not this implementation of the sheaf semantics is most useful in practice is certainly debatable. But something like this could easily be useful as a semantics, i.e. a way of thinking about, the temporal nature of data.

5.3 Monads

Monads would never have been invented without category theory, but they have been quite useful in formalizing algebra, calculating invariants of topological spaces, and imbedding non-functional operations into functional programming languages. We will mainly discuss monads in terms of how they can help us make explicit the context in which we're working, and in so doing relegate it to the background of our discussion.

Much of the following material on monads is taken from [Sp3].

5.3.1 Monads formalize context

Monads can formalize assumptions about the way one will do business throughout a domain. For example, suppose that we want to consider functions that do not have to return a value for all inputs. Such *partial functions* can be composed. Indeed, given a partial function $f: A \rightarrow B$ and a partial function $g: B \rightarrow C$, one gets a function $g \circ f: A \rightarrow C$ in an obvious way.

Here we are drawing arrows as though we are talking about functions, but there is an implicit context in which we are actually talking about partial functions. Monads allow us to write things in the “functional” way while holding the underlying context. What makes them useful is that the notion of *context* we are using here is made formal.

Example 5.3.1.1. Partial functions can be modeled by ordinary functions, if we add a special “no answer” element to the codomain. That is, the set of partial functions $A \rightarrow B$ is in one-to-one correspondence with the set of ordinary functions $A \rightarrow B \sqcup \{\odot\}$. For example, suppose we want to model the partial function $f(x) := \frac{1}{x^2-1}: \mathbb{R} \rightarrow \mathbb{R}$ in this way, we would use the function

$$f(x) := \begin{cases} \odot & \text{if } x = -1, \\ \odot & \text{if } x = 1, \\ \frac{1}{x} & \text{if } x \neq 0. \end{cases}$$

An ordinary function $f: A \rightarrow B$ can be considered a partial function because we can compose with the inclusion

$$B \rightarrow B \sqcup \{\odot\} \quad (5.4)$$

But how do we compose two partial functions written in this way? Suppose $f: A \rightarrow B \sqcup \{\odot\}$ and $g: B \rightarrow C \sqcup \{\odot\}$ are functions. First form a new function

$$g' := g \sqcup \{\odot\}: B \sqcup \{\odot\} \rightarrow C \sqcup \{\odot\} \sqcup \{\odot\} \quad (5.5)$$

then compose to get $(g' \circ f): A \rightarrow C \sqcup \{\odot\} \sqcup \{\odot\}$, and finally send both $\odot$'s to the same element by composing with

$$C \sqcup \{\odot\} \sqcup \{\odot\} \rightarrow C \sqcup \{\odot\}. \quad (5.6)$$

What does this mean? Every element $a \in A$ is sent by f to either an element $b \in B$ or “no answer”. If it has an answer $f(a) \in B$, this is either sent by g to an element $g(f(a)) \in C$ or to “no answer”. We get a partial function $A \rightarrow C$ by sending a to $g(f(a))$ if possible or to “no answer” if it gets stopped along the way.

Application 5.3.1.2. Experiments are supposed to be performed objectively, but suppose we imagine that changing the person who performs the experiment, say in psychology, may change the outcome. Let A be the set of experimenters, let X be the parameter space for the experimental variables (e.g. $X = \text{Age} \times \text{Income}$) and let Y be the observation space (e.g. $Y = \text{propensity for violence}$). Then whereas we want to think of such an experiment as telling us about a function $f: X \rightarrow Y$, we may want to make some of the context explicit by including information about who performed the experiment. That is, we are really finding a function $f: X \times A \rightarrow Y$.

However, it may be the case that even ascertaining someones age or income, which is done by asking that person, is subject to who in A is doing the asking, and so we again want to consider the experimenter as part of the equation. In this case, we can use a monad to hide the fact that everything in sight is assumed to be influenced by A .

We will return to this in Examples 5.3.2.6 and 5.3.3.4.

◇◇

5.3.2 Definition and examples

What aspects of Example 5.3.1.1 are really about monads, and what aspects are just about partial functions in particular? It is a functor and a pair of natural transformations that showed up in (5.5), (5.4), and (5.6). In this section we will give the definition and a few examples. We will return to our story about how monads formalize context in Section 5.3.3.

Definition 5.3.2.1. A *monad on $\mathbf{Set}$* consists of some constituents (A. functor, B. unit map, C. multiplication map) conforming to some laws (1. unit laws, 2. associativity law), as follows:

- A. a functor $T: \mathbf{Set} \rightarrow \mathbf{Set}$,
- B. a natural transformation $\eta: \text{id}_{\mathbf{Set}} \rightarrow T$, and
- C. a natural transformation $\mu: T \circ T \rightarrow T$

We sometimes refer to the functor T as though it were the whole monad; we call η the *unit map* and we call μ the *multiplication map*. The following laws harmonize these constituents

1. The following diagrams of functors $\mathbf{Set} \rightarrow \mathbf{Set}$ commute:

$$\begin{array}{ccc} T \circ \text{id}_{\mathbf{Set}} & \xrightarrow{\text{id}_T \circ \eta} & T \circ T \\ & \searrow = & \downarrow \mu \\ & & T \end{array} \qquad \begin{array}{ccc} \text{id}_{\mathbf{Set}} \circ T & \xrightarrow{\eta \circ \text{id}_T} & T \circ T \\ & \searrow = & \downarrow \mu \\ & & T \end{array}$$

2. The following diagram of functors $\mathbf{Set} \rightarrow \mathbf{Set}$ commutes:

$$\begin{array}{ccc} T \circ T \circ T & \xrightarrow{\mu \circ \text{id}_T} & T \circ T \\ \text{id}_T \circ \mu \downarrow & & \downarrow \mu \\ T \circ T & \xrightarrow{\mu} & T \end{array}$$

Example 5.3.2.2. We now go through Definition 5.3.2.1 using what is called the List monad. The first step is to give a functor $\text{List}: \mathbf{Set} \rightarrow \mathbf{Set}$, which we did in Example 4.1.2.15. Recall that if $X = \{p, q, r\}$ then $\text{List}(X)$ includes the empty list $[]$, singleton lists, such as $[p]$, and any other list of elements in X , such as $[p, p, r, q, p]$. Given a function $f: X \rightarrow Y$, one obtains a function $\text{List}(f): \text{List}(X) \rightarrow \text{List}(Y)$ by entry-wise application of f .

As a monad, the functor List comes with two natural transformations, a unit map η and a multiplication map μ . Given a set X , the unit map $\eta_X: X \rightarrow \text{List}(X)$ returns singleton lists as follows

$$\begin{array}{ccc} X & \xrightarrow{\eta_X} & \text{List}(X) \\ \hline p & \mapsto & [p] \\ q & \mapsto & [q] \\ r & \mapsto & [r] \end{array}$$

Given a set X , the multiplication map $\mu_X: \text{List}(\text{List}(X)) \rightarrow \text{List}(X)$ flattens lists of lists as follows.

$$\begin{array}{ccc} \text{List}(\text{List}(X)) & \xrightarrow{\mu_X} & \text{List}(X) \\ \hline [[q, p, r], [], [q, r, p, r], [r]] & \mapsto & [q, p, r, q, r, p, r, r] \end{array}$$

The naturality of η and μ just mean that these maps work appropriately well under term-by-term replacement by a function $f: X \rightarrow Y$. Finally the three monad laws from Definition 5.3.2.1 can be exemplified as follows:

$$\begin{array}{ccc} [p, q, q] & \xrightarrow{\text{id}_{\text{List}} \circ \eta} & [[p], [q], [q]] \\ & \searrow & \downarrow \mu \\ & & [p, q, q] \end{array} \quad \begin{array}{ccc} [p, q, q] & \xrightarrow{\eta \circ \text{id}_{\text{List}}} & [[p, q, q]] \\ & \searrow & \downarrow \mu \\ & & [p, q, q] \end{array}$$

$$\begin{array}{ccc} [[p, q], [r]], [], [r, q, q] & \xrightarrow{\mu \circ \text{id}_{\text{List}}} & [[p, q], [r], [], [r, q, q]] \\ \downarrow \text{id}_{\text{List}} \circ \mu & & \downarrow \mu \\ [p, q, r], [r, q, q] & \xrightarrow{\mu} & [p, q, r, r, q, q] \end{array}$$

Exercise 5.3.2.3. Let $\mathbb{P}: \mathbf{Set} \rightarrow \mathbf{Set}$ be the powerset functor, so that given a function $f: X \rightarrow Y$ the function $\mathbb{P}(f): \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$ is given by taking images.

- a.) Make sense of the following statement: “with η defined by singleton subsets and with μ defined by union, $\top := (\mathbb{P}, \eta, \mu)$ is a monad”.

- b.) With $X = \{a, b\}$, write down the function η_X as a 2-row, 2-column table, and write down the function μ_X as a 16-row, 2-column table (you can stop after 5 rows if you fully get it).
- c.) Check that you believe the monad laws from Definition 5.3.2.1.

◇

Example 5.3.2.4. Here is the monad for partial functions. The functor $T: \mathbf{Set} \rightarrow \mathbf{Set}$ sends a set X to the set $X \sqcup \{\odot\}$. Clearly, given a function $f: X \rightarrow Y$ there is an induced function $f \sqcup \{\odot\}: X \sqcup \{\odot\} \rightarrow Y \sqcup \{\odot\}$, so this is a functor. The natural transformation $\eta: \text{id} \rightarrow T$ is given on a set X by the component function

$$\eta_X: X \rightarrow X \sqcup \{\odot\}$$

that includes $X \hookrightarrow X \sqcup \{\odot\}$. Finally, the natural transformation $\mu: T \circ T \rightarrow T$ is given on a set X by the component function

$$\mu_X: X \sqcup \{\odot\} \sqcup \{\odot\} \longrightarrow X \sqcup \{\odot\}$$

that collapses both copies of $\odot$.

Exercise 5.3.2.5. Let E be a set, elements we will refer to as *exceptions*. We imagine that a function $f: X \rightarrow Y$ either outputs a value or one of these exceptions, which might be things like “overflow!” or “division by zero!”, etc. Let $T: \mathbf{Set} \rightarrow \mathbf{Set}$ be the functor $X \mapsto X \sqcup E$. Follow Example 5.3.2.4 and come up with a unit map η and a multiplication map μ for which (T, η, μ) is a monad. ◇

Example 5.3.2.6. Fix a set A . Let $T: \mathbf{Set} \rightarrow \mathbf{Set}$ be given by $T(X) = X^A = \text{Hom}_{\mathbf{Set}}(A, X)$; this is a functor. For a set X , let $\eta_X: X \rightarrow T(X)$ be given by the constant function, $x \mapsto c_x: A \rightarrow X$ where $c_x(a) = x$ for all $a \in A$. To specify a function

$$\mu_X: \text{Hom}_{\mathbf{Set}}(A, T(X)) \rightarrow \text{Hom}_{\mathbf{Set}}(A, X),$$

we curry and need a function $A \times \text{Hom}_{\mathbf{Set}}(A, T(X)) \rightarrow X$. We have an evaluation function (see Exercise 2.7.2.5) $ev: A \times \text{Hom}_{\mathbf{Set}}(A, T(X)) \rightarrow T(X)$, and we have an identity function $\text{id}_A: A \rightarrow A$, so we have a function $(\text{id}_A \times ev): A \times \text{Hom}_{\mathbf{Set}}(A, T(X)) \rightarrow A \times T(X)$. Composing that with another evaluation function $A \times \text{Hom}_{\mathbf{Set}}(A, X) \rightarrow X$ yields our desired μ_X .

Remark 5.3.2.7. Monads can be defined on categories other than $\mathbf{Set}$. In fact, for any category $\mathcal{C}$ one can take Definition 5.3.2.1 and replace every occurrence of $\mathbf{Set}$ with $\mathcal{C}$ and obtain the definition for monads on $\mathcal{C}$. We have actually seen a monad $(\text{Paths}, \eta, \mu)$ on the category $\mathbf{Grph}$ of graphs before, namely in Examples 4.3.1.5 and 4.3.1.6. That is, $\text{Paths}: \mathbf{Grph} \rightarrow \mathbf{Grph}$, which sends a graph to its paths-graph is the functor part. The unit map η includes a graph into its paths graph using the observation that every arrow is a path of length 1. And the multiplication map μ concatenates paths of paths. The Kleisli category of this monad (see Definition 5.3.3.1) is used, e.g. in (4.14) to define morphisms of database schemas.

5.3.3 Kleisli category of a monad

Given a monad $\top := (T, \eta, \mu)$, we can form a new category $\mathbf{Kls}(\top)$.

Definition 5.3.3.1. Let $\top = (T, \eta, \mu)$ be a monad on **Set**. Form a new category, called the *Kleisli category for $\top$* , denoted $\mathbf{Kls}(\top)$, with sets as objects, $\text{Ob}(\mathbf{Kls}(\top)) := \text{Ob}(\mathbf{Set})$, and with

$$\text{Hom}_{\mathbf{Kls}(\top)}(X, Y) := \text{Hom}_{\mathbf{Set}}(X, T(Y))$$

for sets X, Y . The identity morphism $\text{id}_X: X \rightarrow X$ in $\mathbf{Kls}(\top)$ is given by $\eta: X \rightarrow T(X)$ in **Set**. The composition of morphisms $f: X \rightarrow Y$ and $g: Y \rightarrow Z$ in $\mathbf{Kls}(\top)$ is given as follows. Writing them as functions, we have $f: X \rightarrow T(Y)$ and $g: Y \rightarrow T(Z)$. The first step is to apply the functor T to g , giving $T(g): T(Y) \rightarrow T(T(Z))$. Then compose with f to get $T(g) \circ f: X \rightarrow T(T(Z))$. Finally, compose with $\mu_Z: T(T(Z)) \rightarrow T(Z)$ to get the required function $X \rightarrow T(Z)$. The associativity of this composition formula follows from the associativity law for monads.

Example 5.3.3.2. Recall the monad $\top$ for partial functions, $T(X) = X \sqcup \{\odot\}$, from Example 5.3.2.4. The Kleisli category $\mathbf{Kls}(\top)$ has sets as objects, but a morphism $f: X \rightarrow Y$ means a function $X \rightarrow Y \sqcup \{\odot\}$, i.e a partial function. Given another morphism $g: Y \rightarrow Z$, the composition formula in $\mathbf{Kls}(\top)$ ensures that $g \circ f: X \rightarrow Z$ has the appropriate behavior.

Note how this monad allows us to make explicit our assumption that all functions are partial, and then hide it away from our notation.

Remark 5.3.3.3. For any monad $\top = (T, \eta, \mu)$ on **Set**, there is a functor $i: \mathbf{Set} \rightarrow \mathbf{Kls}(\top)$ given as follows. On objects we have $\text{Ob}(\mathbf{Kls}(\top)) = \text{Ob}(\mathbf{Set})$, so take $i = \text{id}_{\text{Ob}(\mathbf{Set})}$. Given a morphism $f: X \rightarrow Y$ in **Set**, we need a morphism $i(f): X \rightarrow Y$ in $\mathbf{Kls}(\top)$, i.e. a function $i(f): X \rightarrow T(Y)$. We assign $i(f)$ to be the composite $X \xrightarrow{f} Y \xrightarrow{\eta} T(Y)$. The functoriality of this mapping follows from the unit law for monads.

The point is that any ordinary function (morphism in **Set**) has an interpretation as a morphism in the Kleisli category of any monad. More categorically, there is a functor $\mathbf{Set} \rightarrow \mathbf{Kls}(\top)$.

Example 5.3.3.4. In this example we return to the setting laid out by Application 5.3.1.2 where we had a set A of experimenters and assumed that the person doing the experiment may affect the outcome. We use the monad $\top = (T, \eta, \mu)$ from Example 5.3.2.6 and hope that $\mathbf{Kls}(\top)$ will conform to our understanding of how to manage the affect of the experimenter on data.

The objects of $\mathbf{Kls}(\top)$ are ordinary sets, but a map $f: X \rightarrow Y$ in $\mathbf{Kls}(\top)$ is a function $X \rightarrow Y^A$. By currying this is the same as a function $X \times A \rightarrow Y$, as desired. To compose f with $g: Y \rightarrow Z$ in $\mathbf{Kls}(\top)$, we follow the formula. It turns out to be equivalent to the following. We have a function $X \times A \rightarrow Y$ and a function $Y \times A \rightarrow Z$. Modifying the first slightly, we have a function $X \times A \rightarrow Y \times A$, by identity on A , and we can now compose to get $X \times A \rightarrow Z$.

What does this say in terms of experimenters affecting data gathering? It says that if we work within $\mathbf{Kls}(\top)$ then we will be able to assume that the experimenter is being taken into account; all proposed functions $X \rightarrow Y$ are actually functions $A \times X \rightarrow Y$. The natural way to compose these experiments is that we only consider the data from one experiment to feed into another if the experimenter is the same in both experiments.

8

⁸This requirement seems a bit stringent, but it can be mitigated in a variety of ways. One such way is to notice that by Remark 5.3.3.3 that we have not added any requirement, because any old way of doing business yields a valid new way of doing business (we just say “every experimenter would get the same result”). Another way would be to hand off the experiment results to another person, who could carry it forward (see Example 5.3.3.8).

Exercise 5.3.3.5. In Exercise 5.3.2.3 we discussed the power set monad $\mathbb{T} = (\mathbb{P}, \eta, \mu)$.

- a.) Can you find a way to relate the morphisms in $\mathbf{Kls}(\mathbb{T})$ to relations? That is, given a morphism $f: A \rightarrow B$ in $\mathbf{Kls}(\mathbb{T})$, is there a natural way to associate to it a relation $R \subseteq A \times B$?
- b.) How does the composition formula in $\mathbf{Kls}(\mathbb{T})$ relate to the composition of relations given in Definition 2.5.2.3? ⁹

◇

Exercise 5.3.3.6. Let $\mathbb{T} = (\mathbb{P}, \eta, \mu)$ be the power set monad. The category $\mathbf{Kls}(\mathbb{T})$ is closed under binary products, i.e. every pair of objects $A, B \in \mathbf{Ob}(\mathbf{Kls}(\mathbb{T}))$ have a product in $\mathbf{Kls}(\mathbb{T})$. What is the product of $A = \{1, 2, 3\}$ and $B = \{a, b\}$? ◇

Exercise 5.3.3.7. Let $\mathbb{T} = (\mathbb{P}, \eta, \mu)$ be the power set monad. The category $\mathbf{Kls}(\mathbb{T})$ is closed under binary coproducts, i.e. every pair of objects $A, B \in \mathbf{Ob}(\mathbf{Kls}(\mathbb{T}))$ have a coproduct in $\mathbf{Kls}(\mathbb{T})$. What is the coproduct of $A = \{1, 2, 3\}$ and $B = \{a, b\}$? ◇

Example 5.3.3.8. Suppose given a preorder A , which we speak of throughout this example as though it was the linear order given by time. There is a monad $\mathbb{T} = (T, \eta, \mu)$ that captures the idea that a function $f: X \rightarrow Y$ occurs in the context of time in the following sense: The output of f is determined not only by the element $x \in X$ on which it is applied but also by the time at which it was applied to x ; and the output of f occurs at another time, which is not before the time of input.

The functor part of the monad is given on $X \in \mathbf{Ob}(\mathbf{Set})$ by

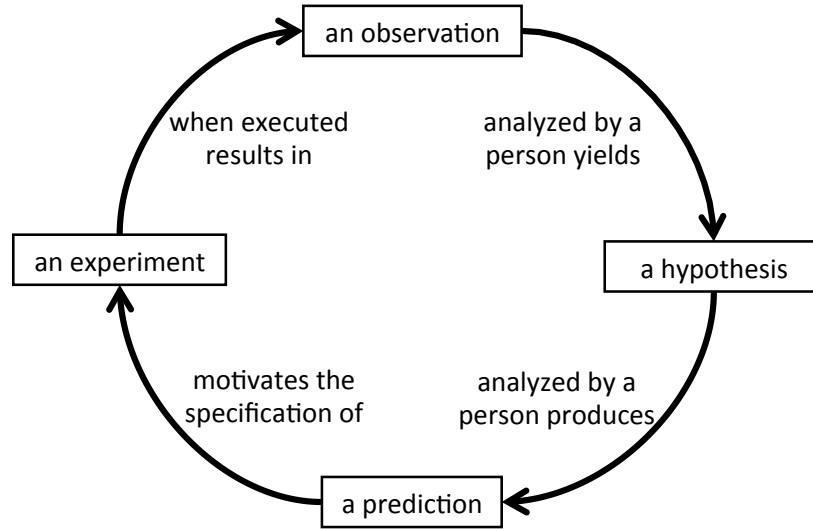
$$T(X) = \{p: A \rightarrow A \times X \mid \text{if } p(a) = (a', x) \text{ then } a' \geq a\}.$$

The unit $\eta_X: X \rightarrow T(X)$ sends x to the function $a \mapsto (a, x)$. The multiplication map $\mu_X: T(T(X)) \rightarrow T(X)$ is roughly described as follows. If for every $a \in A$ you have a later element $a' \geq a$ and a function $p: A \rightarrow A \times X$ that takes elements of A to later elements of A and values of X , then $p(a')$ is a still later element of A and a value of X , as desired.

Morphisms in the Kleisli category $\mathbf{Kls}(\mathbb{T})$ can be curried to look like $f: A \times X \rightarrow A \times Y$ such that if $f(a, x) = (a', y)$ then $a' \geq a$.

Example 5.3.3.9. Here we reconsider the image from the front cover of this book, reproduced here.

⁹Actually, Definition 2.5.2.3 is about composing spans, but a relation $R \subseteq A \times B$ is a kind of span, $R \rightarrow A \times B$.



It looks like an olog, and all ologs are database schemas (see Section 3.5.2.14). But how is “analyzed by a person yields” a function from observations to hypotheses? The very name belies the fact that it is an invalid aspect in the sense of Section 2.3.2.1, because given an observation there may be more than one hypothesis yielded, corresponding to which person is doing the observing. In fact, all of the arrows in this diagram correspond to some hidden context involving people: the prediction is dependent on who analyzes the hypothesis, the specification of an experiment is dependent on who is motivated to specify it, and experiments may result in different observations by different observers.

Without monads, the model of science proposed by this olog would be difficult to believe in. But by choosing a monad we can make explicit (and then hide from discourse) our implicit assumption that “of course everything is dependent on which human is doing the science”. The choice of monad is an additional modeling choice. Do we want to incorporate the partial order of time? Do we want the scientist to be modified by each function (i.e. the person is changed when analyzing an observation to yield a hypothesis)? These are all interesting possibilities.

One reasonable choice would be to use a simplification of the monad from Example 5.3.3.8 in which the preorder A is the set of scientific models with the indiscrete preorder. This implies the following context: every morphism $f: X \rightarrow Y$ is really a morphism $f: X \times A \rightarrow Y \times A$; while ostensibly giving a map from X to Y , it is influenced by the scientific model under which it is performed, and its outcome yields a new scientific model.

Reading the olog in this context might look like this:

A hypothesis (in the presence of a scientific model) analyzed by a person produces a prediction (in the presence of a scientific model), which motivates the specification of an experiment (in the presence of a scientific model), which when executed results in an observation (in the presence of a scientific model), which analyzed by a person yields a hypothesis (in the presence of a scientific model).

The parenthetical statements can be removed if we assume them to always be around, which can be done using the monad above.

5.3.4 Monads in databases

In this section we discuss how to record data in the presence of a monad. The idea is quite simple. Given a schema (category) $\mathcal{C}$, an ordinary instance is a functor $I: \mathcal{C} \rightarrow \mathbf{Set}$. But if $\top = (T, \eta, \mu)$ is a monad, then a *Kleisli $\top$ -instance on $\mathcal{C}$* is a functor $J: \mathcal{C} \rightarrow \mathbf{Kls}(\top)$. Such a functor associates to every object $c \in \text{Ob}(\mathcal{C})$ a set $J(c)$, and to every arrow $f: c \rightarrow c'$ in $\mathcal{C}$ a morphism $J(f): J(c) \rightarrow J(c')$ in $\mathbf{Kls}(\top)$. How does this look in terms of tables?

Recall that to represent an ordinary database instance $I: \mathcal{C} \rightarrow \mathbf{Set}$, we use a tabular format in which every object $c \in \text{Ob}(\mathcal{C})$ is displayed as a table including one ID column and an additional column for every arrow emanating from c . In the ID column of table c were elements of the set $I(c)$ and in the column assigned to some arrow $f: c \rightarrow c'$ the cells were elements of the set $I(c')$.

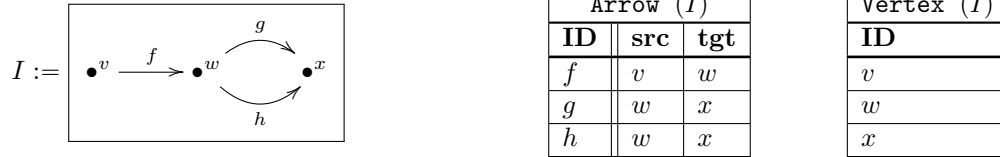
To represent a *Kleisli* database instance $J: \mathcal{C} \rightarrow \mathbf{Kls}\top$ is similar; we again use a tabular format in which every object $c \in \text{Ob}(\mathcal{C})$ is displayed as a table including one ID column and an additional column for every arrow emanating from c . In the ID column of table c are again elements of the set $J(c)$; however in the column assigned to some arrow $f: c \rightarrow c'$ are not elements of $J(c')$ but T -values in $J(c')$, i.e. elements of $T(J(c'))$.

Example 5.3.4.1. Let $\top = (T, \eta, \mu)$ be the monad for partial functions, as discussed in Example 5.3.1.1. Given any schema $\mathcal{C}$, we can represent a Kleisli $\top$ -instance $J: \mathcal{C} \rightarrow \mathbf{Kls}(\top)$ in tabular format. To every object $c \in \text{Ob}(\mathcal{C})$ we'll have a set $J(c)$ of rows, and given a column $c \rightarrow c'$ every row will produce either a value in $J(c')$ or fail to produce a value; this is the essence of partial functions. We might denote the absence of a value using $\odot$.

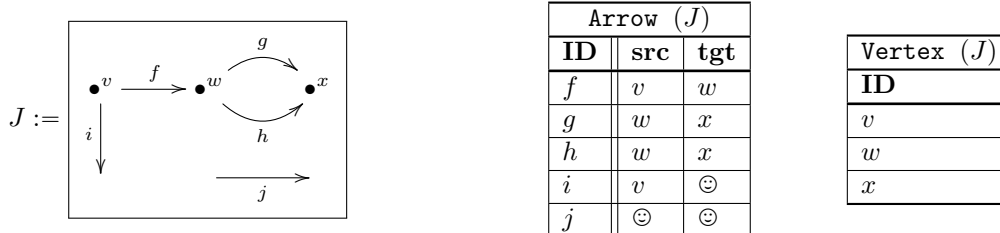
Consider the schema indexing graphs

$$\mathcal{C} := \begin{array}{ccc} \text{Arrow} & \xrightarrow[\text{tgt}]{\text{src}} & \text{Vertex} \\ \bullet & & \bullet \end{array}$$

As we discussed in Section 4.2.1.19, an ordinary instance on $\mathcal{C}$ represents a graph.



A Kleisli $\top$ -instance on $\mathcal{C}$ represents graphs in which edges can fail to have a source vertex, fail to have a target vertex, or both.



The context of these tables is that of partial functions, so we do not need a reference for $\odot$ in the vertex table. Mathematically, the morphism $J(\text{src}): J(\text{Arrow}) \rightarrow J(\text{Vertex})$ needs to be a function $J(\text{Arrow}) \rightarrow J(\text{Vertex}) \sqcup \{\odot\}$, and it is.

5.3.4.2 Probability distributions

Let $[0, 1] \subseteq \mathbb{R}$ denote the set of real numbers between 0 and 1. Let X be a set and $p: X \rightarrow [0, 1]$ a function. We say that p is a *finitary probability distribution on X* if there exists a finite subset $W \subseteq X$ such that

$$\sum_{w \in W} p(w) = 1, \quad (5.7)$$

and such that $p(x) > 0$ if and only $x \in W$. Note that W is unique if it exists; we call it *the support of p* and denote it $\mathbf{Supp}(p)$. Note also that if X is a finite set then every function p satisfying (5.7) is a finitary probability distribution on X .

For any set X , let $\mathbf{Dist}(X)$ denote the set of finitary probability distributions on X . It is easy to check that given a function $f: X \rightarrow Y$ one obtains a function $\mathbf{Dist}(f): \mathbf{Dist}(X) \rightarrow \mathbf{Dist}(Y)$ by $\mathbf{Dist}(f)(y) = \sum_{f(x)=y} p(x)$. Thus we can consider $\mathbf{Dist}: \mathbf{Set} \rightarrow \mathbf{Set}$ as a functor, and in fact the functor part of a monad. Its unit $\eta: X \rightarrow \mathbf{Dist}(X)$ is given by the Kronecker delta function $x \mapsto \delta_x$ where $\delta_x(x) = 1$ and $\delta_x(x') = 0$ for $x' \neq x$. Its multiplication $\mu: \mathbf{Dist}(\mathbf{Dist}(X)) \rightarrow \mathbf{Dist}(X)$ is given by weighted sum: given a finitary probability distribution $w: \mathbf{Dist}(X) \rightarrow [0, 1]$ and $x \in X$, put $\mu(w)(x) = \sum_{p \in \mathbf{Supp}(w)} w(p)p(x)$.

Example 5.3.4.3 (Markov chains). Let $\mathcal{Loop}$ be the loop schema,

$$\mathcal{Loop} := \boxed{\begin{array}{c} f \\ \curvearrowright \\ s \end{array}}$$

as in Example 3.5.2.9. A $\mathbf{Dist}$ -instance on $\mathcal{Loop}$ is equivalent to a time-homogeneous Markov chain. To be explicit, a functor $\delta: \mathcal{Loop} \rightarrow \mathbf{KlsDist}$ assigns to the unique object $s \in \mathbf{Ob}(\mathcal{Loop})$ a set $S = \delta(s)$, which we call the state space, and to $f: s \rightarrow s$ a function $\delta(f): S \rightarrow \mathbf{Dist}(S)$, which sends each element $x \in S$ to some probability distribution on elements of S . For example, the table δ on the left corresponds to the Markov matrix M on the right below:

		s
	ID	f
$\delta :=$	1	.5(1)+.5(2)
	2	1(2)
	3	.7(1)+.3(3)
	4	.4(1)+.3(2)+.3(4)

$$M := \begin{pmatrix} 0.5 & 0.5 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0.7 & 0 & 0.3 & 0 \\ 0.4 & 0.3 & 0 & 0.3 \end{pmatrix} \quad (5.8)$$

As one might hope, for any natural number $n \in \mathbb{N}$ the map $f^n: S \rightarrow \mathbf{Dist}(S)$ corresponds to the matrix M^n , which sends an element in S to its probable location after n iterations of the transition map.

Application 5.3.4.4. Every star **emits a spectrum of light**, which can be understood as a distribution on the electromagnetic spectrum. Given an object B on earth, different parts of B will **absorb radiation** at different rates. Thus B produces a function from the electromagnetic spectrum to distributions of energy absorption. In the context of the probability distributions monad, we can record data on the schema

$$\bullet \xrightarrow{\text{emits}} \bullet \xrightarrow{\text{absorbed by } B} \bullet$$

star wavelengths energies

The composition formula for Kleisli categories is the desired one: to each star we associate the weighted sum of energy absorption rates over the set of wavelengths emitted by the star.

◇◇

5.4 Operads

In this section we briefly introduce operads, which are generalizations of categories. They often are useful for speaking about self-similarity of structure. For example, we will use them to model agents made up of smaller agents, or materials made up of smaller materials. This association with self-similarity is not really inherent in the definition, but it tends to emerge in our thinking about many operads used in practice.

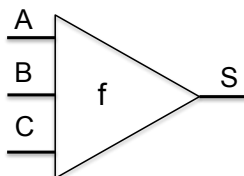
Let me begin with a warning.

Warning 5.4.0.5. My use of the term operad is not entirely standard and conflicts with widespread usage. The more common term for what I am calling an operad is *multicategory* or *colored operad*. An operad classically is a multicategory with one object, and a colored operad is a multicategory. The analogy is that “operad is to multicategory as monoid is to category”. The term multicategory stems from the fact that the morphisms in a multicategory have many, rather than one, input. But there is nothing really “multi” about the multicategory itself, only its morphisms. Probably the real reason though is that I find the term multicategory to be clunky and the term operad to be sleek, clocking in at half the syllables. I apologize if my break with standard terminology causes any confusion.

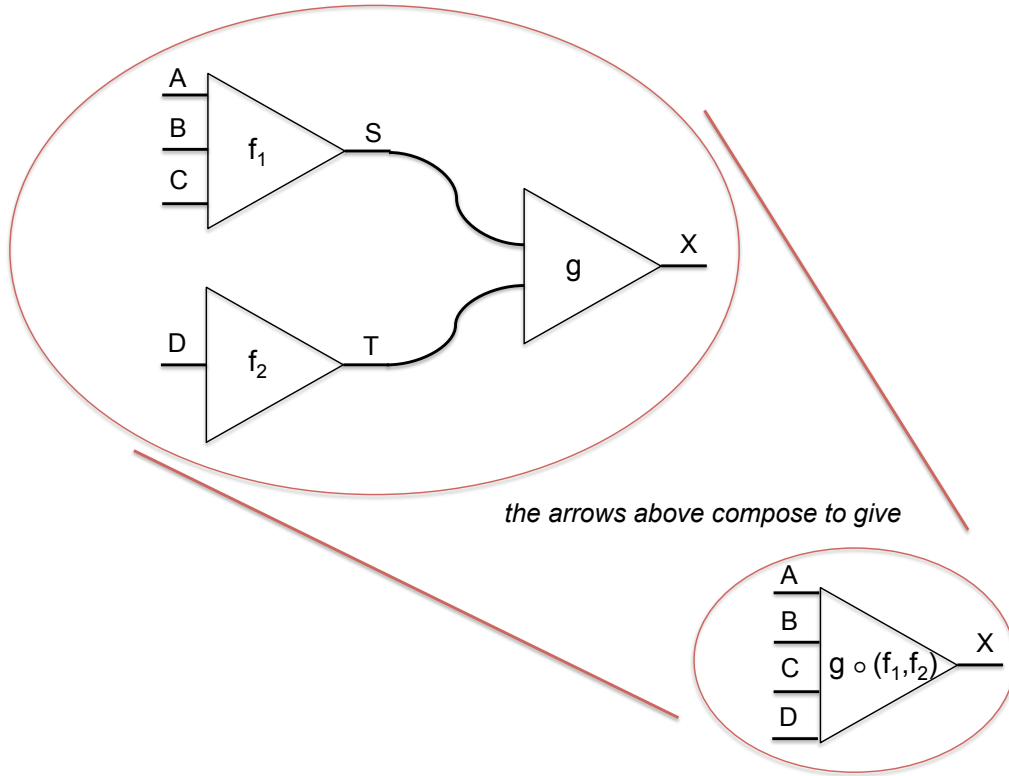
This introduction to operads is quite short. One should see [Le1] for an excellent treatment.

5.4.1 Definition and classical examples

An operad is like a category in that it has objects, morphisms, and a composition formula, and it follows an identity law and an associativity law. The difference is that each morphism has many inputs (and one output).



The description of composition in an operad is a bit heavier than it is in a category, but the idea fairly straightforward. Here is a picture of morphisms being composed.



Note that S and T disappear from the composition, but this is analogous to the way the middle object disappears from the composition of morphisms in a category

$$\boxed{A \xrightarrow{f} S \xrightarrow{g} X} \quad \text{the arrows to the left compose to give} \quad \boxed{A \xrightarrow{g \circ f} X}$$

Here is the definition.

Definition 5.4.1.1. An *operad* $\mathcal{O}$ consists of some constituents (A. objects, B. morphisms, C. identities, D. compositions) conforming to some laws (1. identity law, 2. associativity law), as follows:

- A. A collection $\text{Ob}(\mathcal{O})$, elements of which are called *objects*.
- B. For each natural number $n \in \mathbb{N}$ and sequence $(x_1, \dots, x_n, y)$ of objects in $\text{Ob}(\mathcal{O})$, a set $\text{Hom}_{\mathcal{O}}(x_1, \dots, x_n; y) \in \mathbf{Set}$. Its elements are called *morphisms* from $x_1, \dots, x_n$ to y .
- C. For every object $x \in \text{Ob}(\mathcal{O})$, a specified morphism denoted $\text{id}_x \in \text{Hom}_{\mathcal{O}}(x; x)$ called *the identity morphism on x* .
- D. For every natural number $n \in \mathbb{N}$, natural numbers $m_1, \dots, m_n \in \mathbb{N}$, sequences $(x_{1,1}, x_{1,2}, \dots, x_{1,m_1}, y_1)$ through $(x_{n,1}, x_{n,2}, \dots, x_{n,m_n}, y_n)$, and element z (all in $\text{Ob}(\mathcal{O})$), a function

$$\begin{aligned} \circ: \text{Hom}_{\mathcal{O}}(y_1, \dots, y_n; z) \times \prod_{i=1}^n \text{Hom}_{\mathcal{O}}(x_{i,1}, \dots, x_{i,m_i}; y_i) \\ \longrightarrow \text{Hom}_{\mathcal{O}}(x_{1,1}, x_{1,2}, \dots, x_{n,m_n}; z), \end{aligned}$$

called *the composition formula*.

Given objects $x_1, \dots, x_n, y \in \text{Ob}(\mathcal{C})$, we can denote a morphism $f \in \text{Hom}_{\mathcal{C}}(x_1, \dots, x_n; y)$ by $f: (x_1, \dots, x_n) \rightarrow y$; we say that each x_i is a *domain object* of f and that y is the *codomain object* of f . We use infix notation for the composition formula, such as $g \circ (f_1, \dots, f_n)$.

Here are the laws that harmonize the above constituents:

1. for every $x_1, \dots, x_n, y \in \text{Ob}(\mathcal{C})$ and every morphism $f: (x_1, \dots, x_n) \rightarrow y$, we have

$$f \circ (\text{id}_{x_1}, \dots, \text{id}_{x_n}) = f \quad \text{and} \quad \text{id}_y \circ f = f;$$

and;

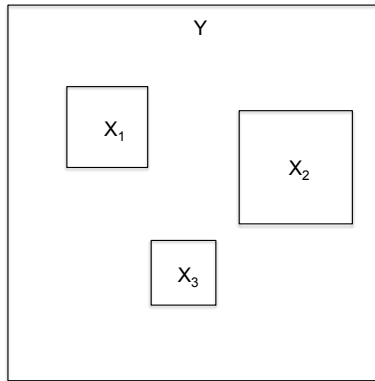
2. an appropriate associative law (see [Le1, Definition 2.1.1]).

Example 5.4.1.2. Let **Sets** denote the operad defined as follows. For objects we put $\text{Ob}(\mathbf{Sets}) = \text{Ob}(\mathbf{Set})$. For a natural number $n \in \mathbb{N}$ and sets $X_1, \dots, X_n, Y$, put

$$\text{Hom}_{\mathbf{Sets}}(X_1, \dots, X_n; Y) := \text{Hom}_{\mathbf{Set}}(X_1 \times \dots \times X_n, Y).$$

Given functions $f_1: (X_{1,1} \times \dots \times X_{1,m_1}) \rightarrow Y_1$ through $f_n: (X_{n,1} \times \dots \times X_{n,m_n}) \rightarrow Y_n$ and a function $Y_1 \times \dots \times Y_n \rightarrow Z$, the universal property provides us a unique function of the form $(X_{1,1} \times \dots \times X_{n,m_n}) \rightarrow Z$, giving rise to our composition formula.

Example 5.4.1.3. An operad commonly used in mathematics is called the *little n -cubes operad*. We'll focus on $n = 2$ and talk about the little squares operad $\mathcal{O}$. Here the set of objects consists of only one object, $\text{Ob}(\mathcal{O}) = \{\square\}$, which we denote by a square. For a natural number $n \in \mathbb{N}$, a morphism $f: (\square, \square, \dots, \square) \rightarrow \square$ is a positioning of n non-overlapping squares inside of a square. Here is a picture of a morphism $(X_1, X_2, X_3) \rightarrow Y$, where $X_1 = X_2 = X_3 = Y = \square$.



The composition law says that given a positioning of small squares inside a large square, and given a positioning of tiny squares inside each of those small squares, we get a positioning of tiny squares inside a large square. A picture is shown in Figure 5.9.

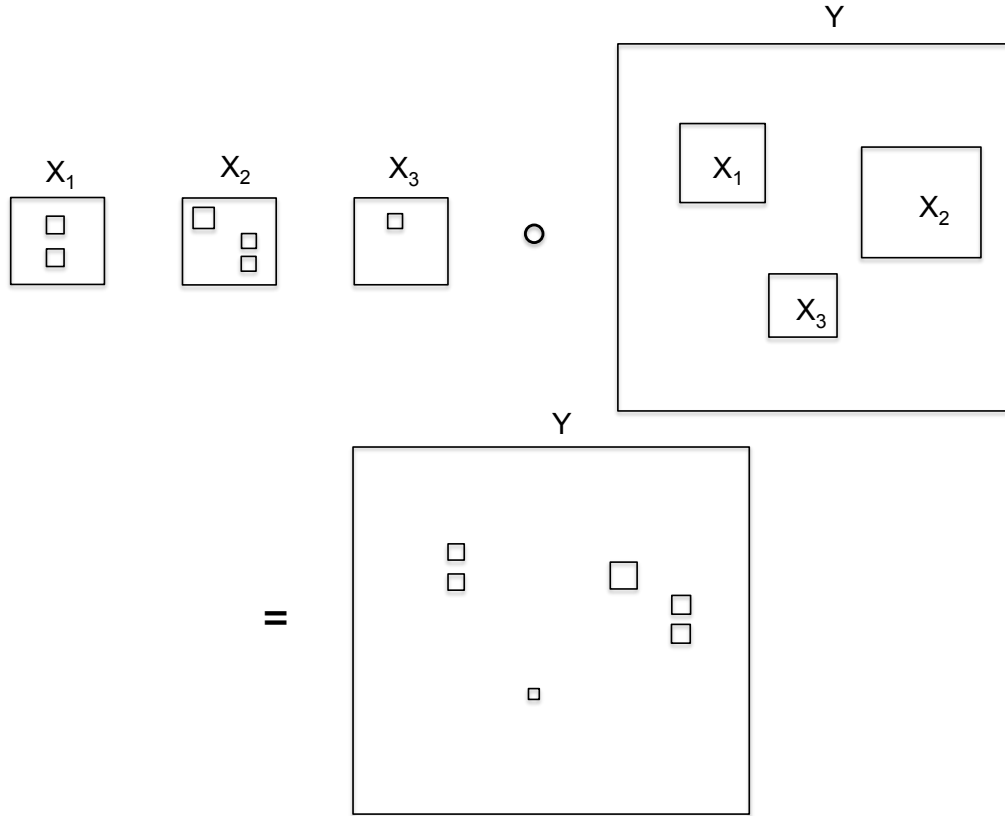


Figure 5.9: Here we show a morphism $(X_1, X_2, X_3) \rightarrow Y$ and morphisms $(W_{1,1}, W_{1,2}) \rightarrow X_1$, $(W_{2,1}, W_{2,2}, W_{2,3}) \rightarrow X_2$, and $(W_{3,1}) \rightarrow X_3$, each of which is a positioning of squares inside a square. The composition law scales and positions the squares in the “obvious” way.

Hopefully, what we meant by “self-similarity” in the introduction to this section (see page 212) is becoming clear.

Exercise 5.4.1.4. Consider an operad $\mathcal{O}$ like the little squares operad from Example 5.4.1.3, except with three objects: square, circle, equilateral triangle. A morphism is again a positioning of shapes inside of a shape.

- Draw an example of a morphism f from two circles and a square to a triangle.
- Find three other morphisms that compose into f , and draw the composite.

◇

5.4.1.5 Operads: morphisms and algebras

If operads are like categories, then we can define things like functors and call such a thing a *morphism of operads* (like a functor is a morphism of categories). We sketch the definition here.

Definition 5.4.1.6. Let $\mathcal{O}$ and $\mathcal{O}'$ be operads. A *morphism of operads from $\mathcal{O}$ to $\mathcal{O}'$* , denoted $F: \mathcal{O} \rightarrow \mathcal{O}'$ consists of some constituents (A. on-objects part, B. on-morphisms part) conforming to some laws (1. preservation of identities, 2. preservation of composition), as follows:

- A. a function $\text{Ob}(F): \text{Ob}(\mathcal{O}) \rightarrow \text{Ob}(\mathcal{O}')$; and
- B. for every sequence of objects $x_1, \dots, x_n, y \in \text{Ob}(\mathcal{O})$, a function

$$\text{Hom}_F(x_1, \dots, x_n; y): \text{Hom}_{\mathcal{O}}(x_1, \dots, x_n; y) \rightarrow \text{Hom}_{\mathcal{O}'}(F(c), F(d)),$$

which we sometimes denote simply by $F: \text{Hom}_{\mathcal{O}}(c, d) \rightarrow \text{Hom}_{\mathcal{O}'}(F(c), F(d))$.

The following laws harmonize the above constituents:

- 1. Identities are preserved by F . That is, for any object $c \in \text{Ob}(\mathcal{O})$, we have $F(\text{id}_c) = \text{id}_{F(c)}$; and
- 2. Composition is preserved by F . That is, for any objects $b_{i,j}, c_i, d \in \text{Ob}(\mathcal{O})$ and morphisms $g_i: b_{i,j} \rightarrow c_i$ and $h: c_i \rightarrow d$, we have $F(h \circ g_i) = F(h) \circ F(g_i)$.¹⁰

Exercise 5.4.1.7. Let $\mathcal{O}$ denote the little squares operad from Example 5.4.1.3 and let $\mathcal{O}'$ denote the operad you constructed in Exercise 5.4.1.4.

- a.) Can you come up with an operad morphism $\mathcal{O} \rightarrow \mathcal{O}'$?
- b.) What about an operad morphism $\mathcal{O}' \rightarrow \mathcal{O}$?

◇

Definition 5.4.1.8. Let $\mathcal{O}$ be an operad. An *algebra on $\mathcal{O}$* is a morphism of operads $A: \mathcal{O} \rightarrow \mathbf{Sets}$.

Remark 5.4.1.9. Every category can be construed as an operad (yes, there is a functor $\mathbf{Cat} \rightarrow \mathbf{Oprd}$), by simply not including non-unary morphisms. That is, given a category $\mathcal{C}$, one makes an operad $\mathcal{O}$ with $\text{Ob}(\mathcal{O}) := \text{Ob}(\mathcal{C})$ and with

$$\text{Hom}_{\mathcal{O}}(x_1, \dots, x_n; y) = \begin{cases} \text{Hom}_{\mathcal{C}}(x_1, y) & \text{if } n = 1; \\ \emptyset & \text{if } n \neq 1 \end{cases}$$

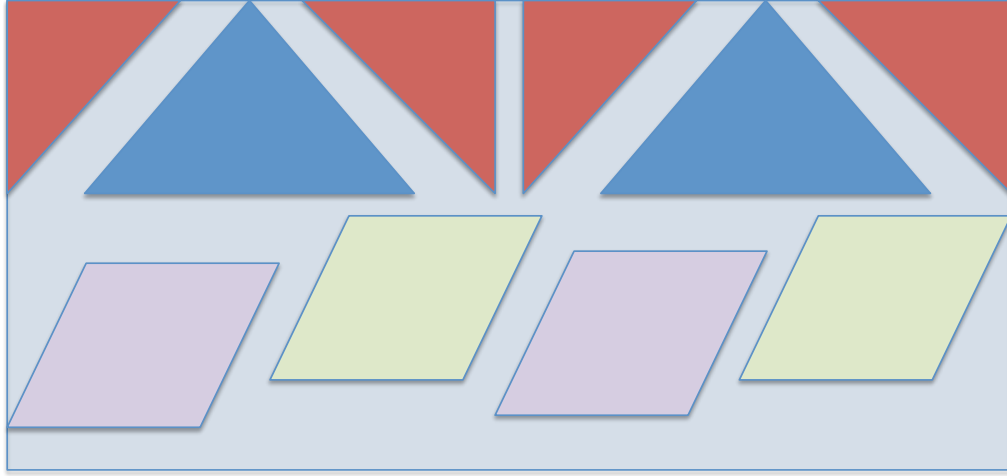
Just like a schema is a category presentation, it is possible to discuss operad presentations by generators and relations. Under this analogy, an algebra on an operad corresponds to an instance on a schema.

5.4.2 Applications of operads and their algebras

Hierarchical structures may be well-modeled by operads. Describing such structures using operads and their algebras allows one to make appropriate distinctions between different types of thinking. For example, the allowable formations are encoded in the operad, whereas the elements that will fit into those formations are encoded in the algebra. Morphisms of algebras are high-level understandings of how elements of very different types (such as materials vs. numbers) can occupy the same place in the structure and be compared. We will give examples below.

¹⁰Here is how the indexing works. Choose $n \in \mathbb{N}$, and for each $i \in \underline{n}$ choose $m_i \in \mathbb{N}$. Then for each $i \in \underline{n}$ we have a morphism $g_i: (b_{i,1}, \dots, b_{i,m_i}) \rightarrow c_i$ and we also have a morphism $h: (c_1, \dots, c_n) \rightarrow d$. The law here is that $F(h \circ (g_1, \dots, g_n)) = F(h) \circ (F(g_1), \dots, F(g_n))$.

Application 5.4.2.1. Every material is composed of constituent materials, arranged in certain patterns. (In case the material is “pure”, we consider the material to consist of itself as the sole constituent.) Each of these constituent materials each is itself an arrangement of constituent materials. Thus we see a kind of self-similarity which we can model with operads.



(5.10)

For example, a tendon is made of collagen fibers that are assembled in series and then in parallel, in a specific way. Each collagen fibre is made of collagen fibrils that are again assembled in series and then in parallel, with slightly different specifications. We can continue down, perhaps indefinitely, though our resolution fails at some point. A collagen fibril is made up of tropocollagen collagen molecules, which are twisted ropes of collagen molecules, etc.¹¹

Here is how operads might be employed. We want the same operad to model both actual materials, theoretical materials, and functional properties; that is we want more than one algebra on the same operad.

The operad $\mathcal{O}$ should abstractly model the structure, but not the substance being structured. Imagine that each of the shapes (including the background “shape”) in Diagram (5.10) is a place-holder, saying something like “*your material here*”. Each morphism (that’s what (5.10) is a picture of) represents a construction of a material out of parts. In our picture, it appears we are only concerned with the spacial arrangements, but there is far more flexibility than that. Whether we want to allow for additional details beyond spacial arrangements is the kinds of choice we make in a meeting called “what operad should we use?”

◇◇

Application 5.4.2.2. Suppose we have chosen an operad $\mathcal{O}$ to model the structure of materials. Each object of $\mathcal{O}$ might correspond to a certain quality of material, and each morphism corresponds to an arrangement of various qualities to form a new quality. An algebra $A: \mathcal{O} \rightarrow \mathbf{Sets}$ on $\mathcal{O}$ forces us to choose what substances will fill in for these qualities. For every object $x \in \text{Ob}(\mathcal{O})$, we want a set $A(x)$ which will be the set of

¹¹Thanks to Sandra Shefelbine for explaining the hierarchical nature of collagen to me. Any errors are my own.

materials with that quality. For every arrangement, i.e. morphism, $f: (x_1, \dots, x_n) \rightarrow y$, and every choice $a_1 \in A(x_1), \dots, a_n \in A(x_n)$ of materials, we need to understand what material $a' = A(f)(a_1, \dots, a_n) \in A(y)$ will emerge when these materials are arranged in accordance with f . We are really pinning ourselves down here.

But there may be more than one interesting algebra on $\mathcal{O}$. Suppose that $B: \mathcal{O} \rightarrow \mathbf{Sets}$ is an algebra of strengths rather than materials. For each object $x \in \text{Ob}(\mathcal{O})$, which represents some quality, we let $B(x)$ be the set of possible strengths that something of quality x can have. Then for each arrangement, i.e. morphism, $f: (x_1, \dots, x_n) \rightarrow y$, and every choice $b_1 \in B(x_1), \dots, b_n \in B(x_n)$ of strengths, we need to understand what strength $b' = B(f)(b_1, \dots, b_n) \in B(y)$ will emerge when these strengths are arranged in accordance with f . Certainly an impressive achievement!

Finally, a morphism of algebras $S: A \rightarrow B$ would consist of a coherent system for assigning to each material $a \in A(X)$ of a given quality x a specific strength $S(a) \in B(X)$, in such a way that morphisms behaved appropriately. In this language we have stated a very precise goal for the field of material mechanics.

◇◇

5.4.2.3 Wiring diagrams

Example 5.4.2.4. Here we describe an *operad of relations*, which we will denote by $\mathcal{R}$. The objects are sets, $\text{Ob}(\mathcal{R}) = \text{Ob}(\mathbf{Set})$. A morphism $f: (x_1, x_2, \dots, x_n) \rightarrow x'$ in $\mathcal{R}$ is a diagram in $\mathbf{Set}$ of the form

$$\begin{array}{c} & & R & & \\ & \swarrow f_1 & \downarrow & \searrow f' & \\ x_1 & & x_2 & \cdots & x_n & & x' \end{array} \quad (5.11)$$

such that the induced function $R \rightarrow (x_1 \times x_2 \times \cdots \times x_n \times x')$ is an injection.

We use a composition formula similar to that in Definition 2.5.2.3. Namely, we form a fiber product

$$\begin{array}{ccccc} & & FP & & \\ & \swarrow & & \searrow & \\ & \prod_{i \in \underline{n}} R_i & & S & \\ \swarrow & & \searrow & \swarrow & \searrow \\ \prod_{i \in \underline{n}} \prod_{j \in \underline{m_i}} x_{i,j} & & \prod_{i \in \underline{n}} y_i & & z \end{array}$$

One can show that the induces function $FP \rightarrow \left(\prod_{i \in \underline{n}} \prod_{j \in \underline{m_i}} x_i \right) \times y$ is an injection, so we have a valid composition formula. Finally, the associativity and identity laws hold.

¹²

Application 5.4.2.5. Suppose we are trying to model **life** in the following way. We define an entity as a set of phenomena, but in order to use colloquial language we say the entity *is able to experience* that set of phenomena. We also want to be able to put entities

¹²Technically we need to use isomorphism classes of cone points, but we don't worry about this here.

together to form a super-entity, so we have a notion of morphism $f: (e_1, \dots, e_n) \longrightarrow e'$ defined as a relation as in (5.11). The idea is that the morphism f is a way of translating between the phenomena that may be experienced by the sub-entities and the phenomena that may be experienced by the super-entity.

The operad $\mathcal{R}$ from Example 5.4.2.4 becomes useful as a language for discussing issues in this domain. $\diamond\diamond$

Example 5.4.2.6. Let $\mathcal{R}$ be the operad of relations from Example 5.4.2.4. Consider the algebra $S: \mathcal{R} \rightarrow \mathbf{Sets}$ given by $S(x) = \mathbb{P}(x)$. Given a morphism $\prod_i x_i \leftarrow R \rightarrow y$ and subsets $x'_i \subseteq x_i$, we have a subset $\prod_i x'_i \subseteq \prod_i x_i$. We take the fiber product

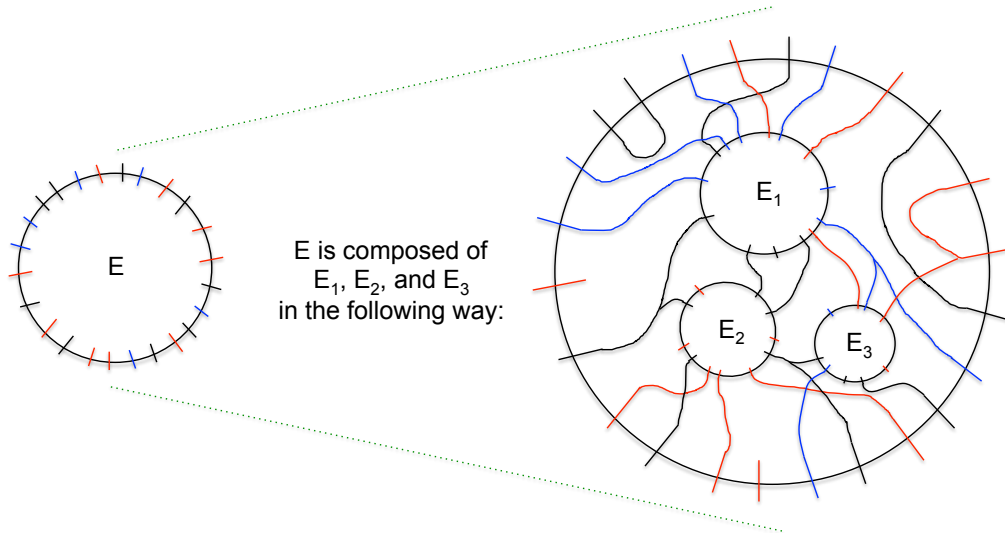
$$\begin{array}{ccc} & FP & \longrightarrow R \\ & \swarrow & \searrow \\ \prod_i x'_i & \longrightarrow & \prod_i x_i \end{array} \quad \begin{array}{c} \\ \\ y \end{array}$$

and the image of $FP \rightarrow y$ is a subset of y .

Application 5.4.2.7. Following Application 5.4.2.5 we can use Example 5.4.2.6 as a model of survival. Each entity survives only for a subset of the phenomena that it can experience. Under this interpretation, the algebra from Example 5.4.2.6 defines survival as the survival of all parts. That is, suppose that we understand how a super-entity is composed of sub-entities in the sense that we have a translation between the set of phenomena that may be experienced across the sub-entities and the set of phenomena that may be experienced by the super-entity. Then the super-entity will survive exactly those phenomena which translate to phenomena for which each sub-entity desires.

Perhaps a better term than survival would be “allowance”. A bureaucracy consists of a set of smaller bureaucracies, each of which allows certain phenomena to pass; the whole bureaucracy allows something to pass if and only if, when translated to the perspective of each sub-bureaucracy, it is allowed to pass there. $\diamond\diamond$

Example 5.4.2.8. In this example we discuss wiring diagrams that look like this:



The operad in question will be denoted $\mathcal{W}$. Its objects are pairs (C, s) where C is a finite set and $v: C \rightarrow \mathbf{Ob}(\mathbf{Set})$ is a function. Think of such an object as a circle with C -many

cables sticking out of it; each cable c is assigned a set $v(c)$ corresponding to the set of values that can be carried on that cable. For example $E_2 = (C, v)$ where $|C| = 11$ and we consider v to be specified by declaring that black wires carry $\mathbb{Z}$ and red wires carry {sweet, sour, salty, bitter, umami}.

The morphisms in $\mathcal{W}$ will be pictures as above, formalized as follows. Given objects $(C_1, v_1), \dots, (C_n, v_n), (D, w)$, a morphism $F: ((C_1, v_1), \dots, (C_n, v_n)) \rightarrow (D, w)$ is a commutative diagram of sets ¹³

$$\begin{array}{ccccc} \bigsqcup_{i \in n} C_i & \xrightarrow{i} & G & \xleftarrow{j} & D \\ & \searrow \sqcup_i v_i & \downarrow x & \swarrow w & \\ & & \text{Ob}(\mathbf{Set}) & & \end{array}$$

such that i and j are jointly surjective.

Composition of morphisms is easily understood in pictures: given wiring diagrams inside of wiring diagrams, we can throw away the intermediary circles. In terms of sets, we perform a pushout.

There is a morphism of operads $\mathcal{W} \rightarrow \mathcal{S}$ given by sending (C, v) to $\prod_{c \in C} v(c)$. The idea is that to an entity defined as having a bunch of cables carrying variables, a phenomenon is the same thing as a choice of value on each cable. A wiring diagram translates between values experienced locally and values experienced globally.

Application 5.4.2.9. In cognitive neuroscience or in economics, it may be that we want to understand the behavior of an entity such as a mind or a society in terms of its structure. Knowing the connection pattern (**connectome**, **supply chain**) of sub-entities should help us understand how big changes are generated from small ones.

Under the morphism $\mathcal{W} \rightarrow \mathcal{S}$ the algebra $\mathcal{S} \rightarrow \mathbf{Sets}$ from Application 5.4.2.7 becomes an algebra $\mathcal{W} \rightarrow \mathbf{Sets}$. To each entity we now associate some subset of the value-assignments it can carry. ♦♦

Application 5.4.2.10. In [RS], **Radul and Sussman** discuss propagator networks. These can presumably be understood in terms of wiring diagrams and their algebra of relations. ♦♦

¹³If one is concerned with cardinality issues, fix a cardinality κ and replace $\text{Ob}(\mathbf{Set})$ everywhere with $\text{Ob}(\mathbf{Set}_{<\kappa})$.

Index

a category

Cat, 112
FLin, 103
Fin, 143
Grp, 103
Grpd, 123
Grph, 104
Mon, 102
PrO, 103
Prop, 125
Sch, 147
Set, 102
Star_n, 161
Top, 122
Vect, 122, 195
 Δ , 143, 172
C-Set, 138
sSet, 172
terminal, 114

a functor

Disc: **Set** $\rightarrow$ **Cat**, 114, 134
Disc: **Set** $\rightarrow$ **Grph**, 114
Ind: **Set** $\rightarrow$ **Cat**, 177
Cat $\rightarrow$ **Grph**, 114, 184
Cat $\rightarrow$ **Sch**, 149
FLin $\rightarrow$ **PrO**, 108
Grp $\rightarrow$ **Cat**, 116
Grp $\rightarrow$ **Grpd**, 123
Grp $\rightarrow$ **Mon**, 108
Grpd $\rightarrow$ **Cat**, 123
Grph $\rightarrow$ **Cat**, 113, 184
Grph $\rightarrow$ **PrO**, 109
Grph $\rightarrow$ **Set**, 109, 132, 184
List: **Set** $\rightarrow$ **Set**, 132
Mon $\rightarrow$ **Cat**, 115
Mon $\rightarrow$ **Set**, 107, 181
Ob: **Cat** $\rightarrow$ **Set**, 114, 134, 184
Paths: **Grph** $\rightarrow$ **Grph**, 111, 113, 131, 147
 Π_1 : **Top** $\rightarrow$ **Grpd**, 124

PrO $\rightarrow$ **Cat**, 118, 142, 154, 158
PrO $\rightarrow$ **Grph**, 108, 118, 184
PrO $\rightarrow$ **Set**, 109, 183
PrO $\rightarrow$ **Top**, 201
Sch $\rightarrow$ **Cat**, 149
Set $\rightarrow$ **Mon**, 110, 181
Set $\rightarrow$ **PrO**, 183
Top $\rightarrow$ **PrO^{op}**, 122
Top $\rightarrow$ **Set**, 122
Vect_R $\rightarrow$ **Grp**, 123
Vect_R $\rightarrow$ **PrO**, 123
Vect_R $\rightarrow$ **Top**, 123
 Δ $\rightarrow$ **FLin**, 143

a group

E_3 , 76
 GL_3 , 76
 $U(1)$, 77
 Σ_X , 77

a monad

Paths, 206
exceptions, 206
List, 205
partial functions, 203

a schema

Loop, 152, 211
department store, 93
indexing graphs, 139

a symbol

$(F \downarrow G)$, 177
 $X/\sim$, 47
Fun, 133
Hom_{Set}, 16
Hom_C, 101, 213
N, 13
Ob, 101, 213
 Ω , 57
 $\mathbb{P}$, 55
Path, 81
 $\mathbb{R}$, 31
 $\mathbb{Z}$, 13

- $\hookrightarrow$, 67
- $\circ$, 16, 101, 214
- colim, 168
- $\times$, 30
- $\diamond$, 137
- $\emptyset$, 13
- $\exists$, 14, 184
- $\exists!$, 14
- $\forall$, 14, 184
- id_X , 16
- $\int$, 173
- $\cong$, 17
- $\triangleleft$, 161
- lim, 166
- $\lrcorner$, 39
- $\mathcal{C}^{\text{op}}$, 172
- $\mathcal{C}/X$, 166
- $\mathcal{C}_{X/}$, 168
- $++$, 65
- $\triangleright$, 161
- $\sim$, 47
- $\simeq$, 27, 142
- $\sqcup$, 34
- $\lceil$, 49
- f^{-1} , 41
- action
 - of a group, 76
 - of a monoid, 67
 - orbit of, 77
- action table, 71
- adjunction, 182
 - mate isomorphism, 182
- algebra
 - operad, 216
- an operad
 - Sets**, 214
 - little n -cubes, 214
- Baez, John, 8
- canonical, 18
- cardinality, 19
- category, 101
 - arithmetic of, 179
 - as equivalent to schema, 146
 - cartesian closed, 126
 - comma, 177
 - coslice, 168
 - discrete, 114
 - equivalence of, 142
 - free category, 113, 196
 - Kleisli, 207
 - of elements, 173
 - opposite, 172
 - slice, 166
 - small, 102
 - underlying graph of, 113
- CCCs, 126
- characteristic function, 57
- coequalizer, 52
- colimit, 168
- common ground, 201
- commuting diagram, 19
- component, 128
- cone
 - left, 161
 - right, 161
- context, 203
- coproducts, 155
 - of sets, 34
 - universal property of, 36
- correspondence
 - one-to-one, 17
- coslice, 168
- cospan, 158
- CPER, 95
- currying, 53
- data, 92
 - valid time, 202
- data migration, 187
 - left pushforward Σ , 190
 - pullback Δ , 188
 - right pushforward Π , 192
- database
 - business rules, 93
 - category of instances on, 138
 - foreign key, 92
 - instance, 99, 121
 - Kleisli, 210
 - primary key, 92
 - schema, 96
 - tables, 91
- diagram
 - commutes, 19
- diagram, 159
 - in **Set**, 19
- Dolan, James, 8
- dynamical system

- continuous, 122
 - discrete, 97
- Eilenberg, Samuel, 7
- entry
 - in list, 65
- epimorphism
 - in **Set**, 58
- equalizer, 46, 167
- equivalence relation, 47
 - as partition, 47
 - equivalence classes, 47
 - generated, 48
 - quotient by, 47
- exceptions, 206
- exponentials
 - evaluation of, 54
- exponentials
 - in **Set**, 53
- fiber product, 38
- fiber sum, 49
- finite state machine, 70, 176
- function, 14
 - bijection, 58
 - codomain, 14
 - composition, 16
 - domain, 14
 - equality of, 16
 - identity, 16
 - injection, 58
 - inverse, 17
 - isomorphism, 17
 - restriction of, 15
 - surjection, 58
- functor, 107
 - adjoint, 182
 - constant, 185
 - contravariant, 171
 - covariant, 171
 - faithful, 145
 - full, 145
 - representable, 194
- gateway, 153
- geography, 91, 196
- graph, 78
 - as functor, 119
 - bipartite, 46
 - free category on, 113, 196
 - homomorphism, 82
 - paths, 81
 - paths-graph, 111, 206
 - symmetric, 119
- graph homomorphism
 - as functor, 139
- Grothendieck, 173
 - construction, 173
 - expanding universes, 102
 - in history, 7
- group, 75
 - action, 76
 - as category, 116
 - homomorphism of, 78
 - of automorphisms, 117
- groupoid, 123
 - fundamental, 124
 - of material states, 123
- hierarchy, 97
- hom-set, 101
- image, 16
 - in olog, 29
- indexed set, 62
 - as functor, 139
- indexing category, 159
- induced function, 33
- infix notation, 64
- information theory, 127
- initial object, 162
- instance, 99, 121
 - Kleisli, 210
- isomorphism, 104
- join, 87
- Joyal, André, 8
- Kan, Daniel, 8
- Kleisli category, 207
- labeled null, 191
- Lambek, Joachim, 8
- Lawvere, William, 8
- leaf table, 189
- limit, 166
- linear order
 - finite, 86
- list, 65, 205
 - as functor, 111

- concatenation, 65
- local-global, 7
- Mac Lane, Saunders, 7
- Markov chain, 211
- mate, 182
- materials
 - force extension curves, 53
 - force-extension curves, 15
- meet, 87
- Moggi, Eugenio, 8
- monad, 203, 204
 - formalizing context, 203
 - Kleisli category of, 207
 - on **Grph**, 206
 - on **Set**, 204
- monoid, 63
 - action, 67
 - additive natural numbers, 64
 - as category, 114
 - commutative, 64
 - cyclic, 67
 - free, 65, 110
 - homomorphism, 72
 - identity element of, 64
 - initial, 163
 - inverse of an element in, 75
 - multiplication formula, 64
 - of endomorphisms, 116
 - olog of, 69
 - presented, 66
 - terminal, 163
 - trivial, 65
 - trivial homomorphism, 73
- monomorphism
 - in **Set**, 58
- morphism, 101
 - inverse, 104
- multicategory, 212
- multiset, 60
- natural isomorphism, 135
- natural transformation, 128
 - as functor, 167
 - as refinement of model, 129
 - for adding functionality, 136
 - horizontal composition of, 137
 - interchange, 137
 - vertical composition of, 133
 - whiskering of, 137
- olog, 20
 - and sheaves, 201
 - as database schema, 98
 - aspects, 22
 - facts, 26
 - facts in English, 28
 - images, 29
 - invalid aspects, 23
 - path in, 26
 - rules, 22, 26, 96
 - types, 21
- one-to-one correspondence, 17
- open cover, 198
- operad, 213
 - algebra of, 216
 - colored, 212
 - morphism of, 216
- orbit, 77
- order, 85
 - linear order, 85
 - morphism, 88
 - opposite, 88
 - partial order, 85
 - preorder, 85
- partial functions, 203
- path, 81
- PED, 95
- permutation, 76
- power set, 55
 - as poset, 87
- preimage, 41, 184
- preorder
 - as category, 117
 - clique in, 87
 - converting to graph, 86
 - discrete, 89
 - generated, 87
 - indiscrete, 89
 - join, 87
 - meet, 87
- presheaf, 197
- product
 - as grid, 30
- products, 150, 153, 164
 - as not always existing, 154
 - of sets, 30
 - universal property of, 32
- pullback, 166

- of sets, 38
- pushout, 168
 - of topological spaces, 170
- RDF, 173
 - as category of elements, 174
- relation
 - binary, 83
 - equivalence, 47
 - graph of, 84
- relative set, 61, 62
 - as slice category, 168
- representable functor, 194
- representation theory, 195
- restriction of scalars, 74
- retraction, 53
- retractions, 53
- RNA transcription, 17
- schema, 96
 - as category presentation, 120
 - as equivalent to category, 146
 - Categorical path equivalence relation (CPER), 95
 - fact table, 188
 - leaf table, 94, 188
 - morphism, 147
 - Path equivalence declaration (PED), 95
- schematically implied reference spread, 195
- security, 90
- set, 13
 - arithmetic of, 54
 - Lawvere's description of, 126
 - permutation of, 76
 - set builder notation, 14
- sheaf
 - condition, 198
 - glueing, 199
- sheaves, 197
- simplex, 56
- simplicial complex, 56, 201
- simplicial set, 172
- skeleton, 144
- Skolem, 195
- Skolem variable, 191
- slice, 166
- space, 90, 121
 - topological, 122
- space group, 76
- span, 44
 - composite, 44
- subcategory
 - full, 177
- subobject classifier, 57
- subset, 13
 - as function, 15
 - characteristic function of, 57
- subway, 171
- taxonomy, 89
- terminal object, 162
 - in **Set**, 46
- topological space, 122
- topology, 122
- trivial homomorphism
 - of monoids, 73
- universal property, 153
 - products, 32
 - pullback, 166
- vector field, 104, 124
 - conservative, 124
- vector space, 122, 195
- wiring diagram, 219
- Yoneda's lemma, 195

Bibliography

- [Awo] S. Awodey. (2010) *Category theory*. Second edition. Oxford Logic Guides, 52. Oxford University Press, Oxford.
- [BD] Baez, J.C.; Dolan, J. (1995) “Higher-dimensional algebra and topological quantum field theory”. *Journal of mathematical physics* vol 36, 6073.
- [BS] Baez, J.C.; Stay, M. (2011) “Physics, topology, logic and computation: a Rosetta Stone.” *New structures for physics*, 95D172. Lecture Notes in Phys., 813, Springer, Heidelberg.
- [BW] M. Barr, C. Wells. (1990) *Category theory for computing science*. Prentice Hall International Series in Computer Science. Prentice Hall International, New York.
- [Big] Biggs, N.M. (2004) *Discrete mathematics*. Oxford University Press, NY.
- [DI] Döring, A.; Isham, C. J. “A topos foundation for theories of physics. I. Formal languages for physics.” *J. Math. Phys.* 49 (2008), no. 5, 053515.
- [Gog] Goguen, J. (1992) “Sheaf semantics for concurrent interacting objects” *Mathematical structures in Computer Science* Vol 2, pp. 159 – 191.
- [Lam] (1980) “From λ -calculus to Cartesian closed categories”. In *Formalism*, Academic Press, London, pp. 375 – 402.
- [Law] Lawvere, F.W. (2005) “An elementary theory of the category of sets (long version) with commentary.” (Reprinted and expanded from Proc. Nat. Acad. Sci. U.S.A. **52** (1964)) *Repr. Theory Appl. Categ.* 11, pp. 1 – 35.
- [Le1] Leinster, T. (2004) *Higher Operads, Higher Categories*. London Mathematical Society Lecture Note Series 298, Cambridge University Press.
- [Le2] Leinster, T. (2012) “Rethinking set theory”. ePrint available <http://arxiv.org/abs/1212.6543>.
- [LS] F.W. Lawvere, S.H. Schanuel. (2009) *Conceptual mathematics. A first introduction to categories*. Second edition. Cambridge University Press, Cambridge.
- [Mac] (1998) S. Mac Lane. *Categories for the working mathematician*. Second edition. Graduate Texts in Mathematics, 5. Springer-Verlag, New York.

- [Mar] Marquis, J-P, “Category Theory”, *The Stanford Encyclopedia of Philosophy* (Spring 2011 Edition), Edward N. Zalta (ed.), <http://plato.stanford.edu/archives/spr2011/entries/category-theory>
- [Mog] Moggi, E. (1989) “A category-theoretic account of program modules.” *Category theory and computer science (Manchester, 1989)*, 101–117, Lecture Notes in Comput. Sci., 389, Springer, Berlin.
- [nLa] nLab authors. <http://ncatlab.org/nlab/show/HomePage>
- [Pen] Penrose, R. (2006) *The road to reality*. Random house.
- [RS] Radul, A.; Sussman, G.J. (2009). “The art of the propagator”. *MIT Computer science and artificial intelligence laboratory technical report*.
- [Sp1] Spivak, D.I. (2012) “Functorial data migration”. *Information and communication*
- [Sp2] Spivak, D.I. (2012) “Queries and constraints via lifting problems”. Submitted to *Mathematical structures in computer science*. ePrint available: <http://arxiv.org/abs/1202.2591>
- [Sp3] Spivak, D.I. (2012) “Kleisli database instances”. ePrint available: <http://arxiv.org/abs/1209.1011>
- [SGWB] Spivak D.I., Giesa T., Wood E., Buehler M.J. (2011) “Category Theoretic Analysis of Hierarchical Protein Materials and Social Networks.” *PLoS ONE* 6(9): e23911. doi:10.1371/journal.pone.0023911
- [SK] Spivak, D.I., Kent, R.E. (2012) “Ologs: A Categorical Framework for Knowledge Representation.” *PLoS ONE* 7(1): e24274. doi:10.1371/journal.pone.0024274.
- [WeS] Weinberger, S. (2011) “What is... Persistent Homology?” AMS.
- [WeA] Weinstein, A. (1996) “Groupoids: unifying internal and external symmetry. *Notices of the AMS* Vol 43, no. 7, pp. 744 – 752.
- [Wik] [Wikipedia](#) (multiple authors). Various articles, all linked with a hyperreference are scattered throughout this text. All accessed December 6, 2012 – April 1, 2018.